

Rüdiger Brause

Betriebssysteme

Grundlagen und Konzepte

4. Auflage



Springer Vieweg

Betriebssysteme

Rüdiger Brause

Betriebssysteme

Grundlagen und Konzepte

4., erweiterte Auflage



Springer Vieweg

Rüdiger Brause
J.W. Goethe-Universität Frankfurt am Main
Frankfurt
Hessen
Deutschland

Die Darstellung von manchen Formeln und Strukturelementen war in einigen elektronischen Ausgaben nicht korrekt, dies ist nun korrigiert. Wir bitten damit verbundene Unannehmlichkeiten zu entschuldigen und danken den Lesern für Hinweise.

ISBN 978-3-662-54099-2 ISBN 978-3-662-54100-5 (eBook)
DOI 10.1007/978-3-662-54100-5

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.d-nb.de> abrufbar.

Springer Vieweg

© Springer-Verlag GmbH Deutschland 1998, 2001, 2004, 2017

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die Wiedergabe von Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. in diesem Werk berechtigt auch ohne besondere Kennzeichnung nicht zu der Annahme, dass solche Namen im Sinne der Warenzeichen- und Markenschutz-Gesetzgebung als frei zu betrachten wären und daher von jedermann benutzt werden dürften. Der Verlag, die Autoren und die Herausgeber gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag, noch die Autoren oder die Herausgeber übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Gedruckt auf säurefreiem und chlorfrei gebleichtem Papier

Springer Vieweg ist Teil von Springer Nature

Die eingetragene Gesellschaft ist Springer-Verlag GmbH Deutschland

Die Anschrift der Gesellschaft ist: Heidelberger Platz 3, 14197 Berlin, Germany

Vorwort

Betriebssysteme sind sehr, sehr beharrlich – fast kein Systemprogrammierer oder Informatiker wird zeit seines Lebens mit der Aufgabe konfrontiert werden, ein komplett neues Betriebssystem zu schreiben. Wozu dient dann dieses Buch?

Seine Zielsetzung besteht hauptsächlich darin, Verständnis für die innere Struktur der heutigen komplexen Betriebssysteme zu wecken. Erst das Wissen um grundsätzliche Aufgaben und mögliche Alternativen versetzt Informatiker in die Lage, mögliche leistungshemmende Strukturen ihres Betriebssystems zu erkennen und die Wirkung von Änderungen in den vielen Parametern und Konfigurationen abzuschätzen.

Statt eine komplette analytische Erfassung der Systemleistung vorzugaukeln, die meist nur über spezielle Simulations- und Leistungsmessprogramme möglich ist, konzentriert sich das Buch darauf, gedankliche Konzepte und Methoden zu vermitteln, die in der Praxis wichtig sind und in eigenen Programmen auch angewendet werden können.

Angesichts der vielfältigen Veränderungen der Betriebssystemlandschaft durch die Vernetzung der Rechner und die Vielzahl parallel eingesetzter Prozessoren ist es schwierig, einen klassischen Text über Betriebssysteme zu verfassen, ohne Rechnernetze und Multiprozessorsysteme einzubeziehen. Andererseits ist ein Verständnis dieser Funktionen ohne die klassischen Betriebssystemthemen nicht möglich. Aus diesem Grund geht das Buch auf beide Seiten ein: Ausgehend von der klassischen Einprozessorsituation über die Mehrprozessorproblematik behandelt es die klassischen Themen wie Prozesse, Speicherverwaltung und Ein-/Ausgabeverwaltung. Über die klassischen Themen hinaus wendet es sich den Netzwerkdiensten, dem Thema Sicherheit und den Benutzeroberflächen zu; alle drei gehören zu einem modernen Betriebssystem.

Dabei werden sowohl Einprozessor- als auch Mehrprozessorsysteme betrachtet und die Konzepte an existierenden Betriebssystemen verdeutlicht. Viele Mechanismen der verteilten Betriebssysteme und Echtzeitsysteme werden bereits in diesem Buch vorgestellt, um damit eine spätere Vertiefung im Bedarfsfall zu erleichtern.

Im Unterschied zu manch anderen Betriebssystembüchern steht bei den Beispielen kein eigenes Betriebssystem im Vordergrund; auch die Beispiele in Pseudocode sind nicht als vollständige Programmoduln eines Betriebssystems gedacht. Bei einem solchen Ansatz käme entweder die Darstellung der Konzepte zu kurz, oder das Buch müsste wesentlich länger sein, ohne mehr aussagen zu können. Aus der Fülle der existierenden

experimentellen und kommerziellen Systeme konzentriert sich das Buch deshalb auf die in der Praxis wichtigsten Systeme wie UNIX und Windows NT und benennt die für das jeweilige Konzept wesentlichen Prozeduren. Die detaillierte Dokumentation der Benutzerschnittstellen bleibt allerdings aus Platzgründen den einschlägigen Handbüchern der Softwaredokumentation vorbehalten und wurde deshalb hier nicht aufgenommen.

Ein spezieller Aspekt des Buches ist den Lernenden gewidmet: Zusätzlich zu den Übungs- und Verständnisaufgaben sind Musterlösungen am Ende des Buches enthalten. Außerdem sind alle wichtigen Begriffe bei der Einführung fett gedruckt, um die Unsicherheit beim Aneignen eines fremden Stoffes zu verringern; Fachwörter und englische Originalbegriffe sind durch Kursivschrift gekennzeichnet. Wo immer es möglich und nötig erschien, sollen Zeichnungen das Verständnis des Textes erleichtern.

Dazu kommen noch für den Lernenden und Lehrenden die Vorlesungsvideos, Übungsfragen, Klausuren und Vortragsfolien, welche die Verwendung des Buches erleichtern sollen. Sie sind elektronisch direkt unter der URL <http://www.asa.cs.uni-frankfurt.de/bs/> herunterladbar.

Ich hoffe, mit den aktuellen Verbesserungen und Ergänzungen der vierten Auflage alle Interessierte motiviert zu haben, sich mit dem Unterbau unseres Computerzeitalters genauer zu beschäftigen. Dabei sollen aber nicht die vielen kritischen Anmerkungen und Hinweise vieler Menschen unterschlagen werden, deren Beiträge dieses Buch erst reifen ließen. Neben den Tutoren und Vorlesungsteilnehmern möchte ich mich dabei besonders bei Herrn Eric Hutter bedanken, der durch seine ausführlichen Anmerkungen die neueste Edition entscheidend verbessert hat.

Frankfurt, im November 2016

Rüdiger Brause

Inhaltsverzeichnis

- 1 Übersicht 1
 - 1.1 Einleitung: Was ist ein Betriebssystem? 2
 - 1.2 Betriebssystemschichten 2
 - 1.3 Betriebssystemaufbau 4
 - 1.3.1 Systemaufrufe 5
 - 1.3.2 Beispiel UNIX 6
 - 1.3.3 Beispiel Mach 7
 - 1.3.4 Beispiel Windows NT 8
 - 1.4 Schnittstellen und virtuelle Maschinen 11
 - 1.5 Software-Hardware-Migration 15
 - 1.6 Virtuelle Betriebssysteme 16
 - 1.6.1 Paravirtualisierung 18
 - 1.6.2 Virtualisierung mit BS-Containern 19
 - 1.6.3 Vollständige Virtualisierung 19
 - 1.6.4 Hardware-Virtualisierung 19
 - 1.6.5 Virtualisierungserweiterungen 20
 - 1.7 Betriebssysteme für eingebettete Systeme 21
 - 1.8 Betriebssysteme in Mehrprozessorarchitekturen 22
 - 1.9 Aufgaben 25
 - Literatur 26
- 2 Prozesse 27
 - 2.1 Prozesszustände 29
 - 2.1.1 Beispiel UNIX 30
 - 2.1.2 Beispiel Windows NT 33
 - 2.1.3 Leichtgewichtsprozesse 34
 - 2.1.4 Aufgaben 37
 - 2.2 Prozessscheduling 39
 - 2.2.1 Zielkonflikte 40
 - 2.2.2 Non-präemptives Scheduling 41
 - 2.2.3 Präemptives Scheduling 44

2.2.4	Mehrere Warteschlangen und Scheduler	46
2.2.5	Scheduling in Echtzeitbetriebssystemen	47
2.2.6	Scheduling in Multiprozessorsystemen	53
2.2.7	Stochastische Schedulingmodelle	60
2.2.8	Beispiel UNIX: Scheduling	62
2.2.9	Beispiel: Scheduling in Windows NT	64
2.2.10	Aufgaben	65
2.3	Prozesssynchronisation	67
2.3.1	<i>Race conditions</i> und kritische Abschnitte	68
2.3.2	Signale, Semaphore und atomare Aktionen	69
2.3.3	Beispiel UNIX: Semaphore	77
2.3.4	Beispiel Windows NT: Semaphore	78
2.3.5	Anwendungen	79
2.3.6	Aufgaben zur Prozesssynchronisation	85
2.3.7	Kritische Bereiche und Monitore	88
2.3.8	Verklemmungen	92
2.3.9	Aufgaben zu Verklemmungen	101
2.4	Prozesskommunikation	103
2.4.1	Kommunikation mit Nachrichten	103
2.4.2	Beispiel UNIX: Interprozesskommunikation mit <i>pipes</i>	107
2.4.3	Beispiel Windows NT: Interprozesskommunikation mit <i>pipes</i>	108
2.4.4	Prozesssynchronisation durch Kommunikation	108
2.4.5	Implizite und explizite Kommunikation	117
2.4.6	Aufgaben zur Prozesskommunikation	118
	Literatur	118
3	Speicherverwaltung	121
3.1	Direkte Speicherbelegung	122
3.1.1	Zuordnung durch feste Tabellen	123
3.1.2	Zuordnung durch verzeigerte Listen	123
3.1.3	Belegungsstrategien	125
3.1.4	Aufgaben zur Speicherbelegung	128
3.2	Logische Adressierung und virtueller Speicher	129
3.2.1	Speicherprobleme und Lösungen	129
3.2.2	Der virtuelle Speicher	130
3.3	Seitenverwaltung (<i>paging</i>)	132
3.3.1	Prinzip der Adresskonversion	132
3.3.2	Adresskonversionsverfahren	133
3.3.3	Gemeinsam genutzter Speicher (<i>shared memory</i>)	137
3.3.4	Virtueller Speicher in UNIX und Windows NT	138
3.3.5	Aufgaben zu virtuellem Speicher	142
3.3.6	Seitenersetzungsstrategien	144

3.3.7	Modellierung und Analyse der Seitenersetzung	151
3.3.8	Beispiel UNIX: Seitenersetzungsstrategien	159
3.3.9	Beispiel Windows NT: Seitenersetzungsstrategien	161
3.3.10	Aufgaben zur Seitenverwaltung	162
3.4	Segmentierung	164
3.5	Cache	167
3.6	Speicherschutzmechanismen	170
3.6.1	Speicherschutz in UNIX	171
3.6.2	Speicherschutz in Windows NT	172
3.6.3	Sicherheitsstufen und <i>virtual mode</i>	173
	Literatur	174
4	Dateiverwaltung	175
4.1	Dateisysteme	176
4.2	Dateinamen	178
4.2.1	Dateitypen und Namensbildung	178
4.2.2	Pfadnamen	182
4.2.3	Beispiel UNIX: Der Namensraum	183
4.2.4	Beispiel Windows NT: Der Namensraum	185
4.2.5	Aufgaben	187
4.3	Dateiattribute und Sicherheitsmechanismen	188
4.3.1	Beispiel UNIX: Zugriffsrechte	188
4.3.2	Beispiel Windows NT: Zugriffsrechte	190
4.3.3	Aufgaben	191
4.4	Dateifunktionen	192
4.4.1	Standardfunktionen	192
4.4.2	Beispiel UNIX: Dateizugriffsfunktionen	193
4.4.3	Beispiel Windows NT: Dateizugriffsfunktionen	194
4.4.4	Strukturierte Zugriffsfunktionen	195
4.4.5	Gemeinsame Nutzung von Bibliotheksdateien	202
4.4.6	Speicherabbildung von Dateien (<i>memory mapped files</i>)	204
4.4.7	Besondere Dateien (<i>special files</i>)	206
4.4.8	Aufgaben zu Dateikonzepten	207
4.5	Implementierung der Dateiorganisation	209
4.5.1	Kontinuierliche Speicherzuweisung	209
4.5.2	Listenartige Speicherzuweisung	210
4.5.3	Zentrale indexbezogene Speicherzuweisung	210
4.5.4	Verteilte indexbezogene Speicherzuweisung	212
4.5.5	Beispiel UNIX: Implementierung des Dateisystems	213
4.5.6	Beispiel Windows NT: Implementierung des Dateisystems	215
4.5.7	Aufgaben zu Dateimplementierungen	219
	Literatur	219

5	Ein- und Ausgabeverwaltung	221
5.1	Die Aufgabenschichtung	222
5.1.1	Beispiel UNIX: I/O-Verarbeitungsschichten	224
5.1.2	Beispiel Windows NT: I/O-Verarbeitungsschichten	227
5.2	Gerätemodelle	229
5.2.1	Geräteschnittstellen	230
5.2.2	Initialisierung der Geräteschnittstellen	231
5.2.3	Plattenspeicher	231
5.2.4	SSD-RAM-Disks	237
5.2.5	Serielle Geräte	239
5.2.6	Aufgaben zu I/O	240
5.3	Multiple Plattenspeicher: RAID	240
5.3.1	RAID-Ausfallwahrscheinlichkeiten	246
5.3.2	Aufgaben zu RAID	248
5.4	Modellierung und Implementierung der Treiber	249
5.4.1	Beispiel UNIX: Treiberschnittstelle	250
5.4.2	Beispiel Windows NT: Treiberschnittstelle	252
5.4.3	I/O Treiber in Großrechnern	255
5.4.4	Aufgaben zu Treibern	255
5.5	Optimierungsstrategien für Treiber	256
5.5.1	Optimierung bei Festplatten	256
5.5.2	Pufferung	257
5.5.3	Synchrone und asynchrone Ein- und Ausgabe	259
5.5.4	Aufgaben zur Treiberoptimierung	261
	Literatur	261
6	Netzwerkdienste	263
6.1	Das Schichtenmodell für Netzwerkdienste	269
6.2	Kommunikation im Netz	274
6.2.1	Namensgebung im Netz	275
6.2.2	Kommunikationsanschlüsse	280
6.2.3	Aufgaben	288
6.3	Dateisysteme im Netz	289
6.3.1	Zugriffssemantik	289
6.3.2	Zustandsbehaftete und zustandslose Server	291
6.3.3	Die Cacheproblematik	293
6.3.4	Implementationskonzepte	295
6.3.5	Die WebDAV-Dienste	299
6.3.6	Sicherheitskonzepte	300
6.3.7	Aufgaben	302
6.4	Massenspeicher im Netz	302
6.5	Arbeitsmodelle im Netz	305

6.5.1	Jobmanagement	305
6.5.2	Netzcomputer	306
6.5.3	Schattenserver	309
6.5.4	Aufgaben zu Arbeitsmodellen im Netz	313
6.6	Middleware und SOA	314
6.6.1	Transparenz durch Middleware	315
6.6.2	Vermittelnde Dienste	316
6.6.3	Service-orientierte Architektur	318
6.6.4	Aufgaben zu Middleware	320
	Literatur	321
7	Sicherheit	323
7.1	Vorgeschichte	324
7.2	Passworte	324
7.2.1	Passwort erfragen	325
7.2.2	Passwort erraten	325
7.2.3	Passwort abhören	326
7.3	Trojanische Pferde	326
7.4	Der <i>buffer overflow</i> -Angriff	327
7.5	Viren	328
7.5.1	Wie kann man einen eingeschleppten Virus entdecken?	330
7.5.2	Was kann man dagegen tun, wenn man einen Virus im System entdeckt hat?	330
7.5.3	Aufgaben zu Viren	331
7.6	Rootkits	332
7.6.1	User mode-Rootkits	332
7.6.2	Kernel mode-Rootkits	333
7.6.3	Ring -1, Ring -2, Ring -3 Rootkits	334
7.6.4	Entdeckung und Bekämpfung der Rootkits	335
7.7	Zugriffsrechte und Rollen	336
7.7.1	Zugriffsrechte im Dateisystem	337
7.7.2	Zugriffsrechte von Systemprogrammen	337
7.7.3	Zugriffslisten und Fähigkeiten	338
7.7.4	Aufgaben	339
7.8	Vertrauensketten	340
7.9	Sicherheit in Unix und Windows NT	341
7.10	Sicherheit im Netz: Firewall-Konfigurationen	343
7.11	Die Kerberos-Authentifizierung	345
7.11.1	Das Kerberos-Protokoll	346
7.11.2	Kerberos Vor- und Nachteile	348
7.11.3	Aufgaben zur Netz-Sicherheit	349
	Literatur	350

8 Benutzeroberflächen	351
8.1 Das Design der Benutzeroberfläche	352
8.2 Die Struktur der Benutzeroberfläche	355
8.2.1 Eingaben	356
8.2.2 Rastergrafik und Skalierung	360
8.2.3 Fenstersysteme und Displaymanagement	362
8.2.4 Virtuelle Realität	365
8.2.5 Das Management der Benutzeroberfläche	365
8.2.6 Aufgaben	367
8.3 Das UNIX-Fenstersystem: Motif und X-Window	368
8.3.1 Das Client-Server-Konzept von X-Window	369
8.3.2 Das Fensterkonzept von X-Window	370
8.3.3 Dialogfenster und Widgets	371
8.3.4 Ereignisbehandlung	373
8.3.5 Sicherheitsaspekte	374
8.4 Das Fenstersystem von Windows NT	375
8.4.1 Das Konzept der Benutzerschnittstelle	375
8.4.2 Die Implementierung	377
8.4.3 Aufgaben	379
Literatur	380
9 Musterlösungen	381
9.1 Lösungen zu Kap. 1	381
9.2 Lösungen zu Kap. 2	386
9.3 Lösungen zu Kap. 3	416
9.4 Lösungen zu Kap. 4	428
9.5 Lösungen zu Kap. 5	439
9.6 Lösungen zu Kap. 6	448
9.7 Lösungen zu Kap. 7	455
9.8 Lösungen zu Kap. 8	460
Literatur	463
10 Anhang	465
10.1 Modellierung von Thrashing	465
Weiterführende Literatur	473
Stichwortverzeichnis	475

Über den Autor

Prof. Dr. rer. nat. habil. Rüdiger W. Brause Von 1970–1978 Studium der Physik und Kybernetik in Saarbrücken und Tübingen mit Abschluß als Diplom-Physiker. 1980–1988 Konzeption und Implementierung des fehlertoleranten, lastverteilten Multi-Mikroprozessorsystems ATTEMPTO in der Arbeitsgruppe von Prof. Dal Cin. 1983 Promotion mit einer Arbeit zum Thema „Fehlertoleranz in verteilten Systemen“. Ab 1985 Akad. Oberrat an der Universität Frankfurt. A. M. im Fachbereich Informatik, in dem er sich 1993 habilitierte. Ab 2005 Professor für Praktische Informatik. Im Ruhestand seit April 2016.

Abbildungsverzeichnis

Abb. 1.1	Benutzungsrelationen von Programmteilen	3
Abb. 1.2	Schichtenmodell und Zwiebelschalenmodell	3
Abb. 1.3	Die Sicherheitsstufen (Ringe) der Intel 80X86-Architektur	4
Abb. 1.4	Überblick über die Rechnersoftwarestruktur	4
Abb. 1.5	Befehlsfolge eines Systemaufrufs	5
Abb. 1.6	UNIX-Schichten	6
Abb. 1.7	Das Mach-Systemmodell.	8
Abb. 1.8	Schichtung und Aufrufe bei Windows NT	9
Abb. 1.9	Echte und virtuelle Maschinen	11
Abb. 1.10	Hierarchie virtueller Maschinen	12
Abb. 1.11	Virtuelle, logische und physikalische Geräte.	13
Abb. 1.12	Ein Netzwerk als virtueller Massenspeicher: die asymmetrische Pool-Lösung	14
Abb. 1.13	Software-Hardware-Migration des Java-Codes	15
Abb. 1.14	Virtualisierungsarten bei Servern.	17
Abb. 1.15	Schichten bei der Paravirtualisierung	18
Abb. 1.16	Schichten bei der vollständigen Virtualisierung	20
Abb. 1.17	Hardwareerweiterung für die Virtualisierung	21
Abb. 1.18	Programmentwicklung bei eingebetteten Systemen	22
Abb. 1.19	Ein Einprozessor- oder Mehrkernsystem.	23
Abb. 1.20	Multiprozessorsystem	23
Abb. 1.21	Mehrrechnersystem.	24
Abb. 1.22	Rechnernetz.	24
Abb. 2.1	Zusammensetzung der Prozessdaten	28
Abb. 2.2	Prozesszustände und Übergänge	30
Abb. 2.3	Prozesszustände und Übergänge bei UNIX	31
Abb. 2.4	Erzeugung und Vernichten eines Prozesses in UNIX	31
Abb. 2.5	Prozesszustände in Windows NT.	33
Abb. 2.6	Ausführung der Prozesse in UNIX und Windows NT	34
Abb. 2.7	I/O-Verhalten (a) leichter und (b) schwerer threads	36
Abb. 2.8	Langzeit- und Kurzzeitscheduling	40

Abb. 2.9	Job-Reihenfolgen	42
Abb. 2.10	Präemptives Scheduling	44
Abb. 2.11	Scheduling mit mehreren Warteschlangen	46
Abb. 2.12	Multi-level Scheduling	46
Abb. 2.13	Präzedenzgraph	54
Abb. 2.14	Gantt-Diagramm der Betriebsmittel	54
Abb. 2.15	Gantt-Diagramm bei der <i>earliest scheduling</i> -Strategie	56
Abb. 2.16	Gantt-Diagramm bei <i>latest scheduling</i> -Strategie.	56
Abb. 2.17	Optimaler Ablaufplan von drei Prozessen auf zwei Prozessoren	57
Abb. 2.18	Asymmetrisches und symmetrisches Multiprocessing.	59
Abb. 2.19	Multi-level-Warteschlangen in UNIX	62
Abb. 2.20	Dispatcher ready queue in Windows NT	64
Abb. 2.21	Race conditions in einer Warteschlange	68
Abb. 2.22	Erster Versuch zur Prozesssynchronisation.	69
Abb. 2.23	Zweiter Versuch zur Prozesssynchronisation.	70
Abb. 2.24	Prozesssynchronisation nach Peterson	71
Abb. 2.25	Ein Beispiel für das <i>Apartment</i> -Modell in Windows NT.	79
Abb. 2.26	Das Erzeuger-Verbraucher-Problem	81
Abb. 2.27	Lösung des Erzeuger-Verbraucher-Problems mit Semaphoren	82
Abb. 2.28	Der Ringpuffer des NYU-Ultracomputers	84
Abb. 2.29	Ein- und Aushängen bei der ready-queue	85
Abb. 2.30	Lösung des Erzeuger-Verbraucher-Problems mit einem Monitor	90
Abb. 2.31	Ein Betriebsmittelgraph	94
Abb. 2.32	Verbindungsarten	104
Abb. 2.33	Ablauf einer verbindungsorientierten Kommunikation	104
Abb. 2.34	Interprozesskommunikation mit <i>pipes</i> in UNIX	108
Abb. 2.35	POSIX- Signale.	110
Abb. 2.36	Datenkonsistenz mit atomic broadcast-Nachrichten	112
Abb. 2.37	Das Rendez-vous-Konzept	113
Abb. 2.38	Erzeuger-Verbraucher-Modell mit kommunizierenden Prozessen	115
Abb. 2.39	Fibonacci-Folge mittels nebenläufiger Berechnung in GO.	115
Abb. 2.40	Producer-Consumer-System in GO	116
Abb. 3.1	Eine direkte Speicherbelegung und ihre Belegungstabelle.	123
Abb. 3.2	Eine Speicherbelegung und die verzeigte Belegungsliste	124
Abb. 3.3	Visualisierung einer Speicherzuteilung im <i>Buddy</i> -System.	126
Abb. 3.4	Die Abbildung physikalischer Speicherbereiche auf virtuelle Bereiche	131
Abb. 3.5	Die Umsetzung einer virtuellen Adresse in eine physikalische Adresse	132
Abb. 3.6	Eine zweistufige Adresskonversion.	135
Abb. 3.7	Zusammenfassung von Seitentabellen zu einer inversen Seitentabelle	136
Abb. 3.8	Funktion eines assoziativen Tabellencache.	136

Abb. 3.9	Die Abbildung auf gemeinsamen Speicher.	138
Abb. 3.10	Die Auslegung des virtuellen Adressraum bei Linux	139
Abb. 3.11	Die Unterteilung des virtuellen Adressraums in Windows NT 32 Bit . .	139
Abb. 3.12	Die optimale Seitenersetzung	145
Abb. 3.13	Die FIFO-Seitenersetzung	145
Abb. 3.14	Der Clock-Algorithmus	147
Abb. 3.15	Die LRU-Seitenersetzung	147
Abb. 3.16	Realisierung von LRU mit Schieberegistern	148
Abb. 3.17	Die working set – Seitenersetzung bei $\Delta t = 3$	150
Abb. 3.18	Die FIFO-Seitenersetzung	153
Abb. 3.19	Die FIFO-Seitenersetzung in Reihenfolge-Notation	153
Abb. 3.20	Die LRU-Seitenersetzung	154
Abb. 3.21	Überlappung von Rechen- und Seitentauschphasen bei $t_s > t_w$	155
Abb. 3.22	Überlappung von Rechen- und Seitentauschphasen bei $t_s < t_w$	155
Abb. 3.23	Das Nutzungsgradmodell.	157
Abb. 3.24	Die logische Unterteilung von Programmen	164
Abb. 3.25	Segment-Speicherverwaltung beim Intel 80286	165
Abb. 3.26	Lokale und globale Seitentabellen beim INTEL 80486	165
Abb. 3.27	Benutzung eines pipeline-burst-Cache	167
Abb. 3.28	Adress- und Datenanschluss des Cache	167
Abb. 3.29	Dateninkonsistenz bei unabhängigen Einheiten	168
Abb. 4.1	Dateiorganisation als Tabelle.	177
Abb. 4.2	Hierarchische Dateiorganisation	177
Abb. 4.3	Querverbindungen in Dateisystemen.	177
Abb. 4.4	Gültigkeit relativer Pfadnamen.	183
Abb. 4.5	Querverbindungen und Probleme in UNIX	184
Abb. 4.6	Die Erweiterung des Dateisystems unter UNIX	185
Abb. 4.7	Ergänzung der Namensräume in Windows NT.	186
Abb. 4.8	Das pipe-System in UNIX	194
Abb. 4.9	Baumstruktur für zweistufigen, index-sequentiellen Dateizugriff	196
Abb. 4.10	Der Aufbau des B-Baums mit $m = 5$	198
Abb. 4.11	Der fertige Baum des Beispiels mit $m = 5$	198
Abb. 4.12	Einfügen des Schlüssels „41“ im (a) B-Baum und (b) im B*-Baum . .	201
Abb. 4.13	Mehrfachlisten einer sequentiellen Datei.	201
Abb. 4.14	Abbildung von Dateibereichen in den virtuellen Adressraum	204
Abb. 4.15	Speicherung von Dateien mittels Listen	210
Abb. 4.16	Listenverzeichnis	211
Abb. 4.17	Verteilung der Dateindexliste auf die Dateiköpfe	212
Abb. 4.18	Mehrstufige Übersetzung von logischer zu physikalischer Blocknummer.	213
Abb. 4.19	Die Struktur einer i-node in UNIX.	214
Abb. 4.20	Die Reihenfolge der ersten 16 Dateien, der NTFS-Metadata Files . . .	216

Abb. 4.21	Die Struktur eines MFT-Eintrags	216
Abb. 4.22	Die Dateistruktur des Wurzelverzeichnisses	217
Abb. 4.23	Datenstrukturen für eine NTFS-Datei	218
Abb. 5.1	Grundschichten der Geräteverwaltung	223
Abb. 5.2	Die Schichtung und Schnittstellen des UDI-Treibers	224
Abb. 5.3	Die Dateisystemschiichtung unter Linux	224
Abb. 5.4	Beispiel für einen SystemFS-Dateibaum.	225
Abb. 5.5	Das Stream-System in UNIX.	226
Abb. 5.6	Einfache und multiple Schichtung in Windows NT	227
Abb. 5.7	Ausfall eines Clusters in NTFS.	229
Abb. 5.8	Ersatz eines Clusters in NTFS	229
Abb. 5.9	Adressraum, Controller und Gerät	230
Abb. 5.10	I/O-Port-Adressen der Standardperipherie eines PC	230
Abb. 5.11	Plattenspeicheraufbau	232
Abb. 5.12	Modellierungsfehler bei Suchzeit und Transferzeit (nach Rummel 94)	234
Abb. 5.13	Neugruppierung der Plattenbereiche in Streifen	241
Abb. 5.14	Spiegelplattenkonfiguration zweier Platten	242
Abb. 5.15	Datenzuweisung beim RAID-2-System	244
Abb. 5.16	Wechselseitige Abspeicherung der Fehlertoleranzinformation.	245
Abb. 5.17	Die Zustände des Gesamtsystems und seine Einzelzustände	246
Abb. 5.18	Entwicklung der Ausfallwahrscheinlichkeit eines Geräts	247
Abb. 5.19	Die Verbindung von Geräte- und Treiberobjekten	254
Abb. 6.1	Ein typisches lokales Netzwerk	264
Abb. 6.2	Subnetze und Backbone eines Intranets	264
Abb. 6.3	Ein verteiltes Betriebssystem.	267
Abb. 6.4	Das Open System Interconnect (OSI)-Modell der International Organization for Standardization (ISO, früher International Standards Organization)	270
Abb. 6.5	Die Kapselung der Daten und Kontrollinformationen	271
Abb. 6.6	Oft benutzte Protokollschichten in UNIX	272
Abb. 6.7	OSI-Modell und Windows NT-Netzwerkkomponenten	273
Abb. 6.8	Virtual Private Network	273
Abb. 6.9	Ein homogener Dateibaum im lokalen Netz	278
Abb. 6.10	Lokale, inhomogene Sicht des Dateibaums	279
Abb. 6.11	Auflösung von Netzwerk-Dateianfragen	280
Abb. 6.12	Zuordnung von Portnummern und Diensten	281
Abb. 6.13	Kommunikation über Transportendpunkte	282
Abb. 6.14	Kommunikationsablauf im Socket-Konzept	282
Abb. 6.15	Das Transportschema eines RPC.	285
Abb. 6.16	Ablauf eines synchronen RPC	286
Abb. 6.17	Die <i>big endian</i> - und <i>little endian</i> -Bytfolgen	286

Abb. 6.18	Die Schichtung der RPC in Windows NT	288
Abb. 6.19	Der Vorgang zum Sperren einer Datei	292
Abb. 6.20	Übertragungsschichten und Pufferorte für Dateizugriffe im Netz	294
Abb. 6.21	Cachekohärenz bei gleichen und unterschiedlichen Puffern	294
Abb. 6.22	Implementierung eines Netzdateiservers auf Prozessebene	296
Abb. 6.23	Implementierung eines Netzdateiservers auf Treiberebene	296
Abb. 6.24	Die Architektur des NFS-Dateisystems	297
Abb. 6.25	Implementierung des Netzdateisystems in Windows NT	298
Abb. 6.26	Ein Speichernetzwerk	303
Abb. 6.27	Das SAN Schichtenmodell nach SNIA.org	303
Abb. 6.28	Das NAS-Modell	304
Abb. 6.29	Das Konzept des Schattenservers (<i>shadow server</i>).	310
Abb. 6.30	Struktur des Coda-Dateisystems	312
Abb. 6.31	Heterogenität als Problem	314
Abb. 6.32	Middleware als Vermittlungsschicht	315
Abb. 6.33	Schichtung der Middleware	315
Abb. 6.34	Beispiel Middleware: SAP/R3 3-tiers Architektur	316
Abb. 6.35	Architekturschichten von Jini	317
Abb. 6.36	Thin clients/Thick server durch Middleware	317
Abb. 6.37	Gesamtschichtung der .NET-Architektur.	318
Abb. 6.38	Ablauf einer Bestellung durch <i>enterprise service bus</i> (IBM)	319
Abb. 7.1	Die Infektion eines Programms	329
Abb. 7.2	Angriff eines <i>user mode</i> -Rootkits unter Windows NT	333
Abb. 7.3	Der Rootkit-Angriff als Filtertreiber	334
Abb. 7.4	Die Matrix der Zugriffsrechte	338
Abb. 7.5	Rollen und Zugriffsrechte	339
Abb. 7.6	Die Vertrauensketten beim Starten.	340
Abb. 7.7	Ablauf beim login	341
Abb. 7.8	Die klassische fire-wall-Konfiguration	344
Abb. 7.9	Die dual-homed-host-Konfiguration	345
Abb. 7.10	Sitzungsausweis und Transaktionsausweis.	347
Abb. 7.11	Transaktions- (Service-) durchführung.	348
Abb. 8.1	Benutzeroberfläche und Gesamtsystemstruktur	356
Abb. 8.2	Die ASCII-Buchstabenkodierung	357
Abb. 8.3	Auslegung des Unicodes	358
Abb. 8.4	Die Pixelkoordinaten der Rastergrafik	360
Abb. 8.5	Farbwertermittlung über eine Color Lookup Table	361
Abb. 8.6	Ein Buchstabe (a) im 5×7-Raster und (b) als Umrandung.	362
Abb. 8.7	Systemstruktur einer traditionellen Benutzeroberfläche	363
Abb. 8.8	Systemstruktur einer fensterorientierten Benutzeroberfläche	363
Abb. 8.9	Die netzübergreifende Darstellung durch das Client-Server-Konzept von X-Window	369

Abb. 8.10	Beispiel einer fensterorientierten Interaktion in Motif	370
Abb. 8.11	Fenster und Unterfenster in X-Window	370
Abb. 8.12	Der Hierarchiebaum der Fenster aus Abb. 8.11	371
Abb. 8.13	Die Schichtung des X-Window-Motif-toolkits.	372
Abb. 8.14	Ausschnitt aus der Hierarchie der Motif-widget-Primitiven	373
Abb. 8.15	Beispiel eines <i>container widgets</i>	373
Abb. 8.16	Beispiel einer fensterorientierten Interaktion in Windows NT.	376
Abb. 8.17	Getrennte und verschmolzene MDI-Fenster	377
Abb. 8.18	Die Struktur der Window-Subsystemschicht.	378
Abb. 10.1	Wahrscheinlichkeitsverteilung der Seitenreferenzen.	466
Abb. 10.2	Der Seitenaustauschgrad ρ in Abhängigkeit vom Speicherangebot σ pro Prozess	468
Abb. 10.3	Anstieg der Bearbeitungszeit mit wachsender Prozesszahl bei $\rho_T \leq \rho_w$.	470
Abb. 10.4	Anstieg der Bearbeitungszeit mit wachsender Prozesszahl bei $\rho_T \geq \rho_w$.	471

Inhaltsverzeichnis

1.1 Einleitung: Was ist ein Betriebssystem? 2

1.2 Betriebssystemsichten 2

1.3 Betriebssystemaufbau 4

 1.3.1 Systemaufrufe 5

 1.3.2 Beispiel UNIX 6

 1.3.3 Beispiel Mach 7

 1.3.4 Beispiel Windows NT 8

1.4 Schnittstellen und virtuelle Maschinen 11

1.5 Software-Hardware-Migration 15

1.6 Virtuelle Betriebssysteme 16

 1.6.1 Paravirtualisierung 18

 1.6.2 Virtualisierung mit BS-Containern 19

 1.6.3 Vollständige Virtualisierung 19

 1.6.4 Hardware-Virtualisierung 19

 1.6.5 Virtualisierungserweiterungen 20

1.7 Betriebssysteme für eingebettete Systeme 21

1.8 Betriebssysteme in Mehrprozessorarchitekturen 22

1.9 Aufgaben 25

Literatur 26

Bevor wir die Einzelteile von Betriebssystemen genauer betrachten, wollen wir uns die einfache Frage stellen: Was ist eigentlich ein Betriebssystem? Naiv betrachtet ist es die Software, die mit einem neuen Rechner zum Betrieb mitgeliefert wird. So gesehen enthält also ein Betriebssystem alle Programme und Programmteile, die nötig sind, einen Rechner für verschiedene Anwendungen zu betreiben. Die Meinungen, was alles in einem Betriebssystem enthalten sein sollte, gehen allerdings weit auseinander. Benutzt beispielsweise

jemand einen Rechner nur zur Textverarbeitung, so erwartet er oder sie, dass der Rechner alle Funktionen der Anwendung „Textverarbeitung“ beherrscht; wird der Rechner zum Spielen verwendet, so soll er alles Wichtige für den Spielbetrieb enthalten. Was ist wichtig?

1.1 Einleitung: Was ist ein Betriebssystem?

Betrachten wir mehrere Anwendungsprogramme, so finden wir gemeinsame Aufgaben, die alle Programme mit den entsprechenden Funktionen abdecken müssen. Statt jedes Mal „das Rad neu zu erfinden“, lassen sich diese gemeinsamen Funktionen auslagern und als „zum Rechner gehörige“ Standardsoftware ansehen, die bereits beim Kauf mitgeliefert wird. Dabei beziehen sich die Funktionen sowohl auf Hardwareeinheiten wie Prozessor, Speicher, Ein- und Ausgabegeräte, als auch auf logische (Software-) Einheiten wie Dateien, Benutzerprogramme usw.

Bezeichnen wir alle diese Einheiten, die zum Rechnerbetrieb nötig sind, als „Ressourcen“ oder „Betriebsmittel“, so können wir ein Betriebssystem auch definieren als „die Gesamtheit der Programnteile, die die Benutzung von Betriebsmitteln steuern und verwalten“. Diese Definition erhebt keinen Anspruch auf Universalität und Vollständigkeit, sondern versucht lediglich, den momentanen Zustand eines sich ständig verändernden Begriffs verständlich zu machen. Im Unterschied zu der ersten Definition, in der Dienstprogramme wie Compiler und Linker zum Betriebssystem gehören, werden diese in der zweiten Definition ausgeschlossen. Im Folgenden wollen wir eine historisch orientierte, mittlere Position einnehmen:

Das Betriebssystem ist die Software (Programnteile), die für den Betrieb eines Rechners anwendungsunabhängig notwendig ist.

Dabei ist allerdings die Interpretation von „anwendungsunabhängig“ (es gibt keinen anwendungsunabhängigen Rechnerbetrieb: Dies wäre ein nutzloser Rechner) und „notwendig“ sehr subjektiv (sind Fenster und Mäuse notwendig?) und lädt zu neuen Spekulationen ein.

1.2 Betriebssystemschichten

Es gibt nicht das Betriebssystem schlechthin, sondern nur eine den Forderungen der Anwenderprogramme entsprechende Unterstützung, die von der benutzerdefinierten Konfiguration abhängig ist und sich im Laufe der Zeit stark gewandelt hat. Gehörte früher nur die Prozessor-, Speicher- und Ein-/Ausgabeverwaltung zum Betriebssystem, so werden heute auch eine grafische Benutzeroberfläche mit verschiedenen Schriftarten und -größen (Fonts) sowie Netzwerkfunktionen verlangt. Einen guten Hinweis auf den Umfang eines Betriebssystems bietet die Auslieferungsliste eines anwendungsunabhängigen Rechnersystems mit allen darauf vermerkten Softwarekomponenten.

Die Beziehungen der Programnteile eines Rechners lassen sich durch das Diagramm in [Abb. 1.1](#) visualisieren.

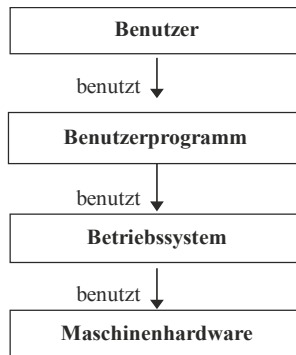


Abb. 1.1 Benutzungsrelationen von Programmteilen

Dies lässt sich auch kompakter zeichnen: in [Abb. 1.2](#) links als Schichtensystem und rechts als System konzentrischer Kreise („Zwiebelschalen“).

Dabei betont die Darstellung als Schichtenmodell den Aspekt der Basis, auf der aufgebaut wird, während das Zwiebelschalenmodell eher Aspekte wie „Abgeschlossenheit“ und „Sichtbarkeit“ von „inneren“ und „äußeren“ Schichten visualisieren.

Beispiel: CPU-Ringe und Schichten

Nach der Initialisierung sind bereits beim 80386 vier Sicherheitsstufen für den Zugriff auf Programme wirksam, siehe [Abb. 1.3](#). Sie lassen sich mit Hilfe von Ringen oder Schichten modellieren. Benutzerprogramme (Stufe 3, äußerster Ring), gemeinsam benutzte Bibliotheken (Stufe 2), Systemaufrufe (Stufe 1) und Kernmodus (Stufe 0, innerster Ring). Die 2 Bits dieser Einstufung werden als Zugriffsinformation in allen Speicherbeschreibungen mitgeführt und entscheiden über die Legalität des Zugriffs. Wird der Zugriff verweigert, so erfolgt eine Fehlerunterbrechung.

Auch Programmsprünge und Prozeduraufrufe in Code einer anderen Stufe werden streng geregelt. Um eine Prozedur einer anderen Stufe aufzurufen, muss eine spezielle Instruktion CALL benutzt werden, die über eine Datenstruktur (*call gate*) den Zugriff überprüft und dann die vorher von der Prozedur eingerichtete Einsprungsadresse verwendet. Unkontrollierte Sprünge werden so verhindert.

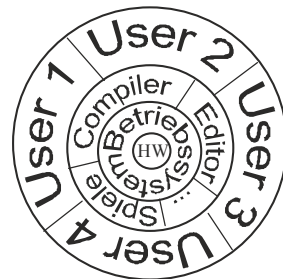
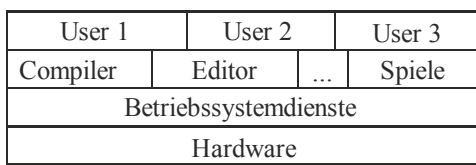


Abb. 1.2 Schichtenmodell und Zwiebelschalenmodell

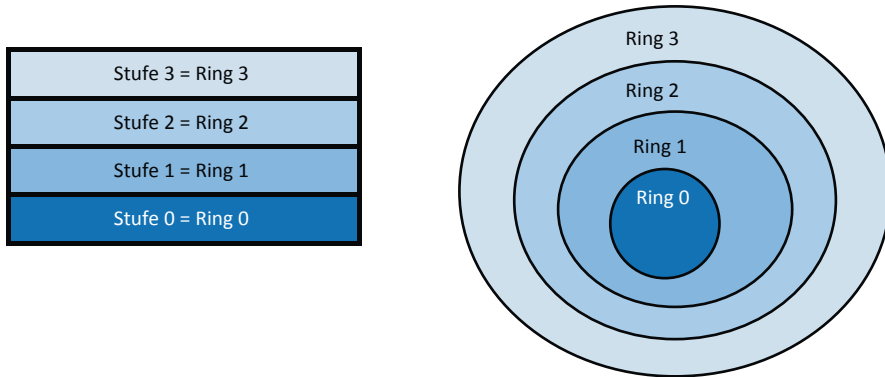


Abb. 1.3 Die Sicherheitsstufen (Ringe) der Intel 80X86-Architektur

1.3 Betriebssystemaufbau

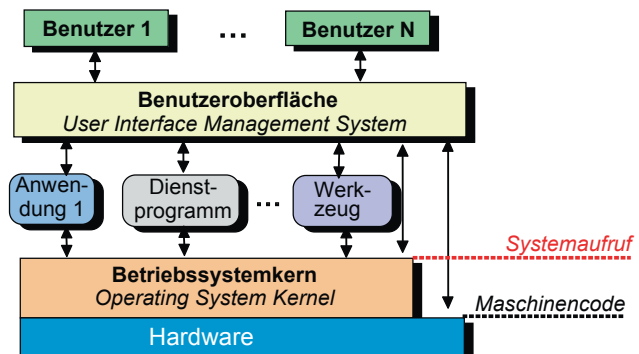
Das Betriebssystem (*operating system*) als Gesamtheit aller Software, die für den anwendungsunabhängigen Betrieb des Rechners notwendig ist, enthält

- *Dienstprogramme, Werkzeuge*: oft benutzte Programme wie Editor, ...
- *Übersetzungsprogramme*: Interpreter, Compiler, Translator, ...
- *Organisationsprogramme*: Speicher-, Prozessor-, Geräte-, Netzverwaltung.
- *Benutzerschnittstelle*: textuelle und graphische Interaktion mit dem Benutzer.

Da ein vollständiges Betriebssystem inzwischen mehrere hundert Megabyte umfassen kann, werden aus diesem Reservoir nur die sehr oft benutzten Funktionen als **Betriebssystemkern** in den Hauptspeicher geladen. In Abb. 1.4 ist die Lage des Betriebssystemkerns innerhalb der Schichtung eines Gesamtsystems gezeigt.

Der Kern umfasst also alle Dienste, die immer präsent sein müssen, wie z. B. große Teile der Prozessor-, Speicher- und Geräteverwaltung (**Treiber**) und wichtige Teile der Netzverwaltung.

Abb. 1.4 Überblick über die Rechnersoftwarestruktur



1.3.1 Systemaufrufe

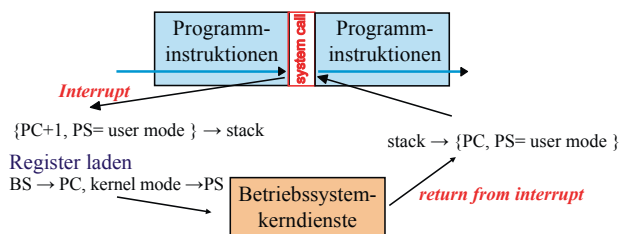
Die Dienste des Kerns werden durch einen Betriebssystemaufruf angefordert und folgen wie ein normaler Prozeduraufruf einem festen Format (Zahl und Typen der Parameter). Da die Lage des Betriebssystemkerns im Hauptspeicher sich ändern kann und die Anwenderprogramme dann immer wieder neu gebunden werden müssten, gibt es bei den meisten Betriebssystemkernen einen speziellen Aufrufmechanismus (**Systemaufruf**, *system call*), der eine Dienstleistung anfordert, ohne die genaue Adresse der entsprechenden Prozedur zu kennen. Dieser Aufrufmechanismus besteht daraus, nach dem Abspeichern der Parameter des Aufrufs auf dem Stack einen speziellen Maschinenbefehl auszuführen, den *Softwareinterrupt*. Wie bei einem Hardwareinterrupt speichert der Prozessor seine augenblickliche Instruktionsadresse (*program counter* $PC + 1$) und den Prozessorstatus PS auf dem Stack, entnimmt das neue Statuswort und die Adresse der nächsten Instruktion einer festen, dem speziellen Interrupt zugeordneten Adresse des Hauptspeichers und setzt mit der Befehlsausführung an dieser Adresse fort.

Wurde beim Initialisieren des Rechnersystems (*bootstrap*) auf dem festen Interrupt-Speicherplatz die Einstiegsadresse des Betriebssystemkerns geschrieben, so findet das Anwenderprogramm prompt das Betriebssystem, egal ob es inzwischen neu kompiliert bzw. im Speicher verschoben wurde oder nicht. In Abb. 1.5 ist der Ablauf eines solchen Systemaufrufs gezeigt. Da nach einem Systemaufruf die nächste Instruktion nicht gleich ausgeführt wird, sondern die Befehlsfolge an dieser Speicheradresse plötzlich „aufhört“, wird der Softwareinterrupt auch als Falltür (**trap door**, kurz *trap*) bezeichnet.

Dabei wird hardwaremäßig der augenblickliche Wert des Programmzählers PC (um eins erhöht für die Adresse der nächsten Instruktion nach der Interrupt-Instruktion) sowie der Status des Programms (Statuswort PS, enthält die Ergebnisbits der CPU und die Information, ob im *user mode* oder *kernel mode*) auf den Stack gerettet. Anschließend wird dann der Inhalt eines Tabelleneintrags für diesen Interrupt, der neue Status PS und die neue Adresse BS des Betriebssystems, in die CPU geladen.

Die Umschaltung vom Benutzerprogramm zum Betriebssystem wird meist zum Anlass genommen, die Zugriffsrechte und -möglichkeiten des Codes drastisch zu erweitern. Wurden große Teile des Rechners vor der Umschaltung hardwaremäßig vor Fehlbedienung durch das Benutzerprogramm geschützt, so sind nun alle Schutzmechanismen abgeschaltet, um den Betriebssystemkern nicht zu behindern. Beim Verlassen des

Abb. 1.5 Befehlsfolge eines Systemaufrufs



Betriebssystemkerns wird automatisch wieder zurückgeschaltet, ohne dass der Benutzer etwas davon merkt. Die Umschaltung zwischen dem Benutzerstatus (**user mode**) und dem Betriebssystemstatus (**kernel mode**) geschieht meist hardwaremäßig durch den Prozessor und lässt sich damit vom Benutzer nicht manipulieren. Manche Prozessoren besitzen sogar noch weitere Sicherheitsstufen, die aber den Hardwareaufwand auf dem Chip erhöhen. Bei der Rückkehr vom Betriebssystemkern werden die erweiterten Rechte durch das Neuladen der auf dem Stack gespeicherten Werte automatisch wieder zurückgesetzt.

Auch für die Behandlung von Fehlern in der Fließkommaeinheit (Floating Point Unit FPU) werden Softwareinterrupts verwendet. Die Fehlerbehandlung ist aber meist anwendungsabhängig und deshalb üblicherweise im Betriebssystem nicht enthalten, sondern dem Anwenderprogramm oder seiner Laufzeitumgebung überlassen.

1.3.2 Beispiel UNIX

In UNIX gibt es traditionellerweise als Benutzeroberfläche einen Kommandointerpreter, die **Shell**. Von dort werden alle Benutzerprogramme sowie die Systemprogramme, die zum Betriebssystem gehören, gestartet. Die Kommunikation zwischen Benutzer und Betriebssystem geschieht durch Ein- und Ausgabekanäle; im einfachsten Fall sind dies die Eingabe von Zeichen durch die Tastatur und die Ausgabe auf dem Terminal. In [Abb. 1.6](#) ist das Grundschemata gezeigt.

Das UNIX-System wies gegenüber den damals verfügbaren Systemen verschiedene Vorteile auf. So ist es nicht nur ein Betriebssystem, das mehrere Benutzer (*Multi-user*) gleichzeitig unterstützt, sondern auch mehrere Programme (*Multiprogramming*) gleichzeitig ausführen konnte. Zusammen mit der Tatsache, dass die Quellen den Universitäten fast kostenlos zur Verfügung gestellt wurden, wurde es bei allen Universitäten

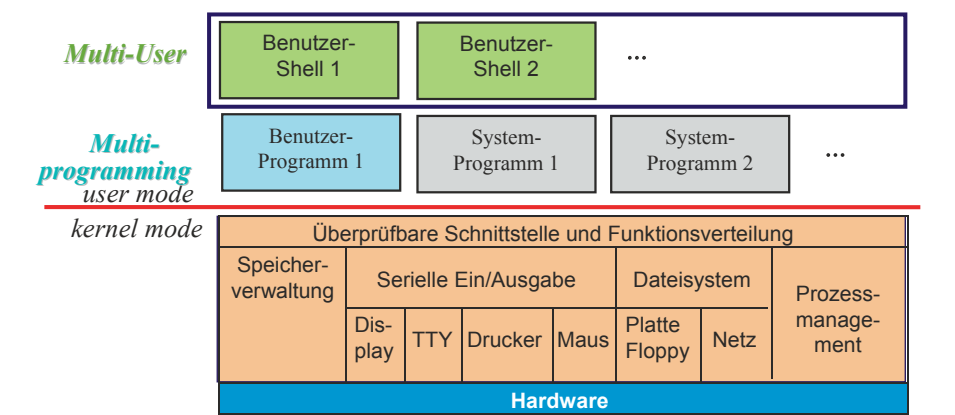


Abb. 1.6 UNIX-Schichten

Standard und dort weiterentwickelt. Durch die überwiegende Implementierung mittels der „höheren“ Programmiersprache „C“ war es leicht veränderbar und konnte schnell auf eine andere Hardware übertragen (*portiert*) werden. Diese Faktoren machten es zum Betriebssystem der Wahl, als für die neuen RISC-Prozessoren schnell ein Betriebssystem benötigt wurde. Hatte man bei einem alten C-Compiler den Codegenerator für den neuen Instruktionssatz geändert, so war die Hauptarbeit zum Portieren des Betriebssystems bereits getan – alle Betriebssystemprogramme sowie große Teile des Kerns sind in C geschrieben und sind damit nach dem Kompilieren auf dem neuen Rechner lauffähig.

Allerdings gibt es trotzdem noch genügend Arbeit bei einer solchen Portierung. Die ersten Versionen von UNIX (Version 6 und 7) waren noch sehr abhängig von der Hardware, vor allem aber von der Wortbreite der CPU. In den folgenden Versionen (System IV und V sowie Berkeley UNIX) wurde zwar viel gelernt und korrigiert, aber bis heute ist die Portierbarkeit nicht problemlos.

Die Grundstruktur von UNIX differiert von Implementierung zu Implementierung. So sind Anzahl und Art der Systemaufrufe sehr variabel, was die Portierbarkeit der Benutzerprogramme zwischen den verschiedenen Versionen ziemlich behindert. Um dem abzuweichen, wurden verschiedene Organisationen gegründet. Eine der bekanntesten ist die X/Open-Gruppe, ein Zusammenschluss verschiedener Firmen und Institutionen, die verschiedene Normen herausgab. Eine der ersten Normen war die Definition der Anforderungen an ein portables UNIX-System (*Portable Operating System Interface based on UNIX*: POSIX, genormt als IEEE 1003.1-2013 bzw. ISO/IEC 9945), das als Menge verschiedener verfügbarer Systemdienste definiert wurde. Allerdings sind dabei nur Dienste, nicht die Systemaufrufe direkt definiert worden. Durch die an dem Namen „UNIX“ auf X/Open übertragenen Rechte konnte ein Zertifikationsprozess institutionalisiert werden, der eine bessere Normierung verspricht. Dabei wurde UNIX auch an das Client-Server-Modell (Seite 27) angepasst: Es gibt eine Spezifikation für UNIX-98 Server und eine für UNIX-98 Client.

1.3.3 Beispiel Mach

Das Betriebssystem „Mach“ wurde an der Universität Berkeley als ein Nachfolger von UNIX entwickelt, der auch für Multiprozessorsysteme einsetzbar sein sollte. Das Konzept von Mach ist radikal verschieden von dem UNIX-Konzept. Statt alle weiteren, als wichtig angesehenen Dienste im Kern unterzubringen, beschränkten die Designer das Betriebssystem auf die allernotwendigsten Funktionen: die Verwaltung der Dienste, die selbst aber aus dem Kern ausgelagert werden, und die Kommunikation zwischen den Diensten. In [Abb. 1.7](#) ist das prinzipielle Schichtenmodell gezeigt, das so auch auf das bekannte Shareware-Amoeba-System von Tanenbaum zutrifft. Beide Systeme besitzen Bibliotheken für UNIX-kompatible Systemaufrufe, so dass sie vom Benutzerprogramm als UNIX-Systeme angesehen werden können.

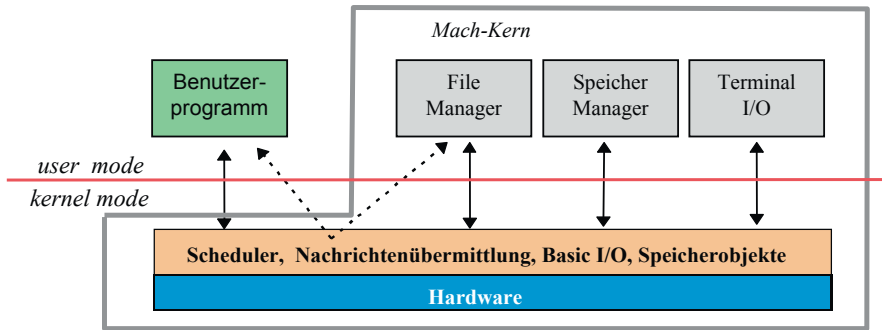


Abb. 1.7 Das Mach-Systemmodell

Der Vorteil einer solchen Lösung, den Kern in einen besonders kleinen, sparsam bemessenen Teil (**Mikrokern**) einerseits und in Systemdienste auf Benutzerebene andererseits aufzuteilen, liegt auf der Hand: Die verschiedenen Systemdienste sind gekapselt und leicht austauschbar; es lassen sich sogar mehrere Versionen nebeneinander benutzen. Die Nachteile sind aber auch klar: Durch die Kapselung als Programme im Benutzerstatus dauert es relativ lange, bis die verschiedenen Systemdienste mittels Systemaufrufen und Umschalten zwischen *user mode* und *kernel mode* sich abgestimmt haben, um einen Systemaufruf des Benutzerprogramms auszuführen.

1.3.4 Beispiel Windows NT

Das Betriebssystem Windows NT der Firma Microsoft ist ein relativ modernes System; es wurde unter der Leitung von David Cutler, einem Betriebssystementwickler von VMS, RSX11-M und MicroVax der Fa. Digital, seit 1988 entwickelt. Das Projekt, mit dem Microsoft erstmals versuchte, ein professionelles Betriebssystem herzustellen, hatte verschiedenen Vorgaben zu genügen:

- Das Betriebssystem musste zu allen bisherigen Standards (MS-DOS, 16 Bit-Windows, UNIX, OS/2) kompatibel sein, um überhaupt am Markt akzeptiert zu werden.
- Es musste zuverlässig und robust sein, d. h. Programme dürfen weder sich gegenseitig noch das Betriebssystem schädigen können; auftretende Fehler dürfen nur begrenzte Auswirkungen haben.
- Es sollte auf verschiedene Hardwareplattformen leicht zu portieren sein.
- Es sollte nicht perfekt alles abdecken, sondern für die sich wandelnden Ansprüche leicht erweiterbar und änderbar sein.
- Sehr wichtig: Es sollte auch leistungsstark sein.

Betrachtet man diese Aussagen, so stellen sie eine Forderung nach der „eierlegenden Wollmilchsau“ dar. Die gleichzeitigen Forderungen von „kompatibel“, „zuverlässig“,

„portabel“, „leistungsstark“ sind schon Widersprüche in sich: MS-DOS ist absolut nicht zuverlässig, portabel oder leistungsstark und überhaupt nicht kompatibel zu UNIX. Trotzdem erreichten die Entwickler ihr Ziel, indem sie Erfahrungen anderer Betriebssysteme nutzten und stark modularisierten. Die Schichtung und die Aufrufbeziehungen des Kerns in der ursprünglichen Konzeption sind in Abb. 1.8 gezeigt. Der gesamte Kern trägt den Namen *Windows NT Executive* und ist der Block unterhalb der *user mode/kernel mode*-Umschaltsschranke.

Zur Lösung der Designproblematik seien hier einige Stichworte genannt:

- Die **Kompatibilität** wird erreicht, indem die Besonderheiten jedes der Betriebssysteme in ein eigenes Subsystem verlagert werden. Diese setzen als virtuelle Betriebssystemmaschinen auf den Dienstleistungen der NT Executive (Systemaufrufe, schwarze Pfeile in Abb. 1.8) auf. Die zeichenorientierte Ein/Ausgabe wird an die Dienste des zentralen Win32-Moduls weitergeleitet. Die Dienste der Subsysteme werden durch Nachrichten (*local procedure calls* LPC, graue Pfeile in Abb. 1.8) von Benutzerprogrammen, ihren *Kunden (Clients)*, angefordert. Als Dienstleister (**Server**) haben sie also eine Client-Server-Beziehung.
- Die **Robustheit** wird durch rigorose Trennung der Programme voneinander und durch Bereitstellen spezieller Ablaufumgebungen („virtuelle DOS-Maschine VDM“) für die MS-DOS/Windows Programme erreicht. Die Funktionen sind gleich, aber der direkte Zugriff auf Hardware wird unterbunden, so dass nur diejenigen alten Programme laufen können, die die Hardware-Ressourcen nicht aus Effizienzgründen direkt anzusprechen versuchen. Zusätzliche Maßnahmen wie ein fehlertolerantes Dateisystem und spezielle Sicherheitsmechanismen zur Zugriffskontrolle von Dateien, Netzwerken und Programmen unterstützen dieses Ziel.
- Die **Portierbarkeit**, Wartbarkeit und Erweiterbarkeit wird dadurch unterstützt, dass das gesamte Betriebssystem bis auf wenige Ausnahmen (z. B. in der Speicherverwaltung)

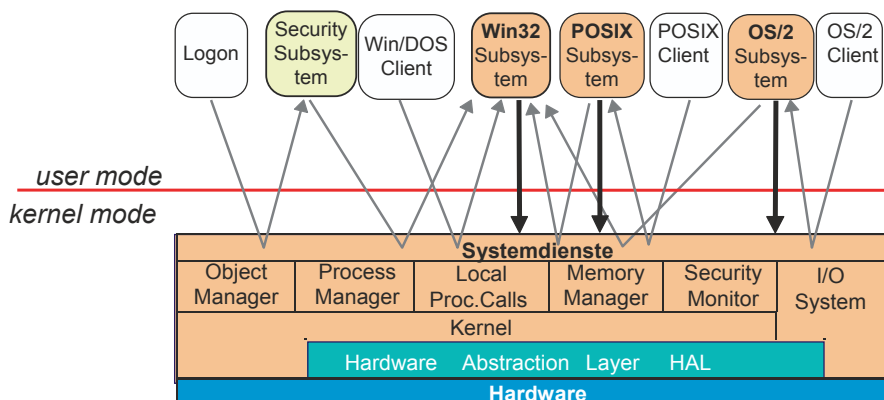


Abb. 1.8 Schichtung und Aufrufe bei Windows NT

in der Sprache C geschrieben, stark modularisiert und von Anfang an geschichtet ist. Eine spezielle Schicht HAL bildet als virtuelle Maschine allgemeine Hardware nach und reduziert bei der Portierung auf andere Prozessoren die notwendigen Änderungen auf wenige Module.

Die detaillierte Diskussion der oben geschilderten Lösungen würde zu weit führen; hier sei auf das Buch von Helen Custer ([1993](#)) verwiesen.

- Obwohl in Windows NT einige wichtige Subsysteme wie z. B. das Sicherheitssystem nicht als Kernbestandteil, sondern als Prozess im *user mode* betrieben werden („Integrale Subsysteme“), ist interessanterweise seit Version 4.0 das Win32-Subsystem aus Effizienzgründen in den Kern verlagert worden, um die Zeit für die Systemaufrufe von Win32 zu NT Executive zu sparen. Auch die Unterstützung anderer Standards wurde eingestellt. Von den gezeigten Betriebssystem-Subsystemen sind im Laufe der Versionen von NT nur noch das Windows/Dos-System und das Windows 32Bit-System („windows over windows“) übrig; zuerst das OS/2-System ab Version NT 4.0 und dann das POSIX-System („Interix“) wurden mit fortschreitender Marktakzeptanz von Windows NT eingestellt.

In Version NT 5.0, erschienen im Februar 2000 („Windows 2000“), wurden zusätzlich viele Dienstprogramme zur Netzdateiverwaltung und Sicherheit integriert. Dies ließ den Umfang von 8 Mio. Codezeilen auf über 40 Mio. anschwellen, was an die Zuverlässigkeit und Testumgebung der Betriebssystemmodule besonders hohe Anforderungen stellt.

Mit den Versionen NT 5.1 („Windows XP“), NT 5.2 („Windows Server 2003“) und Version 6.0 („Windows Vista“) stiegen die Anforderungen an die Grafikhardware weiter; ohne Hardware-Grafikbeschleunigung war MS Windows ab Version NT 6 nicht installierbar.

Auch für die Version 6.1 („Windows 7“), Version NT 6.2 („Windows 8“) und Version NT 6.3 („Windows 8.2“) sowie für die zur Zeit (2016) aktuelle Version NT 10.0 („Windows 10“) wurden die Anforderungen immer weiter ausgeweitet. Zwar wurde der Softwareerstellungprozess komplett umgeändert und auf Module umgestellt, nachdem Windows Vista zunächst als sehr fehleranfällig und langsam von den Benutzern abgelehnt wurde, aber Umfang und Komplexität des Codes blieben erhalten. So wuchs zwar die Anzahl der Codezeilen mit NT 5.2 und 6.0 auf 50 Mio. und wurde nach der Modularisierung NT 6.1 auf die „meist verwendeten“ 40 Mio. Codezeilen von über 60 Mio. Zeilen reduziert, aber durch die Übernahme zusätzlicher Funktionalität (Hilfsprogramme) wächst die gesamte Softwarebasis kontinuierlich an. Zum Vergleich: das zurzeit am meisten verwendete Unix, der Linux-Kern 4.2, hat nur ca. 20 Mio. Codezeilen, und die Smartphone-Variante „Android“ nur 12 Mio. Codezeilen. Das hört sich viel an im Vergleich für das Windows-Betriebssystem, aber: betrachtet man etwa das aus Unix entwickelte Apple-Betriebssystem MAC OS X 10.4 mit ca. 86 Mio. Codezeilen, und vergleicht man dies noch mit dem Google-Suchdienst („Betriebssystem des Internets“) mit dem Umfang aller auf den Servern laufenden Programme von über 2000 Mio. Codezeilen (86 TByte!), also 40 mal mehr als Windows, so relativiert sich diese Einschätzung wieder!

1.4 Schnittstellen und virtuelle Maschinen

Betrachten wir nun das Schichtenmodell etwas näher und versuchen, daraus zu abstrahieren. Die Relation „A benutzt B“ lässt sich dadurch kennzeichnen, dass B für A **Dienstleistungen** erbringt. Dies ist beispielsweise bei der Benutzung einer Unterprozedur B in einem Programm A der Fall. Betrachten wir dazu das Zeichnen einer Figur, etwa eines Vierecks. Die Dienstleistung `DrawRectangle(x0, y0, x1, y1)` hat als Argumente die Koordinaten der linken unteren Ecke und die der rechten oberen. Wir können diesen Aufruf beispielsweise direkt an einen Grafikprozessor GPU richten, der dann auf unserem Bildschirm das Rechteck zeichnet, siehe [Abb. 1.9](#) links. In diesem Fall benutzen wir eine echte Maschine, um die Dienstleistung ausführen zu lassen.

In vielen Rechnern ist aber kein Grafikprozessor vorhanden. Stattdessen ruft das Programm, das diese Funktion ausführen soll, selbst wieder eine Folge von einfachen Befehlen (Dienstleistungen) auf, die diese Funktion implementieren sollen, etwa viermal das Zeichnen einer Linie. Die Dienstleistung `DrawRectangle(x0, y0, x1, y1)` wird also nicht wirklich, sondern mit `DrawLine()` durch den Aufruf einer anderen Maschine erbracht; das aufgerufene Programm ist eine **virtuelle Maschine**, in [Abb. 1.9](#) rechts V1 genannt.

Nun kann das Zeichnen einer Linie auch wieder entweder durch einen echten Grafikprozessor erledigt werden, oder aber die Funktion arbeitet selbst als virtuelle Maschine V2 und ruft für jedes Zeichnen einer Linie eine Befehlsfolge einfacher Befehle der darunter liegenden Schicht für das Setzen aller Bildpunkte zwischen den Anfangs- und Endkoordinaten auf. Diese Schicht ist ebenfalls eine virtuelle Maschine V3 und benutzt Befehle für die CPU, die im Video-RAM Bits setzt. Die endgültige Ausgabe auf den Bildschirm wird durch eine echte Maschine, die Displayhardware (Displayprozessor), durchgeführt.

Erinnern wir uns in diesem Zusammenhang an die Normierung von UNIX mittels POSIX. Interessant ist, dass dabei die Bezeichnung „UNIX“ nur für eine Sammlung von verbindlichen Schnittstellen steht, nicht für eine Implementierung. Dies bedeutet, dass UNIX eigentlich als eine virtuelle und nicht als reelle Betriebssystemmaschine angesehen werden kann.

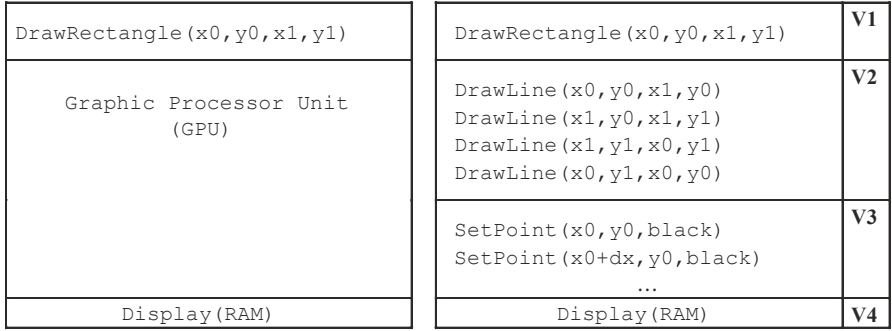


Abb. 1.9 Echte und virtuelle Maschinen

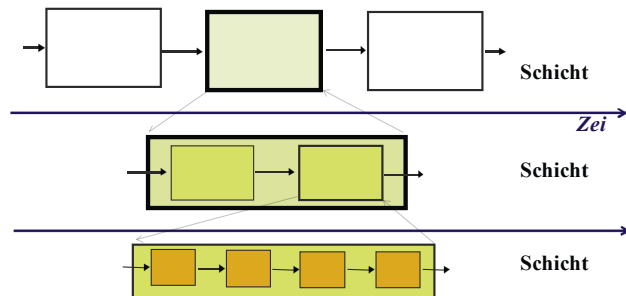
Den Gedankengang der Virtualisierung können wir allgemein formulieren. Gehen wir davon aus, dass alle Dienstleistungen in einer bestimmten Reihenfolge (Sequenz) angefordert werden, so lassen sich im Schichtenmodell die Anforderungen vom Anwender an das Anwenderprogramm, vom Anwenderprogramm an das Betriebssystem, vom Betriebssystem an die Hardware usw. auf Zeitachsen darstellen, die untereinander angeordnet sind.

Jede dieser so entstandenen Schichten bildet nicht nur eine Softwareeinheit wie in [Abb. 1.2](#) links, sondern die Schichtenelemente sind hierarchisch als Untersequenzen oder Dienstleistungen angeordnet. Bei jeder Dienstleistung interessiert sich die anfordernde, darüber liegende Schicht nur dafür, dass sie überhaupt erbracht wird, und nicht, auf welche Weise. Die Dienstleistungsfunktionen einer Schicht, also die Prozeduren bzw. Methoden, Daten und ihre Benutzungsprotokolle, kann man zu einer **Schnittstelle** zusammenfassen. Das Programm, das diese Dienstleistungen erbringt, kann nun selbst wieder als Befehlssequenz aufgefasst werden, die darunter liegende, elementarere Dienstleistungen als eigene Leistungen benutzt. Die allerunterste Schicht, die die Arbeit nun tatsächlich auch ausführt, wird von der „darunter liegenden“ Maschinenhardware gebildet. Da sich ihre Funktionen, so wie bei allen Schichten, über Schnittstellen ansteuern lassen, kann man die darüber liegende Einheit ebenfalls als eine Maschine auffassen; allerdings erbringt sie die Arbeit nicht selbst und wird deshalb als virtuelle Maschine bezeichnet. Die [Abb. 1.10](#) beschreibt also eine Hierarchie virtueller Maschinen. Das allgemeine Schichtenmodell aus [Abb. 1.2](#) zeigt dies ebenfalls, aber ohne Zeitachsen und damit ohne Reihenfolge. Es ist auch als Übersicht über Aktivitäten auf parallel arbeitenden Maschinen geeignet.

Die Funktion der virtuellen Gesamtmaschine ergibt sich aus dem Zusammenwirken virtueller Einzelmaschinen. In diesem Zusammenhang ist es natürlich von außen ununterscheidbar, ob eine Prozedur oder eine Hardwareeinheit eine Funktion innerhalb einer Sequenz ausführt.

Bisher haben wir zwischen physikalischen und virtuellen Maschinen unterschieden. Es gibt nun noch eine dritte Sorte: die **logischen** Maschinen. Für manche Leute sind sie als Abstraktion einer physikalischen Maschine mit den virtuellen Maschinen identisch, andere platzieren sie zwischen physikalische und virtuelle Maschinen.

Abb. 1.10 Hierarchie virtueller Maschinen



Beispiel virtuelle Festplatte

Eine *virtuelle Festplatte* lässt sich als Feld von Speicherblöcken modellieren, die einheitlich mit einer sequentiellen Blocknummer angesprochen werden können. Im Gegensatz dazu modelliert die *logische Festplatte* alles etwas konkreter und berücksichtigt, dass eine Festplatte auch unterschiedlich groß sein kann sowie eine Verzögerungszeit und eine Priorität beim Datentransfer kennt. In [Abb. 1.11](#) ist dies gezeigt. Mit diesen Angaben kann man eine Verwaltung der Speicherblöcke anlegen, in der der Speicherplatz mehrerer Festplatten einheitlich verwaltet wird, ohne dass dies der Schnittstelle der virtuellen Festplatte bekannt sein muss. In der dritten Konkretisierungsstufe beim tatsächlichen Ansprechen der logischen Geräte müssen nun alle Feinheiten der Festplatten (Statusregister, Fehlerinformation, Schreib-/Lesebufferadressen, ...) bekannt sein. Das Verwalten dieser Informationen und Verbergen vor der nächsthöheren Schicht (der Verwaltung der logischen Geräte) besorgt dann der gerätespezifische Treiber. Die Definitionen bedeuten dann in diesem Fall:

virtuelles Gerät = Verwaltungstreiber für logisches Gerät,
logisches Gerät = HW-Treiber für physikalisches Gerät.

Die drei Gerätearten bilden also auch wieder drei Schichten für den Zugriff auf die Daten, vgl. [Abb. 1.11](#). Die logischen Geräte (LUN) gestatten es, unabhängig vom Hersteller auf eine Festplatte zuzugreifen; es kann als eine Abstraktion einer Festplatte aufgefasst werden. Es gibt dabei keinen Unterschied zu einer virtuellen Maschine; der Begriff „logisches Gerät“ ist eher eine historische Bezeichnung für diese Abstraktionsstufe.

Mit Hilfe des Begriffs der virtuellen Maschine können wir uns virtuelle Massenspeicher aber auch ganz anderer Art konstruieren.

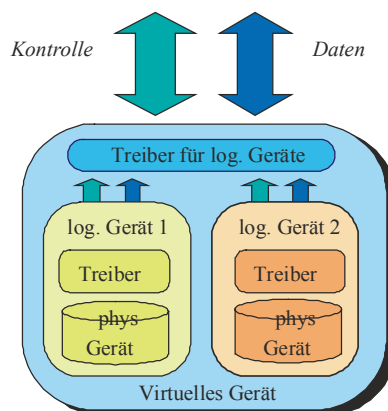


Abb. 1.11 Virtuelle, logische und physikalische Geräte

Beispiel *Disk-to-Disk-Storage*

Ein Bandlaufwerk zur Sicherung von Massendaten kann gut mit einer Menge von Festplatten simuliert werden. Die Simulation eines billigen Bandlaufwerks durch teure Festplatten ist dann sinnvoll, wenn für die Datensicherung (*backup*)

- Kurze Backup-Zeiten
- Kurze Recovery-Zeiten
- Multiple Bandformate
- Hoher Datendurchsatz

gefordert werden. Hier kommen dann kostengünstige Plattenlaufwerke mit großen Kapazitäten zum Einsatz.

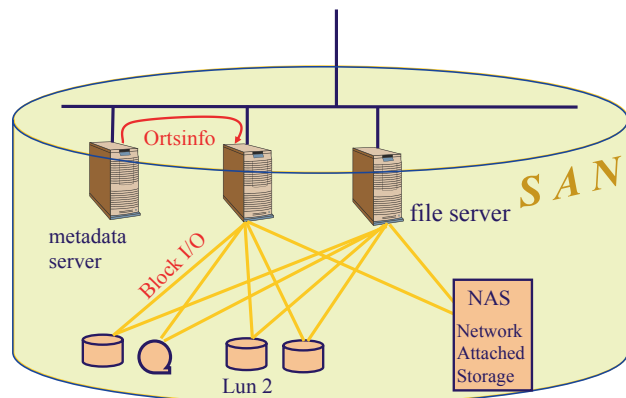
Beispiel *Storage Attached Network SAN*

Aber wir können auch alle Massenspeicher eines Speichernetzes mit den kontrollierenden Rechnern zu einem einzigen virtuellem Massenspeicher zusammenfassen. In [Abb. 1.12](#) ist ein solches Storage Attached Network SAN gezeigt.

Der virtuelle Massenspeicher besteht in diesem Fall aus einem Netzwerk, das nicht nur die Speicherfunktionen intern organisiert, sondern auch die Zugriffsrechte, die Datensicherung (*backup*) und Zugriffsoptimierung (Lastverteilung, Datenmigration) selbst regelt. Konflikte gibt es dabei mit unabhängigen Untereinheiten, die selbst diese Funktionen übernehmen wollen (z. B. NAS oder SAN-in-a-box). Es gibt hauptsächlich zwei verschiedene Konfigurationen:

- Der *asymmetrische Pool*: Ein Server dient als Metadaten-Server (Wo sind welche Daten) und gibt Block-I/O Informationen an die anderen

Abb. 1.12 Ein Netzwerk als virtueller Massenspeicher: die asymmetrische Pool-Lösung



- Der *symmetrische Pool* (3-tiers-Lösung): Die Speichergeräte sind nur mit mehreren Metadaten-Servern über ein extra Netzwerk verbunden; die Metaserver hängen über das SAN an den anfragenden Rechnern.

1.5 Software-Hardware-Migration

Die Konstruktion von virtuellen Maschinen erlaubt es, analog zu den Moduln die Schnittstelle beizubehalten, die Implementierung aber zu verändern. Damit ist es möglich, die Implementierung durch eine wechselnde Mischung aus Hardware und Software zu realisieren: Für die angeforderte Dienstleistung ist dies irrelevant. Da die Hardware meist schneller arbeitet, aber teuer ist, und die Software vergleichsweise langsam abgearbeitet wird, aber (als Kopie) billig ist und schneller geändert werden kann, versucht der Rechnerarchitekt, bei dem Entwurf eines Rechensystems eine Lösung zwischen diesen beiden Extremen anzusiedeln. In [Abb. 1.13](#) ist als Beispiel die Schichtung eines symbolischen Maschinencodes (hier: Java-Code) zu sehen, der entweder softwaremäßig durch einen Compiler oder Interpreter in einen realen Maschinencode umgesetzt werden („*Java virtual machine*“) oder aber auch direkt als Maschineninstruktion hardwaremäßig ausgeführt werden kann. Im zweiten Fall wurde die mittlere Schicht in die Hardware (schraffiert in [Abb. 1.13](#) links) migriert, indem für jeden Java-Code Befehl eine in Mikrocode programmierte Funktion in der CPU ausgeführt wird.

Ein anderes Beispiel für diese Problematik ist die Unterstützung von Netzwerkfunktionen. In billigen Netzwerkcontrollern ist meist nur ein Standard-Chipset enthalten, das die zeitkritischen Aspekte wie Signalerzeugung und -erfassung durchführt. Dies entspricht den virtuellen Maschinen auf unterster Ebene. Alle höheren Funktionen und Protokolle zum Zusammensetzen der Datenpakete, Finden der Netzwerkrouuten und dergleichen (s. [Abschn. 6.1](#)) muss vom Hauptprozessor und entsprechender Software durchgeführt werden, was die Prozessorleistung für alle anderen Aufgaben (wie Benutzeroberfläche, Textverarbeitung etc.) drastisch mindert. Aus diesem Grund sind bei teureren Netzwerkcontrollern viele Funktionen der Netzwerkkontrolle und Datenmanagement auf die Hardwareplatine migriert worden; der Hauptprozessor muss nur wenige Anweisungen ausführen, um auf hohem Niveau (in den oberen Schichten) Funktionen anzustoßen und Resultate entgegenzunehmen.

Dabei spielt es keine Rolle, ob die von dem Netzwerkcontroller übernommenen Funktionen durch einen eigenen Prozessor mit Speicher und Programm erledigt werden oder durch einen dedizierten Satz von Chips: Durch die eindeutige Schnittstellendefinition wird die angeforderte Dienstleistung erbracht, egal wie. Stellt die virtuelle Maschine nicht die originale Schnittstelle bereit, sondern eine vereinfachte, modifizierte Version davon, so

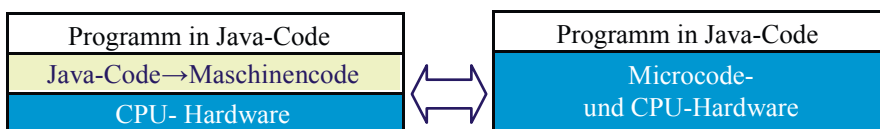


Abb. 1.13 Software-Hardware-Migration des Java-Codes

wird dies „Paravirtualisierung“ genannt. Wird so die darunter liegende Hardware besser ausgenutzt, kann man damit die Leistungsfähigkeit der virtuellen Maschine verbessern.

Wird für das Hardwaredesign auch eine formale Sprache verwendet (z. B. VHDL), so spielen die Unterschiede in der Änderbarkeit der migrierten und nicht-migrierten Implementierungen immer weniger eine Rolle. Entscheidend sind vielmehr andere Aspekte wie Kosten, Standards, Normen und Kundenwünsche.

1.6 Virtuelle Betriebssysteme

Die Virtualisierung, also das Einziehen einer abstrahierenden Zwischenschicht zwischen den Geräten und die bedienenden Programme, macht auch vor dem Betriebssystem nicht Halt. Eine beliebte Maßnahme in Rechenzentren, die Dienstleistungen von Rechnern (*Serverpool*) erbringen, besteht darin, die feste Zuordnung von Dienstleistung zu einem Rechner oder dem Rechner eines Maschinentyps aufzubrechen und die Dienstleistungen geeignet auf die vorhandenen Rechner und Multiprozessorkerne zu verteilen.

Dies hat verschiedene Vorteile:

- Damit ist es möglich, Rechenzentren dynamisch wachsen zu lassen (*Skalierbarkeit*), je nach Anforderungen des Betriebs. Es werden nicht mehr Rechner für bestimmte Betriebssysteme benötigt, sondern man kann alle Anwendungen auf demselben Rechnerpool laufen lassen.
- Da ein großer Teil der Betriebskosten auf die *verbrauchte Energie* für den Rechnerbetrieb und seine Klimatisierung entfällt, kann man erheblich Geld sparen (ca. 30 %), wenn man die Jobs auf wenige Rechner konzentriert und unbenutzte Kapazitäten dynamisch ab oder zuschaltet. Voraussetzung dafür ist, dass mit den Jobs auch der gesamte Prozesskontext des Betriebssystems verschoben werden kann; am besten gleich das ganze Betriebssystem.
- Auch ein *Lastenausgleich* zwischen verschiedenen Rechnern ist dynamisch während des Betriebs möglich.
- Man kann so auch alte Programme (*Legacy-Anwendungen*), die nur auf bestimmten veralteten Betriebssystemen (*Legacy-Betriebssysteme*) oder veralteter Hardware laufen, problemlos weiterhin betreiben.
- Ein weiterer Vorteil einer solchen *Serverkonsolidierung* liegt in der dazu notwendigen Modularisierung und Konzentration der Dienste und Bibliotheken auf wenige, verschiebbare Pakete, die überall eingesetzt werden können.
- Ein wichtiges Merkmal wird auch gern bei der *Softwareentwicklung* ausgenutzt: Möchte man eine Applikation gleichzeitig für eine Vielzahl von Betriebssystemen und Hardwarekonfigurationen entwickeln, so ist das Entwickeln und Testen auf so vielen Konfigurationen sehr aufwändig, teuer und zeitraubend. Besser geht es, wenn man alle Konfigurationen (z. B. Smartphones) als virtuelle Maschinen auf demselben Rechner laufen lassen kann. Man spart sich so die Anschaffung der unterschiedlichen Rechner und, für die Tests auf derselben Hardware, das umständliche Neubooten jeder Konfiguration.

- Insgesamt ist durch die neuere Entwicklung der Virtualisierung und ihrer Hardwareunterstützung durch die CPU nur noch ein geringer Leistungsverlust von ca. 5–10 % gegenüber einer traditionellen Rechnerkonfigurationen zu beobachten (Hardt 2005).

Die Virtualisierung der Rechnerressourcen bedeutet, dass jede Anwendung vollständig unabhängig von der tatsächlichen Rechnerkonfiguration laufen kann. Dies bedeutet, dass Rechnerressourcen universell gehandelt werden können. Beispielsweise ist die Deutsche Börse AG in Frankfurt in den Handel mit Virtuellen Maschinen eingestiegen. Unter dem Schlagwort „*Infrastructure as a Service*“ IaaS vermarktet sie die Angebote von Betreibern großer Rechenzentren wie Amazon. Entscheidend für den Preis einer solchen virtuellen Maschine sind nicht nur Hardwaredaten wie garantierte CPU-Leistung, Hauptspeichergröße und Massenspeicher, sondern auch der Vertragskontext wie Leistungsklasse, Wartungsart oder der dafür gültige Rechtsrahmen, wie etwa das Staatsgebiet. Hier sind europäische IT-Unternehmen durch das europäische Datenschutzrecht in besserer Position!

Problematisch bei solchen Ansätzen ist die Definition der garantierten Leistungsdaten (Benchmarks). Die aktuelle Leistung ist meist von vielen Faktoren abhängig, etwa der Implementierung der virtuellen Maschinen, der Last der parallel laufenden, anderen Aufgaben, CPU-Typ und weiterer Kontextfaktoren. Hier ist noch viel Forschung nötig.

Wie funktioniert die Virtualisierung genau? Je nach Sachlage und Anforderungen gibt es unterschiedliche Virtualisierungsarten für einen Server (*virtual server*). Allen gemeinsam ist, dass jeweils eine Zwischenschicht eingezeichnet wird, die die Funktionalität einer virtuellen Maschine hat. Da sie alles kontrolliert, was an Funktionalität bei der Schicht darunter vorhanden ist, wird sie auch *virtual machine monitor* (VMM) oder *Hypervisor* genannt. In Abb. 1.14 sind drei verschiedene Möglichkeiten gezeigt, wobei jeweils die Grenze zwischen *user mode* und *kernel mode* als roter Strich eingezeichnet ist.

Diese Überlegungen werden in den folgenden Abschnitten genauer erklärt.

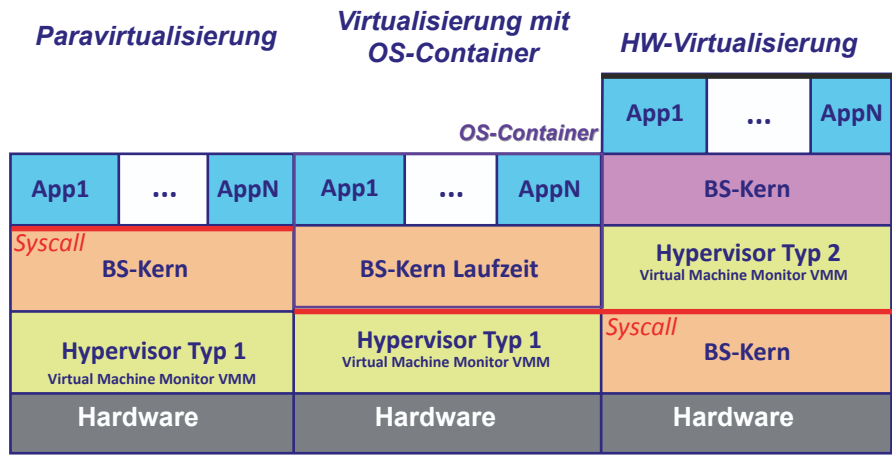


Abb. 1.14 Virtualisierungsarten bei Servern

1.6.1 Paravirtualisierung

Möchte man mehrere verschiedene Betriebssysteme mit den dazu gehörenden Anwendungen auf demselben Rechner gleichzeitig laufen lassen, so kann man den Hypervisor direkt zwischen traditionellem Betriebssystemkern und der Hardware installieren (*bare metal hypervisor*, *hypervisor type 1*), siehe [Abb. 1.14](#) links. Natürlich müssen die Betriebssystemkerne, die darauf ablaufen, Code desselben CPU-Typs enthalten, also beispielsweise für Intel x86.

Diese Art von Virtualisierung wird auch *Paravirtualisierung* genannt, wobei sich Hypervisor und Betriebssystemkern logisch über einander befinden, siehe [Abb. 1.15](#).

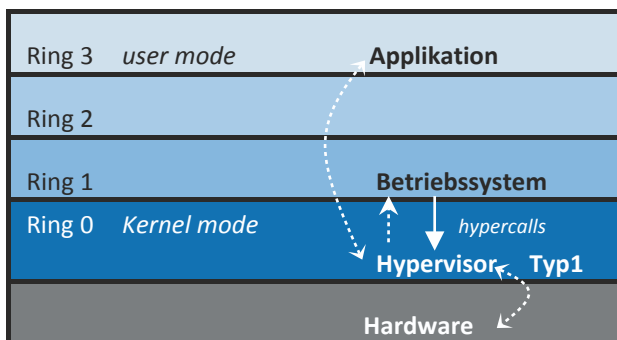
Allerdings werden Ring 2 und 3 meist nicht benutzt, so dass in der 64 Bit-Architektur (x86-64-Architektur) hardwaremäßig auf diese Schichten verzichtet wurde. In solchen Systemen mit nur zwei Sicherheitsstufen sind Betriebssystem und Hypervisor entweder nebeneinander in derselben Schicht 0 (Ring 0) im *kernel mode*, oder das Betriebssystem wird in einen gesonderten Prozess in den *user mode* verschoben.

Da das Betriebssystem nur mittels des Hypervisors auf die Hardware zugreifen sollte, müssen dazu vorher aus dem Betriebssystemcode alle Hardwareinstruktionen entfernt werden, die problematisch sind. Dies sind nach (Popek und Goldberg 1974) alle Befehle, die keine Unterbrechung (*trap*) hervorrufen wie die Systemaufrufe („privilegierte Befehle“), sondern unüberwacht eine Hardwarekontrolle ausüben („sensitive Befehle“). Dazu gehört beispielsweise das Setzen oder Löschen von Bits in Controllern und CPU-Registern. Diese Befehle werden ersetzt durch Aufrufe an den Hypervisor (*Hypercalls*), so dass der Hypervisor die volle Kontrolle erhält. In [Abb. 1.15](#) ist dies visualisiert.

Dabei werden die „normalen“ Betriebssystemaufrufe zunächst in den Hypervisor umgeleitet, der dann wiederum die Betriebssystemfunktionen aufruft, die dann über Hypercalls die eigentlichen Hardwarefunktionen bewirken.

Beispiele sind Red Hat Fedora mit XEN, z/VM, VMware ESX, Hyper-V, die *open source* SUN VirtualBox.org, Citrix XenServer, Microsoft Hyper-V und Virtual Iron.

Abb. 1.15 Schichten bei der Paravirtualisierung



1.6.2 Virtualisierung mit BS-Containern

Bei vielen Anwendungen reicht es auch aus, nur die benötigten Systemfunktionen, also eine Teilmenge des Kerns, den Anwendungen zur Verfügung zu stellen und sie damit unabhängig zu machen. Die Anwendungen und die benötigte Laufzeitumgebung zusammen werden gekapselt und bilden einen Container (*Jail*), siehe [Abb. 1.14](#) mitte. Dies hat den Vorteil, dass alle internen Referenzen der Programme, wie offene Dateien, der Programmzustand, belegte Puffer, andere benötigte Programme usw. nicht gewechselt werden müssen, wenn die Anwendung gewechselt wird. Der Hypervisor ermöglicht dann die parallele Abarbeitung verschiedener Container und damit auch verschiedener Laufzeitanwendungen. Dies wird als *Virtualisierung mit BS-Containern* bezeichnet. Der Übergang zum *kernel mode* geschieht hier erst beim Übergang zum Hypervisor.

Beispiele sind etwa Open Solaris, BSD jails, Mac-on-Linux, OpenVZ, Linux-VServer, ...

1.6.3 Vollständige Virtualisierung

Möchte man nicht das gleiche Betriebssystem nutzen, sondern ein anderes, so müssen die Systemaufrufe für die Verwaltung der Hardware wie CPU, Hauptspeicher, Festplattenspeicher, Netzwerke usw. in die Aufrufe des aktuellen Betriebssystems übersetzt werden, das auf der Hardware läuft. Mit anderen Worten, die gesamte Maschinenhardware wird virtualisiert (*Vollständige Virtualisierung*). Dies ist die Aufgabe des Hypervisors oder Virtuellen Maschinenmonitors, der eine entsprechende Schnittstelle nach oben anbietet und dazu nach unten die Schnittstelle des aktuellen Betriebssystems, die Liste der Systemaufrufe, nutzt.

Auch hier kann man ihn in einen nicht genutzten Ring legen und damit zwischen die Anwendungs- und Betriebssystemschicht.

Beispiele VMware, x86 Version von MS Virtual PC, XEN 3 auf Intel VT-x bzw. AMD-V.

1.6.4 Hardware-Virtualisierung

Möchte man auch Rechner und Betriebssystemkern noch weiter entkoppeln und etwa Betriebssysteme ganz anderer CPU-Typen auf dem Server ablaufen lassen, so dient der Hypervisor als Vermittler zwischen verschiedenen CPU-Typen und Betriebssystemfunktionen. Dies wird als *Hardware-Virtualisierung* bezeichnet. Hier gilt die gleiche Schichtung wie bei der vollständigen Virtualisierung in [Abb. 1.14](#) rechts und [Abb. 1.16](#), aber es werden nicht die Systemanforderungen, sondern die Maschineninstruktionen der anderen CPU emuliert, also aus den echten Instruktionen nachgebildet (*hardware emulation*). Dabei muss, ebenso wie bei der Paravirtualisierung oder der Virtualisierung mit Betriebssystemcontainern, das Betriebssystem angepasst werden, um die sensitiven und

Abb. 1.16 Schichten bei der vollständigen Virtualisierung

Ring 3	<i>user mode</i>	Applikation
Ring 2		Gast-Betriebssystem
Ring 1		Hypervisor Typ2
Ring 0	<i>Kernel mode</i>	Host-Betriebssystem
Hardware		

privilegierten Befehle auch mittels Hypervisor in der Hardware ausführen zu können. Deshalb sind diese Techniken meist bei quelloffenen Betriebssystemen wie Unix, nicht aber bei Windows, üblich.

Beispiel sind Pocket PC auf MS Virtual PC oder eingebettete Systeme auf x86, MIPS, ARM und PowerPC Basis, die emuliert werden.

1.6.5 Virtualisierungserweiterungen

In Absch. 1.2 hatten wir ein Schichten- und Zwiebelschalenmodell für den CPU-Zugriff eingeführt. Diese CPU-Architektur von konzentrischen Ringen stößt an ihre Grenzen beim Versuch, zwischen Hardware und Betriebssystem eine weitere Schicht zur Überwachung und Management des laufenden Betriebs, einen Hypervisor, einzuziehen. Eine Möglichkeit besteht nun darin, den Hypervisor in Ring 0 anzusiedeln und das Betriebssystem in Ring 1 („Vollständige Virtualisierung“). Dazu muss man aber vorher zur Laufzeit alle Systemaufrufe (Traps) zum Hypervisor umlenken bzw. durch „normale“ Aufrufe zum Betriebssystem im *user mode* ersetzen (*binary translation*). Eine weitere mögliche Lösung nutzt Ring 0 für beide: Betriebssystem und Hypervisor („Paravirtualisierung“). Auch hier müssen im Betriebssystem die Aufrufe der Hardware durch entsprechende Aufrufe (*hyper calls*) des Hypervisors ersetzt werden.

Die Ersetzung von Instruktionen im laufenden Betrieb ist umständlich, fehleranfällig und kostet Zeit. Wesentlich einfacher ist nun, die inzwischen für die Virtualisierung übliche Lösung: Man führt einfach eine weitere Schicht (Ring –1) in der Hardware ein, in der der Hypervisor ausgeführt wird (Intel VT, AMD-V), siehe [Abb. 1.17](#).

Damit kann man das unmodifizierte Originalbetriebssystem mittels seiner privilegierten Aufrufe (*system calls*) verwenden, ohne deshalb die Kontrolle aufzugeben: Bei jedem sensitiven oder privilegierten Aufruf wird der Hypervisor angesprungen. Hier gibt es also neben dem eigentlichen *kernel mode* noch einen weiteren, noch stärker privilegierten Modus: den *root mode* in Ring –1, der zusätzliche, privilegierte Hardwareinstruktionen bereitstellt und dem Hypervisor vorbehalten ist, siehe (vmware 2007).

In dem neueren Linuxkern ab Version 2.6.20 (2007) ist bereits eine *kernel-based virtual machine* (KVM) enthalten, die auf obigen Hardwareerweiterungen basiert.

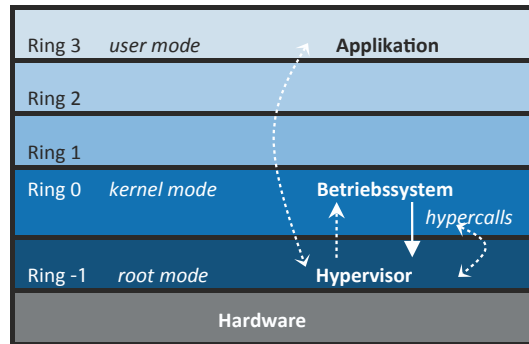


Abb. 1.17 Hardwareerweiterung für die Virtualisierung

1.7 Betriebssysteme für eingebettete Systeme

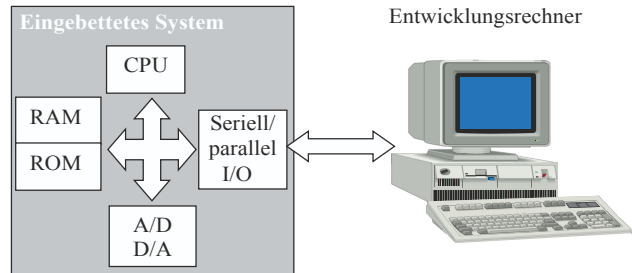
In über 92 % aller Computersysteme werden universelle Betriebssysteme wie Unix oder Windows nicht benötigt, da sie sehr spezielle Aufgaben zu bewältigen haben, etwa einen Fahrstuhl zu betätigen oder einen Fahrkartenautomaten zu managen. Üblicherweise laufen dazu nur wenige, genau festgelegte und gut bekannte Prozesse und Programme, so dass die Interaktion mit der Umgebung und die benötigten Ressourcen genau festgelegt werden können.

Solche Systeme heißen „eingebettete Systeme“ (*embedded systems*) und treten als Rechnersysteme nach außen hin nicht in Erscheinung: Wer vermutet schon in der Motor-elektronik, im Handy, im Radio oder in der Waschmaschine eigene Rechnersysteme? Tatsächlich kommt die „Intelligenz“ solcher Alltagsmaschinen ohne eigene Rechnersysteme mit CPU, Speicher, Ein- und Ausgabebausteinen nicht mehr aus. Natürlich hat ein solches System auch ein Betriebssystem. Es zeichnet sich aus durch

- kleineren Betriebssystemkern
- Eliminierung von Fehlerquellen
- robusteres Gesamtsystem
- Beschleunigung des Systemstarts und -stopps
- Keine Portierung oder Veränderung bestehender Anwendungen nötig

Hier macht eine vollständige Betriebssysteminstallation mit allen Komponenten keinen Sinn, sondern schafft nur unerwünschte zusätzliche Fehlerquellen. Alle unnötigen Teile wie Treiber nicht vorhandener Geräte, PnP, Sicherheitsüberprüfungen, unnötige Protokollschichten usw. können weggelassen werden. Durch eine Reduktion des Betriebssystems auf die tatsächlich erforderlichen Komponenten wird ein System erzeugt, welches erheblich robuster und besser geschützt gegen Fehlbedienungen und Missbrauch ist. Teilweise kann sogar eines der kritischsten Systemkomponenten, der Festplattenspeicher, eingespart werden. Das wesentlich kompaktere System passt auf einen robusten Speicherchip

Abb. 1.18 Programmentwicklung bei eingebetteten Systemen



(FLASH-Memory). So lassen sich sogar Gesamtsysteme ohne bewegliche Teile realisieren und so die Lebensdauer und Zuverlässigkeit erhöhen.

Das speziell angepasste eingebettete Betriebssystem startet in erheblich kürzerer Zeit, als eine Standard-Installation des Betriebssystems. Es existieren auch *Embedded*-Versionen, die ohne BIOS auskommen. Innerhalb weniger, meist einzelner Sekunden ist das System dann voll einsatzbereit. Genauso einfach kann das System abgeschaltet werden. Ein geordnetes „Herunterfahren“, Dogma der Standard-Betriebssysteminstallationen, entfällt vollkommen. Ein Beispiel dafür ist ein Microsystem für Roboter (Koubaa 2017).

Die Programmentwicklung für ein solches System ist nicht ganz unproblematisch. Verwenden wir Betriebssysteme, welche die gleichen Schnittstellen zum Kern anbieten wie die vorhandenen Vollversionen, etwa das kostenlose *embedded Linux* oder lizenzpflichtige *Windows CE*, so können wir unsere auf den Vollversionen entwickelten Programme auch auf dem eingebetteten System einsetzen. Zwar müssen nicht alle eingebetteten Systeme harte Echtzeitbedingungen erfüllen und benötigen deshalb nicht unbedingt die Scheduler aus Kap. 2, aber alle Programme müssen auf dem Zielsystem getestet werden, bevor sie in ROM oder EPROM gebrannt und dort fest eingesetzt werden. In Abb. 1.18 ist dazu eine Skizze gezeigt, wie die Programmentwicklung üblicherweise durchgeführt wird.

Nach jedem Kompilieren und Binden wird das fertige Programm über eine Kommunikationsleitung in das eingebettete System geladen und dort getestet. Spezielle Kommunikationsprogramme bewirken das Übermitteln des Codes an einen Stellvertreterprozess im eingebetteten System, der diesen Kindprozess in den Speicher lädt und dann startet. Erfolgt ein Fehlerabbruch oder das „normale“ Ende des Kindes, so erhält der Stellvertreterprozess die Kontrolle zurück und teilt dieses seinem Pendant auf dem Entwicklungsrechner mit. Kommunikationsprogramme und Stellvertreterprozesse sind üblicherweise Teil einer Entwicklungsumgebung auf dem Entwicklungsrechner, die entweder vom Hersteller des eingebetteten Systems (z. B. Motorola, Intel) oder vom Hersteller des eingebetteten Betriebssystems (z. B. Microsoft) angeboten werden.

1.8 Betriebssysteme in Mehrprozessorarchitekturen

Für die Verwaltung von Betriebsmitteln ist es ziemlich wichtig, welche Beziehungen und Abhängigkeiten zwischen ihnen bestehen. Unabhängig von dem benutzten Prozessortyp,

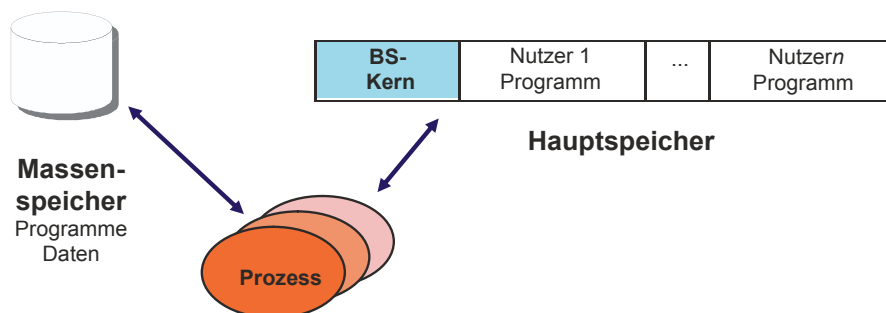


Abb. 1.19 Ein Einprozessor- oder Mehrkernsystem

Bustyp oder Speicherplattenfabrikat muss man einige grundsätzliche Konfigurationen unterscheiden. Im einfachsten, klassischen Fall gibt es nur einen Prozessor, der Haupt- und Massenspeicher benutzt, um das Betriebssystem (BS) und die Programme der Benutzer auszuführen, siehe Abb. 1.19. Die Ein- und Ausgabe (Bildschirm, Tastatur, Maus) ist dabei nicht gezeigt.

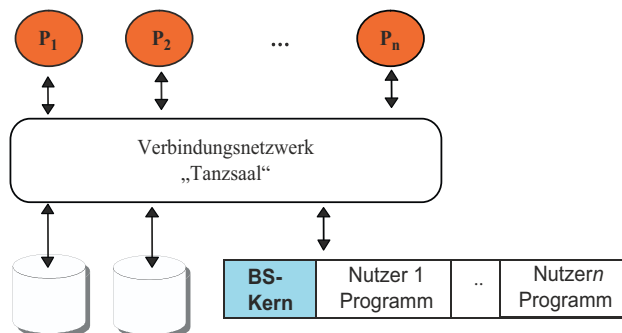
Ein solches **Einprozessorsystem** (*single processor*) kann man mit mehreren Prozessoren (meist bis zu 16 Stück) aufrüsten. Je nachdem, wie man die Prozessoren miteinander koppelt, ergeben sich unterschiedliche Architekturen.

Im einfachsten Fall kann man nur multiple Prozessoren an die Stelle des einzelnen Prozessor setzen, etwa durch Anreicherung mit mehreren Kernen wie sie bereits in einfachen PCs anzutreffen sind. Hierbei ändert sich aber nichts weiter an der Hauptarchitektur, so dass die multiplen Kerne sich die Datenwege zum Hauptspeicher und Ein-/Ausgabe zum Massenspeicher teilen müssen. Dies begrenzt die Leistung des Gesamtsystems stark.

Die nächste Stufe sieht Replikationen der CPU vor, die parallel an einem besonderen Verbindungsnetzwerk (z. B. an einem Multi-Master-Systembus) hängen. In Abb. 1.20 ist ein solches **Multiprozessorsystem** abgebildet.

Die Prozessormodule sind auf der einen Seite des Netzwerks und die Speichermodule auf der anderen lokalisiert. Für jeden Daten- und Programmcodezugriff wird zwischen ihnen eine Verbindung hergestellt, die während der Anforderungszeit bestehen bleibt. Diese Architektur wird deshalb auch „Tanzsaal“-Konzept genannt. Sie ist bei den

Abb.1.20 Multiprozessorsystem



Großrechner (*main frames*, ab 250.000\$) weit verbreitet, wobei die Anzahl der Prozessoren >16 (bis einige tausend CPUs), die Hauptspeichergröße einige tausend GByte und die Anzahl der I/O-Einheiten mehrere hundert bis tausend beträgt.

Eine derartige Architektur ermöglicht zwar einen parallelen Zugriff der Prozessoren auf den Hauptspeicher, führt aber auch leicht zu Leistungs- (*Performance*)-Einbußen, da einzelne Netzwerkknoten für Speicherzugriffe bestimmter, oft benutzter Teile des Betriebssystems stark belastet werden („hot spots“) und sich dort Warteschlangen ausbilden. Abhilfe kommt in diesem Fall aus der Beobachtung, dass die Prozessoren meist nur einen eng begrenzten Programmteil referenzieren. Der Speicher kann deshalb aufgeteilt und der relevante Teil „dichter“ an den jeweiligen Prozessor herangebracht werden (Abb. 1.21). Die feste Aufteilung muss natürlich auch durch den Compiler unterstützt werden, der das Anwenderprogramm auf die Rechner entsprechend aufteilt, siehe Abschn. 6.4. Ein solches **Mehrrechnersystem** ist auch als „Vorzimmer“-Architektur bekannt.

Als dritte Möglichkeit der Anordnung der Verbindungen existiert das **Rechnernetz**. Hier sind vollkommen unabhängige Rechner mit jeweils eigenem (nicht notwendig gleichem!) Betriebssystem lose miteinander über ein Netzwerk gekoppelt, s. Abb. 1.22. Ist das Netzwerk sehr schnell und sind die Rechner räumlich dicht beieinander, so spricht man auch von einem *Cluster*.

Abb. 1.21 Mehrrechnersystem

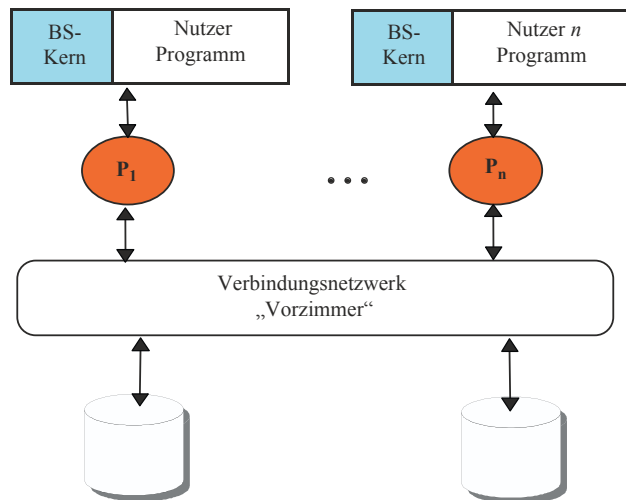
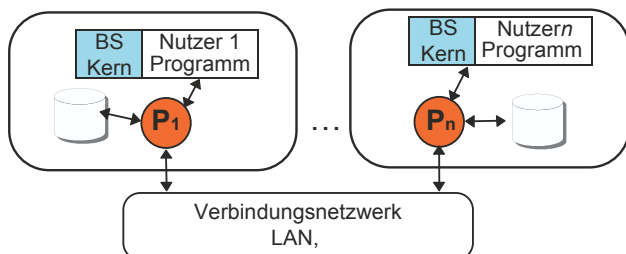


Abb. 1.22 Rechnernetz



Ist eine Software auf einem Rechner installiert, so dass der Rechner als Dienstleister (*Server*) Aufträge für einen anderen Rechner (Kunden, *Client*) ausführen kann, so spricht man von einer **Client-Server**-Architektur. Beispiele für Dienstleistungen sind numerische Rechnungen mit Supercomputern (*number cruncher*), Ausdruck von Dateien (*print server*) oder das Bereithalten von Dateien (*file server*).

Jede der vorgestellten Rechnerarchitekturen besitzt ihre Vor- und Nachteile. Für uns ist dabei wichtig: Jede benötigt spezielle Mechanismen, um einer Interprozessorkommunikation und Zugriffssynchronisation bei den Betriebsmitteln zu erreichen. Im kommenden Kapitel wird darauf näher eingegangen.

1.9 Aufgaben

Aufgabe 1-1 Betriebssystem

Der Zweck eines Betriebssystems besteht beispielsweise in der Verteilung von Betriebsmitteln auf die sich bewerbenden Benutzer.¹

- a) Wie ist der grobe Aufbau eines Betriebssystems?
- b) Welche Betriebsmittel kennen Sie?
- c) Welche Benutzer könnten sich bewerben? (Dabei ist der Begriff „Benutzer“ allgemein gefasst!)
- d) Welche Anforderungen stellt ein menschlicher Benutzer an das Betriebssystem?

Aufgabe 1-2 UNIX System calls

- a) Finden Sie heraus, wie viele System Calls auf einem aktuellen x86_64-Linux-System definiert sind. Beispielsweise können Sie dazu die entsprechenden Header-Dateien, mit denen der Kernel übersetzt wurde, inspizieren. Auf vielen Rechnern finden Sie diese im Verzeichnis `/lib/modules/[aktuelle Kernelversion]/build`, wobei Sie die `[aktuelle Kernelversion]` in einem Terminalfenster mit den Befehl `uname -r` herausfinden können.

Alternativ können Sie auch eine aktuelle Linux-Distribution in der 64-Bit-Variante, wie beispielsweise Ubuntu (<http://www.ubuntu.com/>) in einer virtuellen Maschine installieren. Beachten Sie jedoch, dass unter Umständen das Paket `linux-headers-generic` installiert werden muss, damit die Header-Dateien bereitstehen.

Geben Sie die Zahl der definierten Systemaufrufe an und erläutern Sie, wie Sie auf diese gekommen sind.

- b) In welche Funktionsgruppen lassen sie sich einteilen?

¹verkürzt für Benutzerinnen und Benutzer.

- c) Welcher Assemblerbefehl sorgt bei x86-Prozessoren für die Auslösung der Trap bei einem Systemaufruf, und wie wird dabei die Information übergeben, welcher Systemaufruf ausgeführt werden soll?

Tip: Wälzen Sie entweder das Handbuch (C- oder Assembler-Programmierung!), oder lassen Sie von einem Debugger einen in C oder einer anderen Programmiersprache geschriebenen Betriebssystemaufruf rückübersetzen in Assembler. Auch *include files* von C (syscall.h, trap.h, proc.h, kernel.h, ...) geben interessante Hinweise.

Aufgabe 1-3 Schichtenmodell und Kernel

- Beschreiben Sie mit Hilfe eines Beispiels das Schichtenmodell. (Das Beispiel sollte dabei kein Betriebssystem sein.)
- Erläutern Sie kurz die Begriffe „virtuelle Maschine“ und „Schnittstelle“.
- Warum werden im Schichtenmodell Schnittstellen benötigt?
- Das Mach-Betriebssystem besteht neben dem Kernel aus verschiedenen Modulen. Doch unterstützen die meisten modernen unixoiden Systeme das Laden von Modulen zur Laufzeit.

Eine Übersicht über alle aktuell geladenen Module lässt sich in einem Terminalfenster wie folgt ausgeben:

- Unter Linux: lsmod
- Unter FreeBSD: kldstat
- Unter Solaris: modinfo
- Unter OS X: kextstat

Überzeugen Sie sich auf einem beliebigen der aufgeführten Systeme, dass tatsächlich Module geladen sind.

- Was ist bei diesen Betriebssystemen der Unterschied zu besagten modularen Komponenten im Mach-Betriebssystem?
 - Welche Vor- und Nachteile ergeben sich daraus?
- e) Informieren Sie sich über GNU Hurd. Welches grundlegende Konzept wird bei Hurd verfolgt und wodurch zeichnet es sich aus?

Literatur

- Custer H.: Inside Windows NT. Microsoft Press, Redmond, Washington 1993
- Hardt M.: Virtualisation for Grid Computing. Cracow Grid Workshop, Krakow, Poland 2005
- Koubaa, A.: Robot Operating System (ROS). Springer Verlag 2017
- Popek G.J., Goldberg R.P.: Formal Requirements for Virtualizable Third Generation Architectures. Commun. of the ACM 17, 412–421 (1974)
- Vmware Inc.: Understanding full virtualization, paravirtualization and hardware assist. White paper, vmware, Palo Alto, CA 2007 http://www.vmware.com/files/pdf/VMware_paravirtualization.pdf [10.5.2016]

Inhaltsverzeichnis

2.1	Prozesszustände	29
2.1.1	Beispiel UNIX	30
2.1.2	Beispiel Windows NT	33
2.1.3	Leichtgewichtsprozesse	34
2.1.4	Aufgaben	37
2.2	Prozessscheduling	39
2.2.1	Zielkonflikte	40
2.2.2	Non-präemptives Scheduling	41
2.2.3	Präemptives Scheduling	44
2.2.4	Mehrere Warteschlangen und Scheduler	46
2.2.5	Scheduling in Echtzeitbetriebssystemen	47
2.2.6	Scheduling in Multiprozessorsystemen	53
2.2.7	Stochastische Schedulingmodelle	60
2.2.8	Beispiel UNIX: Scheduling	62
2.2.9	Beispiel: Scheduling in Windows NT	64
2.2.10	Aufgaben	65
2.3	Prozesssynchronisation	67
2.3.1	<i>Race conditions</i> und kritische Abschnitte	68
2.3.2	Signale, Semaphore und atomare Aktionen	69
2.3.3	Beispiel UNIX: Semaphore	77
2.3.4	Beispiel Windows NT: Semaphore	78
2.3.5	Anwendungen	79
2.3.6	Aufgaben zur Prozesssynchronisation	85
2.3.7	Kritische Bereiche und Monitore	88
2.3.8	Verklemmungen	92
2.3.9	Aufgaben zu Verklemmungen	101
2.4	Prozesskommunikation	103
2.4.1	Kommunikation mit Nachrichten	103
2.4.2	Beispiel UNIX: Interprozesskommunikation mit <i>pipes</i>	107

2.4.3	Beispiel Windows NT: Interprozesskommunikation mit pipes	108
2.4.4	Prozesssynchronisation durch Kommunikation.....	108
2.4.5	Implizite und explizite Kommunikation	117
2.4.6	Aufgaben zur Prozesskommunikation	118
Literatur.....		118

In früheren Zeiten waren die Rechner zu jedem Zeitpunkt für nur eine Hauptaufgabe bestimmt. Alle Programme wurden zu einem Paket geschnürt und liefen nacheinander durch (**Stapelverarbeitung** oder *Batch*-Betrieb). Üblicherweise gibt es heutzutage aber nicht nur ein Programm auf einem Rechner, sondern mehrere (**Mehrprogrammbetrieb**, *multi-tasking*). Auch gibt es nicht nur einen Benutzer (*single user*), sondern mehrere (**Mehrbenutzerbetrieb**, *multi-user*).

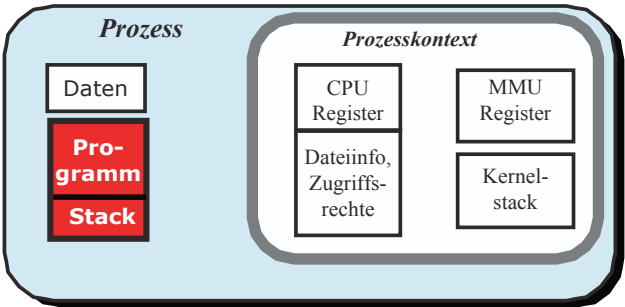
Um Konflikte zwischen ihnen bei der Benutzung des Rechners zu vermeiden, muss die Verteilung der Betriebsmittel auf die Programme geregelt werden. Dies spart außerdem noch Rechnerzeit und erniedrigt damit die Bearbeitungszeiten. Beispielsweise kann die Zuteilung des Hauptprozessors (*Central Processing Unit* CPU) für den Ausdruck von Text parallel zu einer Textverarbeitung so geregelt werden, dass die Textverarbeitung die CPU in der Zeit erhält, in der der Drucker ein Zeichen ausdruckt. Ist dies erledigt, schiebt der Prozessor ein neues Zeichen dem Drucker nach und arbeitet dann weiter an der Textverarbeitung.

Zusätzlich zu jedem Programm muss also gespeichert werden, welche Betriebsmittel es benötigt: Speicherplatz, CPU-Zeit, CPU-Inhalt etc. Die gesamte Zustandsinformation der Betriebsmittel für ein Programm wird als eine Einheit angesehen und als **Prozess** (*task*) bezeichnet (Abb. 2.1).

Ein Prozess kann auch einen anderen Prozess erzeugen, wobei der erzeugende Prozess als **Elternprozess** und der erzeugte Prozess als **Kindsprozess** bezeichnet wird.

Ein Mehrprogrammsystem (*multiprogramming system*) erlaubt also das „gleichzeitige“ Ausführen mehrerer Programme und damit mehrerer Prozesse (MehrProzesssystem, *multi-tasking system*). Ein Programm (**Job**) kann dabei auch selbst mehrere Prozesse erzeugen.

Abb. 2.1 Zusammensetzung der Prozessdaten



Beispiel UNIX

Die Systemprogramme werden in UNIX als Bausteine angesehen, die beliebig miteinander zu neuen, komplexen Lösungen kombiniert werden können. Die Unabhängigkeit der Prozesse und damit auch der Prozesse erlaubt nun in UNIX, mehrere Prozesse gleichzeitig zu starten. Beispielsweise kann man das Programm *cat*, das mehrere Dateien aneinander hängt, das Programm *pr*, das einen Text formatiert, und das Programm *lpr*, das einen Text ausdruckt, durch die Eingabe

```
cat Text1 Text2 | pr | lpr
```

miteinander verbinden: Die Texte „Text1“ und „Text2“ werden aneinandergehängt, formatiert und dann ausgedruckt. Der Interpreter (*shell*), dem der Befehl übergeben wird, startet dazu die drei Programme als drei eigene Prozesse, wobei das Zeichen „|“ ein Umlenken der Ausgabe des einen Programms in die Eingabe des anderen veranlasst. Gibt es mehrere Prozessoren im System, so kann jedem Prozessor ein Prozess zugeordnet werden und die obige Operation tatsächlich parallel ablaufen. Ansonsten bearbeitet der eine Prozessor immer nur ein Stück eines Prozesses und schaltet dann um zum nächsten.

Zu einem diskreten Zeitpunkt ist bei einem Einprozessorsystem nur immer ein Prozess aktiv; die anderen sind blockiert und warten. Dies wollen wir näher betrachten.

2.1 Prozesszustände

Zusätzlich zu dem Zustand „aktiv“ (*running*) für den einen, aktuellen Prozess müssen wir noch unterscheiden, worauf die anderen Prozesse warten. Für jede der zahlreichen Ereignismöglichkeiten gibt es meist eine eigene Warteschlange, in der die Prozesse einsortiert werden.

Ein blockierter Prozess kann darauf warten,

- aktiv den Prozessor zu erhalten, ist aber sonst bereit (*ready*),
- eine Nachricht (*message*) von einem anderen Prozess zu erhalten,
- ein Signal von einem Zeitgeber (*timer*) zu erhalten,
- Daten eines Ein/Ausgabegeräts zu erhalten (*io*).

Üblicherweise ist der *bereit*-Zustand besonders ausgezeichnet: Alle Prozesse, die Ereignisse erhalten und so entblockt werden, werden zunächst in die *bereit*-Liste (*ready-queue*) verschoben und erhalten dann in der Reihenfolge den Prozessor. Die Zustände und ihre Übergänge sind in [Abb. 2.2](#) skizziert.

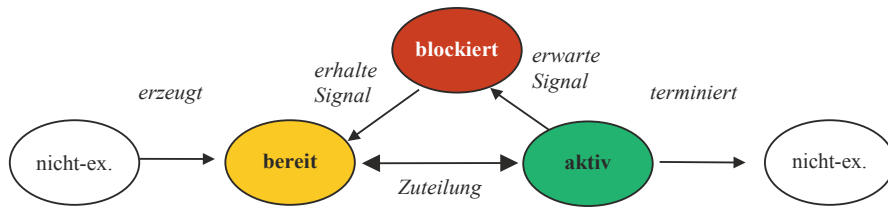


Abb. 2.2 Prozesszustände und Übergänge

Die Zustände „bereit“ und „blockiert“ enthalten eine oder mehrere Warteschlangen (Listen), in die die Prozesse mit diesem Zustand eingetragen werden. Es ist klar, dass ein Prozess immer nur in einer Liste enthalten sein kann.

Die Programme und damit die Prozesse existieren nicht ewig, sondern werden irgendwann erzeugt und auch beendet. Dabei verwalten die Prozesse aus Sicherheitsgründen sich nicht selbst, sondern das Einordnen in die Warteschlangen wird von einer besonderen Instanz des Betriebssystems, dem **Scheduler**, nach einer Strategie geplant. Bei einigen Betriebssystemen gibt es darüber hinaus eine eigene Instanz, den **Dispatcher**, der das eigentliche Überführen von einem Zustand in den nächsten bewirkt. Das Einordnen in eine Warteschlange, die Zustellung der Signale und das Abspeichern der Prozessdaten werden also von einer zentralen Instanz erledigt, die der Benutzer nicht direkt steuern kann und die den Kern des Betriebssystems bildet. Stattdessen werden über die Betriebssystemaufrufe die Wünsche der Prozesse angemeldet, denen im Rahmen der Betriebsmittelverwaltung vom Scheduler mit Rücksicht auf andere Benutzer entsprochen wird. Scheduler und Dispatcher sind in manchen Betriebssystemen systemeigene, spezielle Prozesse, die bei allen Ereignissen zuerst aufgerufen werden. In anderen, wie Unix, sind sie nur Kernfunktionen, die keine eigene Prozessstruktur haben.

Im Unterschied zu dem Maschinencode werden die Zustandsdaten der Hardware (CPU, FPU, MMU), mit denen der Prozess arbeitet, als **Prozesskontext** bezeichnet, s. Abb. 2.1. Der Teil der Daten, der bei einem blockierten Prozess den letzten Zustand der CPU enthält und damit wie ein Abbild der CPU ist, kann als *virtueller Prozessor* angesehen werden und muss bei einer Umschaltung zu einem anderen Prozess bzw. Kontext (*context switch*) neu geladen werden.

Die verschiedenen Betriebssysteme differieren in der Zahl der Ereignisse, auf die gewartet werden kann, und der Anzahl und Typen von Warteschlangen, in denen gewartet werden kann. Sie unterscheiden sich auch darin, welche Strategien sie für das Erzeugen und Terminieren von Prozessen sowie die Zuteilung und Einordnung in Wartelisten vorsehen.

2.1.1 Beispiel UNIX

Im UNIX-Betriebssystem gibt es sechs verschiedene Zustände: die drei oben erwähnten *running* (SRUN), *blocked* (SSLEEP) und *ready* (SWAIT) sowie *stopped* (SSTOP), was einem Warten auf ein Signal des Elternprozesses bei der Fehlersuche (*tracing* und

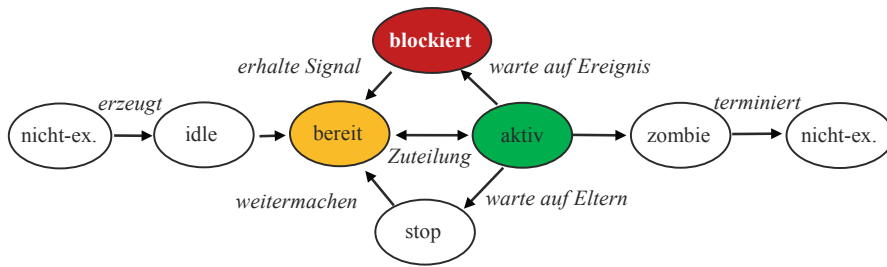


Abb. 2.3 Prozesszustände und Übergänge bei UNIX

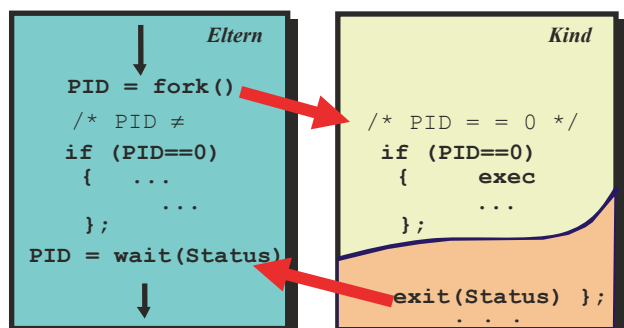
debugging) entspricht. Außerdem existieren noch die zusätzlichen Zwischenzustände *idle* (SIDL) und *zombie* (SZOMB), die beim Erzeugen und Terminieren eines Prozesses entstehen. Die Zustandsübergänge haben dabei die in [Abb. 2.3](#) gezeigte Gestalt.

Der Übergang von einem Zustand in einen nächsten wird durch Anfragen (Systemaufrufe) erreicht. Ruft beispielsweise ein Prozess die Funktion `fork()` auf, so wird vom laufenden Prozess (von sich selbst) eine Kopie gezogen und in die *bereit*-Liste eingehängt. Damit gibt es nun zwei fast identische Prozesse, die beide aus dem `fork()`-Aufruf zurückkehren. Der Unterschied zwischen beiden liegt im Rückgabewert der Funktion: Der Elternprozess erhält die Prozesskennzahl PID des Kindes; das Kind erhält `PID = 0` und erkennt daran, dass es der Kindsprozess ist und den weiteren Programmablauf durch Abfragen anders gestalten kann. Es kann z. B. mit `exec('program')` bewirken, dass das laufende Programm mit Programmcode aus der Datei „program“ überladen wird. Alle Zeiger und Variablen werden initialisiert (z. B. der Programmzähler auf die Anfangsadresse des Programms gesetzt) und der fertige Prozess in die *bereit*-Liste eingehängt. Damit ist im Endeffekt vom Elternprozess ein völlig neues Programm gestartet worden.

Der Elternprozess hat dann die Möglichkeit, auf einen `exit()`-Systemaufruf und damit auf das Ende des Kinds mit einem `waitpid(PID)` zu warten. In [Abb. 2.4](#) ist der Ablauf einer solchen Prozesserzeugung gezeigt.

Man beachte, dass der Kindsprozess den `exit()`-Aufruf im obigen Beispiel nur dann erreicht, wenn ein Fehler bei `exec()` auftritt; z. B. wenn die Datei „program“ nicht existiert, nicht lesbar ist usw. Ansonsten ist der nächste Programmbefehl nach

Abb. 2.4 Erzeugung und Vernichten eines Prozesses in UNIX



`exec()` (im *user mode*) identisch mit dem ersten Befehl des Programms „program“. Der Kindsprozess beendet sich erst, wenn in „program“ selbst ein `exit()`-Aufruf erreicht wird.

Mit diesen Überlegungen wird folgendes Beispiel für einen Prozess für Benutzereingaben am Terminal (*shell*) klar. Der Code ist allerdings nur das Grundgerüst der in UNIX tatsächlich für jeden Benutzer gestarteten *shell*.

Beispiel shell

```

LOOP
  Write(prompt);                (* tippe z. B. '>' *)
  ReadLine(command, params);    (* lese strings, getrennt durch
                                Leertaste *)
  pid := fork();                (* erzeuge Kopie dieses Prozesses *)
  IF (pid=0)
    THEN execve(command,params,0) (* Kind: überlade mit Programm *)
    ELSE waitpid(-1, status,0)    (*Eltern: warte auf Ausführungsende
                                vom Kind*)

  END;
END;
```

Alle Prozesse in UNIX stammen direkt oder indirekt von einem einzigen Prozess ab, dem Init-Prozess mit PID = 1. Ist beim „Ableben“ eines Kindes kein Elternprozess mehr vorhanden, so wird stattdessen „init“ benachrichtigt. In der Zeit zwischen dem `exit()`-Systemaufruf und dem Akzeptieren der Nachricht darüber bei dem Elternprozess gelangt der Kindsprozess in einen besonderen Zustand, genannt „Zombie“, s. [Abb. 2.1](#).

Es gibt in Unix nur Benutzerprozesse, keine Kernprozesse. Ruft ein Prozess durch einen *system call* Kernfunktionen auf, so wechselt der Prozess dabei aus dem *user mode* in den *kernel mode* und führt im *kernel mode* die Funktionen des Kerns aus.

Der interne Prozesskontext ist in zwei Teile geteilt: einer, der als Prozesseintrag (*Prozesskontrollblock* PCB) in einer speicherresidenten Tabelle (*process table*) steht, für das Prozessmanagement wichtig und deshalb immer präsent ist, und ein zweiter (*user structure*), der nur wichtig ist, wenn der Prozess aktiv ist, und der deshalb auf den Massenspeicher mit dem übrigen Code und den Daten ausgelagert werden kann.

Die beiden Teile sind im Wesentlichen

- *speicherresidente Prozesskontrollblöcke* PCB der Prozesstafel
 - Scheduling-Parameter
 - Speicherreferenzen: Code-, Daten-, Stackadressen im Haupt- bzw. Massenspeicher
 - Signaldaten: Masken, Zustände
 - Verschiedenes: Prozesszustand, erwartetes Ereignis, Timerzustand, PID, PID der Eltern, User/Group-IDs

- *auslagerbarer Benutzerkontext (swappable user structure)*
 - Prozessorzustand: Register, FPU-Register, ...
 - Systemaufruf: Parameter, bisherige Ergebnisse, ...
 - Dateinfo-Tabelle (file descriptor table)
 - Benutzungsinfo: CPU-Zeit, max. Stackgröße, ...
 - Kernel-Stack: Stackplatz für Systemaufrufe des Prozesses

Im Unterschied zu dem PCB, den der Prozess nur indirekt über Systemaufrufe abfragen und ändern kann, gestatten spezielle UNIX-Systemaufrufe, die *user structure* direkt zu inspizieren und Teile davon zu verändern.

2.1.2 Beispiel Windows NT

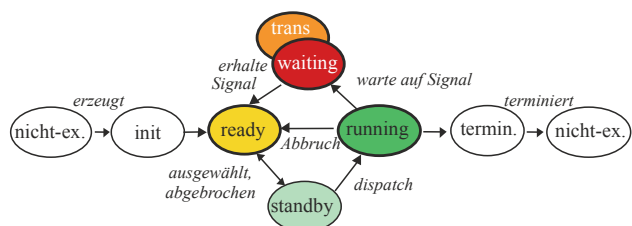
Da in Windows NT verschiedene Arten von Prozessen unterstützt werden müssen, deren vielfältige Ausprägungen sich nicht behindern dürfen, wurde nur für eine einzige, allgemeine Art von Prozessen, den sog. *thread objects*, ein Prozesssystem geschaffen. Die speziellen Ausprägungen der OS/2-Objekte, POSIX-Objekte und Windows32-Objekte sind dabei in den Objekten gekapselt und spielen bei den Zustandsänderungen keine Rolle. Der vereinfachte Graph der Zustandsübergänge ist in Abb. 2.5 gezeigt.

Ist der *thread* zur Ausführung ausgewählt worden, so wartet er im Zustand *standby* auf seine Prozessorzuteilung; maximal ein Prozess pro Prozessor im Multiprozessorsystem. Passt der Prozess nicht zum Prozessor, weil er etwa den falschen Maschinencode hat oder einen Fließkomma-Coprozessor verlangt, den der freie Prozessor nicht hat, so wird er wieder zurück in die Warteschlange geschoben und der nächste zuteilungsreife Prozess der Warteschlange geht in den Zustand *standby*.

Der übliche „blockiert“-Zustand (hier: *waiting*) bedeutet hier das Warten auf ein Signal, genauer: darauf, dass ein oder mehrere Objekte in den Zustand „signalisiert“ übergehen. Fehlen noch Ressourcen zur Ausführung, so geht der *thread* vorher in den Zustand *transition*. Dort wartet er auf die Ressourcen, etwa auf ausgelagerte Seiten seines Kernel-Stacks.

Die Prozesserschöpfung in Windows NT ist etwas komplexer als in UNIX, da als Vorgabe mehrere Prozessmodelle erfüllt werden mussten. Dazu wurden die speziellen Ausprägungen in den Subsystemen gekapselt. Zur Erzeugung von Prozessen gibt es nur einen einzigen Systemaufruf `NtCreateProcess()`, bei dem neben der Initialisierung durch Code

Abb. 2.5 Prozesszustände in Windows NT



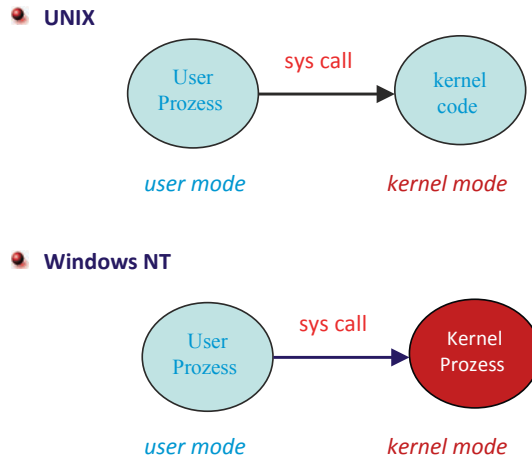


Abb. 2.6 Ausführung der Prozesse in UNIX und Windows NT

auch der Elternprozess angegeben werden kann. Auf diesem bauen alle anderen, subsystemspezifischen Aufrufvarianten auf, die von den Entwicklern benutzt werden.

Dies ermöglicht beispielsweise den POSIX-`fork()`-Mechanismus. Dazu ruft das POSIX-Programm (POSIX-Prozess) über die API (*Application Programming Interface*) den `fork()`-Befehl auf. Dies wird in eine Nachricht umgesetzt und über den Kern an das POSIX-Subsystem geschickt, s. Abb. 1.9. Dieses wiederum ruft `NtCreateProcess()` auf und gibt als ElternPID das POSIX-Programm an. Der vom Kern zurückgegebene Objektschlüssel (*object handle*) wird dann vom POSIX-Subsystem verwaltet; alle Systemaufrufe vom POSIX-Prozess, die als Nachrichten beim POSIX-Subsystem landen, werden dort mit Hilfe von NT-Systemaufrufen erledigt und die Ergebnisse im POSIX-Format an den aufrufenden Prozess zurückgegeben. Analog gilt dies auch für die Prozessaufrufe der anderen Subsysteme.

Es gibt in Windows NT sowohl Benutzerprozesse als – im Unterschied zu Unix – auch Kernprozesse. Die Kernprozesse werden beim Systemstart bereits erzeugt und in einem Pool gehalten. Diese Prozesse werden nur im *kernel mode* ausgeführt. Jeder *system call* beauftragt jeweils einen Kernprozess mit der Ausführung des Dienstes, während der jeweilige Benutzerprozess angehalten wird und auf das Ergebnis wartet. Es können so mehrere *system calls* gleichzeitig abgearbeitet werden, so das Windows NT multiprozessorfähig ist.

In Abb. 2.6 ist dies im Vergleich zu Unix schematisch abgebildet.

2.1.3 Leichtgewichtsprozesse

Der Speicherbedarf eines Prozesses ist meist sehr umfangreich. Er enthält nicht nur wenige Zahlen, wie Prozessnummer und CPU-Daten, sondern auch beispielsweise alle Angaben

über offene Dateien sowie den logischen Status der benutzten Ein- und Ausgabegeräte sowie den Programmcode und seine Daten. Dies ist bei sehr vielen Prozessen meist mehr, als in den Hauptspeicher passt, so dass bis auf wenige Daten die Prozesse auf den Massenspeicher (Festplatte) ausgelagert werden. Da beim Prozesswechsel der aktuelle Speicher ausgelagert und der alte von der Festplatte wieder restauriert werden muss, ist ein Prozesswechsel eine „schwere“ Systemlast und dauert relativ lange.

Bei vielen Anwendungen werden keine völlig neuen Prozesse benötigt, sondern nur unabhängige Codestücke (*threads*), die im gleichen Prozesskontext agieren, beispielsweise Prozeduren eines Programms. In diesem Fall spricht man auch von **Coroutinen**. Ein typisches Anwendungsbeispiel ist das parallele Ausführen unterschiedlicher Funktionen bei Texteditoren, bei denen „gleichzeitig“ zur Eingabe der Zeichen auch bereits der Text auf korrekte Rechtschreibung überprüft wird.

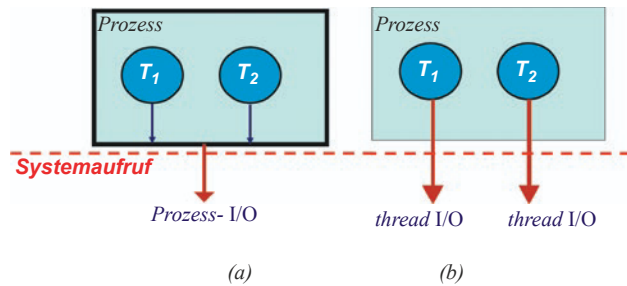
Die Verwendung von *threads* ermöglicht es, innerhalb eines Prozesses ein weiteres Prozesssystem aus sog. **Leichtgewichtsprozessen** (LWP) zu schaffen. In der einfachsten Form übergeben sich die Prozesse gegenseitig explizit die Kontrolle („Coroutinen-Konzept“). Dies bedingt, dass die anderen Prozesse bzw. ihre Namen (ID) den aufrufenden Prozessen auch bekannt sind. Je mehr Prozesse erzeugt werden, desto schwieriger wird dies. Aus diesem Grund kann man auch hier einen Scheduler einführen, der immer die Kontrolle erhält und sie nur leihweise an einen Prozess aus seiner *bereit*-Liste weitergibt. Wird dies nicht vom Anwender programmiert, sondern die Funktionalität ist bereits im Betriebssystem enthalten und wird über Systemaufrufe benutzt, so werden die *thread*-Prozesse durch die Umschaltzeiten der Systemaufrufe etwas langsamer und damit „schwergewichtiger“.

Jeder Prozess muss seine eigenen Daten unabhängig vom anderen halten. Dies gilt auch für Leichtgewichtsprozesse, auch wenn sie die gleichen Dateien und den gleichen Speicher (genauer: den gleichen virtuellen Adressraum, s. [Kap. 3](#)) mit den anderen Leichtgewichtsprozessen teilen. Allerdings muss die Trennung der Daten vom Programmierer eingehalten werden und ist nicht automatisch. Hält er sich nicht daran und lässt beispielsweise mehrere Threads auf den gleichen, global gültigen Daten arbeiten, so können sich *race conditions* ergeben; die Threads müssen extra synchronisiert werden, siehe [Abschn. 2.3](#).

Für jeden Threads wird bei der Erzeugung ein Stack eingerichtet. Im Unterschied zu den „echten“ Prozessen benötigen Leichtgewichtsprozesse nur wenige Kontextdaten, die beim Umschalten geändert werden müssen. Die wichtigsten sind das Prozessorstatuswort PS und der Stackpointer SP, die vor dem Umschalten auf einen anderen Thread auf dem eigenen Stack abgelegt werden. Selbst der aktuelle Programmzähler PC kann auf dem Stack abgelegt sein, so dass er nicht explizit übergeben werden muss. Effiziente Implementierungen des Umschaltens in Assemblersprache machen diese Art von Prozesssystemen sehr schnell.

Die Einrichtung eines eigenen Stacks ermöglicht den Threads, beliebig zu anderen Threads innerhalb eines Prozesses umzuschalten. Dies ist ein großer Unterschied zu Prozeduren oder Subroutinen, die beim Aufruf anderer Prozeduren einen einzigen Stack verwenden und deshalb nach mehreren Aufrufen in der inversen Aufrufreihenfolge wieder beendet werden müssen.

Abb. 2.7 I/O-Verhalten (a) leichter und (b) schwerer threads



Leichtgewichtsprozesse sind meist in Form einer Programmbibliothek organisiert, die von allen Benutzern genutzt werden kann. Der Vorteil einer Implementierung von *threads* als Bibliothek (z. B. Northrup 1996) besteht in einer sehr schnellen Prozessumschaltung (*lightweight threads*, *user threads*), da die Mechanismen des Betriebssystemaufrufs und seiner Dekodierung nach Dienstnummer und Parametern nicht durchlaufen werden müssen. Der Nachteil besteht darin, dass ein *thread*, der auf ein Ereignis (z. B. I/O) wartet, den gesamten Prozess mit allen *threads* blockiert. In Abb. 2.7 ist dies visualisiert. In Abb. 2.7a ist der Fall gezeigt, dass nicht die beiden *threads*, sondern nur der ganze Prozess dem Betriebssystem bekannt und deshalb bei I/O mit dem Prozess beide *threads* blockiert werden.

Im oben erwähnten Beispiel eines Texteditors würde der Teil, der die Rechtschreibüberprüfung durchführt, während einer Tipp-Pause blockiert werden; die Vorteile der Benutzung von *threads* (parallele Ausführung und Ausnutzung freier CPU-Zeit) wären dahin. Eine solche *thread*-Bibliothek eignet sich deshalb eher für unabhängige Programmstücke, die keine Benutzereingabe benötigen.

Im zweiten Fall in Abb. 2.7 (b) werden alle *threads* über BS-Aufrufe erzeugt und kontrolliert. Zwar dauert damit die Umschaltung zwischen den *threads* länger (*heavyweight threads*, *kernel threads*), aber zum einen hat dies den Vorteil, dass der Systemprogrammierer eine feste, verbindliche Schnittstelle hat, an die er oder sie sich halten kann. Es erleichtert die Portierung von Programmen, die solche LWP benutzen, und verhindert die Versuchung, eigene abweichende Systeme zu entwickeln, wie dies für bibliotheksbasierte *threads* lange Zeit der Fall war. Zum anderen hat der BS-Kern damit auch die Kontrolle über die LWP, was besonders wichtig ist, wenn in Multiprozessorsystemen die LWP echt parallel ausgeführt werden oder für I/O nur ein *thread* eines Prozesses blockiert werden soll, wie z. B. bei unserem Texteditor.

Beispiel UNIX

In UNIX sind die Leichtgewichtsprozesse (*threads*) als Benutzerbibliothek in C oder C++ implementiert, siehe UNIX-Manual Kapitel 3. Je nach Implementierung gibt es einfache Systeme mit direkter Kontrollübergabe (Coroutinen-Konzept) oder komplexere mit einem extra Scheduler und all seinen Möglichkeiten, siehe Abschn. 2.2.

Der Aufruf `vfork()` in UNIX, der einen echten, neuen Prozess bei altem Prozesskontext schafft, ist nicht identisch mit einem *thread*; er übernimmt den Kontrollfluss des

Elternprozesses und blockiert ihn damit. In LINUX ist er identisch mit einem `fork()`. Unabhängig davon sind *threads* in Linux auch als *kernel threads* implementiert und werden wie „Prozesse mit gemeinsamem Kontext“ behandelt. Beim Erzeugen wird über Argumente festgelegt, wie viel der neue *thread* an den gemeinsamen Ressourcen des Prozesses teilhaben soll: die Daten, die offenen Dateien oder auch die Signale. Damit enthält Linux neben den Leichtgewichtsprozess-Bibliotheken mit dem *pthread*-System auch Schwergewichtsprozesse.

Es gibt Versuche, die *threads* zu normieren, um Programmportierungen zu erleichtern (siehe IEEE 1992). In den UNIX-Spezifikationen für ein 64-Bit UNIX (UNIX-98) sind sie deshalb enthalten und gehören fest zu „UNIX“. In LINUX wird POSIX –Kompatibilität angestrebt.

Beispiel Windows NT

In Windows NT sind im Unterschied zu UNIX zunächst die *threads* über Betriebssystemaufrufe implementiert. Da ein solcher Schwergewichts-*thread* wieder zu unnötigen Verzögerungen bei unkritischen Teilen innerhalb einer Anwendung führt, wurden in Windows NT ab Version 4.0 Bibliotheken mit sogenannten *fibers* eingeführt. Dies sind parallel ablaufende Prozeduren, die nach dem Coroutinen-Konzept funktionieren: Die Abgabe der Kontrolle von einer *fiber* an eine andere innerhalb eines *threads* erfolgt freiwillig; ist der *thread* blockiert, so sind alle *fibers* darin ebenfalls blockiert. Dies erleichtert auch die Portierung von Programmen auf UNIX-Systeme.

2.1.4 Aufgaben

Aufgabe 2.1-1 Betriebsarten

Nennen Sie einige Betriebsarten eines Betriebssystems (inklusive der drei wichtigsten).

Aufgabe 2.1-2 Prozesse

- Erläutern Sie nochmals die wesentlichen Unterschiede zwischen Programm, Prozess und *thread*.
- Welche schwerwiegenden Konsequenzen kann es haben, wenn ein Betriebssystem keine Prozesse, sondern lediglich Threads unterstützen würde?
- Wie sehen im UNIX-System des Bereichsrechners ein Prozesskontrollblock (PCB) und die *user structure* aus? Inspizieren Sie dazu die *include files* `/include /sys/proc.h` und `/include/sys/user.h` und charakterisieren Sie grob die Einträge darin.

Aufgabe 2.1-3 Prozesszustände

- Welche Prozesszustände durchläuft ein Prozess?
- Was sind typische Wartebedingungen und Ereignisse?
- Ändern Sie die Zustandsübergänge im Zustandsdiagramm von Abb. 2.2 durch ein abweichendes, aber ebenso sinnvolles Schema. Begründen Sie Ihre Änderungen.

Aufgabe 2.1-4 UNIX-Prozesse

Wie ließe sich in UNIX mit der Sprache C oder MODULA-2 ein Systemaufruf „ExecuteProgram(prg)“ als Prozedur mit Hilfe der Systemaufrufe `fork()` und `waitpid()` realisieren?

Aufgabe 2.1-5 Android activities

Recherchieren Sie,

- welche Zustände eine *activity* unter Android annehmen kann und wofür diese verwendet werden.
- Wie werden dabei die aktuell nicht sichtbaren *activities* verwaltet?

Aufgabe 2.1-6 Threads

Realisieren Sie zwei Prozeduren als Leichtgewichtsprozesse, die sich wechselseitig die Kontrolle übergeben. Der eine Prozess gebe „Ha“ und der andere „tschi!“ aus.

In MODULA-2 ist die Verwendung von Prozeduren als LWP mit Hilfe der Konstrukte `NEWPROCESS` und `TRANSFER` leicht möglich. Ein LWP ist in diesem Fall eine **Coroutine**, die parallel zu einer anderen ausgeführt werden kann. Dabei gelten folgende Schnittstellen:

```
PROCEDURE NEWPROCESS ( Prozedurname: PROCEDURE;
                       Stackadresse: ADDRESS;
                       Stacklänge: CARDINAL;
                       Coroutine: ADDRESS )
```

Die Prozedur `NEWPROCESS` wird nur einmal aufgerufen und initialisiert die Datenstrukturen für die als Coroutine zu verwendende Prozedur. Dazu verwendet sie einen (vorher zu reservierenden) Speicherplatz.

Die Coroutinvariable vom Typ `ADDRESS` hat dabei die Funktion eines Zeigers in diesen Speicherbereich, in dem alle Registerinhalte gerettet werden, bevor die Kontrolle einer anderen Coroutine übergeben wird.

```
PROCEDURE TRANSFER ( QuellCoroutine,
                    ZielCoroutine: ADDRESS)
```

Bei der Prozedur `TRANSFER` wird die Prozessorkontrolle von einer Coroutine `QuellCoroutine` zu einer Coroutine `ZielCoroutine` übergeben.

Aufgabe 2.1-7 Kernel Threads

Auf einem Linux-System mit verschlüsselter Festplatte tauchen bei Eingabe von `ps -ef` in einem Terminal unter anderem die Einträge `[kcryptd]` und `[dmccrypt_write]` auf. Dabei handelt es sich um Kernel-Threads.

Recherchieren Sie, was Kernel-Threads auszeichnet und erläutern Sie in eigenen Worten, worin der Unterschied zwischen Kernel-Threads und den in der Vorlesung vorgestellten Threadarten liegt.

Aufgabe 2.1-8 Prozesse vs. Threads

Ihr Arbeitgeber hat Sie damit beauftragt, für die firmeninterne Verwendung eine integrierte Entwicklungsumgebung (IDE) zu programmieren. Für welche der folgenden Anforderungen ist es sinnvoller, Prozesse zu verwenden, zur Umsetzung welcher sollten besser Threads verwendet werden? Begründen Sie Ihre Antwort.

- Einer Ihrer Kollegen ist ein begnadeter Programmierer, macht jedoch sehr viele Syntaxfehler. Um diese so früh wie möglich zu erkennen, fordert er, dass die neue IDE ständig den aktuellen Stand des Codes einer Syntaxanalyse unterzieht.
- Zwei Ihrer Kollegen arbeiten an mehr als einem Projekt gleichzeitig und möchten daher die IDE mehrfach öffnen sowie die Fenster auf verschiedenen Bildschirmen darstellen können, um einen besseren Überblick zu haben. Allerdings trauen die beiden Ihren Programmierfähigkeiten nicht, weshalb sie betonen, dass bei einem Absturz durch einen Programmierfehler in der IDE nicht die Änderungen aller Projekte verloren sein dürfen, sondern höchstens eines einzigen Projekts. Es soll also jeder Programmierfehler höchstens eine einzige IDE gleichzeitig zum Absturz bringen können.

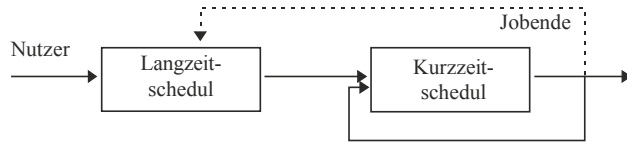
2.2 Prozesssscheduling

Gibt es in einem System mehr Bedarf an Betriebsmitteln, als vorhanden sind, so muss der Zugriff koordiniert werden. Eine wichtige Rolle spielt dabei der in [Abschn. 2.1](#) erwähnte Scheduler und seine Strategie beim Einordnen der Prozesse in Warteschlangen. Betrachten wir dazu die Einprozessorsysteme, auf denen voneinander unabhängige Prozesse hintereinander (*sequentiell*) ablaufen.

Das Scheduling besteht dabei aus einem Verfahren, was aus vielen Größen wie bekannte Prozessdauer, Wichtigkeit des Prozesses, der Kontextsituation, Abhängigkeit von anderen Prozessen, Anzahl der freien CPUs eines bestimmten Typs, usw. die Reihenfolge der bekannten, rechenwilligen Prozesse erstellt. Ist diese Liste einmal und für immer erstellt, so spricht man von „*statischem Scheduling*“. Dies ist aber meist nicht möglich, da die Aufgaben eines Computersystems rasch wechseln können. Stattdessen wird die Liste immer wieder neu erstellt und der gerade herrschenden Situation angepasst. Hier spricht man dann von „*dynamischem Scheduling*“.

Im Gegensatz zu dem rechenintensiven, zeitaufwändigen Scheduling ist die tatsächliche Zuteilung eines Prozesses zum Prozessor, also das Laden der Prozessdaten, relativ schnell. Dies wird von einer anderen Instanz, dem Dispatcher, erledigt.

Abb. 2.8 Langzeit- und Kurzzeitscheduling



In einem normalen Rechensystem können wir zwei Arten von Scheduling-Aufgaben unterscheiden: das Planen der Jobausführung (**Langzeitscheduling**, Jobscheduling) und das Planen der aktuellen Prozessorzuteilung (**Kurzzeitscheduling**). In Abb. 2.8 ist die Verkettung beider Verfahren visualisiert. Beim Langzeitscheduling wird darauf geachtet, dass nur so viele Benutzer mit ihren Prozessen neu ins System kommen (*log in*) dürfen, wie Benutzer sich abmelden (*log out*). Sind es zu viele, so muss der Zugang gesperrt werden, bis die Systemlast erträglich wird.

Ein Beispiel dafür ist die Zugangskontrolle von Benutzern über das Netz zu Dateiservern (ftp-server, www-server). Eine weitere Aufgabe für Langzeitscheduling ist das Ausführen von nicht-interaktiv ablaufenden Prozessen („Batch-Jobs“) zu bestimmten Zeiten, z. B. nachts.

Die meiste Arbeit wird allerdings in das Kurzzeitscheduling, die Strategie zur Zuweisung des Prozessors an Prozesse, gesteckt.

Im Folgenden betrachten wir einige der gängigsten Strategien für dynamisches Kurzzeitscheduling.

2.2.1 Zielkonflikte

Alle Schedulingstrategien versuchen, gewisse Ziele zu verwirklichen. Die gängigsten Ziele sind:

- *Auslastung der CPU*
Ist die CPU das Betriebsmittel, das am wenigsten vorhanden ist, so wollen wir den Gebrauch möglichst effizient gestalten. Ziel ist die 100 %ige **Auslastung** der CPU, normal sind 40–90 %.
- *Durchsatz*
Die Zahl der Prozesse pro Zeiteinheit, der **Durchsatz** (*throughput*), ist ein weiteres Maß für die Systemauslastung und sollte maximal sein.
- *Faire Behandlung*
Kein Prozess sollte dem anderen bevorzugt werden, wenn dies nicht ausdrücklich vereinbart wird (**fairness**). Dies bedeutet, dass jeder Benutzer im Mittel den gleichen CPU-Zeitanteil erhalten sollte.
- *Ausführungszeit*
Die Zeitspanne vom Jobbeginn bis zum Jobende, die **Ausführungszeit** (*turnaround time*), enthält alle Zeiten in Warteschlangen, der Ausführung (**Bedienzeit**) und der Ein- und Ausgabe. Natürlich sollte sie minimal sein.

- *Wartezeit*

Der Scheduler kann von der gesamten Ausführungszeit nur die **Wartezeit** (*waiting time*) in der *bereit*-Liste beeinflussen. Für die Schedulerstrategie kann man sich also als Ziel darauf beschränken, die Wartezeit zu minimieren.

- *Antwortzeit*

In interaktiven Systemen empfindet der Benutzer es als besonders unangenehm, wenn nach einer Eingabe die Reaktion des Rechners lange auf sich warten lässt. Unabhängig von der gesamten Ausführungszeit des Prozesses sollte besonders die Zeit zwischen einer Eingabe und der Übergabe der Antwortdaten an die Ausgabegeräte, die **Antwortzeit** (*response time*), auch minimal werden.

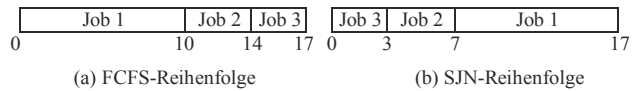
Die Liste der möglichen Zielvorgaben ist weder vollständig noch konsistent. Beispielsweise benötigt jede Prozessumschaltung einen Wechsel des Prozesskontextes (*context switch*). Werden die kurzen Prozesse bevorzugt, so verkürzt sich zwar die Antwortzeit mit der Ablaufzeit zwischen zwei Eingaben und der Durchsatz steigt an, aber der relative Zeitanteil der langen Prozesse wird geringer; sie werden benachteiligt und damit die Fairness verletzt. Wird andererseits die Auslastung erhöht, so verlängern sich die Antwortzeiten bei interaktiven Prozessen, weil durch die Vorplanung nicht jeder neuen Anforderung sogleich entsprochen werden kann.

Dies kann man analog dazu auch im täglichen Leben sehen: Werden bei einer Autovermietung bestimmte Kunden trotz großen Andrangs bevorzugt bedient, so müssen andere Kunden länger warten. Werden die Mietwagen gut ausgelastet, so muss ein neu hinzukommender Kunde meist warten, bis er einen bekommt; für eine kurze Reaktionszeit müssen ausreichend viele Wagen eines Modells zur Verfügung stehen.

Da sich für jeden Benutzerkreis die Zielvorgaben verändern können, gibt es keinen idealen Schedulingalgorithmus für jede Situation. Aus diesem Grund ist es sehr sinnvoll, die Mechanismen des Scheduling (Umhängen von Prozessen aus einer Warteschlange in die nächste etc.) von der eigentlichen Schedulingstrategie und ihren Parametern abzutrennen. Erzeugt beispielsweise ein Datenbank-Prozess einige Hilfsprozesse, deren Charakteristika er kennt, so sollte es möglich sein, die Schedulingstrategie der Kindsprozesse durch den Eltern-Prozess zu beeinflussen. Die BS-Kern-eigenen, internen Scheduling- und Dispatching-Mechanismen sollten dazu über eine genormte Schnittstelle (Systemaufrufe) benutzt werden; die Schedulingstrategie selbst aber sollte vom Benutzer programmiert werden können.

2.2.2 Non-präemptives Scheduling

Im einfachsten Fall können die Prozesse so lange laufen, bis sie von sich aus den Aktivzustand verlassen und auf ein Ereignis (I/O, Nachricht etc.) warten, die Kontrolle an andere Prozesse abgeben oder sich selbst beenden: Sie werden nicht vorzeitig unterbrochen (**non-präemptive scheduling**). Diese Art von Scheduling ist bei allen Systemen sinnvoll, bei denen man genau weiß, welche Prozesse existieren und welche Charakteristika sie haben. Ein Beispiel dafür ist das oben erwähnte Datenbankprogramm, das genau weiß, wie lange

Abb. 2.9 Job-Reihenfolgen

eine Transaktion normalerweise dauert. In diesem Fall kann man ein Leichtgewichts-Prozesssystem zur Realisierung verwenden. Für diese Art von Scheduling gibt es die folgenden, am häufigsten verwendeten Strategien:

- *First Come First Serve (FCFS)*

Eine sehr einfache Strategie besteht darin, die Prozesse in der Reihenfolge ihres Eintreffens in die Warteschlange einzusortieren. Damit kommen alle Prozesse an die Reihe, egal wie viel Zeit sie verbrauchen. Die Implementierung dieser Strategie mit einer FIFO Warteschlange ist sehr einfach.

Die Leistungsfähigkeit dieses Algorithmus ist allerdings auch sehr begrenzt. Nehmen wir an, wir haben 3 Prozesse der Längen 10, 4 und 3, die fast gleichzeitig eintreffen. Diese werden nach FCFS eingeordnet und bearbeitet. In [Abb. 2.9a](#) ist dies gezeigt. Die Ausführungszeit (*turnaround time*) von Prozess 1 ist in diesem Beispiel 10, von Prozess 2 ist sie 14 und 17 von Prozess 3, so dass die mittlere Ausführungszeit $(10 + 14 + 17):3 = 13,67$ beträgt. Ordnen wir allerdings die Prozesse so an, dass die kürzesten zuerst bearbeitet werden, so ist für unser Beispiel in [Abb. 2.9 \(b\)](#) die mittlere Ausführungszeit $(3 + 7 + 17):3 = 9$ und damit kürzer. Dieser Sachverhalt führt uns zur nachfolgenden Strategie.

- *Shortest Job First (SJF)*

Der Prozess mit der (geschätzt) kürzesten Bedienzeit wird allen anderen vorgezogen. Diese Strategie vermeidet zum einen die oben beschriebenen Nachteile von FCFS. Zum anderen bevorzugt sie stark interaktive Prozesse, die wenig CPU-Zeit brauchen und meist in Ein-/Ausgabeschlangen auf den Abschluss der parallel ablaufenden Aktionen der Ein-/Ausgabekanäle (DMA) warten, so dass die mittlere Antwortzeit gering gehalten wird.

Man kann zeigen, dass SJF die mittlere Wartezeit eines Prozesses für eine gegebene Menge von Prozessen minimiert, da beim Vorziehen des kurzen Prozesses seine Wartezeit stärker absinkt, als sich die Wartezeit des langen Prozesses erhöht. Aus diesem Grund ist es die Strategie der Wahl, wenn keine anderen Gesichtspunkte dazukommen.

Ein Problem dieser Strategie besteht darin, dass bei großem Zustrom von kurzen Prozessen ein Prozess mit großen CPU-Anforderungen nie die CPU erhält, obwohl er nicht blockiert in der „bereit“-Warteschlange ist. Dieses als **Verhungern** (*starvation*) bekannte Problem tritt übrigens auch in anderen Situationen auf.

- *Highest Response Ratio Next (HRN)*

Hier werden die Prozesse mit großem Verhältnis (Antwortzeit/Bedienzeit) bevorzugt bearbeitet, wobei für Antwortzeit und Bedienzeit die Schätzungen verwendet werden, die auf vorher gemessenen Werten basieren. Diese Strategie zieht Prozesse mit kurzer

Bedienzeit vor, begrenzt aber auch die Wartezeit von Prozessen mit langen Bedienzeiten, weil bei einer Benachteiligung auch deren Antwortzeit zunimmt.

- *Priority Scheduling (PS)*

Jedem Prozess wird initial beim Start eine **Priorität** zugeordnet. Kommt ein neuer Prozess in die Warteschlange, so wird er so einsortiert, dass die Prozesse mit höchster Priorität am Anfang der Schlange stehen; die mit geringster am Ende. Sind mehrere Prozesse gleicher Priorität da, so muss innerhalb dieser Prozesse die Reihenfolge nach einer anderen Strategie, z. B. FCFS, entschieden werden.

Man beachte, dass auch hier wieder benachteiligte Prozesse verhungern können. Bei Prioritäten lässt sich dieses Problem dadurch umgehen, dass die Prioritäten der Prozesse nicht fest sind („*statische Prioritäten*“), sondern dynamisch verändert werden („*dynamische Prioritäten*“). Erhält ein Prozess in regelmäßiger Folge zusätzliche Priorität, so wird er irgendwann die höchste Priorität haben und so ebenfalls die CPU erhalten.

Die Voraussetzung von *SJF* und *HRN* (einer überschaubaren Menge von Prozessen mit bekannten Charakteristika) wird durch die Tatsache in Frage gestellt, dass die Ausführungszeiten von Prozessen sehr uneinheitlich sind und häufig wechseln. Der Nutzen der Strategien bei sehr uneinheitlichen Systemen ist deshalb begrenzt. Anders ist dies im Fall von häufig auftretenden, gut überschaubaren und bekannten Prozessen wie sie beispielsweise in Datenbanken oder Prozesssystemen (Echtzeitsysteme) vorkommen. Hier lohnt es sich, die unbekannten Parameter ständig neu zu schätzen und so das Scheduling zu optimieren.

Die ständige Anpassung dieser Parameter (wie Ausführungszeit und Bedienzeit) an die Realität kann man mit verschiedenen Algorithmen erreichen. Einer der bekanntesten besteht darin, für einen Parameter a eines Prozesses zu jedem Zeitpunkt t den gewichteten Mittelwert aus dem aktuellen Wert b_t und dem früheren Wert $a(t-1)$ zu bilden:

$$a(t) = (1 - \alpha)a(t-1) + \alpha b_t$$

Es ergibt sich hier die Reihe

$$a(0) \equiv b_0$$

$$a(1) = (1 - \alpha) b_0 + \alpha b_1$$

$$a(2) = (1 - \alpha)^2 b_0 + (1 - \alpha) \alpha b_1 + \alpha b_2$$

$$a(3) = (1 - \alpha)^3 b_0 + (1 - \alpha)^2 \alpha b_1 + (1 - \alpha) \alpha b_2 + \alpha b_3$$

...

$$a(n) = (1 - \alpha)^n b_0 + (1 - \alpha)^{n-1} \alpha b_1 + \dots + (1 - \alpha)^{n-i} \alpha b_i + \dots + \alpha b_n$$

Wie man sieht, schwindet mit $\alpha < 1$ der Einfluss der frühen Messungen exponentiell, der Parameter *altert*. Dieses Prinzip findet man bei vielen adaptiven Verfahren. Die geringere

Gewichtung der früheren Messungen bewirkt, dass sich Verhaltensänderungen des Prozesses auch in den aktuellen Parameterschätzungen wiederfinden. Betrachtet man die Folge der Parameterwerte als Ereignisse einer stochastischen Variablen, so heißt dies, dass die Verteilung der Variablen nicht konstant ist, sondern sich mit der Zeit verändert. Ein solcher adaptiver Algorithmus ermittelt also nicht einfach den Mittelwert der Messungen (erwarteter Parameterwert, siehe Aufgabe 2.2-1), sondern schätzt den aktuellen Stand einer zeitabhängigen Verteilungsfunktion.

Für den Fall $\alpha = 1/2$ lässt sich der Algorithmus besonders schnell implementieren: Die Division durch zwei entspricht gerade einer Shift-Operation der Zahl nach rechts um eine Stelle.

Die adaptive Schätzung der Parameter eines Prozesses für einen Schedulingalgorithmus (*adaptive Prozessorzuteilung*) muss für jeden Prozess extra durchgeführt werden; in unserem Beispiel müsste a also einen doppelten Index bekommen: einen für die Parameternummer pro Prozess und einen für die Prozessnummer. Die Schätzmethode für die Parameter ist unabhängig vom Algorithmus, der für das Scheduling verwendet wird; aus diesem Grund gilt das obige nicht nur für die Algorithmen des non-präemptiven Scheduling, sondern auch für die Verfahren des präemptiven Scheduling des nächsten Abschnitts.

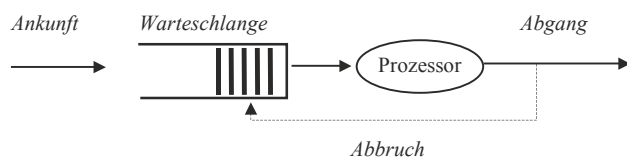
2.2.3 Präemptives Scheduling

In einem normalen Mehrbenutzersystem, bei dem die verschiedenen Prozesse von verschiedenen Benutzern gestartet werden, gibt es zwangsläufig Ärger, wenn ein Prozess alle anderen blockiert. Hier ist eine andere Art von Schedulingalgorithmen erforderlich, bei dem jeder Prozess vorzeitig unterbrochen werden kann: das **präemptive Scheduling**.

Eine der wichtigsten Maßnahmen dieses Verfahrens besteht darin, die verfügbare Zeitspanne für das Betriebsmittel, meist die CPU, in einzelne, gleich große Zeitabschnitte (**Zeitscheiben**) aufzuteilen. Wird ein Prozess bereit, so wird er in die Warteschlange an einer Stelle einsortiert. Zu Beginn einer neuen Zeitscheibe wird der Dispatcher per Zeitinterrupt aufgerufen, der bisher laufende Prozess wird abgebrochen und wie ein neuer *bereit*-Prozess in die Warteschlange eingereiht. Dann wird der Prozess am Anfang der Schlange in den Aktivzustand versetzt. Dies ist in Abb. 2.10 dargestellt. Die senkrechten Striche symbolisieren die Prozesse, die von links in das „Gefäß“, die Warteschlange, hineingeschoben werden. Die Bearbeitungseinheit, hier der Prozessor, ist als Ellipse visualisiert. Nach einem Abbruch wird der Prozess an einen Platz zwischen die anderen Prozesse in der Warteschlange eingereiht.

Für die Wahl dieses Platzes gibt es verschiedene Schedulingstrategien:

Abb. 2.10 Präemptives Scheduling



- *Round Robin (RR)*

Die einfachste Strategie beim Zeitscheibenverfahren ist die FCFS-Strategie mit der FIFO-Schlange. Die Kombination von FCFS und Zeitscheibenverfahren ist auch als der **Round-Robin**-Algorithmus bekannt. Die analytische Untersuchung zeigt, dass hier die Antwortzeiten proportional zu den Bedienzeiten sind und unabhängig von der Verteilung der Bedienzeiten nur von der mittleren Bedienzeit abhängen.

Es ist klar, dass die Leistungsfähigkeit des RR stark von der Größe der Zeitscheibe abhängt. Wählt man die Zeitscheibe unendlich groß, so wird nur noch der einfache FCFS Algorithmus ausgeführt. Wählt man die Zeitscheibe dagegen sehr klein (z. B. gerade eine Instruktion), so erhalten alle n Prozesse jeweils $1/n$ Prozessorleistung; der Prozessor teilt sich in n virtuelle Prozessoren auf. Dies funktioniert allerdings nur dann so, wenn der Prozessor sehr schnell gegenüber der Peripherie (Speicher) ist und die Prozessumschaltung durch Hardwaremechanismen sehr schnell bewirkt wird (z. B. bei der CDC6600) und praktisch keine Zeit beansprucht. Auf Standardsystemen ist dies aber meist nicht der Fall. Hier geschieht die Umschaltung des Prozesskontextes durch Softwaremechanismen und benötigt extra Zeit, in der Größenordnung von 10–100 Mikrosekunden. Wählen wir die Zeitscheibe zu klein, so wird das Verhältnis Arbeitszeit/Umschaltzeit zu klein, der Durchsatz verschlechtert sich, und die Wartezeiten steigen an. Im Extremfall schaltet der Prozessor nur noch um und bearbeitet den Prozess nie.

Für ein vernünftiges Verhalten zwischen FCFS und Dauerumschalten ist die Kenntnis verschiedener Parameter nötig. Eine bewährte Daumenregel besteht darin, die Zeitscheibe größer zu machen als der mittlere CPU-Zeitbedarf zwischen zwei I/O-Aktivitäten (*cpu-burst*) von 80 % der Prozesse. Dies entspricht meist einem Wert von ca. 100 ms.

- *Dynamic Priority Round Robin (DPRR)*

Das RR-Scheduling lässt sich noch durch eine Vorstufe, einer prioritätsgeführten Warteschlange für die Prozesse, ergänzen. Die Priorität der Prozesse in der Vorstufe wächst nach jeder Zeitscheibe so lange, bis sie die Schwellenpriorität des eigentlichen RR-Verfahrens erreicht haben und in die Hauptschlange einsortiert werden. Damit wird eine unterschiedliche Bearbeitung der Prozesse nach Systemprioritäten erreicht, ohne das RR-Verfahren direkt zu verändern.

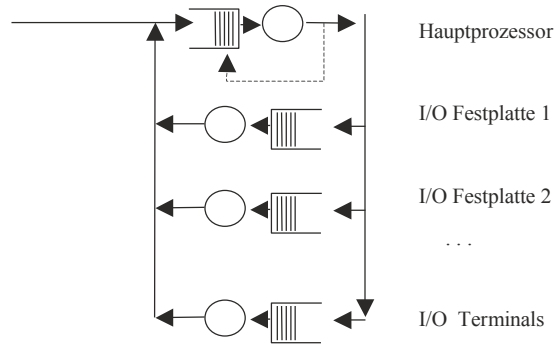
- *Shortest Remaining Time First*

Unsere SJF-Strategie zum Einsortieren in die Schlange bedeutet hier, dass derjenige Prozess bevorteilt wird, der die kleinste noch verbleibende Bedienzeit vorweist.

Eine weitere Gruppe von Schedulingalgorithmen erhalten wir, wenn wir zum Abbrechen anstelle der Zeitscheibe die Prioritäten verwenden. Das **Priority Scheduling** bedeutet im präemptiven Fall, dass der laufende Prozess durch einen neu hinzukommenden Prozess (z. B. aus einer I/O-Warteschlange) höherer Priorität verdrängt werden kann und wieder in die *bereit*-Liste einsortiert wird.

Auch eine Kombination beider Verfahren ist üblich. Dabei wird die FCFS-Strategie der Round-Robin-Warteschlange auf Prioritäten umgestellt.

Abb. 2.11 Scheduling mit mehreren Warteschlangen



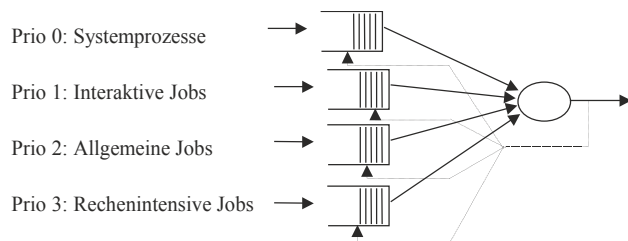
2.2.4 Mehrere Warteschlangen und Scheduler

In einem modernen Einprozessorsystem gibt es zwar nur einen Hauptprozessor, aber fast alle schnelleren Ein- und Ausgabegeräte verfügen über einen Controller, der unabhängig vom Hauptprozessor Daten aus dem Hauptspeicher auf den Massenspeicher und umgekehrt schaffen kann (*Direct Memory Access DMA*). Diese DMA-Controller wirken damit wie eigene, spezialisierte Prozessoren und können als eigene, unabhängige Betriebsmittel betrachtet werden. Es ist deshalb sinnvoll, für jeden Ein-/Ausgabekanal eine eigene Warteschlange einzurichten, die von dem DMA-Controller bedient wird. Das gesamte Dispatching besteht also aus einem Umhängen von Prozessen von einer Warteschlange in die nächste, wobei kurze CPU-Aktivitäten (*CPU bursts*) dazwischenliegen. In [Abb. 2.11](#) ist ein solches System von Warteschlangen gezeigt. Auch für solche speziellen Warteschlangen lassen sich Schedulingstrategien entwickeln, etwa für Festplatten (Teorey und Pinkerton 1972).

Eine weitere Variante besteht darin, dass wir nicht nur eine Art von Prozessen haben, sondern mehrere unterschiedlicher Priorität und deshalb für jede Prozessart eine eigene Warteschlange eröffnen (**Multi-level-Scheduling**).

Durch die unterschiedlichen Prioritäten sind die Warteschlangen in einer bestimmten Reihenfolge angeordnet, die die Abarbeitungsreihenfolge bestimmt: Ist die erste Schlange höchster Priorität abgearbeitet, so folgt die zweite usw. Da ständig neue Prozesse hinzukommen, wechselt die Abarbeitungsreihenfolge auch ständig. In [Abb. 2.12](#) ist dies an

Abb. 2.12 Multi-level Scheduling



einem 4-Ebenen-System visualisiert. Können die Prozesse bei längerer Wartezeit in eine Warteschlange höherer Ordnung überwechseln, etwa durch dynamische, mit der Zeit wachsende Prioritäten, so spricht man von *multi-level feedback Scheduling*.

Bei unseren Schedulingalgorithmen haben wir bisher die meistvorhandene Situation vernachlässigt, dass nicht alle Prozesse im Hauptspeicher gleichzeitig verfügbar sein können. Um trotzdem ein Scheduling der Prozesse durchführen zu können, werden nur die wichtigsten Daten eines Prozesses, zusammengefasst im **Prozesskontrollblock** PCB, im Hauptspeicher gehalten; alle anderen Daten werden auf den Massenspeicher verlagert (**geswappt**). Wird ein Prozess aktiviert, so muss er erst durch Kopieren (*swapping*) vom Massenspeicher in den Hauptspeicher geholt und dann erst ausgeführt werden. Dies erfordert erhebliche, zusätzliche Zeit beim Wechsel des Prozesskontextes und erhöht die Bearbeitungsdauer. Am besten ist es dagegen, jeweils den „richtigen“ Prozess bereits im Hauptspeicher zu haben. Wie soll dazu der Übergang eines Prozesses vom Massenspeicher zum Hauptspeicher geregelt werden?

Eine Lösung für dieses Problem ist die Einführung eines zweiten Schedulers, der nur für das Ein- und Auslagern der Prozesse verantwortlich ist. Der erste Scheduler managt die Zuordnung der Prozesse zum Betriebsmittel „Prozessor“ und ist kurzzeitig tätig; der zweite Scheduler ist ein Mittel- bzw. Langzeitscheduler (ähnlich wie in [Abb. 2.8](#)) und regelt die Zuordnung der Prozesse zum Betriebsmittel „Hauptspeicher“, indem er die Größe des vom Prozess belegten Hauptspeicherbereichs reguliert. Er wird in größeren Zeitabständen aufgerufen als der Kurzzeitscheduler. Beide verfügen über eigene Warteschlangen und regeln Zugang und Ordnung dafür. Strategien für einen solchen Scheduler werden wir in [Abschn. 3.3](#) kennenlernen.

Angenommen, wir haben **keine Prioritäten**, oder alle Jobs haben gleiche Priorität, ähnlich wie bei Datenpaketen in Netzwerkpuffern, und sind aber in mehreren Warteschlangen organisiert. Welche Warteschlange (Puffer) sollten wir mit einem zentralen Prozessor als erstes bedienen? Einen guten Erfolg verspricht die *Greedy-Strategie*, bei der immer diejenige Warteschlange genommen wird, die am vollsten ist. Zwar wissen wir nicht, wann zukünftig welcher Job in welcher Warteschlange landen wird, aber verglichen mit einer optimalen, allwissenden Strategie benötigen wir dann im *worst case* nur maximal die doppelte Zeit zum Abarbeiten aller Pakete; man sagt, der Greedy-Algorithmus ist *2-kompetitiv* (Albers [2010](#)).

2.2.5 Scheduling in Echtzeitbetriebssystemen

Es gibt eine Reihe von Computersystemen, die als **Echtzeitsysteme (real time systems)** bezeichnet werden. Was fällt unter diese Bezeichnung? Eine intuitive Auffassung davon betrachtet alle Systeme, die schnell reagieren müssen, als Echtzeitsysteme. Was heisst aber „schnell“? Dies können wir genauer fassen: ein System, das Prozesse ausführt, die expliziten Zeitbedingungen gehorchen müssen. Aber auch dies ist noch nicht präzise genug. Fast jedes System muss Zeitbedingungen gehorchen: Ein Editor sollte nicht länger als

zwei Sekunden benötigen, um ein Zeichen auf dem Bildschirm sichtbar einzufügen; eine Bank sollte eine Überweisung innerhalb einer Woche durchführen, um Ärger zu vermeiden. Im Unterschied zu diesen auch als „*Soft*-Echtzeitsysteme“ bezeichneten Systemen, bei denen die Zeitschranken nur „weich“ und unspezifiziert sind und eine Nichterfüllung keine schweren Konsequenzen nach sich zieht, sind es die „*Hart*-Echtzeitsysteme“ oder auch kurz nur „Echtzeitsysteme“ genannten Rechner, die in Kernkraftwerken, Flugzeugen und Fahrzeugsteuerungen eingesetzt werden. Ein solches Echtzeitsystem ist ein System, das explizite, genau festgelegte, **endliche Zeitschranken** für die Prozesse erfüllen muss, um ernsthafte Konsequenzen, z. B. einen Misserfolg oder Ausfall des Systems zu verhindern. Dabei bezeichnen wir mit „Misserfolg“ jedes Systemverhalten, das die formale Systemspezifikation nicht mehr erfüllt.

Aus der Forderung nach festen Zeitschranken für das Gesamtsystem können wir drei Forderungen folgern (Coolings 2014):

- *Rechtzeitigkeit*
Jeder Prozess (Task) muss seine Zeitschranken erfüllen.
- *Gleichzeitigkeit*
Auch wenn alle Prozesse gleichzeitig (parallel) laufen, müssen die Zeitschranken für alle Prozesse eingehalten werden.
- *Verfügbarkeit*
Die Einhaltung der Zeitschranken dürfen nicht durch Hardwareausfälle, Wartungsarbeiten oder Rekonfigurationsarbeiten (z. B. *garbage collection*!) beeinträchtigt werden.

Die Schedulingalgorithmen dafür orientieren sich an der Art der Prozesse. Typisch für Echtzeitsysteme ist die Situation, dass die zeitkritischen *Prozesse* zu genau festgesetzten Zeiten immer wiederkehren und damit sowohl in der Häufigkeit ihres Auftretens als auch in der Dauer, Betriebsmittelbelegung etc. vorhersehbar sind und es sich deshalb lohnt, einen festen Ablaufplan dafür zu konstruieren.

Beispiel *Periodische Prozesse*

Ein Flugzeug (z. B. Airbus A-340) wird von Rechnern gesteuert (*flight-by-wire*). Zur Steuerung benötigen die Rechner verschiedene Flugdaten, die in unterschiedlichen Intervallen ermittelt und verarbeitet werden müssen: die Beschleunigungswerte in x-, y- und z-Richtung alle 5 ms, die drei Werte der Drehungen alle 40 ms, die Temperatur alle Sekunde und die absolute GPS-Position zur Kontrolle alle 10 Sekunden. Der Display auf den Monitorschirmen wird jede Sekunde aktualisiert.

Die Auslastung H des Prozessors bei periodischen Prozessen ist einfach zu bestimmen: Tritt ein Prozess alle T Sekunden auf und dauert dann t Sekunden, so trägt er mit t/T Prozent zur Prozessorauslastung bei.

Beispiel Prozessorauslastung

Im obigen Beispiel benötigen die Beschleunigungswerte 1 ms alle 5 ms, die Drehwerte 5 ms alle 40 ms und die absolute GPS-Position 20 ms alle 10.000 ms. Damit bedeuten die Beschleunigungswerte im Mittel eine Auslastung von $H = 1/5 = 20\%$, die Drehwerte $H = 5/40 = 12,5\%$ und die GPS-Position $H = 0,2\%$. Zusammen belasten sie den Prozessor also mit $H = 20\% + 12,5\% + 0,2\% = 32,7\%$.

Die wichtigsten Schedulingstrategien für Echtzeitsysteme teilen sich auf in statische Verfahren, die für kleine, komplett vorbestimmte Systeme feste Ablauf Tabellen erstellen, und dynamische Verfahren, die auch nicht-periodische Ereignisse verarbeiten können (s. Laplante 1993; Stankovic et al. 1998). Wir betrachten hier vorwiegend die wichtige Gruppe der dynamischen Verfahren, bei denen in regelmäßigen Abschnitten der Ablauf-schedul neu berechnet wird. Zwar erfordert dies einen höheren Laufzeit-Overhead, gestattet aber, flexibel auf die sich ändernden Anforderungen einzugehen.

Die wichtigsten Verfahren sind:

- *Polled Loop*

Der Prozessor führt eine Schleife aus, bei der er immer wiederkehrend die Geräte abfragt, ob neue Daten vorhanden sind. Wenn ja, so verarbeitet er sie sofort. Diese Strategie eignet sich für Einzelgeräte, versagt aber, wenn andere Ereignisse während der Bearbeitung auftreten und die Daten nicht gepuffert werden.

- *Interruptgesteuerte Systeme*

Der Prozessor führt als Prozess eine Warteschleife (*idle loop*) aus. Treffen neue Daten ein, so wird jeweils ein Interrupt von dem Gerät ausgelöst (*Interrupt Request IRQ*) und die *Interrupt Service Routine* ISR bearbeitet die neuen Daten. Werden den ISR Prioritäten zugeordnet, so findet damit automatisch durch die Unterbrechungslogik der Interruptbehandlung ein Prioritätsscheduling statt. Problematisch ist diese Strategie, wenn sich die Ereignisse häufen: Interrupts geringerer Priorität unterbrechen nicht und können so überschrieben werden, bevor sie ausgewertet werden können. Werden so sehr viele asynchrone Interrupts von sehr vielen kleinen Datenpaketen erzeugt, etwa beim Empfang von UDP-Datagrammen, so können Datenverluste eintreten. Gegen diese IRQ-Last hilft eine konstante maximale Interruptrate, die von Treiber beim Gerät, etwa der Netzwerkkarte, fest eingestellt wird.

- *FIFO- Systeme*

Wie beim allgemeinen Scheduling in Abschn. 2.2 besteht die einfachste Art der systematischen Warteschlangenbildung darin, alle Prozesse gemäß ihren Auftrittsszeiten in eine First-Come-First-Serve-Reihenfolge einzugliedern. Dies ist zwar einfach, aber absolut nicht optimal und garantiert keine Einhaltung der Zeitschranken. Sie sollte deshalb möglichst nicht verwendet werden.

Stattdessen ist es sinnvoll, den Prozessen unterschiedliche Wichtigkeiten (Prioritäten) zuzuweisen. Betrachten wir zuerst den Fall **fester**, sich nicht ändernder **Prioritäten**. Dies

ist ein sehr einfacher Ansatz, der erlaubt, bei einmal zugeordneten Prioritäten schnell einen Schedul zu bilden, garantiert aber keine 100 %-ge Prozessorauslastung.

- *Rate Monotonic-Scheduling (RMS)*

Haben wir ein festes Prioritätensystem, unterbrechbare Prozesse (*Präemption*) sowie feste Ausführungsraten der beteiligten Prozesse (s. oben das Beispiel des Flugzeugs), ist es optimal, wenn wir die hohen Prioritäten den hohen Ausführungsraten zuordnen und niedrige Prioritäten den niedrigen Raten (*rate monotonic-scheduling*).

$$\text{Priorität} \sim 1/\text{Periodendauer}$$

Falls kein *rate monotonic*-Ablaufplan für eine Menge von Prozessen gefunden werden kann, so ist bewiesen (Liu und Layland 1973), dass dann auch kein anderer Ablaufplan mit festen Prioritäten existiert, der erfolgreich ist. Hat die CPU eine Auslastung kleiner als 70 %, so werden mit dem RMS alle Zeitschranken garantiert eingehalten. Allerdings ist damit kein optimaler Schedul für volle Prozessorauslastung garantiert: nach (Liu und Layland 1973) ist bei n Tasks die maximale Prozessorauslastung durch $n(2^{1/n} - 1)$ gegeben.

Auch kann nötig sein, dass ein wichtiger Prozess mit geringer Ausführungsfrequenz, und damit eigentlich geringer Priorität, trotzdem ausgeführt und dazu die Priorität zur Laufzeit extra angehoben werden muss (*Prioritätsinversion*).

Betrachten wir nun Systeme mit **dynamischen** sowie **keinen Prioritäten**, die eine optimale, 100 %-ge Auslastung ermöglichen.

- *Minimal Deadline First MDF, Earliest Deadline First EDF*

Derjenige Prozess wird zuerst abgearbeitet, der die kleinste, also nächste Zeitschranke (*deadline*) T_D besitzt. Diese Strategie wird zwar oft bei Anwendungen benutzt (z. B. bei der Abwicklung von Softwareprojekten), hat aber auch Nachteile. So ist sie beispielsweise nutzlos, wenn alle Prozesse die gleiche Zeitschranke besitzen (Beispiel: Prozessmenge = alle Motorsteuerungen mit dem Ziel „Haltet den Roboter an, bevor er gegen die Wand rast“). Trotzdem ist das präemptive EDF beliebt, da es eine bessere Prozessorauslastung anbietet. Wie Liu und Layland (1973) gezeigt haben, ist dies für Einprozessorsysteme eine optimale Strategie mit maximal 100 %-ger Prozessorauslastung.

- *Least Laxity First LLF*

Derjenige Prozess wird ausgewählt, der gerade die minimale restliche Zeit (*laxity*) zwischen Ende des Prozesses und der Zeitschranke besitzt. Auch diese Strategie ist optimal auf Einprozessorsystemen. Allerdings ist der Rechenaufwand bei diesem Verfahren höher, da in jedem Zeittakt alle Restzeiten neu bestimmt werden müssen.

- *Minimal Processing Time First*

Derjenige Prozess wird ausgewählt, der die minimale restliche Bedienzeit T_C besitzt. Dies entspricht der SJF-Strategie und hat den Nachteil, dass der kurze, unwichtige Prozess dem langen, aber wichtigen Prozess vorgezogen wird.

- *Time Slice Scheduling TSS*

Eine präemptive Prioritätssteuerung kann man auch dadurch verwirklichen, indem man den einzelnen Prozessen verschieden lange Zeitscheiben entsprechend ihrer Prozessorauslastung zuordnet. Benötigt beispielsweise ein Prozess bei einer Periode von 10 ms eine Zeit von 2 ms, so lastet er in dieser Zeit den Prozessor $H = 2/10 = 20\%$ aus. Ist die maximale Zeitscheibe der Dauer 5 ms in 1 %-Schritten auslegbar, so sollte er also eine Zeitscheibe von 20% von 5 ms $= 20 \cdot 0,05 = 1$ ms bekommen.

Vorteile des Verfahrens sind einfache Realisierung und Vermeiden von Verhungern von Prozessen. Nachteilig ist dagegen die Tatsache, dass bei wachsender Anzahl von Prozessen die tatsächliche Ausführungshäufigkeit trotz gleichbleibender Zeitscheibenlänge sinkt. Auch die Abhängigkeit der Antwortzeit von der Zeitscheibe kann Probleme bereiten: Ist sie gerade abgelaufen, wenn ein Ereignis eintrifft, so dauert es lange, bevor mit der nächsten Zeitscheibe das Ereignis bearbeitet werden kann. Hier ist also ein Hardware-Datenpuffer wichtig (Wörn und Brinkschulte 2005). Auch sollte die Zeitscheibe fein genug unterteilt werden können, um Verschnitt zu vermeiden.

- *Guaranteed Percentage Scheduling GPS*

Als Alternative zum TSS können wir auch den einzelnen Prozessen die Prozessorauslastung direkt fest zuordnen, ohne dies nur indirekt über die Zeitscheibenlänge zu versuchen. Es wird eine einheitliche, kleine Zeitscheibenlänge verwendet, öfters umgeschaltet und damit der Prozessor in quasi mehrere Prozessoren aufgeteilt. Der resultierende Schedul muss nur die Anzahl der Zeitscheiben eines Prozesses entsprechend der festen, vorher bestimmten Prozessorauslastung in der Warteschlange vorsehen. Damit haben die Prozesse nicht eine feste oder dynamische Priorität, sondern werden je nach Kontext unterschiedlich zum Ablauf vorgesehen.

Beispiel

Haben wir Prozesse, die 20 %, 10 % und 30 % Auslastung bedeuten, so müssen 20 %, 10 % und 30 % der Anzahl der Zeitscheiben für den jeweiligen Prozess in der Warteschlange enthalten sein.

Dieses Verfahren ist unabhängig von der Anzahl der Prozesse und ermöglicht ebenfalls eine Auslastung von 100 % des Prozessors. Auch die Behandlung der Datenraten wird vereinfacht, da durch die schnelle Umschaltung kleinere Puffer nötig sind. Damit wird bei dynamischem Scheduling GPS das Verfahren der Wahl.

Ein Echtzeitbetriebssystem enthält nicht nur kritische Prozesse, deren Zeitschranken unbedingt eingehalten werden müssen, sondern auch nicht-kritische, aber notwendige Prozesse. Diese können im Hintergrund abgewickelt werden, sobald der Prozessor frei ist und nicht anderweitig benötigt wird. Jeder notwendige Prozess kann sie unterbrechen. Beispiele für dieses *Foreground-Background Scheduling* sind:

- *Selbsttests*, um defekte Teile zu entdecken.
- *RAM scrubbing*: Auslesen und Zurückschreiben vom RAM-Inhalt. Dies ist in Systemen nützlich, die einen fehlerkorrigierenden Datenbus haben und so die durch Höhenstrahlung (space shuttle!) entstandenen Bitfehler im RAM korrigieren.
- *Auslastungsmonitore*, um Fehler zu frühzeitig zu entdecken, z. B. durch Zeitüberwachung (*watch dog timer*) mittels „Totmannschalter“, also Bits (Flags), die periodisch zurückgesetzt werden müssen, um einen Alarm zu verhindern.

Neben den zeitkritischen sowie den notwendigen, unkritischen Prozessen gibt es nun noch einen Rest, der ebenfalls abgearbeitet werden sollte, falls noch Zeit dafür ist. Nehmen wir an, dass die zeitkritischen, notwendigen Prozesse nach einem festen RMS oder GPS abgewickelt werden und die Prozesse der dritten Kategorie im Hintergrund sind, so bleibt noch die große Menge der Prozesse der zweiten Kategorie, die zeitunkritisch, aber notwendig sind. Für diese Art von Systemen entwickelten die Designer des „spring kernel“ zusätzliche Strategien (Stankovic und Ramamritham 1991). Sie kombinierten dazu die Variablen *Wichtigkeit* w , *Zeitschranke* T_D und *benötigte Betriebsmittel* (z. B. restliche Bedienzeit T_C oder den Zeitpunkt T_S , zu der für einen Prozess alle notwendigen Betriebsmittel frei werden) zu neuen Strategien

- *Minimum Earliest Start*
Derjenige Prozess wird gewählt, der die kleinste Zeit T_S besitzt.
- *Minimum Laxity First*
Der Prozess mit der kleinsten freien Zeit $T_D - (T_S + T_C)$ wird gewählt.
- *Kombiniertes Kriterium 1*
Der Prozess mit der kleinsten freien Zeit $T_D + wT_C$ wird gewählt.
- *Kombiniertes Kriterium 2*
Der Prozess mit der kleinsten freien Zeit $T_D + wT_S$ wird gewählt.

Simulationen ergaben, dass alle Ablaufpläne, die nur T_D allein verwenden, schlechte Resultate für Single- und Multiprozessorsysteme erbrachten. Gut dagegen schnitten die kombinierten Kriterien ab, wobei Kriterium 2 den besten Ablaufplan lieferte, wohl weil es auch die Betriebsmittel mitberücksichtigte.

Real-time software

Angenommen, wir können zeigen, dass die Gesamtlänge der kritischen (mit ihrer Wiederholfrequenz multiplizierten) Prozesse die Zeitschranken verletzt – was können wir dann tun? Eine wichtige Arbeit besteht darin, alle verwendeten Programme, und damit auch das Betriebssystem, echtzeitfähig zu machen. Was bedeutet das?

Wir müssen dafür sorgen, dass alle Algorithmen, die auf nicht-beschränktem Zeitverbrauch beruhen, ausgetauscht werden. Dies sind im Falle des Betriebssystems zum einen alle Schedulingalgorithmen, die auf *swapping* beruhen – eine Unterbrechung der Programmausführung durch langdauernde, nicht zeitdeterminierte Plattenaktivität

ist nicht hinnehmbar. Ein anderes Beispiel ist die automatische Speicherbereinigung (*garbage collection*), etwa bei Java: Auch hier sollte man im Echtzeitprogramm die Speicheranforderung und Speicherfreigabe der Objekte explizit selbst verwalten und die automatische *garbage collection* der *Java Virtual Machine* abschalten, um ohne automatische Inkarnation nur mit der schnellen Speichervergabe des Heap die Zeit kontrolliert einzuteilen. Dies ermöglicht für Java eine Geschwindigkeitssteigerung um etwa den Faktor drei.

Prominente Beispiele für Echtzeit-Betriebssysteme sind die Echtzeit-Erweiterungen von POSIX. Beispielsweise sind in POSIX 4a neben Threads auch Echtzeiterweiterungen vorgesehen; in POSIX 4b das Echtzeitscheduling mittels FPN, FPP, *time slices* und allgemein ein benutzerdefinierbares Scheduling, Zeitgeber und asynchrones I/O.

2.2.6 Scheduling in Multiprozessorsystemen

Im allgemeinen Fall existiert für jedes Betriebsmittel, und damit für jeden Prozessor, eine eigene Warteschlange und ein eigener Scheduler. Allerdings sind die Übergänge zwischen den Warteschlangen nicht beliebig, sondern es ist bei mehreren, zusammenarbeitenden und parallel ablaufenden Prozessen eine Koordination notwendig. Diese muss berücksichtigen, dass zwischen den einzelnen Prozessen Abhängigkeiten bestehen in der Reihenfolge der Abarbeitung.

Bezeichnen wir die Betriebsmittel mit A, B, C und die Anforderungen an sie mit A_i , B_j , C_k , so lässt sich dies durch eine *Präzedenzrelation* „ $\gg$ “ ausdrücken. Dabei soll $A_i \gg B_j$ bedeuten, dass zuerst A_i und dann (irgendwann) B_j ausgeführt werden muss. Ist A_i die direkte, letzte Aktion vor B_j , so sei die *direkte* Präzedenzrelation mit „ $\gg$ “ notiert.

Beispiel

$$\begin{aligned} A_1 &\gg B_1 \gg C_1 \gg A_5 \gg B_3 \gg A_6 \\ B_1 &\gg B_4 \gg C_3 \gg B_3 \\ A_2 &\gg A_3 \gg B_4 \\ A_3 &\gg C_2 \gg B_2 \gg B_3 \\ A_4 &\gg C_2 \end{aligned}$$

Eine solche Menge von Relationen lässt sich durch einen gerichteten, gewichteten Graphen visualisieren, bei dem ein Knoten die Anforderung und die Kante „ $\rightarrow$ “ die Relation „ $\gg$ “ darstellt. Die Dauer t_i der Betriebsmittelanforderung ist jeweils mit einer Zahl (Gewichtung) neben der Anforderung (Knoten) notiert.

Für unser Beispiel sei dies durch [Abb. 2.13](#) gegeben. Eine Reihenfolge ist genau dann nicht notwendig, wenn es sich um **unabhängige**, disjunkte Prozesse handelt, die keine gemeinsamen Daten haben, die verändert werden. Benötigen sie dagegen gemeinsame Betriebsmittel, so handelt es sich um **konkurrente** Prozesse.

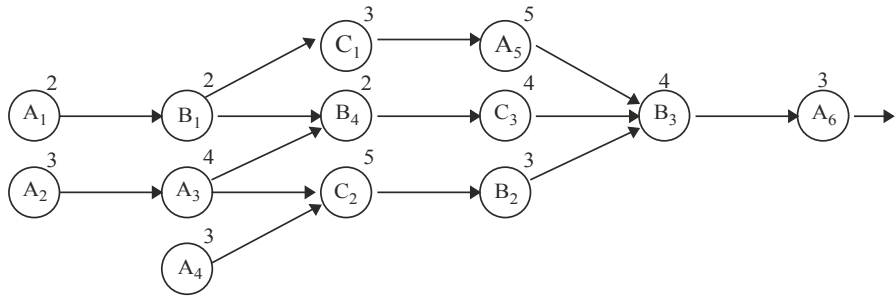


Abb. 2.13 Präzedenzgraph

Ein möglicher Ablaufplan lässt sich durch ein Balkendiagramm (**Gantt-Diagramm**) visualisieren, in dem jedem Betriebsmittel ein Balken zugeordnet ist. Für unser Beispiel ist dies in Abb. 2.14 zu sehen.

Was können wir bei einer guten Schedulingstrategie an Gesamtlaufzeit T erwarten? Bei abhängigen Prozessen gibt uns unser zyklensfreier Präzedenzgraph die Antwort. Bilden wir alle möglichen Pfade, die vom einem Anfangsknoten (Knoten ohne Pfeilspitze) zu einem Endknoten (Knoten ohne Pfeilanzfang) führen und addieren dabei die Ausführungszeiten entlang des Pfades, so ist der Pfad mit der größten Summe, der *kritische Pfad*, entscheidend: Kürzer kann die Gesamtausführungszeit nicht sein, auch wenn wir beliebig viele Parallelprozessoren verwenden.

Bei n unabhängigen Prozessen mit den Gewichten $t_1, \dots, t_n$ und m Prozessoren ist klar, dass die optimale Gesamtausführungszeit T_{opt} für ein präemptives Scheduling entweder von der Ausführungszeit aller Prozesse, verteilt auf die Prozessoren, oder aber von der Ausführungszeit des längsten Prozesses bestimmt wird:

$$T_{\text{opt}} = \max \left\{ \max_{1 \leq i \leq n} t_i, \frac{1}{m} \sum_{i=1}^n t_i \right\}$$

Dies ist der günstigste Fall, den wir erwarten können. Für unser obiges Beispiel in Abb. 2.13 mit $n = 13$ Prozessen und $m = 3$ Prozessoren ist günstigstenfalls $T = 43/3 \leq 15$

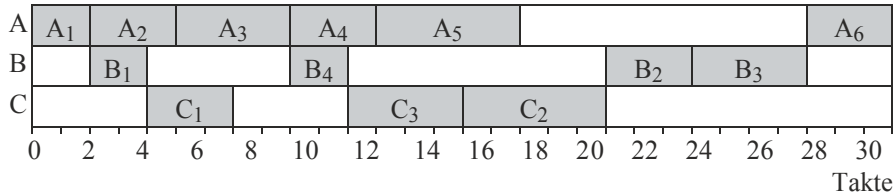


Abb. 2.14 Gantt-Diagramm der Betriebsmittel

im Unterschied zu $T = 30$ für den Ablaufplan in [Abb. 2.14](#). Führen wir nämlich zusätzliche Nebenbedingungen ein, wie beispielsweise die Bedingung, dass ein Prozess vom Typ A_i nur auf Prozessor A, ein Prozess B_j nur auf Prozessor B und ein Prozess C_k nur auf Prozessor C ablaufen darf, so verlängert sich die Ausführungszeit T .

Mit den oben eingeführten Hilfsmitteln wollen wir nun einige Schedulingstrategien für die parallele Ausführung von Prozessen auf mehreren Prozessoren näher betrachten.

Deterministisches Parallelscheduling

Angenommen, die Betriebsmittelanforderung (das Knotengewicht) ist jeweils das Mehrfache einer festen Zeiteinheit Δt : Die Zeitabschnitte sind „*mutually commensurable*“. Die Menge aller Knoten des Graphen lässt sich nun derart in Untermengen zerteilen, dass alle Knoten einer Untermenge unabhängig voneinander sind, also keine Präzedenzrelation zwischen ihnen besteht.

Beispiel

Die Knotenmenge des Graphen aus [Abb. 2.13](#) lässt sich beispielsweise in die unabhängigen Knotenmengen $\{A_1, A_2\}$, $\{B_1, A_3, A_4\}$, $\{C_1, B_4, C_2\}$, $\{A_5, C_3, B_2\}$, $\{B_3\}$, $\{A_6\}$ unterteilen. Dabei ist zwar B_4 von A_3 und B_1 und damit auch $\{C_1, B_4, C_2\}$ von $\{B_1, A_3, A_4\}$ abhängig, aber nicht die Knoten innerhalb einer Untermenge.

Eine solche Einteilung kann auf verschiedene Weise vorgenommen werden; die Unterteilung ist nicht eindeutig festgelegt; in unserem Beispiel ist auch $\{A_1, A_2, A_4\}$ eine Lösung für die erste Untermenge. Die N Untermengen bezeichnen wir als N Stufen (*levels*), wobei dem Eingangsknoten die Stufe 1, der davon abhängigen Knotenmenge die Stufe 2 usw. und dem letzten Knoten (*terminal node*) die N -te Stufe zugewiesen wird (**Präzedenzpartition**).

Beispiel

Die obige Zerteilung hat $N = 6$ Stufen, wobei die Menge $\{A_1, A_2\}$ der Stufe 1 und dem Terminalknoten A_6 die Stufe 6 zugewiesen wird.

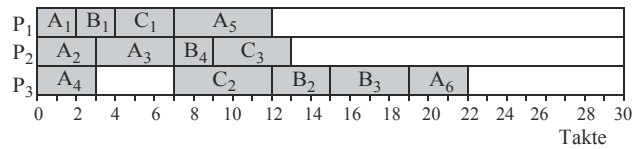
Man beachte, dass zwar das allgemeine Problem, einen optimalen Ablaufplan zu finden, NP-vollständig ist, aber für „normale“ Prozesse unter „normalen“ Bedingungen gibt es durchaus Algorithmen, die einen brauchbaren Ablaufplan entwerfen.

Es existieren nun drei klassische Strategien (Muntz und Coffman [1969](#)), um Prozesse präemptiv zu planen:

- *Earliest Scheduling*

Ein Prozess wird bearbeitet, sobald ein Prozessor frei wird. Dazu beginnen wir mit der ersten Stufe und besetzen die Prozessoren mit den Prozessen der Knotenmenge.

Abb. 2.15 Gantt-Diagramm bei der *earliest scheduling*-Strategie



Alle noch freien Prozessoren werden dann mit den Prozessen der zweiten Stufe (sofern möglich) belegt usw. Frei werdende Prozessoren erhalten darauf in der gleichen Weise die Prozesse der gleichen bzw. nächsten Stufe, bis alle Stufen abgearbeitet sind.

Beispiel

Das Gantt-Diagramm der Präzedenzpartition des obigen Beispiels aus [Abb. 2.13](#), interpretiert als Ablaufplan für die Prozessoren P_1 , P_2 und P_3 , nimmt mit der *earliest scheduling*-Strategie die Gestalt in [Abb. 2.15](#) an.

Mit diesem Ablaufplan erreichen wir für das Beispiel eine Länge $T = 22$.

- *Latest Scheduling*

Ein Prozess wird zum spätesten Zeitpunkt ausgeführt, an dem dies noch möglich ist. Dazu ändern wir die Reihenfolge der Stufenbezeichnungen in der Präzedenzpartition um; die N -te Stufe wird nun zur ersten und die erste Stufe wird zur N -ten. Nun weisen wir wieder den Prozessoren zuerst die Prozesse der Stufe 1 zu, dann diejenigen der Stufe 2 usw.

Beispiel

Mit der *latest-scheduling*-Strategie ergibt sich für obiges Beispiel aus [Abb. 2.13](#) das Gantt-Diagramm in [Abb. 2.16](#).

Mit diesem deterministischen Ablaufplan erreichen wir für das gleiche Beispiel nur eine Länge $T = 22$.

- *List Scheduling*

Alle Prozesse werden mit Prioritäten versehen in eine zentrale Liste aufgenommen. Wird ein Prozessor frei, so nimmt er aus der Liste von allen Prozessen, deren Vorgänger ausgeführt wurden und die deshalb lauffähig sind, denjenigen mit der höchsten Priorität und führt ihn aus. Wird die Priorität bereits beim Einsortieren in die *bereit*-Liste beachtet und werden Prozesse, die auf Vorgänger warten, in die *blockiert*-Warteschlange einsortiert, so erhalten wir eine zentrale Multiprozessor-Prozesswarteschlange ähnlich der

Abb. 2.16 Gantt-Diagramm bei *latest scheduling*-Strategie

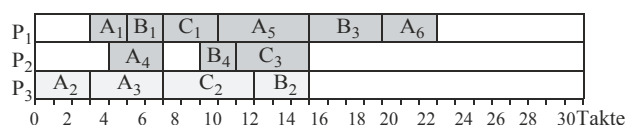
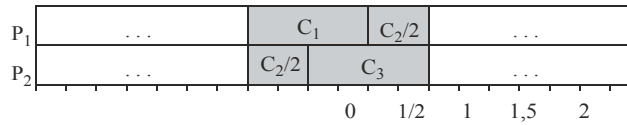


Abb. 2.17 Optimaler Ablaufplan von drei Prozessen auf zwei Prozessoren



Prozesswarteschlange in unserem Einprozessorsystem. Im Unterschied dazu wird sie aber von allen Prozessoren parallel abgearbeitet. In [Abschn. 2.3.5](#) ist als Beispiel das Warteschlangenmanagement beim Ultracomputer der NYU beschrieben.

Man sieht, dass man mit den Algorithmen zwar keinen optimalen, aber durchaus einen funktionsfähigen Ablaufplan finden kann. Für $m = 2$ Prozessoren und Prozesse einheitlicher Länge ($t_i = t_k$) können wir sogar einen optimalen, präemptiven Ablaufplan der Prozess-Gesamtmenge erreichen, indem wir für jede Untermenge (Stufe) einen optimalen präemptiven Ablaufplan finden (Muntz und Coffman 1969). Diesen Gedanken können wir auf den Fall $t_i \neq t_k$ erweitern, indem wir jeden Prozess der Länge $s \cdot \Delta t$ virtuell aus einer Folge von s Einheitsprozessen bestehen lassen. Wir erhalten so aus unserem Präzedenzgraphen einen neuen Präzedenzgraphen, der nur Prozesse gleicher Länge enthält und damit für $m = 2$ Prozessoren ein optimales präemptives Scheduling erlaubt. Allerdings ist es bei einer ungeraden Anzahl von Prozessen nötig, bei drei Prozessen C_1, C_2, C_3 aus einer der Stufen einen der Prozesse (z. B. C_2) zu teilen und beide Hälften auf die Prozessoren zu verteilen, s. [Abb. 2.17](#). Dieser ist wieder optimal, so dass der Gesamtschedul auch optimal wird.

Die Betrachtungen können auch ausgedehnt werden auf Präzedenzgraphen mit vielen parallelen, einfachen Prozessketten, also Graphen mit einem Startknoten, einem Endknoten und (bis auf den Startknoten) nur jeweils maximal einem Nachfolgeknoten pro Knoten.

Die minimale Ausführungszeit bei Multiprozessorscheduling

Angenommen, wir benutzen die prioritäts- und präzedenzorientierte, zentrale Warteschlange, was ist dann die kleinste Ausführungszeit T_{prio} , die wir für eine Liste von Prozessen, versehen mit Prioritäten und Präzedenzen, erwarten können?

Angenommen, wir notieren mit T_{prio} die Ausführungszeit für einen Ablaufplan, der einem freiwerdenden Prozessor den ausführbaren Prozess mit der höchsten Priorität aus der Liste zuweist, und der Gesamtausführungszeit T_o den optimalen Ablaufplan. Für gleichartige Prozessortypen und m Prozessoren gilt mit (Liu und Liu 1978)

$$\frac{T_{\text{prio}}}{T_o} \leq 2 - \frac{1}{m}$$

Wie man aus obiger Formel sieht, ist bei gleichartigen Prozessoren ein prioritätsgeführter Ablaufplan im schlechtesten Fall doppelt so lang wie ein optimaler Ablaufplan; im besten Fall gleich lang. Bei nur einem Prozessor sind sie natürlich identisch, da der eine Prozessor immer die gesamte Arbeit machen muss, egal, in welcher Reihenfolge die Prozesse abgearbeitet werden.

Würde man allgemein mehr Prozessoren (Betriebsmittel) einführen, so würde zwar durch die zusätzlichen, parallel wirkenden Bearbeitungsmöglichkeiten meistens die Gesamtausführungszeit verkleinert werden. Da mit diesen zusätzlichen Betriebsmitteln aber die Auslastung sinkt, wäre dann natürlich auch die mittlere Leerlaufzeit der Betriebsmittel höher.

Interessanterweise führt das Lockern von Restriktionen, wie z. B.

- das Aufheben einiger Präzedenzbedingungen
- das Verkleinern einiger Ausführungszeiten
- die Erhöhung der Prozessoranzahl ($m' \geq m$)

nicht automatisch zu einer kürzeren Gesamtausführungsdauer T , sondern kann auch zu ungünstigeren Schedules führen (**Multiprozessoranomalien**). In diesem Zusammenhang sind die Studien von Graham (1972) interessant. Er fand heraus, dass in einem System mit m identischen Prozessoren, in dem die Prozesse völlig willkürlich den Prozessoren zugeordnet werden, die Gesamtausführungsdauer T der Prozessmenge nicht mehr als das Doppelte derjenigen eines optimalen Schedules werden kann: Für die Gesamtdauer T' eines veränderten Schedules mit gelockerten Restriktionen gilt die Relation

$$\frac{T'}{T} \leq 1 + \frac{m-1}{m'}$$

so dass für $m = m'$ die obere Schranke durch $2 - \frac{1}{m}$ gegeben ist.

Eine gute Methode, um erfolgreiche Ablaufpläne zu konstruieren, besteht in der Zusammenfassung mehrerer Prozesse (meist von einem gemeinsamen Elternprozess erzeugt) zu einer **Prozessgruppe**, die gemeinsam auf einem Multiprozessorsystem ausgeführt wird (**Gruppenscheduling**). Da es meist Abhängigkeiten in Form von Kommunikationsmechanismen unter den Prozessen einer Gruppe gibt, die über gemeinsamen Speicher (*shared memory*, s. Abschn. 3.3) laufen können, verhindert man so ein Ein- und Ausladen der Speicherseiten und des Prozesskontextes der beteiligten Prozesse.

Eine wichtige Voraussetzung für die präemptiven Ablaufpläne ist ein geringer Kommunikationsaufwand, um einen Prozess mit seinem Kontext auf einem Prozessor starten bzw. fortsetzen zu können. Dies ist in verteilten Systemen nicht immer gegeben, sondern meist nur in eng gekoppelten Multiprozessorsystemen oder bei Nutzung von Prozessgruppen. Benutzt man non-präemptive Ablaufpläne, so wird dieser Nachteil vermieden; allerdings ist das Scheduling auch schwieriger. Zwar gelten die obigen präemptiven Schedulingstrategien auch für non-präemptives Scheduling von Prozessen der Einheitslänge, aber allgemeines non-präemptives Scheduling muss extra berücksichtigt werden.

Weitere deterministische Schedulingalgorithmen sind in Gonzalez (1977) zu finden.

Multiprozessorlastverteilung

Die maximal erreichbare Beschleunigung (**speedup** = das Verhältnis AlteZeit zu NeueZeit) der Programmausführung durch zusätzliche parallele Prozessoren, I/O-Kanäle usw.

ist sehr begrenzt. Maximal können wir die Last auf m Betriebsmittel (z. B. Prozessoren) verteilen, also $\text{NeueZeit} = \text{AlteZeit}/m$, was einen linearen *speedup* von m bewirkt. In der Praxis ist aber eine andere Angabe noch viel entscheidender: der Anteil von sequentiell, nicht-parallelisierbarem Code. Seien die sequentiell ausgeführten Zeiten von sequentiell Code mit T_{seq} und von parallelisierbarem Code mit T_{par} notiert, so ist mit $m \rightarrow \infty$ Prozessoren mit $T_{\text{par}} \rightarrow 0$ der *speedup* bei sehr starker paralleler Ausführung

$$\text{speedup} = \frac{\text{AlteZeit}}{\text{NeueZeit}} \rightarrow \frac{T_{\text{seq}} + T_{\text{par}}}{T_{\text{seq}} + 0} = 1 + \frac{T_{\text{par}}}{T_{\text{seq}}}$$

fast ausschließlich vom Verhältnis des parallelisierbaren zum sequentiellen Code bestimmt.

Beispiel

Angenommen, wir haben ein Programm, das zu 90 % der Laufzeit parallelisierbar ist und nur für 10 % Laufzeit sequentiellen Code enthält. Auch mit dem besten Schedulingalgorithmus und der schnellsten Parallelhardware können wir nur den Code für 90 % der Laufzeit beschleunigen; die restlichen 10 % müssen hintereinander ausgeführt werden und erlauben damit nur einen maximalen *speedup* von $(90 \% + 10 \%) / 10 \% = 10$.

Da die meisten Programme weniger rechenintensiv sind und einen hohen I/O-Anteil (20–80 %) besitzen, bedeutet dies, dass auch die Software des Betriebssystems in das parallele Scheduling einbezogen werden sollte, um die tatsächlichen Ausführungszeiten deutlich zu verringern. In Abb. 2.18 sind die zwei Möglichkeiten dafür gezeigt.

Links ist die Situation aus Abb. 1.12 vereinfacht dargestellt: Jeder Prozessor bearbeitet streng getrennt einen der parallel existierenden Prozesse, wobei das Betriebssystem als sequentieller Code nur einem Prozessor zugeordnet wird. Diese Konfiguration wird als **asymmetrisches Multiprocessing** bezeichnet.

Im Gegensatz dazu kann man das Betriebssystem – wie auch die Anwenderprogramme – in parallel ausführbare Codestücke aufteilen, die von jedem Prozessor ausgeführt werden können. So können auch die Betriebssystemteile parallel ausgeführt werden,

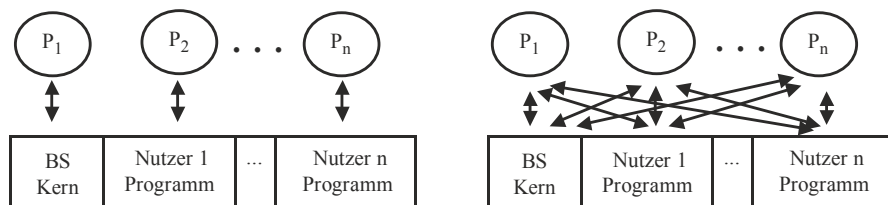


Abb. 2.18 Asymmetrisches und symmetrisches Multiprocessing

was den Durchsatz deutlich erhöht. Diese Konfiguration bezeichnet man als **symmetrisches Multiprocessing SMP**.

Für das Scheduling bei Multiprozessorsystemen wird üblicherweise eine *globale Warteschlange* benutzt, in der alle bereiten Prozesse eingehängt werden. Jeder Prozessor, der seinen Prozess beendet hat, entnimmt den nächsten Prozess der einen Liste. Dies hat nicht nur den Vorteil, dass alle Prozessoren gleichmäßig ausgelastet werden, sondern führt bei den Prozessorfehlern, die sich nicht in einzelnen Fehlfunktionen, sondern in einem völligen Ausfall des Prozessors äußern (*fail-safe* Verhalten), dazu, dass bis auf den betroffenen Prozesse alle anderen unbehelligt weiter ausgeführt werden und der Rechenbetrieb so weiterläuft (*Ausfalltoleranz*).

Das allgemeine Problem, n Prozesse auf m Prozessoren zu verteilen, ist bisher ungelöst; es ist ein typisches Rucksackproblem und damit NP-vollständig. Die verschiedenen Strategien dafür sind stark durch die Annahmen über die Prozesseigenschaften geprägt. Da die Abhängigkeiten der Prozesse untereinander sowie der benötigten Betriebsmittel im Ablauf bei unregelmäßigen Prozessen nur Schätzungen sein können, sind die Schedulingalgorithmen meist nur mehr oder weniger stark heuristisch geprägte Approximationen. Bei stark variierenden Anforderungen ist ihr Nutzen sehr begrenzt.

2.2.7 Stochastische Schedulingmodelle

Aus diesem Grund gibt es zwei prinzipielle Ansätze, gute Schedulingalgorithmen auf der Basis von (statistischen) Beobachtungen der Prozesseigenschaften zu konstruieren, die sich nicht ausschließen, sondern ergänzen. Zum einen kann man mit plausiblen Annahmen über die Prozesse und ihre Bearbeitung ein mathematisches Modell erstellen. Dies ist die Aufgabe des *analytischen* Ansatzes, der meist die **Warteschlangentheorie** (s. z. B. Brinch Hansen 1973) benutzt.

Beispiel

Die Wahrscheinlichkeit P_j , mit der ein neuer Prozess bei einer Warteschlange in der Zeitspanne Δt eintrifft, sei proportional zur (kleinen) Zeitspanne und unabhängig von der Vorgeschichte (Annahme einer Poisson-Verteilung)

$$P_j \sim \Delta t \text{ oder } P_j = \lambda \Delta t$$

Die Wahrscheinlichkeit P_N , dass kein Prozess in Δt ankommt, ist also

$$P_N(\Delta t) = 1 - P_j = 1 - \lambda \Delta t$$

Die Wahrscheinlichkeit, dass kein Prozess im Intervall $t + \Delta t$ ankommt, ist also

$$P_N(t + \Delta t) = P(\text{Kein Prozess in } t \cap \text{kein Prozess in } \Delta t) = P_N(t) P_N(\Delta t) = P_N(t) (1 - \lambda \Delta t)$$

und somit
$$\frac{P_N(t + \Delta t) - P_N(t)}{\Delta t} = -\lambda P_N(t)$$

Im Grenzübergang für $\Delta t \rightarrow 0$ bedeutet dies

$$\frac{dP_N(t)}{dt} = -\lambda P_N(t)$$

so dass mit $P_N(0) = 1$ die Integration $P_N(t) = e^{-\lambda t}$ ergibt und somit

$$P_j(t) = 1 - P_N(t) = 1 - e^{-\lambda t} \quad (\text{Exponentialverteilung})$$

Damit haben wir eine wichtige statistische Annahme erhalten, die so nicht nur für die Ankunftszeit, sondern auch für die Bearbeitungszeit von Prozessen gemacht wird. Die Wahrscheinlichkeitsdichte $p(t)$ für die Ankunft bzw. Bearbeitung der Prozesse ist mit

$$P_j(t) = 1 - e^{-\lambda t} = \int_0^t \lambda e^{-\lambda s} ds = \int_0^t p_j(s) ds \Leftrightarrow p(t) = \lambda e^{-\lambda t}$$

Die mittlere oder erwartete Ankunftszeit T_j ist somit

$$T_j = \int_0^\infty p(t) t dt = \int_0^\infty t \lambda e^{-\lambda t} dt = \underbrace{-te^{-\lambda t}}_0 \Big|_0^\infty - \int_0^\infty -e^{-\lambda t} dt = -\frac{1}{\lambda} e^{-\lambda t} \Big|_0^\infty = \frac{1}{\lambda}$$

Die Umkehrung $1/T = \lambda$ ist eine Frequenz oder Rate, die **Ankunftsrate** λ . Analog dazu erhalten wir die **Bedienrate** μ .

Ist $\lambda < \mu$, so ist die Warteschlange im Gleichgewichtszustand. Für diesen Fall gilt die Littlesche Formel über die mittlere Länge L der Warteschlange und die mittlere Wartezeit T_w eines Prozesses:

$$L = \lambda T_w \quad (\text{Littlesche Formel})$$

was auch intuitiv einleuchtet.

Ein Ansatz für stochastisches Multiprozessorscheduling ist beispielsweise in Robinson (1979) zu finden. Leider treffen die Annahmen, die solche mathematischen Behandlungen ermöglichen, in dieser Form meist nicht zu. Stattdessen müssen komplexere Annahmen gemacht werden, die aber durch ihre Vielfalt die mathematische Behandlung erschweren und teilweise unmöglich machen. Deshalb sind die Ergebnisse einer mathematischen Modellierung meist nur eingeschränkt anwendbar; sie sind nur im Idealfall gültig und stellen eine Extremsituation dar.

Aus diesem Grund ist es sehr sinnvoll, als zweiten Ansatz auch die *Simulation* des untersuchten Systems durchzuführen. Es gibt dafür bereits verschiedene Softwarepakete, die dies erleichtern. Bei der Simulation werden alle parallelen Betriebsmittel wie Prozessoren, I/O-Kanäle usw. mit jeweils einer Warteschlange, Auftrittsrate und Bedienrate (*Bedienstation*) modelliert. Das reale Computersystem wird auf ein Netz aus Bedienstationen abgebildet, so dass man die einzelnen Parameter gut an die Realität anpassen kann. Verklemmungen und Leistungsengpässe („Flaschenhälse“) lassen sich so simulativ erfassen, geeignet interpretieren und beheben.

2.2.8 Beispiel UNIX: Scheduling

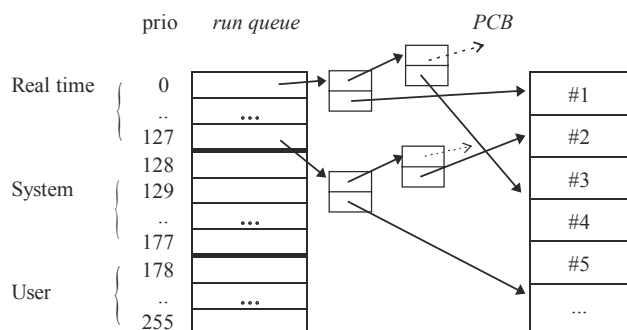
In UNIX kommt meist das Round-Robin-Verfahren zur Anwendung, das durch Prioritäten geeignet modifiziert wird. Jeder Prozess bekommt eine initiale Priorität zugeteilt, die sich beim Warten noch mit der Zeit erhöht. Dadurch sind einerseits die Systemprozesse bevorzugt, zum anderen wird aber sichergestellt, dass auch langwartende Prozesse einmal an die Reihe kommen.

In UNIX entspricht jeder Priorität eine ganze Zahl. Frühere Versionen hatten Prioritäten von -127 bis $+127$, wobei kleine Zahlen hohe Prioritäten bedeuten. Dabei bedeuten negative Zahlen Systemprozesse, die normalerweise nicht unterbrochen werden können. Ein Kommando „*nice*“ erlaubt es Benutzern, die eigene Standardpriorität herabzusetzen, um bei unkritischen Prozessen nett zu den anderen Benutzern zu sein. Typischerweise wird dies meist nicht benutzt.

Da bei ganzen Zahlen mehrere Prozesse mit gleicher Priorität existieren, gibt es für alle Prozesse gleicher Priorität jeweils eine eigene FCFS-Warteschlange. Sind alle Prozesse einer Warteschlange abgearbeitet, so wird die nächste Warteschlange mit geringerer Priorität bearbeitet. Zusätzlich werden Prozesse, deren Priorität sich durch Warten erhöht, in andere Schlangen umgehängt (*multi-level feedback scheduling*).

In neuen UNIX-Varianten erstreckt sich die Priorität von 0 bis 255 und ist zusätzlich unterteilt. In Abb. 2.19 sind die Prioritätswarteschlangen dieser *multi-level queue* für

Abb. 2.19 Multi-level-Warteschlangen in UNIX



HP-UX (Hewlett-Packard 1991) gezeigt. Die Warteschlangen bilden eine verkettete Liste, deren inhaltliche Zeiger auf die Prozesskontrollblöcke PCB der Prozesstafel zeigen. Alle Prozesse der Systemprioritäten (128–177) und der *User* (178–255) werden nach dem Zeitscheibenverfahren zugeteilt; die Prozesse, die mit einem speziellen Systemaufruf `rtprio()` gestartet werden, werden extra behandelt. Ihre Priorität (0–127) ist initial höher und wird nicht mehr verändert.

Diese und andere zusätzliche, früher in UNIX nicht existierende Eigenschaften wie das präemptive Scheduling von Prozessen, die sich gerade im *kernel mode* befinden und deshalb normalerweise ununterbrechbar sind (es existiert nur ein einziger *kernel stack*, der überlaufen könnte), fest reservierte Hauptspeicherbelegung (*memory lock*) zur Vermeidung des *swapping* sowie Dateien mit konsekutiven Blöcken ermöglichen auch ein Echtzeitverhalten in Real-Time-UNIX, siehe Furht et al. (1991).

In **Linux** ab Version 2.6 gibt es im Unterschied dazu 140 Prioritäten, wobei die hohen Prioritäten 0,...,100 den Realzeitprozessen vorbehalten sind. Die Prozesse in jeder der 140 möglichen Warteschlangen sind auch in FCFS-Reihenfolge geordnet und werden so zuerst nach Priorität und dann nach Reihenfolge abgearbeitet. Eine 140-Bit-Folge verzeichnet eine „1“ genau dann, wenn die Warteschlange nicht leer ist; sonst ist das Bit null. Damit wird das Durchsuchen der Warteschlangen aller lauffähigen Prozesse nach dem nächsten auszuführenden Prozess sehr schnell gemacht.

Um die Interaktivität (sichtbare Antwortverhalten bei einem PC) zu fördern, wird die Priorität eines Prozesses dynamisch nach dem Grad der Interaktivität verändert. Eine Variable `sleep_avg` wird für jeden Prozess mitgeführt. Sie kann Werte zwischen –5 und +5 annehmen und entspricht der Anzahl der Prioritätsstufen, die ein Prozess gegenüber der initialen Zuordnung zu- oder abnehmen kann. Beim Aufwachen nach einem I/O wird `sleep_avg` um die Anzahl der Zeitticks (auf max 10) erhöht, die der Prozess geschlafen hat. Umgekehrt wird für jeden Zeittick, den der Prozess läuft, ein Punkt von `sleep_avg` abgezogen.

Diese Priorität wirkt sich auch auf die Länge der Zeitscheibe des Prozesses aus: Sie variiert zwischen 10 ms für unwichtige und 200 ms für wichtige Prozesse.

Realzeitprozessen werden allerdings stark bevorzugt: Wurden sie mit der Strategie `SCHED_RR` gestartet, so wird nur ein einfaches Round-Robin ohne Prioritätsveränderung gestartet. Bei dem Start mit `SCHED_FIFO` dagegen werden für den Prozess sogar die Zeitscheiben abgeschaltet und ein roter Teppich ausgerollt: Er kann laufen, bis er freiwillig die Kontrolle abgibt.

Beim Einsatz in Symmetrischen Multiprozessoranlagen verwaltet jeder Prozessor sein eigenes Warteschlangensystem. Einen global agierende Funktion zur Lastverteilung `load_balance()` sorgt dafür, dass bei starken Ungleichheiten der Lastverteilung (>25 %) eine leere Warteschlange eines Prozessors wieder aufgefüllt wird. Diese Funktion wird mindestens alle 200 ms von einem Prozessor aufgerufen, ansonsten immer dann, wenn einer untätig ist. Näheres zum Linux-Kern findet man in Love (2003) und Maurer (2004).

2.2.10 Aufgaben

Aufgabe 2.2-1 Scheduling

- a) Was ist der Unterschied zwischen der Ausführungszeit und der Bedienzeit eines Prozesses?
- b) Mac OS X ist Apples erstes Betriebssystem, das präemptives Scheduling unterstützt: bis einschließlich Mac OS 9 wurde non-präemptives Scheduling verwendet. Erläutern Sie, worin sich diese beiden Schedulingvarianten unterscheiden.
- c) Geben Sie den Unterschied an zwischen einem Algorithmus für ein *Foreground/background*-Scheduling, das RR für den Vordergrund und einen präemptiven Prioritätsschedul für den Hintergrund benutzt, und einem Algorithmus für *Multi-level-feedback*-Scheduling.

Aufgabe 2.2-2 Scheduling

Fünf Stapelaufträge treffen in einem Computer fast zur gleichen Zeit ein. Sie besitzen in ihrer Reihenfolge geschätzte Ausführungszeiten von 10, 6, 4, 2 und 8 Minuten und die Prioritäten 3, 5, 2, 1 und 4, wobei 5 die höchste Priorität ist. Geben sie für jeden der folgenden Schedulingalgorithmen die durchschnittliche Verweilzeit an. Vernachlässigen sie dabei die Kosten für einen Prozesswechsel.

- a) Round Robin
- b) Priority Scheduling
- c) First Come First Serve
- d) Shortest Job First

Nehmen Sie für a) an, dass das System Mehrprogrammbetrieb verwendet und jeder Auftrag einen fairen Anteil an Prozessorzeit (Zeitscheibe in FIFO-Reihenfolge!) erhält. Die Länge der Zeitscheiben sei dabei unendlich kurz in Relation zu der Joblänge. Nehmen sie weiterhin für b)–d) an, dass die Aufträge nacheinander ausgeführt werden. Alle Aufträge sind reine Rechenaufträge.

Aufgabe 2.2-3 Adaptive Parameterschätzung

Beweisen Sie: Wird eine Größe a mit der Gleichung

$$a(t) = a(t-1) - 1/t (a(t-1) - b(t))$$

aktualisiert, so stellt $a(t)$ in jedem Schritt t den arithmetischen Mittelwert der Größe $b(t)$ über alle t dar.

Aufgabe 2.2-4 Echtzeitscheduling

Um die Ruine eines Atomkraftwerks gefahrlos untersuchen zu können, möchte ein Forscherteam eine autonome Drohne einsetzen. Diese besteht aus verschiedenen Komponenten, die von einem Prozessor gesteuert werden müssen. Dazu muss dieser die folgenden Aufgaben periodisch bearbeiten:

- Alle 50 ms müssen die Beschleunigungssensoren abgefragt und daraus die Ansteuerungsparameter der Rotoren berechnet werden, was 10 ms dauert. Geschieht dies nicht rechtzeitig, so stürzt die Drohne ab und ist verloren.
- Alle 100 ms müssen zur Wegfindung die Abstandssensoren ausgelesen werden, was 10 ms dauert. Geschieht dies nicht rechtzeitig, so kann die Drohne gegen eine Wand fliegen und abstürzen.
- Alle 60 s muss der aktuelle Akkustand überprüft werden, um gegebenenfalls die Rückkehr zum Ausgangspunkt einzuleiten. Diese Überprüfung dauert insgesamt 3 s, da hier zur verlässlichen Vorhersage mehrere Messungen nötig sind. Geschieht dies nicht rechtzeitig, so muss die Drohne im schlimmsten Fall in einem Gebiet notlanden, das stark verstrahlt ist und somit nicht von Menschen betreten werden kann, weshalb die Drohne somit auch verloren wäre.
- Alle 100 ms muss der Temperatursensor ausgelesen werden, um zu verhindern, dass die Drohne zu weit in Richtung des heißen Reaktorkerns fliegt. Dies dauert 5 ms. Geschieht dies nicht rechtzeitig, so fallen wichtige Komponenten möglicherweise aus, was den Absturz der Drohne zur Folge hat.
- Ebenfalls alle 100 ms muss der Geigerzähler abgefragt werden, um die aktuellen Strahlungswerte zu erhalten. Dies dauert 15 ms. Geschieht dies nicht, so kann die Drohne in sehr stark kontaminierte Gebiete fliegen, in denen die Strahlung Fehlfunktionen im Prozessor auslösen kann, die zum Absturz führen können.
- Alle 10 Sekunden soll die integrierte Kamera ein Foto der Umgebung aufnehmen und speichern, was 750 ms dauert. Obwohl dies für die Flugsicherheit der Drohne nicht entscheidend ist, wird gefordert, dass diese Zeitschranke unbedingt eingehalten werden muss, da geschätzt wurde, dass die Entwicklungskosten der Drohne sehr hoch sein werden und ein Drohnenflug mit zu wenigen Bildern nicht genügend Ergebnisse liefert.
- Alle 100 ms muss aus den vorliegenden Daten die Flugroute neu berechnet werden, was 25 ms dauert. Geschieht dies nicht rechtzeitig, so geht die Drohne höchstwahrscheinlich verloren.
- Alle 50 ms müssen die Ansteuerungsparameter der Rotoren an eine separate Flugkontrollereinheit übertragen werden, was 5 ms dauert. Geschieht dies nicht, so stürzt die Drohne ab.
- Aufgrund der zu erwartenden Strahlung, die Inhalte des Arbeitsspeichers verändern könnte, soll der gesamte Arbeitsspeicher immer dann, wenn Zeit zur Verfügung steht, einer Fehlerkorrektur unterzogen werden. Ein Durchlauf durch den gesamten

Arbeitsspeicher dauert dabei 20 s. Wegen der guten Abschirmung der Elektronik wird in der durch die Akkukapazität begrenzten maximalen Drohnenflugzeit jedoch statistisch weniger als ein Fehler pro Flugmission erwartet, weshalb diese Anforderung als nicht drohnenmissionskritisch eingestuft wird.

- Nach Möglichkeit soll in regelmäßigen Abständen eines der gespeicherten Bilder per Funk an die Basisstation übertragen werden. Die Forscher hoffen, somit bereits während des Fluges mit einer ersten Auswertung der Drohnenmission beginnen zu können, sehen diese Anforderung jedoch nicht als kritisch an.

Überprüfen Sie, ob sich diese Anforderungen mit den geplanten Komponenten realisieren lassen oder ein neuer Entwurf mit einem schnelleren Prozessor nötig ist. Falls sich die Anforderungen realisieren lassen, so geben Sie ein Schedulingkonzept dafür an und begründen Sie es.

Aufgabe 2.2.-5 Multiprozessor-Scheduling

Gegeben seien die Präzedenzen $P1 \gg P3$, $P2 \gg P3$, $P3 \gg P5$, $P4 \gg P5$, $P5 \gg P8$, $P6 \gg P7$, $P6 \gg P9$, $P7 \gg P8$, $P8 \gg P9$ mit den Prozessdauern $t(P1) = 1$, $t(P2) = 2$, $t(P3) = 2$, $t(P4) = 2$, $t(P5) = 4$, $t(P6) = 3$, $t(P7) = 2$, $t(P8) = 1$, $t(P9) = 1$.

- Zeichnen Sie den dazugehörigen Präzedenzgraphen und geben Sie den kritischen Pfad samt seiner Länge an.
- Erstellen Sie für dieses Szenario sowohl einen *Earliest Schedul*
- als auch einen *Latest Schedul* und zeichnen Sie die zugehörigen Gantt-Diagramme.

Ihnen stehen unbegrenzt viele Prozessoren zur Verfügung.

Aufgabe 2.2.-6 Paralleles Scheduling

- a) Sei ein Programm gegeben mit 40 % sequentiell, nicht-parallelisierbarem Code. Wie groß ist der maximale *Speedup* ?
- b) Angenommen, ein weiteres Betriebsmittel A' ist verfügbar. Wie kann man dann das Gantt-Diagramm in [Abb. 2.14](#) so abändern, dass weniger Zeit verbraucht wird?

2.3 Prozesssynchronisation

Die Einführung von Prozessen als unabhängige, quasi-parallel ausführbare Programm-einheiten bringt viel Flexibilität und Effizienz in die Rechnerorganisation; sie beschert uns aber neben dem Prozessscheduling auch zusätzliche Probleme. Betrachten wir dazu zunächst folgende Situation.

2.3.1 Race conditions und kritische Abschnitte

Angenommen, in einer der vielen Warteschlangen des Prozesssystems sind mehrere Prozesse eingehängt und warten auf eine Bearbeitung. Diese Situation tritt in allen Warteschlangen immer wieder auf und ist in [Abb. 2.21](#) gezeigt.

Dabei können wir die Situationen unterscheiden, dass entweder ein Prozess (hier Prozess A) neu eingehängt wird, oder aber ein bestehender (hier Prozess B) ausgehängt wird.

Für das Ein- bzw. Aushängen sollen folgende Schritte gelten:

Einhängen

- (1) Lesen des Ankers: `PointToB`
- (2) Setzen des `NextZeigers` := `PointToB`
- (3) Setzen des Ankers := `PointToA`

Aushängen

- (1) Lesen des Ankers: `PointToB`
- (2) Lesen des `NextZeigers`: `PointToC`
- (3) Setzen des Ankers := `PointToC`

Nach dem Aushängen wird evtl. der Speicherplatz des Listeneintrags (Doppelkästchen in der Zeichnung) freigegeben und mit dem Zeiger zum PCB eine Prozessumschaltung vorgenommen.

Jede der Operationen geht gut, solange sie nur für sich an einem Stück durchgeführt wird. Haben wir aber ein Prozesssystem, das ein beliebiges Umschalten nach einer Instruktion innerhalb einer Operation auf einen anderen Prozess und damit das sofortige Umschalten auf eine andere Operation ermöglicht, so erhalten wir Probleme. Betrachten wir dazu die obigen Schritte bei der Operation *Aushängen*. Angenommen, Prozess B soll ausgehängt werden und Schritte (1) und (2) wurden fürs Aushängen durchgeführt. Nun trete ein *timer*-Interrupt auf; die Zeitscheibe von B ist zu Ende. Prozess A tritt auf, hängt sich in die Liste mittels der Operation *Einhängen* ein, verrichtet noch etwas Arbeit oder legt sich schlafen. Wenn nun B wieder die Kontrolle erhält, so arbeitet B fatalerweise mit den alten Daten weiter: In Schritt (3) wird der Anker auf C gesetzt, und damit ist der Prozess A nicht mehr in der Liste; ist dies die *bereit*-Liste, so erhält er nie mehr die Kontrolle und arbeitet nicht mehr.

Das Heimtückische an diesem Fehler besteht darin, dass er nicht immer auftreten muss, sondern nur sporadisch, „unerklärlich“, nicht reproduzierbar und an Nebenbedingungen gebunden (wie große Systemlast etc.) auftreten kann. Da dies geschieht, wenn ein Prozess den anderen „überholt“, wird dies auch als **race condition** bezeichnet; die Codeabschnitte, in denen der Fehler entstehen kann und die deshalb nicht unterbrochen werden dürfen, sind die **kritischen Abschnitte** (*critical sections*).

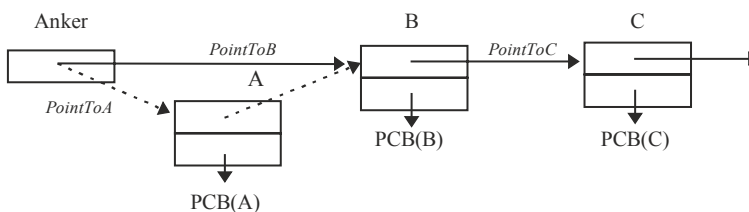


Abb. 2.21 Race conditions in einer Warteschlange

Diese Problematik ist typisch für alle Systeme, bei denen mehrere unabhängige Betriebsmittel auf ein und denselben gemeinsamen Datenbereich arbeiten. Besonders bei Multiprozessorsystemen ist dies ein wichtiges Problem, das von den Systemdesignern erkannt und berücksichtigt werden muss.

Die Hauptproblematik besteht darin, für jeden kritischen Abschnitt im Benutzerprogramm oder Betriebssystem sicherzustellen, dass immer nur ein Prozess oder Prozessor sich darin befindet. Das Betreten und Verlassen des Codes muss zwischen den Prozessen abgestimmt (**synchronisiert**) sein und einen **gegenseitigen Ausschluss** (*mutual exclusion*) in dem kritischen Abschnitt garantieren.

Dieses Problem wurde schon in den 60er Jahren erkannt und mit verschiedenen Algorithmen bearbeitet. An eine Lösung wurden folgende Anforderungen gestellt (Dijkstra 1965):

- Keine zwei Prozesse dürfen gleichzeitig in ihren kritischen Abschnitten sein (*mutual exclusion*).
- Jeder Prozess, der am Eingang eines kritischen Abschnitts wartet, muss irgendwann den Abschnitt auch betreten dürfen: Ein ewiges Warten muss ausgeschlossen sein (*fairness condition*).
- Kein Prozess darf außerhalb eines kritischen Abschnitts einen anderen Prozess blockieren.
- Es dürfen keine Annahmen über die Abarbeitungsgeschwindigkeit oder Anzahl der Prozesse bzw. Prozessoren gemacht werden.

2.3.2 Signale, Semaphore und atomare Aktionen

Die einfachste Idee zur Einrichtung eines gegenseitigen Ausschlusses besteht darin, einen Prozess beim Eintreten in einen kritischen Abschnitt so lange warten zu lassen, bis der Abschnitt wieder frei ist. Für zwei parallel ablaufende Prozesse ist der Code in [Abb. 2.22](#) gezeigt. Die grau schraffierten Bereiche in der Abbildung bedeuten jeweils, dass diese Befehlssequenz nicht unterbrochen werden kann.

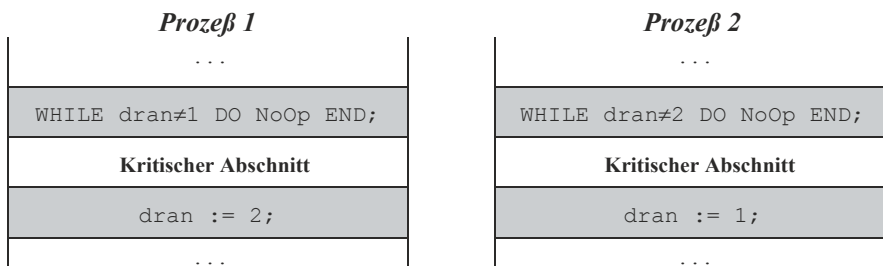


Abb. 2.22 Erster Versuch zur Prozesssynchronisation

Initialisieren wir die gemeinsame (globale) Variable `dran` beispielsweise mit 1, so erreicht der obige Code zwar den gegenseitigen Ausschluss, aber beide Prozesse können immer nur abwechselnd den kritischen Abschnitt durchlaufen. Dies widerspricht der Forderung nach Nicht-blockieren, da ein Prozess sich selbst daran hindert, ein zweites Mal hintereinander den kritischen Abschnitt zu betreten, ohne dabei im kritischen Abschnitt zu sein. Stattdessen muss er solange warten, bis der andere Prozess seinen kritischen Abschnitt durchlaufen hat und die gemeinsame Variable wieder zurückgesetzt hat. Geschieht dies nie, so wartet der eine Prozess ewig: ein weiterer Verstoß, diesmal gegen die *fairness* –Bedingung.

Versuchen wir nun, die beiden Prozesse in beliebiger Reihenfolge zu synchronisieren, so bemerken wir, dass dies nicht so einfach ist. Beispielsweise hat die Konstruktion in [Abb. 2.23](#) den Vorteil, dass ein Prozess nicht mehr auf eine besondere Zuweisung warten, sondern nur noch darauf achten muss, dass nicht ein anderer Prozess im kritischen Abschnitt ist.

Allerdings tritt hier ein anderer Fehler auf: Angenommen, die Synchronisationsvariablen seien mit `drin2:= drin1:= FALSE` initialisiert. Prozess P_1 findet `drin2=FALSE` und Prozess P_2 findet `drin1=FALSE`. Also setzen die Prozesse jeweils `drin1:=TRUE` bzw. `drin2:=TRUE`, und schon sind beide gleichzeitig im kritischen Abschnitt, im Widerspruch zu der Forderung nach gegenseitigem Ausschluss (1).

Auch ein Vertauschen der beiden Zeilen des ersten grau schraffierten Bereichs bringt keine Abhilfe. Erst der Vorschlag von T. Dekker, einem holländischen Mathematiker, zeigte einen Weg auf, das Problem für zwei Prozesse zu lösen, s. Dijkstra ([1965](#)). Eine einfachere Lösung stammt von G. Peterson ([1981](#)) und hat die in [Abb. 2.24](#) gezeigte Form:

Dabei sind `Interesse1`, `Interesse2` und `dran` globale, gemeinsame Variablen. Die beiden Variablen `Interesse1` und `Interesse2` werden mit `FALSE` initialisiert. Angenommen, ein Prozess arbeitet die grau schraffierte Befehlssequenz vor dem kritischen Abschnitt ab. Da der andere Prozess kein Interesse gezeigt hat, ist die Bedingung der `WHILE`-Schleife nicht erfüllt, und der Prozess betritt den kritischen Abschnitt. Erscheint der andere Prozess etwas später, so muss er so lange warten, bis der andere Prozess den kritischen Abschnitt verlassen hat und kein Interesse mehr bekundet.

Für die Situation, in der beide Prozesse (fast) gleichzeitig die grauen Bereiche betreten, ist die Abarbeitung der einen Instruktion entscheidend, mit der die gemeinsame Variable

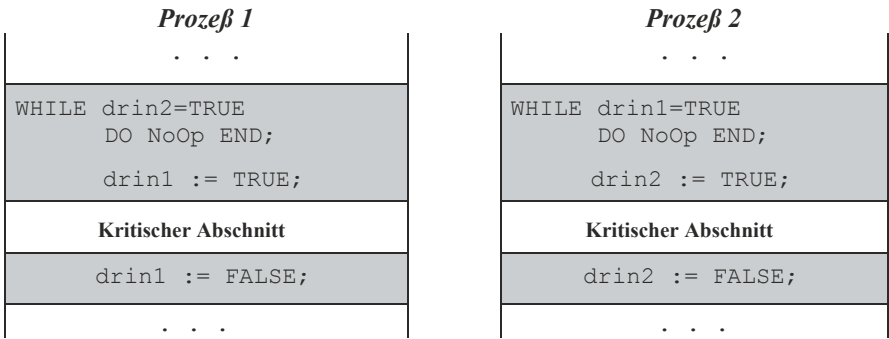
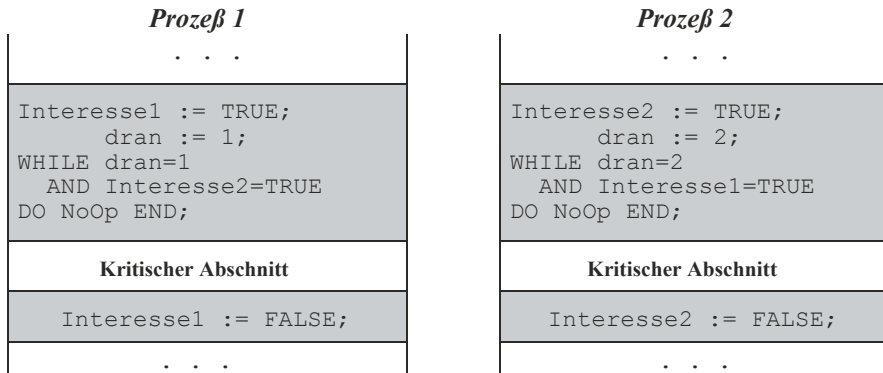


Abb. 2.23 Zweiter Versuch zur Prozesssynchronisation

**Abb. 2.24** Prozesssynchronisation nach Peterson

`dran` belegt wird. Derjenige Prozess, der sie zuletzt belegt, erlässt den anderen Prozess und muss warten; der andere kann den kritischen Abschnitt betreten.

Dieses Konzept kann man auch kompakter mit einem Sprachkonstrukt formulieren. Eine Idee dafür besteht darin, für jeden kritischen Abschnitt ein Signal zu vereinbaren. Die Prozesse synchronisieren sich dadurch, dass vor dem Betreten des kritischen Abschnitts das dafür vereinbarte Signal abgefragt wird. Es wird initial mit dem Wert „gesetzt“ belegt. Mit dem Befehl

```
waitFor(Signal)
```

blockiert sich der aufrufende Prozess, falls kein Signal gesetzt war, ansonsten setzt er das Signal zurück und betritt den Abschnitt. Ist er fertig, so aktiviert er mit

```
send(Signal)
```

einen der Prozesse, die auf das Signal warten. Welcher davon aktiviert wird, ist Sache einer zusätzlich zu vereinbarenden Strategie.

Ein Konstrukt, das einen exklusiven Ausschluss für einen kritischen Abschnitt ermöglicht, wurde von Dijkstra (1965) vorgeschlagen. Er nannte die Signale **Semaphore**. Diese Signalbaken aus der Seefahrt lassen sich hier als eine Art Verkehrsampeln oder Zeichen auffassen. Sie werden von den folgenden zwei elementaren Operationen verwaltet:

- *Passieren* $P(s)$
Beim Eintritt in den kritischen Abschnitt wird $P(s)$ aufgerufen mit dem Semaphore s . Der aufrufende Prozess wird in den Wartezustand versetzt, falls sich ein anderer Prozess in dem korrespondierenden Abschnitt des Semaphors befindet.
- *Verlassen* $V(s)$
Beim Verlassen des kritischen Abschnitts ruft der Prozess $V(s)$ auf und bewirkt damit, dass einer der (evtl.) wartenden Prozesse aktiviert wird und den kritischen Abschnitt betreten darf.

Interessanterweise ist es dabei unwesentlich, ob für ein Signal oder ein Semaphore nur ein kritischer Abschnitt existiert, der von allen Prozessoren im Hauptspeicher durchlaufen wird (Multiprozessorsysteme); ob es mehrere Kopien davon gibt in den Prozessen (z. B. Mehrrechnersysteme), oder aber ob es Abschnitte verschiedenen Codes sind, die aber zueinander funktionell korrespondieren und mit einem gemeinsamen Signal abgesichert werden müssen, wie im obigen Beispiel das Ein- und Aushängen in einer Warteschlange.

Ein ausführliche Darstellung ist in Maurer (1999) zu finden.

Beispiel Benutzung der P(.)- und V(.)-Operationen

Mehrere Prozesse sollen eine Zahl inkrementieren und ihren Wert ausgeben, so dass normal „1,2,3, ...“ hochgezählt wird. Der kritische Abschnitt lautet also hier

```
...
z := z+1;
WriteInt(z,3)
...
```

Mit einem Semaphore als globaler Variable wird dies zu

```
...
P(s);      (* Abfragen: kritischer Abschnitt besetzt? *)
z := z+1;  (* kritischer Abschnitt *)
WriteInt(z,3)
V(s);      (* andere wartende Prozesse aufwecken *)
...
```

Man beachte, dass die Codestücke sowohl für `send(Signal)` und `waitFor(Signal)` als auch für `P(s)` und `V(s)` selbst wieder ununterbrechbare, kritische Abschnitte darstellen, da sie ebenfalls einen globalen Speicherbereich, die Semaphore `Signal` bzw. `s`, bearbeiten. Allerdings sind sie nur sehr kurz und lassen sich deshalb leichter als ununterbrechbare, sog. **atomare Aktionen** implementieren. Eine atomare Aktion wird immer entweder vollständig (alle Operationen des Abschnitts) oder gar nicht (keine einzige Operation) durchgeführt. Spezielle Puffer ermöglichen es, den Anfangszustand zu speichern. Muss die atomare Aktion abgebrochen werden, so wird der Ursprungszustand wieder hergestellt.

Beispiel Atomare Aktionen

Für eine Überweisung werden von Ihrem Konto 2000 € abgebucht. Leider tritt in diesem Augenblick ein Defekt im Computersystem auf, bevor der Betrag dem Zielkonto gutgeschrieben werden kann; das Computersystem hält an. Nach dem erneuten Systemstart haben Sie nur die Auswahl, entweder erneut 2000 € abgebucht zu bekommen und sie dem Zielkonto zu überweisen, oder aber nichts zu tun. Da das Geld nicht beim Empfänger

ankam, bedeutet dies für Sie das gleiche: auf jeden Fall Geldverlust. Als Kunde werden Sie dabei sicher nicht von den Segnungen des Computerzeitalters überzeugt.

Diese Situation lässt sich entschärfen, wenn die Geldtransaktion als atomare Aktion konzipiert ist. Da die Aktion nicht abgeschlossen war, garantiert die Eigenschaft der Atomizität Ihnen, dass die erste Abbuchung nicht durchgeführt wurde und nur als temporäre Änderung bestand: Sie verlieren keine 2000 €. Die eigentliche Abbuchung wird erst nach dem erneuten Systemstart durchgeführt.

Dieses Zurücksetzen (*roll back*) ist aber nicht unproblematisch in verteilten Datenbanken. Aus diesem Grund bevorzugen Banken meist zentrale, von einem Großrechner (*Server*) verwaltete Datenbanken.

Die bisherigen Ideen zur Prozesssynchronisation leiden an einem großen Problem: Der jeweilige Prozess muss aktiv in einer Befehlsschleife warten (**busy waiting**), bis die Bedingung erfüllt ist und er in den kritischen Abschnitt eintreten kann. Diese Art von Warteoperationen heißen deshalb auch **spin locks**. Das Warten belastet aber nicht nur den Prozessor (und damit den Durchsatz) unnötig, sondern kann auch unter Umständen die *fairness*-Bedingung verletzen. Betrachten wir dazu die Situation, dass ein hochprioritärer Prozess gerade dann die Kontrolle erhalten hat, wenn ein niedrigprioritärer Prozess in seinem kritischen Abschnitt ist. In diesem Fall wird der eine Prozess so lange im *spin lock* verbleiben, bis der andere Prozess den kritischen Abschnitt verlässt. Da er aber geringere Priorität hat, wird dies bei statischen Prioritäten nie der Fall sein, und die *fairness*-Bedingung wird verletzt.

Das *busy waiting* lässt sich vermeiden, indem der wartende Prozess sich „schlafen“ legt, also die Kontrolle beim Blockieren in `waitFor(Signal)` oder `P(s)` abgibt.

Software-Implementierung

Eine der typischen Implementierungen von Semaphoren modelliert ein Semaphor als Zähler, der bei der Ankunft eines Prozesses dekrementiert wird. Im nachfolgenden ist eine *busy wait* Implementierung gezeigt, wie sie im Betriebssystemkern für sehr kurze Sperren verwendet werden kann. Dabei ist `s` mit 1 initialisiert.

Die Operationen `P(s)` und `V(s)` selbst sind als Ganzes atomar, was sich etwa durch Sperren aller Interrupts während der gesamten Dauer der Operation (hohe Priorität der Prozeduren) erreichen lässt.

```
PROCEDURE P(VAR s:INTEGER)
BEGIN
    WHILE s<=0 DO NoOp END;
    s:=s-1;
END P;
```

```
PROCEDURE V(VAR s:INTEGER)
BEGIN
    s:=s+1;
END V;
```

Eine komfortablere Lösung für Benutzerprozesse und lange Sperrzeiten ist mit den Betriebssystemaufrufen `sleep` und `wakeup()` realisiert und betrachtet den Semaphor als zusammengesetzte Variable, die auch eine Liste der wartenden Prozesse enthält. Die Implementierung der `ProcessList` ist dabei hier nicht gezeigt. `s.value` ist mit 1 initialisiert.

```

TYPE Semaphor = RECORD
    value: INTEGER;
    list : ProcessList;
END;

PROCEDURE P(VAR s:Semaphor)
BEGIN
    s.value:=s.value-1;
    IF s.value < 0 THEN
        einhängen(MyID,s.list); sleep;
    END;
END P;

PROCEDURE V(VAR s:Semaphor)
VAR PID: ProcessId;
BEGIN
    IF s.value < 0 THEN
        PID:=aushängen(s.list); wakeup(PID);
    END;
    s.value:=s.value +1;
END V;

```

Hardware-Implementierung

Eine wichtige Unterstützung für die Systemprogrammierung, gerade in einer Multiprozessorumgebung, stellt die Implementierung der atomaren Aktionen mittels Hardware dar. Es gibt dabei verschiedene Versionen, die kurz vorgestellt werden sollen:

- *Interrupts ausschalten*

Eine der einfachsten Methoden für atomare Aktionen in Monoprozessorsystemen besteht darin, alle Unterbrechungen durch Setzen der entsprechenden Statusbits in der CPU bzw. im Interruptcontroller auszuschalten. Dies kann aber Nebeneffekte mit sich bringen, z. B. wenn der *timer*-Interrupt ausgeschaltet wird und die Zeitzählung durcheinander kommt, bei Stromausfall der *power failure*-Interrupt versagt und die Register nicht mehr gerettet werden können usw. Wird eine solche atomare Aktion bei einem Round-Robin-Scheduling für die Synchronisierung mehrerer Prozesse eingesetzt, so ist durch das Abschalten des *timer*-Interrupts auch kein Prozesswechsel mehr möglich und ein in P(s) blockierter Prozess wartet ewig. Besser ist es deshalb, die Interrupts nicht

zu sperren und stattdessen dem Prozess im kritischen Abschnitt die höchstmögliche Priorität zu verleihen.

- *Atomare Instruktionsfolgen*

In Multiprozessoranlagen stellt das Sperren der Interrupts sowieso kein geeignetes Mittel dar. Hier haben sich eher atomare Aktionen in Form *nicht-unterbrechbarer Maschinenbefehle* (*atomic operation, atomic action*) bewährt, die einen komplexen Maschinenbefehl in einem einzigen, ununterbrechbaren Speicherzyklus durchführen. Beispiele dafür sind

- *Test And Set*

Die *test and set*-Instruktion (auch *test and set lock tsl* genannt) liest den Inhalt einer Speicherzelle aus und ersetzt ihn innerhalb desselben Speicherzyklus durch einen anderen Wert. Dies lässt sich mit einer Funktion spezifizieren:

```
PROCEDURE TestAndSet (VAR target: BOOLEAN):BOOLEAN
VAR tmp:BOOLEAN;
BEGIN
    tmp:=target; target:= TRUE; RETURN tmp;
END TestAndSet;
```

Das Rücksetzen auf *FALSE* wird nur von einem Prozess durchgeführt und kann daher normal ohne *atomic action* geschehen.

- *Swap*

Man kann auch die obige Operation verallgemeinern und den hineinzuschreibenden Wert spezifizieren. In diesem Fall tauscht swap die Werte der Variablen *source* und *target* miteinander aus.

```
PROCEDURE swap (VAR source, target: BOOLEAN)
VAR tmp:BOOLEAN;
BEGIN
    tmp:=target; target:=source; source:=tmp;
END swap;
```

- *Fetch And Add*

Diese Instruktion liest den Inhalt einer Speicherzelle aus und schreibt im selben ununterbrechbaren Speicherzyklus den neuen, um eine Zahl inkrementierten ausgelesenen Wert hinein.

```
PROCEDURE fetchAndAdd (VAR a, value:INTEGER): INTEGER;
VAR tmp:INTEGER;
BEGIN
    tmp:=a; a:=tmp+value; RETURN tmp;
END fetchAndAdd;
```

Es ist klar, dass mit diesen atomaren HW-Befehlen leicht das *busy wait* eines *spin locks* für einen exklusiven Ausschluss implementiert werden kann. Verwenden wir dies, um die atomaren Operationen $P(\cdot)$ und $V(\cdot)$ des Semaphors zu implementieren, so lassen sich auch die Semaphoroperationen hardwaremäßig absichern, ohne komplexe Maschineninstruktionen für die gesamten Prozeduren $P(\cdot)$ und $V(\cdot)$ zu implementieren.

Beispiel Multiprozessorsynchronisation

In Multiprozessoranlagen mit gemeinsamem Speicher lässt sich eine HW-Synchronisation nur mit einer atomaren Instruktion durchführen, da gemeinsame Interrupts nicht existieren müssen. Die parallel ausgeführten atomaren Instruktionen der Einzelprozessoren wirken deshalb untrennbar, weil sie im engen Verbund mit dem Speicherzugriffssystem erfolgt, das für eine Speicherstelle nur einmal existiert und damit für kurze Zeit (einen Speicherzyklus) exklusiven Zugriff auf die globale Variable garantiert.

Die Implementierung von Multiprozessor-Semaphoroperationen mit einem atomaren Maschinenbefehl kann folgendermaßen aussehen:

```
VAR s: INTEGER;                                (* initial = 1*)
PROCEDURE P(s);
BEGIN
  LOOP
    WHILE s<1 DO NoOp END; (a)
    IF (FetchAndAdd(s,-1) >= 1) THEN RETURN END; (b)
    FetchAndAdd(s,1); (c)
  END;
END P;

PROCEDURE V(s);
BEGIN
  FetchAndAdd(s,1)
END V;
```

Diese Implementierung (Gottlieb et al. 1981) hat neben dem `LOOP` nicht nur eine Warteschleife in $P(\cdot)$, sondern in Zeile (a) mit `WHILE` eine zweite – warum?

Dies ist durch die Multiprozessorumgebung bedingt. Entfernen wir die `WHILE`-Schleife, so reicht der Test mit `FetchAndAdd` (FAA) in Zeile (b) nicht mehr aus. Betrachten wir dazu den Fall, bei dem drei Prozessoren $P(s)$ zur gleichen Zeit die Prozedur durchlaufen, wobei $\text{initial } s = 1$ sei. Die drei Prozessoren sollen als Rückgabe für s die Werte 1,0 und -1 erhalten, wobei zum Schluß $s = -2$ gesetzt wird. Der erste Prozessor betritt und verlässt den kritischen Abschnitt, die beiden anderen warten in der Schleife. Soweit, so gut.

Angenommen, Prozessor 2 hat nun mit der ersten FAA in Zeile (b) gerade erst den Semaphor s auf -1 erniedrigt, wenn der erste Prozessor seine $V(\cdot)$ -Operation durchführt und damit den Semaphor auf $s = 0$ erhöht. Würde nun Prozessor 2 die zweite FAA-Instruktion in (c) durchführen und so auf $s = 1$ erhöhen, so könnte der Prozessor, der danach

die FAA in (b) als erster durchläuft, normal die $P()$ -Operation verlassen. Diese Gelegenheit kann aber durch den 3. Prozessor verhindert werden, indem er die FAA in (b) durchläuft noch bevor der 2. Prozessor (c) durchlaufen hat und damit den Semaphore wieder auf -1 erniedrigt, so dass er anschließend vom 2. Prozessor nur auf 0 statt auf 1 erhöht wird.

So ist durch die Sequenz (*timing*) $[P_3(b), P_2(c), P_2(b), P_3(c)]$, ... eine alternierende Sequenz für den Semaphore mit $s = 0, -1, 0, -1, \dots$ denkbar, in der der Semaphore nie den richtigen Wert hat, um einen der Prozessoren aus der Warteschleife zu entlassen: Die beiden Prozessoren bleiben blockiert, obwohl der kritische Abschnitt frei ist.

Dies wird nun durch die `while`-Schleife verhindert, da hier s nur gelesen und nicht verändert wird, so dass alle „Verlierer“-Prozesse zunächst einfach warten, ohne den Weg für andere blockieren zu können.

2.3.3 Beispiel UNIX: Semaphore

In UNIX gibt es in einigen Versionen den Zugriff auf Semaphore als Systemaufruf. Beispielsweise existieren in POSIX die folgenden Aufrufe :

<code>sem_init</code>	Initialisierung eines Semaphors (Speicherreservierung)
<code>sem_wait</code>	$P(s)$ Operation
<code>sem_post</code>	$V(s)$ Operation
<code>sem_getvalue</code>	Auslesen des Wertes eines Semaphors
<code>sem_destroy</code>	Entfernen des Semaphors (Speicherfreigabe)

Die Syntax und Semantik der Semaphoreoperationen ist dabei von UNIX-Version zu -Version verschieden.

Wichtig neben den expliziten Semaphoren sind die impliziten atomaren Operationen, die immer standardmäßig angeboten werden und mit denen sich auch Semaphore-Funktionen implementieren lassen. Beispielsweise ist das Erzeugen einer Datei (genauer: einer Datei-verwaltungsinformation mit Dateinamen etc.) eine atomare Operation: Entweder existiert eine neue Datei mit dem angegebenen Namen nach dem Systemaufruf, oder die Prozedur liefert eine Fehlermeldung zurück, beispielsweise „Name bereits vorhanden“. Dies ist nötig, um zu verhindern, dass zwei Prozesse gleichzeitig zwei verschiedene Dateien mit demselben Namen erzeugen. Es wird deshalb dazu verwendet, um das nötige Zusammenspiel zwischen Datenerstellung und Ausdruckprozess (*Erzeuger-Verbraucher-Modell*) für den Ausdruck von Dateien (*printer demon*) zu ermöglichen. Die dafür erzeugten Dateien bzw. Semaphore werden als Schlösser oder Schlossdateien (*lock files*) bezeichnet.

Atomare Aktionen werden in UNIX dadurch begünstigt, dass ein Prozess, der im Kernmodus ist, nicht abgebrochen werden kann. Er behält so lange die Kontrolle, bis er in einer Kernprozedur schlafen gelegt wird, oder aber in den *user mode* zurückkehrt.

In den Multiprozessor-UNIX-Systemen werden Semaphore im Kern auch zur Koordination der Prozessoren eingesetzt.

2.3.4 Beispiel Windows NT: Semaphore

Es gibt verschiedene Konstrukte zur Synchronisation in Windows NT. Für die klassische Synchronisation von Prozessen und *threads* gibt es die `CreateSemaphore()`- und `OpenSemaphore()`-Betriebssystemaufrufe, die Semaphore im globalen Namensraum erzeugen (ähnlich wie eine Datei) bzw. den Zugriff darauf initialisieren. Den `P()`- und `V()`-Operationen entsprechen die Prozeduren `WaitForSingleObject(Sema, TimeOutValue)` und `ReleaseSemaphore()`.

Dabei kann man die Semaphore als Zähler mit einem maximalen endlichen Wert oder als wechselseitige ausschließende (binäre) Untervariable wählen. Durch den möglichen systemweiten Zugriff im Namensraum sind diese Semaphorkonstrukte mit einem erhöhten Aufwand ansprechbar. Deshalb sind für die Koordination der *threads* innerhalb eines Prozesses zusätzliche „Leichtgewichtssemaphoren“ geschaffen worden, die nur innerhalb eines Prozesses (innerhalb eines Programms) bekannt sind. Diese als „critical section“ benannte Semaphore werden vom Typ `CRITICAL_SECTION` deklariert und mit `InitializeCriticalSection(S)` initialisiert. Die `P()`- und `V()`-Operationen heißen hier `EnterCriticalSection(S)` und `LeaveCriticalSection(S)`.

Da Windows NT speziell für enggekoppelte Multiprozessorsysteme mit gemeinsamem Speicher gedacht ist, benötigt der Kern besonders Primitive zur Multiprozessorsynchronisation. Hier werden als Semaphore *spin locks* eingesetzt, so dass ein *thread* mit einem *spin lock* den Prozessor so lange blockiert, bis es den *spin lock* wieder verlässt. Deshalb werden die *spin lock*-Operationen nur für Dienste innerhalb der NT Executive zur Verfügung gestellt und unterliegen einigen Einschränkungen. Beispielsweise dürfen im kritischen Abschnitt keine Referenzen zu Speicherbereichen gemacht werden, die auf den Massenspeicher ausgelagert wurden, man kann keine externe Prozeduren aufrufen (incl. Systemaufrufe) und kann keine Interrupts oder Ausnahmebehandlungen (*exceptions*) auslösen.

Für höhere Dienste, die auch vom Anwender genutzt werden können, gibt es deshalb als Primitive sog. *kernel objects*, die verschiedene Synchronisationseigenschaften haben und in das normale Schedulingssystem integriert sind. Beispiele für Kernobjekte sind die Semaphore, Ereignisse (*events*), Ereignispaare, *timer*, *threads*, sog. Mutanten (benutzerdefinierte *mutual exclusion*-Objekte) usw.

Die Objekte können in zwei Zuständen sein: *signalisiert* und *nicht-signalisiert*. Beispielsweise geht ein Semaphorobjekt in den „signalisiert“-Zustand über, wenn der Zähler auf null geht und bewirkt, dass alle darauf wartenden *threads* „befreit“ werden. Ein *thread* kann in Windows NT auf andere *threads*, auf Ereignisse, Semaphore etc. warten und sich dadurch mit ihnen synchronisieren. Die Win32 API stellt dafür die Prozeduren `WaitForSingleObject()` und `WaitForMultipleObjects()` zur Verfügung. Beispielsweise kann ein *thread* in einem Spreadsheet-Programm ein anderes *thread* aufrufen, das ein Spreadsheet ausdruckt. Beendet der Benutzer das Programm, so wartet der Hauptprozess (*main thread*) mit `WaitForSingleObject()` auf das Ende des Druckprozesses, bevor das Programm vollständig beendet wird und alle internen Daten verloren gehen.

In Windows NT gibt es noch ein weiteres Konzept, den Zugriff auf globale Ressourcen zu koordinieren: Die Gruppierung von *threads* und Objekten in sogenannte *Apartments*. Enthält ein Apartment nur einen *thread* (Single-Threaded Apartment STA), so ist ein Zugriff auf alle Objekte des Apartments unproblematisch und ohne besondere Sicherheitsvorkehrungen möglich. Versucht ein *thread* von außerhalb des Apartments auf ein Objekt zuzugreifen, so muss er dazu einen Systemaufruf durchführen; ein direkter Zugriff auf diese Objektinstanz ist nicht möglich. Dazu wird er in eine Warteschlange („Marshalling“) für das gewünschte Objekt gehängt, die durch Semaphoren abgesichert ist. In Abb. 2.25 ist ein Beispiel mit verschiedenen Apartments gezeigt.

Abgesehen von einem Apartment für den Haupt-Thread des Programms (main STA) gibt es noch Apartments, in denen mehrere *threads* nebeneinander existieren (Multi-Threaded Apartment MTA). Hier muss der Zugriff auf gemeinsame Objekte vom Programmierer selbst geregelt werden, um Probleme zu verhindern.

2.3.5 Anwendungen

Semaphore lassen sich für eine Vielzahl von unterschiedlichen Synchronisationsproblemen anwenden. Hier sollen einige wichtige und instruktive Beispiele aufgeführt werden.

Synchronisation von Prozessen

Die $P()$ -Operation für Semaphore blockiert einen Prozess so lange, bis er durch die $V()$ -Operation freigegeben wird. Dies können wir benutzen, um beispielsweise den gegenseitigen Abhängigkeiten in einem Prozesssystem Rechnung zu tragen und die Reihenfolge der Prozessaktivierung exakt festzulegen.

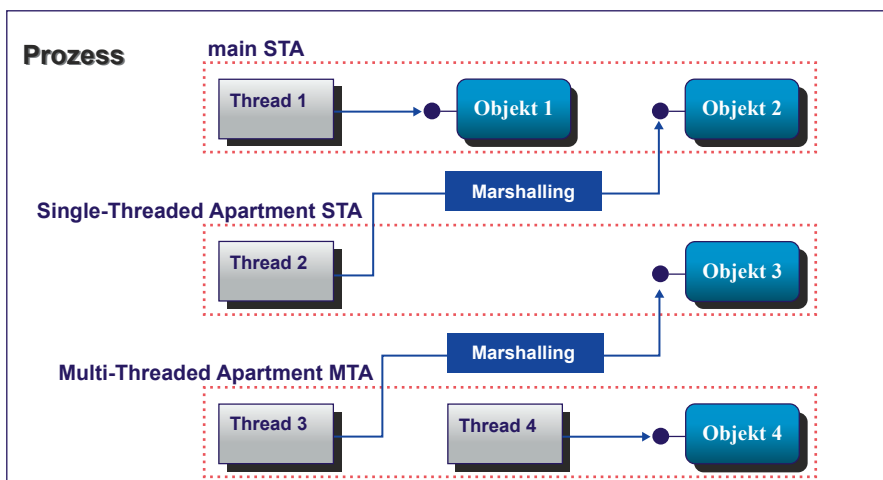
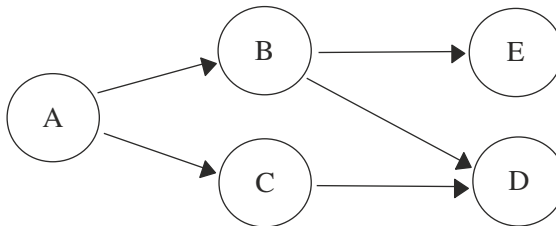


Abb. 2.25 Ein Beispiel für das *Apartment*-Modell in Windows NT

Dazu definieren wir uns für jede Abhängigkeit einen besonderen Semaphor, der am Anfang bzw. am Ende der Prozesse zum Warten bzw. Start benutzt wird.

Beispiel *Synchronisation mit Semaphoren*

Der Präzedenzgraph der Prozesse A,B,C,D,E sei folgendermaßen gegeben:



Dann lassen sich die Prozesse einzeln so formulieren:

```

PROCESS A; BEGIN                ProzessBodyA; V(b); V(c);    END A;
PROCESS B; BEGIN P(b);          ProzessBodyB; V(d1);V(e);    END B;
PROCESS C; BEGIN P(c);          ProzessBodyC; V(d2);        END C;
PROCESS D; BEGIN P(d1); P(d2);  ProzessBodyD;              END D;
PROCESS E; BEGIN P(e);          ProzessBodyE;              END E;
  
```

Die Semaphore *b, c, d1, d2, e* müssen als globale Variablen definiert und mit 0 initialisiert sein. Dadurch warten alle Prozesse, auch wenn sie zu unterschiedlichen Zeiten loslaufen, an den Semaphoren auf ihre Aktivierung. Interessanterweise macht es nichts aus, ob hier ein Prozess zu spät kommt: Das Wecksignal wird in dem Semaphor gespeichert, so dass es nicht verloren gehen kann.

Das Erzeuger-Verbraucher- (bounded buffer)-Problem

Viele Aufgaben der Datenerstellung, -verwaltung oder -filterung lassen sich als ein System von zwei Einheiten oder Prozessen formulieren, von denen einer Daten produziert und der andere sie abnimmt. Ein praktisches Beispiel ist das Aneinanderheften, Formatieren und Ausdrucken von Text in UNIX, s. S. 32.

Bezeichnen wir den Erzeuger als *producer* und den Verbraucher als *consumer*, so besteht das *producer-consumer*-Problem darin, dass die Erzeugung und Abnahme der Daten jeweils mit unterschiedlichen Raten (Daten pro Zeiteinheit) erfolgt. Üblicherweise wird man einen Puffer zwischen die Prozesse schalten, der vom Erzeuger gefüllt und vom Verbraucher geleert wird. Da der Puffer nur endlich ist (**bounded buffer**), muss der Strom der Daten zwischen Erzeuger und Verbraucher durch eine Flusskontrolle synchronisiert werden. Beispielsweise legt sich der Erzeuger schlafen, wenn der Puffer voll ist und wird vom Verbraucher wieder aufgeweckt, sobald der erste Platz frei wird. Entsprechend lässt

sich auch analog für den Verbraucher umgekehrt verfahren: Bei leerem Puffer legt sich der Verbraucher schlafen; sobald das erste Item erzeugt wurde, wird er aufgeweckt. In Abb. 2.26 ist zunächst der einfache, naive Ansatz gezeigt, wobei die Puffergröße mit N und die globale Variable `used` mit 0 initialisiert werden:

Diese Lösungsvariante hat ein Problem: Es kann leicht zu einer *race condition* kommen. Um dies zu sehen, betrachten wir den folgenden Fall. Angenommen, der Puffer ist leer und der Verbraucher hat gerade `used=0` gelesen, als auf den Erzeuger umgeschaltet wird. Dieser produziert ein `item`, schiebt es in den Puffer und inkrementiert `used` um 1. Da `used=1`, will er den Verbraucher wecken. Dieser muss aber nicht aufgeweckt werden; erst bei der nächsten Prozessorzuteilung legt er sich schlafen, da `used=0` war. Nachdem der Erzeuger den Puffer gefüllt hat, legt er sich ebenfalls schlafen. Beide schlafen dann ewig.

Diese Situation entstand, weil das *wakeup*-Signal zu früh kam und damit verlorenging. Würden wir es statt dessen speichern, beispielsweise mit einem *wakeup*-Bit, hilft das kurzfristig bei diesem Beispiel, aber bei weiteren Prozessen müssen weitere Bits eingeführt werden, um das Problem zu verhindern. Haben wir eine unbekannte Anzahl von Prozessen, wie dies in normalen Systemen der Fall ist, so fällt diese Lösung aus.

Statt dessen führt die Einführung von Semaphoren zu einer Lösung. Da Semaphore inkrementiert und dekrementiert werden, sind alle *wakeup*-Signale gespeichert und entbinden uns von der Notwendigkeit spezieller Statusbits. Die in Abb. 2.27 gezeigte Lösung mit Semaphoren wird mit `belegt:=0`, `frei:=N` und `mutex:=1` initialisiert.

Der *mutual exclusion*-Zugang zum Puffer (schattierter Codebereich) ist jeweils durch den gemeinsamen Semaphor `mutex` abgesichert. Mit dem Semaphor `belegt` wird die Zahl der belegten Pufferplätze gezählt und mit `frei` die Zahl der freien Plätze. Anstelle der Abfrage des Zählers und dem *sleep*- bzw. *wakeup*-Aufruf werden nun die Semaphoreoperationen aufgerufen, wobei für die Schlafbedingungen `used=N` bzw. `used=0` jeweils ein eigener Semaphor verwendet wird. Der produzierende Prozess dekrementiert nur die Zahl der freien Plätze; wenn sie null sind, wird er blockiert, bevor er den nächsten Platz belegen kann. Kann er den Platz belegen, so inkrementiert er die Zahl der belegten Plätze und weckt so den Verbraucherprozess auf, der sich bei null belegten Plätzen schlafen gelegt hatte.

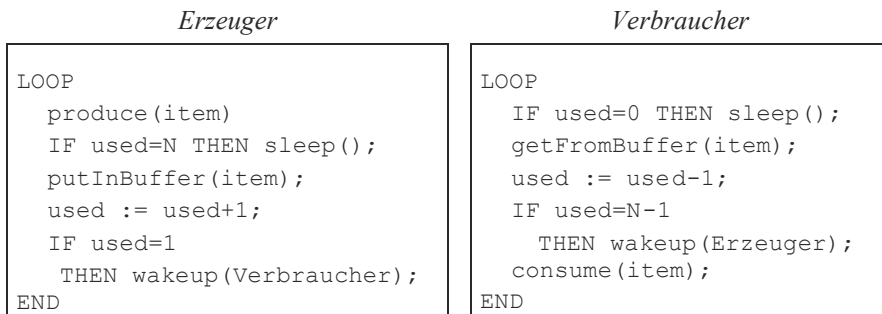


Abb. 2.26 Das Erzeuger-Verbraucher-Problem

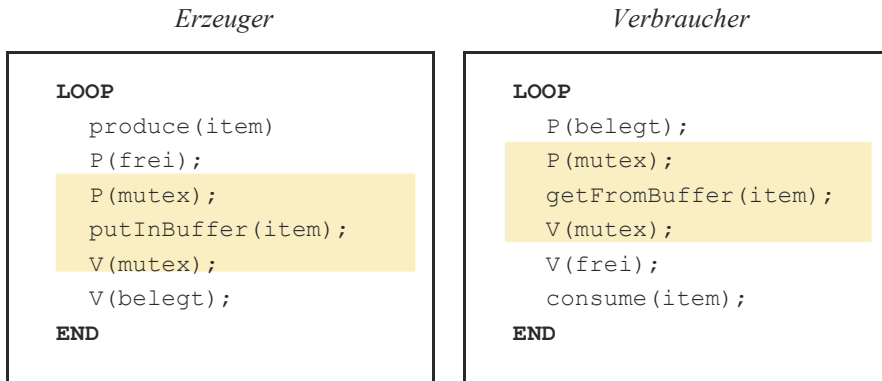


Abb. 2.27 Lösung des Erzeuger-Verbraucher-Problems mit Semaphoren

Man beachte, dass diese Lösung durch die Initialisierung vom `mutex` auf eins nur einen gegenseitigen Ausschluss im kritischen Abschnitt der Pufferverwaltung bewirkt. Bei der Erzeugung und Abnahme der `items` dagegen können auch mehrere Produzentenprozesse und Konsumentenprozesse mit dem gleichen Code existieren und den Puffer füllen bzw. leeren, ohne sich gegenseitig zu stören.

Das readers/writers-Problem

Das Grundproblem der Synchronisation, Schreiben und Lesen von mehreren Prozessen auf gemeinsamen Speicherbereichen zu koordinieren, lässt sich auch beim Schreiben und Lesen von Dateien wiederfinden und wird dann als das **readers/writers-Problem** bezeichnet. Grundforderung ist, das gleichzeitige Lesen und Schreiben auf dem gemeinsamen Datenbereich auszuschließen, um Dateninkonsistenzen zu vermeiden. Dies lässt sich mit einem einzigen Semaphor regeln. Stellen wir allerdings zusätzliche Forderungen auf, so werden die Lösungen komplizierter. Folgende Versionen des Problems existieren:

- Ein Leser soll nur warten, falls ein Schreiber bereits Zugriffsrecht zum Schreiben bekommen hat (**erstes readers/writers-Problem**). Dies bedeutet, dass kein Leser auf einen anderen Leser warten muss, nur weil ein Schreiber wartet.
- Wenn ein Schreiber bereit ist, führt er das Schreiben so schnell wie möglich durch (**zweites readers/writers-Problem**). Wenn also ein Schreiber bereit ist, die Zugriffsrechte zu bekommen, dürfen keine neuen Leser mit Lesen beginnen.

Man beachte, dass eine Lösung des ersten Problems darin resultieren kann, dass ein Schreiber ewig wartet; eine Lösung des zweiten Problems kann dazu führen, dass ein Leser ewig wartet. In beiden Fällen *verhungern* die Prozesse, da das Warten nur von dem Strom der Leser bzw. Schreiber abhängt und keine prinzipielle Blockade vorliegt.

Eine mögliche Lösung für das erste Problem führt einen gemeinsamen Zähler, der die Anzahl der Leser im kritischen Abschnitt registriert. Ist ein Leser in dem Abschnitt, wird

der Schreiber ausgesperrt, sonst nicht. Dazu gibt es zwei Semaphore `ReadSem` und `RWSem`: Einer, der den Zähler fürs Lesen schützt, und einer, der den exklusiven Zugang Lesen/Schreiben regelt. Der Pseudocode fürs Lesen ist

```
P(ReadSem);                # Schütze die Abfrage:
    readcount:=readcount+1; # bin ich der erste Leser?
    IF readcount=1 THEN P(RWSem) END; # sperre das Schreiben
V(ReadSem);
...
Reading_Data();
...
P(ReadSem);
    readcount:=readcount-1; # War dies der letzte Leser?
    IF readcount=0 THEN V(RWSem) END; # ermögliche Schreiben
V(ReadSem);
```

und fürs Schreiben

```
P(RWSem);
...
Writing_Data();
...
V(RWSem);
```

Der erste Leser auf `RWSem` wird nur blockiert, falls ein Schreiber im kritischen Abschnitt ist. Alle anderen Leser warten auf `ReadSem`. Außerdem: Warten sowohl ein Leseprozess als auch ein Schreibprozess auf `RWSem`, so entscheidet der Scheduler, welcher von beiden bei der Operation `V(RWSem)` aufgeweckt wird.

Multiprozessor-Warteschlangenmanagement beim NYU-Ultracomputer

Eines der interessantesten Parallelprozessor-Systeme der 80er Jahre war der Ultracomputer am Courant Institute der New York University (Gottlieb et al. 1983). Er hat eine Multiprozessorarchitektur nach Abb. 1.13. Für das Scheduling wurde ein spezieller Algorithmus implementiert, der die *fetch and add*-Operation zur Synchronisierung der Prozessoren auf dem *shared memory* benutzt und der hier kurz geschildert werden soll.

Die Warteschlange der bereiten Prozesse (*ready queue*) sollte auch bei parallelem Zugriff durch Multiprozessoren so funktionieren, dass eine Art FIFO-Eigenschaft eingehalten wird: Falls die Einfügeoperation für einen Prozess *p* abgeschlossen wird, bevor sie für einen Prozess *q* anfängt, dann darf es nicht möglich sein, dass die Aushängeoperation für *q* vor derjenigen für *p* endet.

Der folgende Algorithmus gilt für einen Ringpuffer der Länge *N*, s. Abb. 2.28.

Dazu existieren die globalen Variablen `InSlot` und `OutSlot`, die den Platz (*Slot*) im Ringpuffer bezeichnen, wo ein neuer Prozess eingehängt bzw. ein alter entnommen

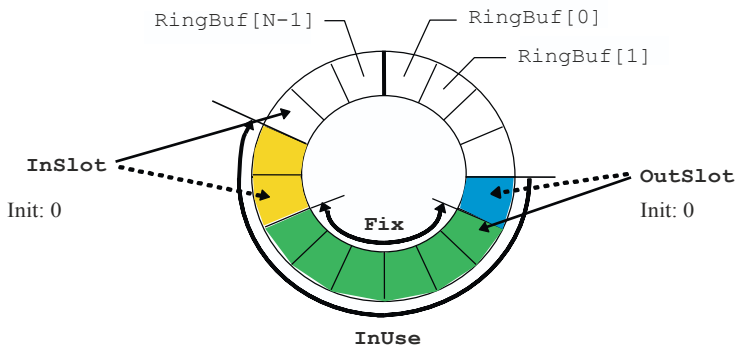


Abb. 2.28 Der Ringpuffer des NYU-Ultracomputers

werden kann. Der Puffer hat drei Zonen: eine Einfügezone, eine Aushängezone und eine fixe Zone, in der keine Aktivität herrscht.

Am Anfang ist der Puffer leer und deshalb $\text{InSlot} = \text{OutSlot} = 0$. Beim fortlaufenden Betrieb verschiebt sich der fixe Abschnitt langsam in Uhrzeigerrichtung im Ringpuffer, wobei die InSlot - und OutSlot -Variablen mittels *fetch and add*-Primitiven geschützt werden. Da noch zusätzlich das Problem des Pufferüberlaufs $\text{Full} := \text{TRUE}$ bzw. Leerpuffers $\text{Empty} := \text{TRUE}$ berücksichtigt werden muss, wird das Warteschlangenmanagement schwieriger. Da beim Einhängen zuerst InSlot inkrementiert wird und beim Aushängen danach OutSlot , stimmt es leider nicht, dass $\text{Full} = (\text{InSlot} - \text{OutSlot} = N)$ oder $\text{Empty} = (\text{InSlot} - \text{OutSlot} = 0)$ gilt, da es eine Reihe von Slots geben kann, die gerade gefüllt werden und noch nicht fertig sind, um geleert zu werden. Aus diesem Grund wird statt dessen für Full die anspruchsvollere Bedingung $(\text{InUse} = N)$ und für Empty die Bedingung $(\text{Fix} = 0)$ abgefragt. Die Zahl der Prozessoren, die gerade auf der Schlange arbeiten, ist damit durch $\text{InUse} - \text{Fix}$ gegeben.

Der Code in Abb. 2.29 hat verschiedene Feinheiten. Ohne die Semaphore entspricht dies im wesentlichen der Implementierung der Semaphoreoperationen, wie sie im Beispiel der Multiprozessorsynchronisation auf Seite 83 für *fetch and add* gezeigt sind, um den kritischen Abschnitt, das Einhängen und Aushängen der Daten des Gesamtpuffers, zu schützen und einen kontrollierten Zugang bereitzustellen. Die zweite Abfrage (grauer Bereich in Abb. 2.29) vorher, die mit einer Fehlermeldung Full oder Empty zurückkehrt, ist aus den schon bei diesem Beispiel erwähnten Gründen nötig. Der aufrufende Prozess kann dann selbst entscheiden, was in diesen Ausnahmefällen zu tun ist und muss nicht automatisch in einer Schleife warten.

Die Existenz der beiden Semaphoreoperationen, die durch *fetch and add*-Operationen implementiert werden können und im Originalbeitrag (Gottlieb et al. 1983) es auch sind, haben einen anderen Sinn. Angenommen, von den beiden ersten Einträgen, die in die leere Liste gefüllt werden, wird der zweite Prozessor zuerst fertig. Ein nachfolgendes Aushängen würde nun sofort Eintrag 1 aushängen, ohne zu merken, dass dieser noch nicht aktuell vorhanden ist,

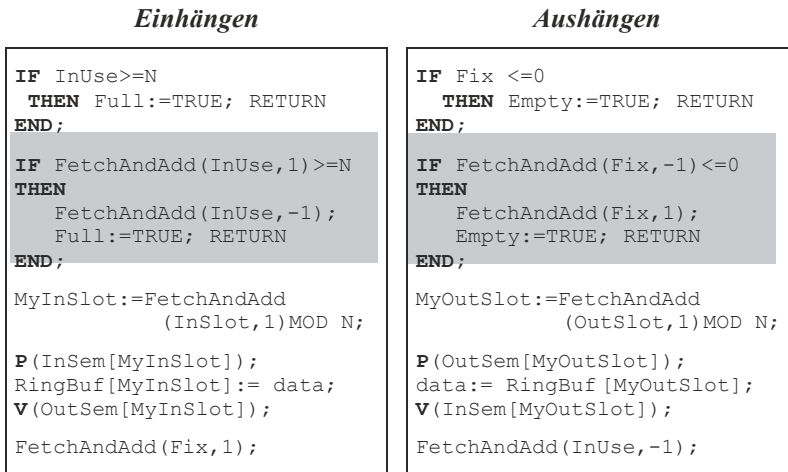


Abb. 2.29 Ein- und Aushängen bei der ready-queue

und die ungültigen Daten weiterverwenden. Aus diesem Grund wird nicht nur die Zahl der Einträge und damit das *bounded buffer*-Problem geregelt, sondern auch jeder einzelne Slot mit einem Semaphor versehen, der initial auf eins gesetzt wird. Dies hat zur Folge, dass entweder ein Einhängen oder ein Aushängen auf dem Slot durchgeführt wird, aber nicht beides gleichzeitig. Die Verwendung von *zwei* Semaphoren, deren Aufrufe wechselseitig verschränkt sind, garantiert nun mit den Anfangsbedingungen $InSem[i] := 1$, $OutSem[i] := 0$ zusätzlich, dass bei jedem Slot zuerst ein Einfügen stattfindet, dann ein Aushängen und so wechselseitig fort.

Man beachte, dass sowohl *InSlot* als auch *OutSlot* nur erhöht werden, also irgendwann überlaufen. Wird aber *N* als Zweierpotenz gewählt, so bedeutet die Modulo-Operation, dass der Überlauf nicht weiter bemerkt wird, da sowieso nur die unteren Bits ausgelesen werden.

2.3.6 Aufgaben zur Prozesssynchronisation

Aufgabe 2.3-1 *Race Conditions*

- Was wäre, wenn in [Abschn. 2.3.1](#) zuerst Prozess A die Kontrolle erhält und nach Schritt (2) die Umschaltung erfolgt?
- Gibt es noch weitere Möglichkeiten der Fehlererzeugung?

Aufgabe 2.3-2 *mutual exclusion*

Formulieren Sie die Prozeduren `betrete_Abschnitt(Prozess: INTEGER)` und `verlasse_Abschnitt(prozess: INTEGER)`, die die Lösung von Peterson für das *mutual exclusion*-Problem enthalten. Definieren Sie dazu die zwei globalen, gemeinsamen

Variablen `Interesse[1..2]` und `dran`. Könnte eine Verallgemeinerung auf n Prozesse existieren und wenn ja, wie?

Aufgabe 2.3-3 Synchronisation

Betrachten Sie die folgenden fünf Prozesse, die gleichzeitig ausgeführt werden. Jeder Prozess besteht aus vier Abschnitten, wobei Abschnitte der Typen A bis G jeweils korrespondierend kritisch sind:

P1	P2	P3	P4	P5
A	D	B	G	A
B	E	E	H	D
C	A	E	G	E
D	D	F	A	H

- Welche Abschnitte müssen Sie tatsächlich mit Semaphoren synchronisieren?
- Wie viele Semaphore benötigen Sie?

Aufgabe 2.3-4 Semaphoreoperationen

- Wie würden Sie die Semaphoreoperationen P und V formulieren, wenn s die Anzahl der jeweils wartenden Prozesse enthalten soll?
- Wie ändert sich die Lösung des Erzeuger-Verbraucher-Problems dadurch?
- Wie muss man s verändern, wenn mehrere Betriebsmittel existieren?

Aufgabe 2.3-5 Prozesssynchronisation

Wie lautet die Synchronisierung der Betriebsmittel aus [Abb. 2.13](#) mit Hilfe von Semaphoren?

Aufgabe 2.3-6 Spinlocks

- Was ist der Unterschied zwischen Blockieren und Spinlocks, um auf das Eintreten eines Ereignisses zu warten?
- Unter welcher Bedingung kann es vertretbar sein, Spinlocks zu verwenden?

Aufgabe 2.3-7 Erzeuger-Verbraucher-Synchronisation

Gegeben sei der folgende unvollständige Programmcode zur Lösung des Erzeuger-Verbraucher-Problems. Ordnen Sie die folgenden 8 fehlenden Befehle den Zeilen (10, 11, 14, 15, 19, 20, 23, 24) im Code zu.

s.mutex.acquire()	gehört in Zeile _____
s.mutex.acquire()	gehört in Zeile _____
s.mutex.release()	gehört in Zeile _____
s.mutex.release()	gehört in Zeile _____
s.empty.acquire()	gehört in Zeile _____
s.empty.release()	gehört in Zeile _____
s.full.acquire()	gehört in Zeile _____
s.full.release()	gehört in Zeile _____

Code:

```

1  REGAL = []
2  REGAL_KAPAZITAET = 5
3
4  s_empty = threading.Semaphore(REGAL_KAPAZITAET)
5  s_full = threading.Semaphore(0)
6  s_mutex = threading.Semaphore(1)
7
8  def verbraucher():
9      while True:
10         # Zeile 10
11         # Zeile 11
12         portion = REGAL.pop()
13         print('Lager hat noch "%s", %i Portionen übrig.' %
14               (portion, len(REGAL)))
15         # Zeile 14
16         # Zeile 15
17
18  def erzeuge():
19      while True:
20         # Zeile 19
21         # Zeile 20
22         portion = "McBurger"
23         REGAL.append(portion)
24         # Zeile 23
25         # Zeile 24

```

Aufgabe 2.3-8 Das readers/writers-Problem

Entwerfen Sie einen Pseudocode, um das zweite *readers/writers*-Problem zu lösen.

Aufgabe 2.3-9 Chinesische, dinierende Philosophen

N Philosophen sitzen um einen Esstisch und wollen speisen. Auf dem Tisch befindet sich eine große Schale mit Reis und Gemüse. Jeder Philosoph meditiert und isst abwechselnd.

Zum Essen benötigt er jeweils 2 Stäbchen, auf dem Tisch liegen aber nur N Stäbchen, so dass er sich ein Stäbchen mit seinem Nachbarn teilen muss. Also können nicht alle Philosophen zur selben Zeit essen; sie müssen sich koordinieren. Wenn ein Philosoph hungrig wird, versucht er in beliebiger Reihenfolge das Stäbchen links und rechts von ihm aufzunehmen, um mit dem Essen zu beginnen. Gelingt ihm das, so isst er eine Weile, legt die Stäbchen nach seinem Mahl wieder zurück und meditiert. Dabei sind allerdings Situationen denkbar, in der alle gleichzeitig agieren und sich gegenseitig behindern.

Geben Sie eine Vorschrift für jeden Philosophen (ein Programm in einem MODULA oder C-Pseudocode) an, das dieses Problem löst!

2.3.7 Kritische Bereiche und Monitore

Obwohl die Einführung von Semaphoren die Synchronisation bei Prozessen stark erleichtert, ist der Umgang mit Ihnen nicht unproblematisch. Vertauschen wir beispielsweise die Reihenfolge der Operationen zu

```
V(mutex); .. krit. Abschnitt ..; P(mutex);
```

so ist das Ergebnis gerade invers zum Gewünschten: Alle sind nur im kritischen Abschnitt zugelassen. Auch der Fehler

```
P(mutex); .. krit. Abschnitt ..; P(mutex);
```

oder das einfache Weglassen einer der beiden Operationen führt zu starken Problemen. Dabei reicht schon eine subtile, in großen Programmen fast nicht bemerkbare Änderung aus, um die unerwünschten, nur sporadisch auftretenden Fehler zu erzeugen. Beispielsweise ist eine Vertauschung der Operationen in der Lösung des *Erzeuger-Verbraucher-Problems*

```
von   P(frei); P(mutex); putInBuffer(item); V(mutex); V(belegt);
zu     P(mutex); P(frei); putInBuffer(item); V(mutex); V(belegt);
```

äußerst problematisch (warum?).

Aus diesem Grund schlugen Hoare (1972) und Brinch Hansen (1972) vor, die richtige Erzeugung und Anordnung der Semaphoroperationen dem Compiler zu überlassen und statt dessen ein neues Sprachkonstrukt, den **kritischen Bereich (critical region)**, einzuführen. Die Syntax für eine gemeinsame Variable (Semaphor) s eines beliebigen Typs ist

region s **do** <statement>

was der Folge entspricht $P(s); \text{<statement>}; V(s)$

Gibt es also mehrere Prozesse, die für sich einen kritischen Bereich mit derselben gemeinsamen Variablen s (deklariert mit **shared** s) betreten wollen, so garantieren die vom

Compiler erzeugen Synchronisationsmechanismen dafür, dass immer nur ein Prozess dies kann. Zwei parallel ablaufende Prozesse mit

region s do S1

region s do S2

erzeugen also Sequenzen der Form S1; S2; ... oder S2; S1; ..., wobei zwar die Reihenfolge der Befehle bzw. Befehlssequenzen S1 und S2 beliebig, die Befehlsausführungen aber nicht gemischt sein dürfen.

Führt man noch zusätzlich eine Bedingung B ein (Hoare [1972](#)), die den Zugang zu einem solchen kritischen Abschnitt regelt, in der Form

region s when B do <statement>

so erhalten wir einen *bedingten* kritischen Bereich (*conditional critical region*). Die Semantik des bedingten kritischen Bereichs dieses Konstrukts sieht vor, dass ein Prozess nach Belegen von s nur dann den kritischen Abschnitt betreten kann, wenn auch die Bedingung B erfüllt ist. Ist dies nicht der Fall, wird s wieder aufgegeben, und der Prozess legt sich schlafen, bis die Bedingung erfüllt ist. Sodann versucht er wieder, s zu belegen.

Dieses Konzept wurde in objektorientierter Weise erweitert, um auch passive Daten mitzuschützen. Dazu schlugen Brinch Hansen ([1973](#)) und Hoare ([1974](#)) einen neuen abstrakten Datentyp (Klasse) vor, der die Aufgabe übernimmt, alle darin enthaltenen, von außen ansprechbaren Prozeduren (*Methoden*, abstrakte Datentypen ADT) und die lokalen, dazu benutzten Daten und Variablen durch Synchronisationsprimitive automatisch zu schützen und den wechselseitigen Ausschluss von Prozessen darin zu garantieren. Diese Klasse nannten sie einen **Monitor**, was aber nichts mit einem Bildschirm oder einem Softwaremonitor aus [Abschn. 2.2.5](#) zu tun hat.

Das Monitorkonstrukt folgt syntaktisch dem Schema

```
MONITOR <name>
  (* lokale Daten und Prozeduren *)
PUBLIC
  (*Prozeduren der Schnittstelle nach außen *)
BEGIN (*Initialisierung *)
...
ENDMONITOR
```

Der Aufruf der allgemein zugänglichen (also unter PUBLIC deklarierten), geschützten Prozeduren erfolgt durch Voranstellen des Prozedurnamens.

Beispiel Erzeuger-Verbraucher

```
MONITOR Buffer;
TYPE      Item = ARRAY[1..M] of BYTE;
```

```

VAR      RingBuf: ARRAY[1..N] of Item;
         InSlot, (* 1. freier Platz *)
         OutSlot, (* 1. belegter Platz *)
         used: INTEGER;
PUBLIC PROCEDURE putInBuffer(item:Item);
        BEGIN ... END putInBuffer;
        PROCEDURE getFromBuffer(VAR item:Item);
        BEGIN ... END getFromBuffer;
BEGIN
  used:=0; InSlot:=1; OutSlot:=1;
END MONITOR;

```

Der Gebrauch des Monitors sieht dann wie in [Abb. 2.30](#) aus:

Allerdings ist der Code noch nicht vollständig. Im Beispiel von [Abb. 2.27](#) sahen wir, dass es nötig sein kann, zusätzliche Wartebedingungen (hier: zur Flusskontrolle des Pufferüberlaufs) innerhalb eines kritischen Abschnitts vorzusehen. Dies ist im Monitor prinzipiell durch spezielle Variablen vom Typ `CONDITION` vorgesehen. Die einzigen Operationen, die auf diesen Variablen zugelassen sind, sind die beiden folgenden Signaloperationen:

- *signal(s)*
sendet ein Signal *s* innerhalb eines Monitors an alle, die darauf warten, unabhängig davon, ob überhaupt jemand wartet.
- *wait(s)*
wartet so lange, bis ein Signal *s* gesendet wird, und führt danach die nächsten Instruktionen aus.

Wir erweitern deshalb unseren Monitor um die Bedingungsvariablen

```
VAR frei, belegt: CONDITION
```

die für die Flusskontrolle nötig sind. Die beiden vom Monitor geschützten Zugriffsoperationen sehen damit dann folgendermaßen aus:

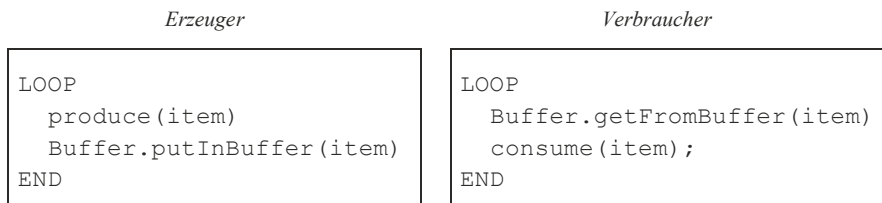


Abb. 2.30 Lösung des Erzeuger-Verbraucher-Problems mit einem Monitor

```
PROCEDURE putInBuffer(item: Item);
BEGIN
    IF used=N THEN wait(frei) END;
    RingBuf[InSlot]:=item;
    used := used+1;
    InSlot := (InSlot+1) MOD N;
    signal (belegt);
END putInBuffer;

PROCEDURE getFromBuffer(VAR item: Item);
BEGIN
    IF used=0 THEN wait(belegt) END;
    item := RingBuf[OutSlot];
    used := used-1;
    OutSlot := (OutSlot+1) MOD N;
    signal(frei);
END getFromBuffer;
```

Diese Implementation erinnert uns an den naiven Ansatz in [Abb. 2.26](#), S. 90. Im Unterschied zu diesem Ansatz haben wir allerdings nun die Abfragen zur Flusssteuerung mit gegenseitigem Ausschluss geschützt, so dass sich keine *race conditions* bilden können.

Die Anwendung des Monitorkonstrukts in Betriebssystemen ist allerdings nicht unproblematisch (Keedy 1979). Die wesentlichsten Kritikpunkte sind die unerwartete und unnötige Sequentialisierung von Betriebssystemabläufen, die bei der (ungewollten) Schachtelung von Monitorkaufrufen durch die Funktionsaufrufe an sich unabhängiger, separat geschriebener Betriebssystemteile auftritt (Lister 1977) sowie die besonderen Wartezustände der Prozesse in den Monitoren, die es schwierig machen, bei Zeitüberschreitungen, Abschalten des Gesamtsystems (*shutdown*) oder Prozessabbrüchen die Wartezustände konsistent aufzulösen (Lampson und Redell 1980). Mit den allgemeinen Konstrukten wie *Prozesse* und *Signale* kann man bei geschickter Programmierung die genannten Probleme vermeiden.

Ein weiteres praktisches Problem besteht darin, dass die Semaphoroperationen als Zusatzmodule (z. B. mit speziellen Maschinenbefehlen) einer Anwenderbibliothek geschrieben und damit leicht in fast allen Programmiersprachen zur Verfügung gestellt werden können, im Gegensatz dazu aber für kritische Bereiche und Monitore die Compiler erweitert werden müssen. Dies bringt zusätzliche Komplikationen mit sich und hat sich deshalb kaum durchgesetzt.

Beispiel Java

Eine wichtige objektorientierte Version von C bzw. C++ ist die Programmiersprache *Java*. Ihre Anwendungsmodule (**applets**) sind dazu gedacht, als Softwaremodule kleiner Systeme zu fungieren sowie als Bestandteil einer World-Wide-Web-Seite beim Benutzer spezielle Funktionen zu erfüllen, die der normale Web-Browser nicht erfüllen kann.

Dazu wird der Sourcecode (oder der kompilierte, symbolische Code) über das Netz auf den Benutzerrechner geladen und dort von einem Java-Interpreter interpretiert, der Bestandteil des Browsers oder des Betriebssystems (z. B. wie in OS/2) sein kann.

Der Java-Code soll auf sehr unterschiedlichen Rechnersystemen gleiche Ergebnisse liefern, wobei auch mehrere *threads* parallel abgearbeitet werden können. Die Koordination mehrerer dieser Leichtgewichtsprozesse für den Zugriff auf ein gemeinsames Objekt wird mit Hilfe des Schlüsselwortes `synchronized` erreicht. Die Syntax ist mit

```
synchronized (<expression>) <statement>
```

sehr ähnlich einem kritischen Bereich. Ein *thread* kann nur dann die Instruktion (bzw. Instruktionsfolge) `<statement>` ausführen, wenn vorher das Objekt oder Feld `<expression>` (durch Semaphore geschützt) belegt wurde. Ist das Objekt schon belegt, so wird er so lange blockiert, bis der andere *thread* den kritischen Bereich wieder verlässt. Innerhalb eines `synchronized`-Bereichs kann man zusätzlich mit den Methoden `wait()` auf ein Signal warten, das mit `notify()` gesendet wird.

2.3.8 Verklemmungen

Auch in einem wohlsynchronisierten Betriebssystem mit guten Schedulingalgorithmen können Situationen auftreten, in denen Prozesse sich gegenseitig behindern und blockieren, so dass die Programme nicht ausgeführt werden können. Wie ist das möglich?

Betrachten wir dazu folgende Situation: Angenommen, Prozess 1 sammelt Daten, aktualisiert dann Einträge in einer Datei und protokolliert dies auf einem Drucker. Nun beginnt ein Prozess 2, der die gesamte Datei ausdrucken soll. Beide Betriebsmittel, der Drucker und die Datei, sind mit wechselseitigem Ausschluss geschützt, um einen Wirrwarr beim Schreiben und Lesen zu verhindern. Nun geschieht folgendes: Nach dem Datensammeln sperrt Prozess 1 die Datei, aktualisiert einen Eintrag und will ihn vor der Aktualisierung des zweiten Eintrags ausdrucken. Währenddessen hat Prozess 2 den Drucker gesperrt, die Überschrift ausgedruckt und beantragt die Datei. Da diese gerade benutzt wird, legt sich Prozess 2 mit dem Semaphor schlafen. Aber auch Prozess 1 legt sich beim Versuch, den Drucker zu bekommen, schlafen. Beide schlafen nun ewig – im Prozesssystem ist eine **Verklemmung (deadlock)** aufgetreten.

Wir haben also die Situation, dass ein Prozess Betriebsmittel A (die Datei) belegt und ein weiteres Betriebsmittel B (den Drucker) haben will, das seinerseits von einem anderen Prozess belegt ist, der wiederum das bereits belegte Betriebsmittel A haben will.

Aus unserem Beispiel lassen sich einige Eigenschaften abstrahieren, die nach Coffman et al. (1971) notwendige und hinreichende Bedingungen für diese Art von Störungen sind:

(1) *Beschränkte Belegung (mutual exclusion)*

Jedes involvierte Betriebsmittel ist entweder exklusiv belegt oder aber frei. *Also:* Semaphoren sind notwendig für ein verklemmungsfreies System, aber nicht hinreichend.

(2) *Zusätzliche Belegung (hold-and-wait)*

Die Prozesse haben bereits Betriebsmittel belegt, wollen zusätzlich weitere Betriebsmittel belegen und warten darauf, dass sie frei werden. Die insgesamt nötigen Betriebsmittel werden also nicht auf einmal angefordert.

(3) *Keine vorzeitige Rückgabe (no preemption)*

Die bereits belegten Betriebsmittel können den Prozessen nicht einfach wieder entzogen werden, ähnlich dem präemptiven Scheduling der CPU, sondern müssen von den Prozessen selbst explizit wieder zurückgegeben werden.

(4) *Gegenseitiges Warten (circular wait)*

Es muss eine geschlossene Kette von zwei oder mehr Prozessen existieren, bei denen jeweils einer Betriebsmittel vom nächsten haben will, die dieser belegt hat und die deshalb nicht mehr frei sind.

Eine solche Situation gibt es nicht nur bei solchen Betriebsmitteln wie Druckern, Bandgeräten oder Scannern, sondern auch bei logischen Einheiten wie Semaphoren. Betrachten wir beispielsweise das *Erzeuger-Verbraucher*-Beispiel in [Abb. 2.27](#) auf Seite 91 und nehmen wir an, dass zusätzlich zu `belegt:=0` aus Versehen auch `frei:=0` gesetzt wurde. Nun haben wir die Situation

<i>Erzeuger</i>	<i>Verbraucher</i>
...	...
<code>wait(frei)</code>	<code>wait(belegt)</code>
...	...
<code>send(belegt)</code>	<code>send(frei)</code>

Beide warten auf die Freigabe des Semaphors, den der andere jeweils „besitzt“, aber nur freigeben kann, wenn seinen eigenen Wünschen Genüge getan wurde.

Für die Behandlungen von Verklemmungen gibt es folgende vier erfolgreiche Strategien:

- das Problem ignorieren,
- die Verklemmungen erkennen und beseitigen,
- die Verklemmungen vermeiden,
- die Verklemmungen unmöglich machen, indem man eine der obigen Bedingungen (1)–(4) verhindert.

Wir wollen im folgenden diese vier Strategien etwas näher betrachten.

Mögliche Verklemmungen ignorieren

Auf den ersten Blick scheint die Strategie, Probleme zu ignorieren statt sie zu lösen, absolut inakzeptabel zu sein, besonders für Mathematiker. Berücksichtigen wir aber die Tatsache, dass es in großen Systemen noch sehr viele weitere potentielle Störungsmöglichkeiten gibt, die uns das Leben schwer machen können, angefangen bei HW-Problemen (schlechten

Lötstellen, wackeligen Kabeln, partiellen Chipausfällen, Netzspannungsschwankungen,...) bis zu SW-Problemen (Fehlern im Betriebssystem, neue Compilerversionen, Umkonfiguration der verwendeten Software, falsche Dateneingaben etc.), so lassen sich die Fälle, bei denen zwei Prozesse sich verklemmen, in der täglichen Praxis als „unbedeutend“ klassifizieren, ähnlich wie der potentielle Überlauf der Prozesstafeln oder das Überschreiten des Maximums der gleichzeitig offenen Dateien. Physiker betrachten dies eher als „Schmutzeffekt“, den man ignoriert, solange er nicht zu viel stört; EDV-Manager sehen in der Vermeidung von Verklemmungen eher eine „Ablenkung von wichtigeren Aufgaben“. Aus diesem Grund sind in UNIX keine Extramaßnahmen für Verklemmungen vorgesehen.

Unabhängig von der Häufigkeit des Auftretens wäre es aber trotzdem besser, auch Verklemmungen im Betriebssystem automatisch zu behandeln, um wenigstens den vorhersehbaren Ärger zu vermeiden. Leider aber ist dies, wie wir sehen werden, einerseits nicht einfach und andererseits nicht ohne Aufwand für den laufenden Betrieb. Aus diesem Grund muss man beim Einsatz der folgenden Maßnahmen immer Anlass und Aufwand gegeneinander abwägen.

Verklemmungen erkennen und beseitigen

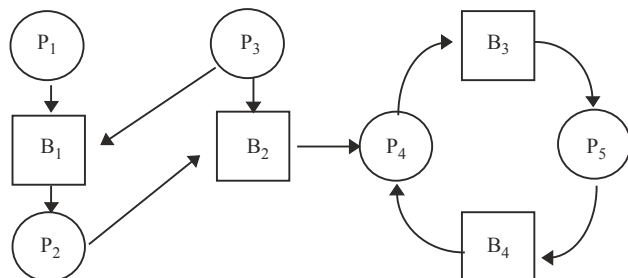
Eine Strategie zur Behandlung von Verklemmungen besteht darin, die normalen Betriebssystemfunktionen zu überwachen und beim Auftreten von verdächtigen Symptomen einen Algorithmus zu starten, der systematisch die bestehende Situation untersucht und ermittelt, ob tatsächlich eine Verklemmung vorliegt.

Verdächtige Symptome können dabei sein,

- wenn sehr viele Prozesse warten und der Prozessor unbeschäftigt (*idle*-Prozess!) ist. Dies erfordert eine genaue Definition von „sehr viele“, die man z. B. über eine obere Grenze (Schwellwert) herstellen kann.
- wenn mindestens zwei Prozesse zu lange auf Betriebsmittel warten müssen. Auch hier muss eine obere Grenze für „zu lange“ definiert werden.

Die Situation der Prozesse und der Betriebsmittel lässt sich durch einen Betriebsmittelgraphen (*resource allocation graph*) visualisieren, s. [Abb. 2.31](#).

Abb. 2.31 Ein Betriebsmittelgraph



Hierbei sei ein Prozess als Kreis, ein Betriebsmittel (*resource*) als Viereck und die Belegung als Pfeil symbolisiert. Die Pfeilrichtung Prozess $\rightarrow$ Betriebsmittel bedeutet, dass ein Prozess das Betriebsmittel haben möchte; die Richtung Prozess $\leftarrow$ Betriebsmittel deutet an, dass der Prozess das Betriebsmittel fest hat.

Die vier notwendigen und hinreichenden Verklemmungsbedingungen bedeuten für den Betriebsmittelgraphen, dass es bei Verklemmungen Zyklen in dem Graphen gibt, was wir im obigen Beispiel direkt bei den Knoten P_4, B_3, P_5, B_4 bemerken. Dies lässt sich bei größeren Graphen durch entsprechende Graphenalgorithmen nachprüfen. Einen Algorithmus für den Fall, dass es mehrere Betriebsmittel derselben Art gibt, wurde von Holt (1972) vorgestellt.

Ein anderer, einfacher Algorithmus (Habermann 1969) prüft nach, ob die Zahlen A_s der noch verfügbaren Betriebsmittel des Typs s ausreichen, um einen verklemmungsfreien Ablauf zu ermöglichen. Dazu prüfen wir, ob ein Prozess existiert, der mit den noch freien Betriebsmitteln auskommt. Ist ein solcher Prozess gefunden, so wird angenommen, dass er nach seinem Abschluss die belegten Betriebsmittel wieder zurückgibt und die Menge $\{A_s\}$ der freien Betriebsmittel anwächst. Dazu wird er markiert und der nächste Prozess gesucht. Dies wiederholen wir so lange, bis entweder alle Prozesse markiert sind und damit feststeht, dass keine Verklemmung existiert, oder aber mehrere unmarkierte Prozesse übrigbleiben, die sich gegenseitig mit ihren Anforderungen blockieren. Dabei gehen wir natürlich davon aus, dass bei vollständig unbelegten Betriebsmitteln jeder Prozess für sich genügend Betriebsmittel zur Verfügung hätte und so allein vollständig abgearbeitet werden könnte.

Mit der Notation

- E_s Zahl der existierenden Einheiten pro Betriebsmittel s ,
z. B. $E_2 = 5$ für 5 existierende Einheiten vom Typ $s = 2$ (Drucker)
- B_{ks} Zahl der Einheiten vom Typ s , die der Prozess k belegt hat
- C_{ks} Zahl der Einheiten vom Typ s , die Prozess k zusätzlich fordert

können wir diesen Algorithmus genauer formulieren, wobei wir als Summe der belegten Betriebsmittel $\sum_k B_{ks}$, die Spaltensumme der Matrix (B_{ks}) , erhalten. Die Anzahl A_s der noch freien Betriebsmittel des Typs s ist somit

$$A_s = E_s - \sum_k B_{ks}$$

Die einzelnen Schritte des Algorithmus lauten also:

- (0) Alle Prozesse sind unmarkiert.
- (1) Suche einen unmarkierten Prozess k , bei dem für alle Betriebsmittel s gilt $A_s \geq C_{ks}$.
- (2) Falls ein solcher Prozess existiert, markiere ihn und bilde $A_s := A_s + B_{ks}$.
- (3) Gibt es keinen solchen Prozess, STOP. Ansonsten durchlaufe erneut (1) und (2).

In Pseudocode lässt sich dies wie folgt formulieren:

```

FOR k:=1 TO N DO unmarkProcess(k) END; d:=0;      (*Step 0*)
REPEAT
  k:= satisfiedProcess();                          (*Step 1*)
  IF k#0 THEN d:=d+1; markProcess(k);              (*Step 2*)
    FOR s:=1 TO M DO A[s]:= A[s]+B[k,s] END;
  END;
UNTIL k=0;                                          (*Step 3*)
IF d<N THEN Error('Deadlock exists!') END;

```

Geht der Algorithmus ohne Fehlermeldung zu Ende, so ist nur festgestellt, dass keine ausweglose Situation entstehen muss; wie sich die Situation tatsächlich nach dem Test weiterentwickeln wird, ist nicht gesagt. Erhalten wir dagegen eine Fehlermeldung, so lässt sich zeigen, dass tatsächlich eine Verklemmung vorhanden ist, egal, in welcher kombinatorischen Reihenfolge wir die Prozesse markiert haben. Der Grund für die Unabhängigkeit dieser Tatsache von der Markierungsreihenfolge ist das stetige Anwachsen der Betriebsmittel beim Markieren.

Natürlich läuft der Algorithmus zu einem Zeitpunkt ab und berücksichtigt nur die zu diesem Zeitpunkt existierende Situation. Würden die Ressourcen sich während der Laufzeit ändern, so wäre die Aussage nicht mehr zuverlässig.

Beispiel Verklemmungen

		plot- print- CD						plot- print- CD			
		tapes		ters	ers	ROM		tapes		ters	ers
		A	B	C	D			A	B	C	D
Seien	P ₁	3	0	1	1	und	P ₁	1	1	0	0
(B _{ij}) =	P ₂	0	1	0	0		P ₂	0	1	1	2
	P ₃	1	1	1	0		P ₃	3	1	0	0
	P ₄	1	1	0	1		P ₄	0	0	1	0
	P ₅	0	0	0	0		P ₅	2	1	1	0
<i>bestehende Belegung</i>						<i>zusätzlich gewünschte Belegung</i>					

wobei $E = (6\ 3\ 4\ 2)$

Die Gesamtbelegung ist $(5\ 3\ 2\ 2)$, so dass also noch $A = (1\ 0\ 2\ 0)$ übrigbleibt. Von allen Prozessen kann also nur P_4 zusätzlich $(0\ 0\ 1\ 0)$ belegen, seine Arbeit vollständig durchführen und seine gesamten Betriebsmittel $(1\ 1\ 1\ 1)$ zurückgeben. Damit erhalten wir $A = (2\ 1\ 2\ 1)$. Nun können P_1 und P_5 die Arbeit beenden, so dass wir $A = (5\ 1\ 3\ 2)$ erhalten, und, zuletzt, wird mit P_2 und P_3 $A = (6\ 3\ 4\ 2)$. Man beachte, dass die Reihenfolge P_4, P_1, P_5, P_2, P_3 nicht zwangsläufig ist; es hätte auch P_4, P_5, P_1, P_2, P_3 sein können.

Wie beseitigen wir nun eine aufgetretene Verklemmung? Aus dem obigen Algorithmus ist leicht ersichtlich, welche Betriebsmittel und Prozesse verklemmt sind. Nun können wir

- *Prozesse abbrechen*

Die einfachste Methode besteht darin, entweder einen der verklemmten Prozesse oder einen unbeteiligten Prozess, der aber das benötigte Betriebsmittel hat, einfach abzubrechen und zu hoffen, dass sich alles selbst wieder arrangiert. Welche Prozesse sinnvoll abgebrochen werden können, muss man selbst ermitteln. Einen Hinweis darauf gibt der obige Algorithmus, der mit der Folge der markierten Prozesse auch eine Schedulingreihenfolge liefert, die verklemmungsfrei die Arbeit beendet.

- *Prozesse zurücksetzen*

In Datenbanksystemen gibt es meist aus Sicherheitsgründen regelmäßige Zeitpunkte, an denen der gesamte Systemzustand abgespeichert wird. Tritt nun eine Verklemmung auf, so kann man einen oder mehrere Prozesse, anstatt sie brutal abzubrechen, auf einen Zustand zurücksetzen, in dem sie die Betriebsmittel noch nicht belegt hatten, und die Prozessausführung so lange verzögern, bis die benötigten Betriebsmittel von den verklemmten Prozessen wieder freigegeben werden.

- *Betriebsmittel entziehen*

In manchen Fällen ist es möglich, einem Prozess das von anderen Prozessen benötigte Betriebsmittel zu entziehen und es zunächst den anderen Prozessen zur Verfügung zu stellen. Sobald es wieder frei wird, kann sich dann auch dieser Prozess wieder darum bewerben und an der abgebrochenen Programmstelle weitermachen.

Diese Strategie ist nicht immer möglich, wie z. B. beim Beschreiben einer Datei, oder führt zu manuellem Mehraufwand, beispielsweise bei einem Drucker, dessen bisheriger Papierausdruck per Hand weggeräumt und wieder im richtigen Augenblick hingelegt werden muss.

Die Nachteile dieser Methoden sind klar: Brechen wir Prozesse einfach ab, so geht alle weitere Arbeit verloren. Setzen wir die Prozesse zurück, so geht alle bisher geleistete Arbeit seit dem Sicherungszeitpunkt verloren. Außerdem gibt es dabei viele Zusatzprobleme: Sowohl der abgebrochene als auch der zurückgesetzte Prozess können Daten geschrieben oder Nachrichten verschickt haben, die nicht zurückgenommen werden und unter Umständen zu inkonsistenten Dateien führen können. Die Auswahl des entsprechenden Prozesses sollte sich deshalb nach dem Kriterium des geringsten angerichteten Schadens richten.

Verklemmungen vermeiden

Eine der einfachsten Strategien gegen Verklemmungen besteht darin, beim Auftreten eines Fehlers („Betriebsmittel bereits belegt“) dies dem Benutzer anzuzeigen und den Benutzer selbst die richtige Reaktion darauf vornehmen zu lassen. Das ist meistens bei Einbenutzersystemen (*Personal Computer* PC) der Fall, in denen der Benutzer alles überschauen kann, genügend Zugriffsrechte besitzt und so Verklemmungen vermeiden kann.

In Mehrbenutzersystemen ist dies aber eine unbrauchbare Strategie. Hier werden ausgefeiltere Techniken benötigt. Eine Idee besteht darin, zwischen **unsicheren, verklemmungsbedrohten Zuständen** und **sicheren Zuständen** zu unterscheiden und immer nur solche Belegungen zuzulassen, die das System in sicheren Zuständen lassen. Was sind unsichere Zustände? Betrachten wir *sichere* Zustände als solche, bei denen es immer einen Weg (eine Schedulingstrategie) für die vorhandenen Prozesse gibt, um alle normal zu beenden, und *unsicherere* als solche, die nicht zwangsläufig zu einer Verklemmung führen müssen, aber können. Dann haben wir bereits einen Algorithmus kennengelernt, um dies zu testen: unseren Erkennungsalgorithmus für Verklemmungen.

Wir betrachten dazu die Betriebsmittelanforderungen der Prozesse im Algorithmus als Maximalforderungen, die alle auf einmal auftreten können, aber nicht müssen. Die Diagnose „Verklemmung“ des Testalgorithmus bedeutet nun, dass eine Verklemmung auftreten kann, aber nicht muss – es könnte ja sein, dass die restlichen Betriebsmittel von den anderen Prozessen einzeln nacheinander und nicht gleichzeitig gefordert und zurückgegeben werden. Im Unterschied zur Belegung im vorigen Abschnitt, die auf einmal realisiert und zurückgegeben wird, muss hier keine Verklemmung auftreten. Immerhin ist damit klar: Der Zustand ist verklemmungsbedroht und damit abzulehnen. Beispielsweise kann P_2 im obigen Beispiel einen Drucker zusätzlich bekommen, ohne dass das System verklemmungsbedroht wäre und damit den sicheren Zustand verlässt. Geben wir aber P_5 den letzten Drucker, so kann P_4 nicht mehr vollständig abgearbeitet werden: Das System ist verklemmungsbedroht und damit in einen unsicheren Zustand übergegangen. Die letzte der beschriebenen Zuteilungen sollte also vermieden werden. Dabei ist das System nicht verklemmt, sondern nur bedroht: Gibt z. B. P_1 seinen Drucker vor dem Ende wieder zurück, so droht keine Gefahr mehr.

Der Testalgorithmus wurde zuerst von Dijkstra (1965) angegeben und wird als „**Banker-Algorithmus**“ bezeichnet, da er das Verhalten eines Bankiers in einer Kleinstadt widerspiegelt, der mit begrenzten Betriebsmitteln (Kapital) versucht, die Kreditwünsche seiner Kunden zu befriedigen. Seine Strategie besteht darin, bei jedem Kreditwunsch eines Kunden zu prüfen, ob dann noch eine Reihenfolge besteht, um die Wünsche eines anderen Kunden im vollen, maximalen Kreditrahmen zu berücksichtigen und mit der anschließenden Rückzahlung die weiteren Wünsche abzudecken.

Die Anwendung dieses Algorithmus, um Verklemmungen zu vermeiden, bereitet allerdings praktische Probleme:

- Bei den meisten Anwendungen ist die Zahl der maximal nötigen Betriebsmittel unbekannt.
- Die Anzahl der Prozesse im System ist dynamisch und wechselt ständig.
- Die Anzahl der verfügbaren Betriebsmittel ist ebenfalls veränderlich.
- Der Algorithmus ist laufzeit- und speicherintensiv.

Aus diesen Gründen wird der bekannte Algorithmus in der Praxis kaum eingesetzt.

Verklemmungen unmöglich machen

Die Grundidee bei dieser Strategie, Verklemmungen zu behandeln, besteht darin, eine der vier auf Seite 101 genannten Bedingungen, die hinreichend und notwendig für eine Verklemmung sind, unmöglich zu machen. Dies lässt sich bei jeder Bedingung unterschiedlich durchführen.

(1) *Mutual Exclusion*

Die Konkurrenz für ein Betriebsmittel lässt sich abschaffen, indem man es beispielsweise einem einzigen, speziell dafür zuständigen Prozess übergibt und alle Anfragen für dieses Betriebsmittel als Dienstleistung an den Prozess umwandelt.

Ein Beispiel dafür ist der Drucker: Ein spezieller Prozess, der **Druckerdämon** (*printer demon*), erhält dauerhaft den Drucker; alle anderen Prozesse übergeben die auszudruckenden Daten diesem Prozess in eine Warteschlange. Der Druckerdämon arbeitet stellvertretend für die ihn beauftragenden Prozesse die Warteschlange ab (*spooling*), so lange ein Auftrag vorliegt, und legt sich dann bei leerer Warteschlange schlafen. Damit wird zunächst jede Verklemmung vermieden.

Diese Strategie ist nicht unproblematisch. Zum einen ist dies nicht für alle Betriebsmittel möglich (z. B. für Semaphore, die ja gerade für den wechselseitigen Aufruf da sind), zum anderen wird das Problem nur vorverlagert. In unserem Beispiel geht der wechselseitige Ausschluss vom Drucker auf den Druckerpuffer im Massenspeicher über, der als endlicher Platz ebenfalls ein Betriebsmittel darstellt. Haben wir zwei Prozesse, deren Pufferbedarf jeweils die Hälfte des verfügbaren Platzes überschreitet, so stellt sich hier wieder das gleiche Problem wie vorher. Jeder Prozess schreibt seine Daten in den Puffer, bis kein Platz mehr da ist. Da der Auftrag erst nach dem vollständigen Übermitteln der Daten in die Warteschlange eingehängt wird, ist die Auftragsschlange leer und beide Prozesse warten, bis wieder Platz verfügbar ist, um die Übermittlung abzuschließen – sie bleiben verklemmt. Aus diesem Grunde werden *spooling*-Systeme auch manchmal so programmiert, dass nur die Originaldatei zum Drucken verwendet wird und nicht vorher eine Kopie davon gemacht wird.

Allgemein ist aber der Übergang von der direkten Belegung auf ein Stellvertreter- und Auftragssystem (*client-server*) üblich.

(2) *Hold And Wait*

Können wir verhindern, dass die Prozesse zusätzliche Ressourcen anfordern, so verhindern wir Verklemmungen. Eine Möglichkeit dafür besteht darin, nur ein einziges Betriebsmittel pro Prozess zuzulassen. Dies ist aber inakzeptabel: Ein Prozess, der Daten von einem Band liest und sie ausdruckt, wäre damit verboten.

Eine andere Möglichkeit sieht vor, immer alle Betriebsmittel, die für den Prozess nötig sind, am Anfang des Prozesses auf einmal anzufordern. Dies erfordert allerdings zusätzlichen Programmieraufwand, da dies anfangs weder dem Prozess noch dem Betriebssystem, sondern nur dem Programmautor bekannt sein kann. Der Nachteil dieser Methode besteht darin, dass immer alle benötigten Betriebsmittel für die gesamte Laufzeit des Prozesses belegt sind und für keine anderen Prozesse mehr

zur Verfügung stehen. Ein Programm, das für fünf Stunden rechnet und dann einen kurzen Ausdruck macht, verhindert damit fünf Stunden lang jeglichen Ausdruck – ein inakzeptables Verhalten in einer Mehrbenutzerumgebung und außerdem eine schlechte Betriebsmittelauslastung.

Eine interessante Variante davon sieht vor, bei jeder zusätzlichen Betriebsmittelbelegung zunächst einmal alle bereits belegten Betriebsmittel zurückzugeben und dann in einer Anfrage alle für die nächste Phase benötigten Betriebsmittel anzufordern. Dies erhöht zwar den Belegungsaufwand, verhindert aber eine Verklemmung.

(3) *No Preemption*

Ein vorzeitiger Entzug eines Betriebsmittels ist, wie bereits angedeutet, nicht einfach so möglich, da bereits Veränderungen durchgeführt sein können, beispielsweise beim Schreiben von Daten oder beim Ausdrucken von Papier. Ein vorzeitiger Entzug der Betriebsmittel etwa durch eine Zeitüberwachung muss deshalb im Programm explizit mitberücksichtigt werden und kann nicht automatisch transparent für das Programm geschehen.

(4) *Circular Wait*

Eine Möglichkeit, die Zyklen zu durchbrechen, besteht darin, bereits bei der Anforderung den ersten Schritt zu einem Zyklus zu vermeiden und eine Ordnungsrelation für die Betriebsmittelanforderung zu definieren (*Linearisierung* der Betriebsmittelanforderung).

- Dazu werden alle Betriebsmittel durchnumeriert und einem Prozess nur gestattet, zusätzliche Betriebsmittel einer höheren Nummer zu belegen als er bereits hat. Dadurch kann kein Prozess auf ein früher durch einen anderen Prozess bereits belegtes Betriebsmittel mit einer kleineren Nummer warten – es ist prinzipiell nicht verfügbar.

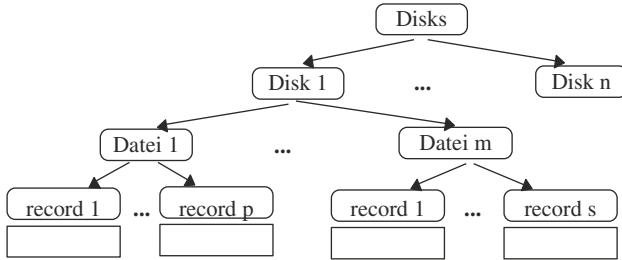
Mit dieser Annahme betrachten wir nun einen Zyklus. Schreiben wir in unserem Betriebsmittelgraphen in jedes Kästchen die Nummer, so können wir die in einem Zyklus durchlaufenen Betriebsmittelnummern notieren. Bei der obigen Einschränkung ist es nicht möglich, von den gehaltenen Betriebsmitteln zu den angeforderten Mitteln immer nur höhere Nummern zu erreichen – irgendwann schließt sich der Kreis, und dann folgt die kleinste Nummer als Anforderung auf eine größere, die belegt wurde. Dies widerspricht aber der Voraussetzung. Also kann sich kein Zyklus ausbilden.

Der Nachteil der Linearisierungsmethode liegt darin, dass so leicht keine Ordnung gefunden werden kann, die für alle Anwendungen brauchbar ist. Ist eine bestimmte gewünschte Anforderungsreihenfolge nicht möglich, so muss der Prozess alle benötigten Ressourcen statt dessen auf einmal anfordern, was wieder die Auslastung der Ressourcen verschlechtert.

- Eine weitere Möglichkeit bietet eine Betriebsmittelzuteilung, bei der die Betriebsmittel in einer Hierarchie angeordnet sind und jeweils bestimmten Verwaltungsprozessen (Druckerdaemon usw.) zugeordnet werden. Diese Prozesse erfüllen die Benutzeranforderungen und achten dabei auf die Verklemmungsfreiheit, wobei jeder Benutzerauftrag zeitlich begrenzt wird.

Die Gesamtheit der Betriebsmittel (z. B. alle Dateien) kann aufgeteilt und ihre Verwaltung Unterprozessen übertragen werden. Da dies auch für den Unterprozess gilt, entsteht so eine baumartige Hierarchie der Belegung, die verklemmungsfrei ist.

Beispiel Dateiverwaltung



Dabei wird jeder *record* nur von einem einzigen Prozess alloziert. Die Aufträge (schreiben, lesen) werden von der Wurzel an die Unterprozesse weitergeleitet. Die Baumstruktur garantiert, dass die Reihenfolge der Bearbeitungen auch bei den Unteraufträgen überall gleich ist. Dadurch bleiben die Zustände der *records* konsistent.

2.3.9 Aufgaben zu Verklemmungen

Aufgabe 2.3-10 Begriffe

- Was ist der Unterschied zwischen Blockieren, Verklemmen und Verhungern von Prozessen?
- Worin liegt der Unterschied zwischen aktivem (*busy wait*) und passivem Warten?
- Geben Sie ein praktisches Beispiel für eine Verklemmung an.

Aufgabe 2.3-11 Verklemmung/Blockierung

Ein Student S_1 hat ein Buch A in der Bibliothek ausgeliehen. In Buch A findet er einen Literaturhinweis auf Buch B. Deshalb möchte er auch Buch B ausleihen. Dies ist momentan von Student S_2 ausgeliehen, der wiederum in Buch B einen Verweis auf Buch A findet. Also versucht er, Buch A auszuleihen. Stellt diese Situation eine Verklemmung, eine Blockierung oder keines von beiden dar? Bitte begründen Sie Ihre Antwort.

Aufgabe 2.3-12 Monitore

- Implementieren Sie das Leser/Schreiber-Problem als Monitorlösung.
- Was sind die Vor- und Nachteile einer Monitorlösung?

Aufgabe 2.3-13 Banker-Algorithmus

- a) Welche der beiden Reihenfolgen wären im Beispiel auf Seite 106 für den Banker-Algorithmus für die fünf Prozesse auch möglich?
- b) Angenommen, Prozess P_1 bekommt ein Bandlaufwerk zusätzlich zugestanden. Ist das System dann verklemmungsbedroht?

Aufgabe 2.3-14 Verklemmungstest

Gegeben seien fünf Prozesse $P_1, \dots, P_5$ mit den angegebenen Anforderungen an die Betriebsmittel A, ..., E, wobei ein Eintrag „a/b“ bedeutet, dass der Prozess derzeit a Einheiten des Betriebsmittels hält und noch zusätzliche b Einheiten anfordert.

	Betriebsmittel					Betriebsmittel	maximal verfügbar
	A	B	C	D	E		
P1	1/0	2/4	1/1	1/0	1/0	A	4
P2	0/2	0/1	0/3	0/0	0/1	B	7
P3	2/0	0/2	1/2	0/2	3/2	C	3
P4	0/1	0/4	0/1	0/2	0/0	D	2
P5	1/3	1/3	0/1	1/1	1/4	E	5

Überprüfen Sie, ob in diesem Szenario eine Verklemmung vorliegt. Falls ja, so geben Sie an, wie diese behoben werden kann.

Aufgabe 2.3-15 Verklemmungsfreiheit

Beweisen oder widerlegen Sie:

In einem System mit $n > 1$ verschiedenen Betriebsmitteltypen, in dem jeder Prozess direkt nach seinem Start nacheinander (jedoch in beliebiger Reihenfolge) für jeden dieser Typen alle $m_i \geq 0$ maximal benötigten Betriebsmittel auf einmal anfordern muss und diese erst bei Prozessende wieder freigegeben werden können, sind Verklemmungen unmöglich.

Aufgabe 2.3-16 Sichere Systeme

- a) Ein Computersystem habe sechs Laufwerke und n Prozesse, wobei ein Prozess jeweils zwei Laufwerke benötigt. Wie groß darf n sein für ein sicheres System?
- b) Für große Werte für m Betriebsmittel und n Prozesse ist die Anzahl der Operationen, um einen Zustand als „sicher“ zu klassifizieren, proportional zu $m^n n^b$. Wie groß sind a und b ?

Aufgabe 2.3-17 Verklemmungen unmöglich machen

In einem Transaktionssystem einer Bank gibt es für jede der vielen parallel stattfindenden Überweisungen von Konto S nach Konto E einen eigenen Prozess. Die Lese-/

Schreiboperationen pro Konto müssen deshalb vor parallelem Zugriff durch einen anderen Prozess geschützt werden. Wird dazu ein Semaphore pro Konto verwendet, so kann es bei gleichzeitigen Rücküberweisungen leicht zu Verklemmungen kommen, da zuerst S und dann E von einem Prozess und zuerst E und dann S vom anderen Prozess belegt werden. Entwerfen Sie ein Zugriffsschema, um eine Verklemmung unmöglich zu machen.

Beachten Sie bitte, dass Lösungen, die zuerst das eine Konto sperren, es verändern sowie entsperren und dann das andere Konto belegen und verändern, gefährlich sind, da bei Computerstörungen der Überweisungsbetrag verschwinden kann. Diese Art von Lösungen scheidet also aus.

2.4 Prozesskommunikation

Ein wichtiges Mittel, um in einem Betriebssystem die Aktionen mehrerer Prozesse zu koordinieren und ein gemeinsames Arbeitsziel zu erreichen, ist der Nachrichtenaustausch. In den klassischen Betriebssystemen wurde dies sehr vernachlässigt; erst in UNIX wurden Programme als Bausteine angesehen, deren koordiniertes Zusammenwirken ein neues Programm ergeben kann. Die Mittel dafür waren in älteren UNIX-Versionen aber sehr beschränkt. Erst der Zwang, auch über Rechengrenzen hinweg in Netzen die Arbeit zu koordinieren, führte dazu, auch auf einem einzelnen Rechner die Interprozesskommunikation als Betriebssystemdienst zu ermöglichen.

In diesem Abschnitt sind deshalb die grundsätzlichen Überlegungen präsentiert, wie sie sowohl für Mono- als auch für Multiprozessorsysteme und Netzwerke zutreffen. Die stärker speziellen für Netzwerke zutreffenden Aspekte für die Betriebssysteme sind gesondert in [Kap. 6](#) beschrieben.

Der Nachrichtenaustausch zwischen Prozessen folgt dabei bestimmten Überlegungen.

2.4.1 Kommunikation mit Nachrichten

Bei dem Nachrichtenaustausch unterscheidet man drei verschiedene Arten der Verbindung: die Punkt-zu-Punkt-(**unicast**)-Verbindung von einem einzelnen Prozess zu einem anderen, die **multicast-Verbindung** von einem Prozess zu mehreren anderen und der Rundspruch (**broadcast-Verbindung**) von einem Prozess zu allen anderen. In [Abb. 2.32](#) sind diese Verbindungsarten visualisiert.

Dabei lässt sich jede Verbindungsart durch eine andere implementieren. Beispielsweise kann man einen *broadcast* und *multicast* durch mehrere *unicast*-Verbindungen realisieren oder durch eine Verbindung, bei der in der Adresse eine Liste aller Empfänger mitgeschickt wird und diese vom Empfänger benutzt wird, um die nächste Verbindung aufzubauen. Umgekehrt kann man eine Nachricht an alle Empfänger schicken, in der als Adressat nur ein einziger Empfänger angegeben ist; die anderen Empfänger sehen dies und ignorieren daraufhin die Nachricht.

Abb. 2.32 Verbindungsarten

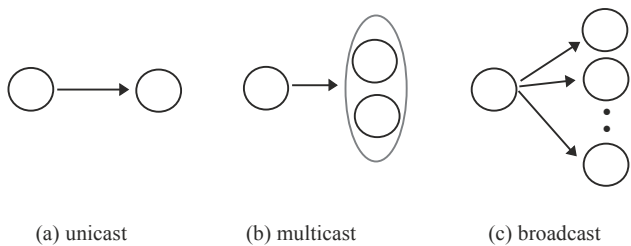


Abb. 2.33 Ablauf einer verbindungsorientierten Kommunikation

<code>openConnection (Adresse)</code>	Feststellen, ob der Empfänger existiert und bereit ist; Aufbau der Verbindung;
<code>send (Message) / receive (Message)</code>	Nachrichtenaustausch;
<code>closeConnection</code>	Leeren der Nachrichtenpuffer, Beenden der Verbindung.

Die Kommunikation kann man dabei verschieden implementieren. Traditionellerweise wird eine Kommunikation, ähnlich einer Telefonverbindung, durch einen Verbindungsaufbau charakterisiert (**verbindungsorientierte Kommunikation**) wie dies in [Abb. 2.33](#) gezeigt ist.

Der Aufwand für den Aufbau und den Unterhalt einer derartigen Verbindung (*Kanal*) ist allerdings ziemlich groß. Aus diesem Grund wurde eine andere Art von Nachrichtenaustausch modern: die **verbindungslose Kommunikation**. Im Unterschied zu dem **synchronen** Verbindungsaufbau, bei dem Sender und Empfänger ihre Bereitschaft für die Verbindung bekunden müssen, werden hierbei die Nachrichten **asynchron** verschickt:

```
send (Adresse, Message) / receive (Adresse, Message)
```

Da nun nicht mehr sichergestellt ist, dass die Nachricht auch tatsächlich ankommt (Übermittlung und Empfänger können gestört sein, was sowohl für verbindungslose als auch verbindungsorientierte Kommunikation gilt), wird normalerweise jede Nachricht quittiert. Trifft innerhalb einer Zeitspanne keine Quittungsnachricht ein, so sendet der Sender seine Nachricht erneut sooft, bis er eine Quittung erhält. Erst dann sendet er eine neue Nachricht. Ist die Quittungsnachricht verloren gegangen, so erhält der Empfänger mehrere Kopien der gleichen Nachricht. Um neue von alten Nachrichten unterscheiden zu können, sind die Nachrichten in diesem Fall mit einer fortlaufenden Nummer versehen. Eine Nachricht kann somit aus folgender Grundstruktur bestehen:

```
TYPE tMessage= RECORD
    EmpfängerAdresse:  STRING;
    AbsenderAdresse:   STRING;
    NachrichtenTyp:    tMsgTyp;
```

```

SequenzNummer    INTEGER;
Laenge:          CARDINAL;
Data:            POINTER TO tBlock;
END;
```

Dies wird auch als *Nachrichtenkopf* (*message header*) bezeichnet und entspricht einem Umschlag, ähnlich wie bei einem Brief. Dabei ist nur ein Minimum an Angaben gezeigt; reale Nachrichten zwischen Prozessen haben meist noch mehr Einträge. Man beachte, dass zwischen Sender und Empfänger Übereinstimmung herrschen muss, wie die Nachrichtenfelder zu interpretieren sind; sowohl der Datentyp (wie z. B. bei `tMsgTyp`) als auch die Reihenfolge der Felder und die Interpretation der Zahlen (kommt bei einem `INTEGER` zuerst das höchstwertige (*high order*) Byte oder zuerst das niedrigstwertige?). Aus diesem Grund werden beim Datenaustausch zwischen Rechnern verschiedenen Typs zuerst die Datenbeschreibungen und dann die Daten selbst geschickt, vgl. [Abschn. 6.2.3](#).

Die interne Verwaltung von Nachrichten verschiedener Länge ist dabei nicht so einfach. Zwar ist der Nachrichtenkopf immer gleich lang, aber die unterschiedliche Länge der Daten gestattet keine feste Längenangabe im `Data` Feld. Beim Empfänger muss deshalb zunächst der Nachrichtenkopf gelesen und dann erst der Speicherplatz für den Datenblock alloziert werden.

Das Senden bzw. Empfangen kann dabei in verschiedener Weise vorgenommen werden. Nach dem Senden wird entweder der Sender so lange verzögert, bis eine Rückmeldung erfolgt ist (blockierendes oder **synchrones** Senden), oder das System puffert die Nachricht (als Kopie oder direkt), kümmert sich um die korrekte Ablieferung und lässt den Sender weitermachen (nicht-blockierendes oder **asynchrones** Senden). Allerdings muss auch beim nicht-blockierenden Senden eine Fehlermeldung zurückgegeben werden, wenn der systeminterne Puffer überläuft, oder aber der Sender muss in diesem Fall verzögert werden.

Auch für das Empfangen und Lesen der Nachrichten gibt es blockierende und nicht-blockierende Versionen: Entweder wird der Empfänger verzögert, bis eine Nachricht vorliegt, oder aber der Empfänger erhält beim nicht-blockierenden Lesen eine entsprechende Rückmeldung, wenn keine Nachricht vorliegt.

Adressierung

Die Adressierung der Empfänger ist sehr unterschiedlich. Bei einer Punkt-zu-Punkt-Verbindung besteht die Adresse zunächst aus einer Prozessnummer. Ist der Prozess auf einem anderen Rechner, so kommt noch die Nummer oder der Name des anderen Rechners hinzu, eventuell auch die Firma, die Region und das Land, die man in der Zeichenkette des Namens durch Punkte voneinander trennen kann:

Adresse = Prozess-ID.RechnerName.Firma.Land
z. B. 5024.hera.rbi.uni-frankfurt.de

Für eine *multicast*-Verbindung benötigt man dann nur eine Liste aller Prozesse bzw. Rechner, die mit der Nachricht gleich mitgeschickt werden kann.

Allerdings ist eine solche Art von Adressierung sehr ungünstig, wenn die Kommunikationspartner sich nicht direkt kennen und ihre Prozess-ID wissen. Möchte beispielsweise ein Formatierungsprogramm einem Drucker die fertigen Daten zum Ausdrucken schicken, so kennt es den PID des Druckprozesses nicht, aber es weiß, dass ein Drucker existiert. Es ist deshalb besser, einen feststehenden logischen Empfängernamen „Drucker“ im System zu definieren, dessen Zuordnung zu einem aktuellen Prozess-ID je nach Systemzustand wechselt und in einer Zuordnungstabelle des Betriebssystemkerns beim Empfänger festgehalten wird. Alle Aufrufe zu „Drucker“ werden damit transparent für den sendenden Prozess an die richtige Adresse geleitet. Dieses Prinzip der logischen und nicht physikalischen Namensgebung kann man auch auf die allgemeine Kommunikation über Rechnernetze ausdehnen. Da die komplette Information über alle Prozesse, Rechner und Institutionen sinnvollerweise nicht auf jedem Rechner eines Netzes gehalten und aktualisiert werden sollte, gibt es für die Adressenzuordnung (Adressauflösung, *address resolution*) bei weiten Entfernungen besondere Rechner (**name server**), die dies durchführen, siehe [Abschn. 6.2](#).

Das bisher Gesagte gilt natürlich nicht nur für die *unicast*-Verbindung, sondern auch für die *multicast*-Verbindung, bei der mehrere Prozesse und Rechner in Gruppen zusammengefaßt und unter ihrem Gruppennamen adressiert werden können.

Eine besondere Variante stellt die bedingte Adressierung durch eine **Prädikatsadresse** dar. Hier gibt man ein logisches Prädikat an, das von logischen Bedingungen abhängt, beispielsweise vom Maschinen- oder CPU-Typ, freien RAM-Speicher, vorhandenen Peripheriegeräten usw. Der Empfänger evaluiert dies wie eine IF-Abfrage; ist das Ergebnis WAHR, so fühlt er sich angesprochen; ist es FALSCH, so kommt er als Empfänger nicht in Frage, und die Nachricht wird ignoriert. Dies ermöglicht es besser, dynamisch Arbeit zu verteilen und freie Peripherie zu nutzen.

Mailboxen

Beim asynchronen Nachrichtenaustausch müssen die Nachrichten, da der Empfänger sie nicht sofort liest, in einem Puffer zwischengelagert werden. Diese Nachrichtenpuffer können auch mit einem Namen versehen werden, so dass man den Prozess nicht mehr zu kennen braucht, dem man die Nachricht schickt. Auch die Systemverwaltung ändert sich; anstelle der Zuordnung von logischen Prozessnamen zu physikalischen Prozess-IDs wird eine Zuordnung von logischen Puffernamen zu physikalischen Pufferadressen durchgeführt.

Ein solcher Nachrichtenpuffer (**Mailbox**) kann auch als Warteschlange strukturiert werden, in die Nachrichten eingehängt und vom Empfänger ausgelesen werden. Sehen wir noch eine weitere Warteschlange für die Empfangsprozesse einer Prozessgruppe vor, falls keine Nachrichten verfügbar sind, sowie eine Warteschlange für die Sendeprozesse, falls die Nachrichtenschlange voll ist, so erhalten wir eine Mailbox mit der Struktur

```
TYPE Mailbox = RECORD
    SenderQueue :      tList;
    EmpfängerQueue :   tList;
    MsgQueue :        tList;
    MsgZahl :         INTEGER;
END
```

Der Zugriff auf die Warteschlangen (`einhängen(Msg)` und `aushängen(Msg)`) muss durch Semaphoreoperationen geschützt werden, so dass pro Mailbox ein Semaphore vorgesehen werden muss. Die Variable „`MsgZahl`“ dient dabei als Zähler zur Flusskontrolle innerhalb der kritischen Abschnitte `einhängen(Msg)` und `aushängen(Msg)`. Ist mit `MsgZahl=N` die maximale Kapazität der Mailbox erreicht, so wird der Sendeprozess eingehängt und schlafen gelegt; umgekehrt ist dies bei `msgZahl=0` für den Empfänger der Fall.

Wird also bei `MsgZahl=N` gelesen, so muss der Empfänger noch zusätzlich ein `wakeup(SenderQueue)` durchführen, um evtl. vorhandene Senderprozesse aufzuwecken. Entsprechend muss bei `MsgZahl=0` der Sendeprozess eventuell wartende Empfänger aufwecken.

Für die Koordination der Sende- und Empfangsprozesse ist es sinnvoll, die Prozeduren `einhängen(Msg)` und `aushängen(Msg)` ebenfalls in die Mailboxdeklaration aufzunehmen und den Zugang zur Mailbox nur über diese kontrollierten Operationen zu gestatten. Die gesamte Datenstruktur wird damit zu einem abstrakten Datentyp. In objektorientierten Sprachen gestattet die PUBLIC-Deklaration ein Verbergen der Datenstrukturen und Kapselung, so dass wir eine Datenstruktur erhalten, die ähnlich einem `MONITOR`-Konstrukt ist.

2.4.2 Beispiel UNIX: Interprozesskommunikation mit *pipes*

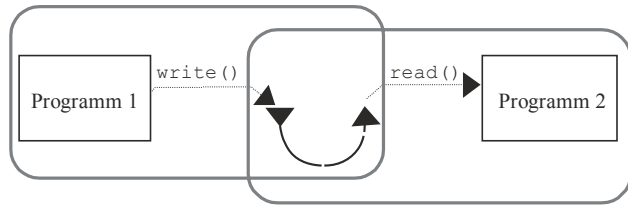
Die Interprozesskommunikation benutzt seit den ältesten Versionen von UNIX einen speziellen, gepufferten Kanal: eine **pipe**. Im einleitenden Beispiel von [Kap. 2](#) sahen wir, dass mehrere UNIX-Programme, in diesem Fall Prozesse, durch eine Anweisung

Programm1 | Programm2 | ... | Programm N

zusammenwirken können. Hier initiiert der senkrechte Strich | einen Übergabemechanismus der Daten von einem Programm zum nächsten, der jeweils durch eine solche *pipe* realisiert wird. Ein solcher Kommunikationskanal funktioniert folgendermaßen:

Mit dem Systemaufruf `pipe()` wird ein Nachrichtenkanal eröffnet, auf den mit den normalen Lese- und Schreiboperationen `read(fileId,buffer)` und `write(fileId,buffer)` zugegriffen werden kann. Für jede Kommunikationsverbindung eröffnet das Hauptprogramm (hier: die Benutzer-*shell*) eine eigene *pipe* und vererbt sie mit dem Prozesskontext an die Kindsprozesse weiter; die Sende-Kindsprozesse benutzen nur den `fileId` zum Schreiben und die Empfangsprozesse nur den zum Lesen. Da die Dateikennungen `fileId` Nummern von internen Tabellen sind, die beim `fork()`-Aufruf dem Kindsprozess vererbt werden, kann in UNIX durch *pipes* nur eine Interprozesskommunikation (IPC) zwischen Eltern- und Kindsprozessen eingerichtet werden; sie gehören derselben **Prozessgruppe** an. In [Abb. 2.34](#) sind zwei Prozesse gezeigt, die durch eine anonyme *pipe*, die sie von dem Elternprozess im Prozesskontext geerbt haben, miteinander kommunizieren. Die *pipe* ist dabei in der Schnittmenge der Prozesskontexte (graue Linien), dem noch gemeinsamen Teil des kopierten Prozesskontextes, enthalten.

Abb. 2.34 Interprozesskommunikation mit *pipes* in UNIX



Eine *pipe* ist in UNIX nur unidirektional; für einen Nachrichtenaustausch zwischen zwei Prozessen werden immer zwei *pipes* benötigt.

Im Normalmodus blockiert sich der Sendeprozess nur, wenn die *pipe* voll ist; der Leseprozess nur, wenn sie leer ist. Im nicht-blockierenden Modus erhält der Leseprozess eine Null zurück, wenn keine Nachricht vorlag; ansonsten die Anzahl der gelesenen Bytes, die in der Puffervariablen gespeichert wurden.

Eine IPC zwischen beliebigen Prozessen und über Rechnergrenzen hinweg ist erst bei neueren UNIX-Versionen über globale Namen möglich, die für eine *pipe* vergeben werden (**named pipes**), sowie über die sog. **Socket**-Konstrukte, die in [Abschn. 6.2](#) näher erläutert werden.

2.4.3 Beispiel Windows NT: Interprozesskommunikation mit *pipes*

Auch in Windows NT gibt es IPC mit *pipes*. Sie wird durch den Aufruf `CreatePipe()` eingerichtet. Danach kann man auch durch die normalen Lese- und Schreiboperationen `WriteFile()` und `ReadFile()` darauf zugreifen. Mit einem `CloseHandle()`-Aufruf wird sie abgeschlossen.

Da auch hier bei der namenslosen *anonymous pipe* der Zugriff auf diese Kanäle durch eine äußere Kennung nicht existiert, ist sie ebenfalls auf die Eltern/Kind-Prozessgruppe beschränkt.

Zusätzlich zu möglichen Statusänderungen *blocking/non-blocking* gibt es auch in Windows NT (bidirektionale) *named pipes* sowie Socket-Konstrukte, die eine Interprozesskommunikation über Rechnergrenzen hinweg ermöglichen.

2.4.4 Prozesssynchronisation durch Kommunikation

Im vorigen Abschnitt sahen wir, dass Semaphore als Synchronisationsprimitive zu elementar sind und Monitore meist nicht verfügbar. Beides ist in einer Mono- und Multiprozessorumgebung sinnvoll, nicht aber in einem Rechnernetz. Wir benötigen deshalb Synchronisationsprimitive, die auch in verteilten Systemen vorliegen. Dies lässt sich vor allem durch Nachrichtenaustausch mit sehr kurzen Nachrichten (**Signale**) und durch Warten auf das Senden einer Nachricht erreichen. Genau genommen besteht ja das Empfangen einer Nachricht aus zwei Teilen: dem Warten, d. h. Synchronisieren mit der Nachricht, und dem Lesen der Nachricht. Bei Nachrichten der Länge null bleibt also noch die Synchronisation dabei übrig. Die Funktionen `send(Message)` und

`receive(Message)` sind mit den Operationen `send(Signal)` und `waitFor(Signal)` für solche Nachrichten identisch. In beiden Fällen gehen wir davon aus, dass die übermittelten Nachrichten zwischengespeichert werden und vor dem Lesen nicht verloren gehen. Eine reine Synchronisation kann also leicht zur Kommunikation mit Nachrichten erweitert und umgekehrt eine allgemeine Kommunikation zur reinen Synchronisation benutzt werden. Betrachten wir zunächst die Synchronisation durch Signale.

Synchronisation durch Signale

Einer der wichtigsten Gründe, um Nachrichten zu empfangen, ist das Auftreten von Ereignissen. Dies sind Alarme über auftretende Fehler im System (z. B. Datenfehler, unbekannte Speicheradressen etc.), Ausnahmebehandlungen (*exceptions*, z. B. Division durch Null) und Signale anderer Prozesse. Dabei kann der empfangende Prozess entweder *synchron* auf das Ereignis warten, bis es eintrifft, oder aber nur die Verarbeitung der Ereignismeldung einrichten (initialisieren) und die Bearbeitung der *asynchron* eintreffenden Nachricht der angemeldeten Verarbeitungsprozedur überlassen.

Der zweite Fall ist besonders in Ausnahmebehandlungen üblich, die Prozessspezifisch behandelt werden müssen wie Division-durch-Null, Stacküberlauf, Verletzung von Feldgrenzen usw. Um der Vielzahl der unterschiedlichen Ausnahmebehandlungen gerecht zu werden, wird deshalb sinnvollerweise eine einzige Schnittstelle zum Betriebssystem definiert, die programmintern für jedes Modul extra initialisiert und aufgesetzt wird.

Beispiel UNIX Signale

In UNIX gibt es ein Signalsystem, mit dem die Existenz eines Ereignisses einem Prozess mitgeteilt werden kann. Dazu sind Signale definiert, von denen in [Abb. 2.35](#) einige übersichtsweise aufgelistet sind (s. *signal.h*):

Standardmäßig gibt es in UNIX mindestens 16 Signale, was durch die frühere Wortbreite des Signalregisters im PCB von 16 Bit bedingt war. Heutige Unixversionen haben 32 oder 64 Signale, je nach Wortbreite der verwendeten Hardware. Der allgemeine Betriebssystemdienst `send(Signal)` ist in UNIX aus dem Senden des Signals SIGKILL entstanden, um einen Prozess abzuberechnen, und heisst deshalb immer noch `kill()`. Die Verwendung der obigen Signale ist zwar, wie an ihrer Namensgebung zu bemerken, für bestimmte Dinge gedacht, aber mit den meisten Signalen (außer dem Abbruchsignal) lässt sich eine selbst definierte Prozedur mittels des Systemaufrufs `sigaction()` verknüpfen, die beim Auftreten eines Signals im Prozess (*software interrupt*) angesprungen wird, so dass sich sowohl beim Senden als auch beim Empfangen verschiedene Signale zur Kommunikation verwenden lassen.

Bei Signalen unterscheidet man noch zusätzlich, ob nur auf ein bestimmtes aus einer Menge gewartet werden soll, oder aber darauf gewartet wird, dass alle Ereignisse einer Menge stattgefunden haben. Die verschiedenen Betriebssysteme haben dazu unterschiedliche

Abb. 2.35 POSIX- Signale

SIGABRT	<i>abort process</i> : Aufforderung zum sofortigen Prozessabbruch
SIGTERM	<i>terminate</i> : geordnetes Beenden des Prozesses erwünscht
SIGQUIT	Aufforderung zum Prozessabbruch mit Speicherabzug (<i>core dump</i>)
SIG FPE	<i>floating point error</i>
SIGALRM	<i>alarm</i> -Signal: Zeituhr abgelaufen
SIGHUP	<i>hang up</i> : Telefonverbindung aufgelegt
SIGKILL	<i>kill</i> -Signal: bricht den Prozess auf jeden Fall ab
SIGILL	<i>illegal instruction</i> : nichtexistenter Maschinenbefehl
SIGPIPE	Es existiert kein Empfänger für <i>pipe</i> -Daten.
SIGSEGV	<i>segmentation violation</i> : nicht verfügbare Speicher-adresse
SIGINT	<i>interrupt</i> -Signal (CTRL C)
SIGSTOP	hält den Prozess an
SIGBUS	Busfehler

Dienste, bei denen mit UND oder ODER Bedingungen für die Prozessaktivierung durch Ereignisse bzw. Signale gesetzt werden können.

Beispiel

Die Ereignisse {MausDoppelClick, LinkeMausTaste, RechteMausTaste, AsciiTaste, MenuAuswahl, FensterKontrolle} erfordern sehr unterschiedliche Reaktionen. Üblicherweise erwarten die interaktiven Programme, beispielsweise die Manager der Fenstersysteme, eines der Ereignisse als Eingabe. Dazu setzen sie einen Aufruf (Warten auf Multi-Event) ab, bei dem angegeben wird, auf welche Ereignisse sie warten wollen. Warten sie auf einen Mausklick ODER einen ASCII-Tastendruck, so wird eine entsprechende Maske gesetzt. Aber auch eine UND-Maske ist möglich, wenn z. B. auf die Tastenkombination SHIFT-MausDoppelClick reagiert werden soll.

Die Signale können aber auch dazu benutzt werden, eine Prozesssynchronisation einzurichten. Beispielsweise lässt sich mit Hilfe von `send(Signal)` und `waitFor(Signal)` leicht die Semaphoroperation implementieren. In MODULA-2 ist dies auf einem Rechner durch die Typdefinition

```

TYPE      Semaphor = RECORD
                besetzt : BOOLEAN;
                frei    : SIGNAL;
            END;

```

und die Operationen

```
PROCEDURE P(VAR S:Semaphor);
BEGIN
    IF S.besetzt THEN waitFor(S.frei) END;
    S.besetzt:= TRUE;
END P;

PROCEDURE V(VAR S:Semaphor);
BEGIN
    S.besetzt:= FALSE;
    send(S.frei)
END V;
```

möglich. Das gesamte Semaphormodul muss allerdings eine höhere Priorität (Interrupt!) als die übrigen Prozesse aufweisen, um die $P()$ - und $V()$ -Operation atomar werden zu lassen.

Dies lässt sich entweder in MODULA-2 durch eine Prioritätsangabe bei der Moduldeklaration erreichen oder in einer anderen Sprache durch einen Aufruf `setPrio(high)` jeweils direkt nach `BEGIN`, ergänzt durch `setPrio(low)` jeweils direkt vor `END`.

Das damit errichtete Semaphorsystem hat allerdings den Nachteil, dass es nur zwischen Prozessen auf demselben Prozessor funktioniert, wo die Erhöhung der Priorität eine Unterbrechung ausschließen kann. Sowohl in Multiprozessorsystemen als auch in Mehrrechnersystemen müssen deshalb andere Mechanismen angewendet werden, um eine Synchronisation zu erreichen.

Synchronisation durch atomic broadcast

Ein wichtiger Aspekt der Synchronisierung durch Nachrichten ist die Frage, ob eine Nachricht überhaupt beim Empfänger angekommen ist oder nicht. Haben einige Empfänger sie erhalten, andere aber nicht, so ist es für den Sender schwierig, die Kommunikation richtig zu verwalten und mit seinen Nachrichten eine einheitliche, konsistente Datenbasis bei den Empfängern zu garantieren. Um diesen Zusatzaufwand zu verringern und gerade bei Transaktionen in verteilten Datenbanken den vollständigen Abschluss einer Transaktion zu gewährleisten, ist es deshalb sinnvoll, die *broadcast*-Nachrichtenübermittlung als *atomare Aktion* zu konzipieren. Ein **atomarer Rundspruch (atomic broadcast)** definiert sich durch folgende Forderungen (Cristian et al. 1985):

- Die Übertragungszeit der Nachrichten ist endlich.
- Entweder erhalten alle Empfänger die Nachrichten oder keiner.
- Die Reihenfolge der Nachrichten ist bei allen Empfängern gleich.

Die gleiche Reihenfolge der Nachrichten bei allen Empfängern unabhängig von bestehenden Kausalitäten („Totale Ordnung“) lässt sich als nicht-blockierender Grundmechanismus einer konsistenten Datenhaltung verwenden. Sind nämlich überall Reihenfolge und Inhalt der Nachrichten gleich, so resultieren bei gleichem Anfangszustand und gleichen Änderungen bei allen beteiligten Prozessen im Gesamtsystem die gleichen Zustände

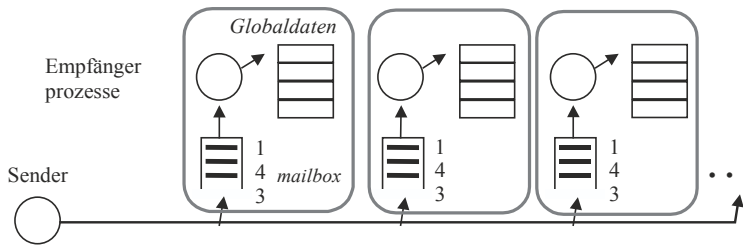


Abb. 2.36 Datenkonsistenz mit atomic broadcast-Nachrichten

der gemeinsam geführten Daten, also der verteilt aktualisierten globalen Variablen und Dateien, s. Dal Cin et al. (1987). In Abb. 2.36 ist dies visualisiert.

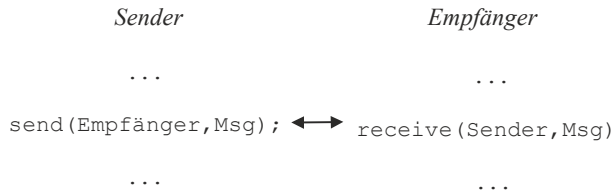
Man beachte, dass die Reihenfolge der Nachrichten 1, 4, 3 den Zustand der globalen Variablen festlegt, unabhängig von der Nummer der Nachricht, die ja von der Zählung des jeweiligen Senders abhängt. Ist der Sender auch Mitglied der Gruppe, so muss natürlich zur Datenkonsistenz innerhalb der Gesamtgruppe der Sender die Nachricht ebenfalls an sich senden und darf erst dann, beim Empfang der eigenen Nachricht, die globale Variable ändern. Tut er das vorher, so kann es sein, dass seine Nachricht bei den anderen erst nach einer bestimmten Änderungsmeldung ankommt, bei sich aber vorher und damit die Wirkung bei ihm anders ist als bei allen anderen Prozessoren: Eine Inkonsistenz ist da. Dies gilt beispielsweise für die Belegungswünsche verschiedener Prozesse für ein Betriebsmittel, die überall streng in der gleichen Reihenfolge abgearbeitet werden müssen, um überall den gleichen Zustand für die Belegungsvariable zu erzeugen.

Das obige Verfahren benötigt für die konsistente Reihenfolge der Nachrichten keine explizite globale Nummerierung der Nachrichten. Es vermeidet damit Probleme beim Nachrichtenaustausch zwischen wechselnden lokalen Gruppen. Man beachte allerdings, dass damit nicht unbedingt die Reihenfolge kausal aufeinander folgender Nachrichten garantiert wird („Kausale Ordnung“). Diese kausale Sequentialisierung des Nachrichtenverkehrs lässt sich nur durch kausale Zusatzbedingungen erreichen.

Die Frage der Konsistenz von Daten durch atomaren *broadcast* oder *multicast* spielt allgemein bei verteilten Systemen eine wichtige Rolle. Man denke dabei z. B. an verteilte Prozesse (Reihenfolge der Aktivierung), verteilten Speicher (Wer schreibt welche Daten in welcher Reihenfolge in den Speicher?) und dezentrale Synchronität (Jeder Rechner hat seine eigene, lokale Zeit. Was bedeutet „gleichzeitig“?). Mehr zu diesem Thema ist beispielsweise im Buch von F. Mattern (1989) zu finden.

Synchronisation von Programmen

Für das Senden und Empfangen von Signalen kann man auch eine ungepufferte Kommunikation verwenden. In diesem Fall wird das empfangende Programm so lange verzögert, bis das sendende Programm die Nachricht sendet (*synchrones bzw. blockierendes* Empfangen). Dies führt zu einer Synchronisierung zwischen Sender und Empfänger, die in manchen Prozesssystemen gewollt ist und unter dem Namen **Rendez-vous-Konzept**

Abb. 2.37 Das Rendez-vous-Konzept

bekannt ist. Die Programmiersprache ADA hat ein solches Konzept bereits im Sprachumfang enthalten und ermöglicht damit den Ablauf eines Programms aus kommunizierenden Prozessen auf allen Systemen, auf denen ein ADA-Compiler installiert ist. In [Abb. 2.37](#) ist der Synchronisationsverlauf gezeigt.

Ein anderes wichtiges Konzept ist die parallele Programmausführung durch kommunizierende Prozesse. Viele Programme können einfacher, erweiterbar und wartungsfreundlicher gestaltet werden, wenn man den Sachverhalt als eine Aufgabe formuliert, die von vielen kleinen, unabhängigen Spezialeinheiten erledigt wird, die miteinander kommunizieren.

Zerteilen wir also ein Programm in kurze Codestücke, z. B. *threads*, so benötigen diese Programmstücke auch eine effiziente Kommunikation, um die gemeinsame Aufgabe des Gesamtprogramms zu erfüllen. Das Kommunikationsmodell innerhalb eines Programms sollte dabei so einfach und klar wie möglich für den Programmierer sein, um Fehler zu vermeiden und eine effiziente Programmierung zu ermöglichen.

Zu diesem Zweck konzipierte Hoare ([1978](#)) sein sprachliches Modell der kommunizierenden sequentiellen Prozesse **CSP**, in der Ereignisse (z. B. Nachrichten) und Zustände eines endlichen Automaten miteinander verbunden werden. In CSP gibt es dazu die sprachlichen Konstrukte

$s \rightarrow A$ Auf ein Ereignis s folgt der Zustand A

$P = (s \rightarrow A)$ Wenn s kommt, so wird Zustand A erreicht, und P nimmt A an.

Für den parallelen Fall ist

$P = (s_1 \rightarrow A \mid s_2 \rightarrow B \mid \dots \mid s_n \rightarrow D)$

Wenn s_1 kommt, nimmt P den Wert A an. Wenn s_2 kommt, dann B , usw.

Jedes Prozessobjekt wird so durch Kommunikation in seinem Zustand gesteuert. Ähnliche Konstrukte wurden von Dijkstra ([1975](#)) vorgeschlagen als **bedingte Befehle** (*guarded commands*):

IF $C_1 \rightarrow \text{command1}$ $C_2 \rightarrow \text{command2}$ FI	IF $(a \geq b) \rightarrow \text{print „größer / gleich“}$ $(a < b) \rightarrow \text{print „kleiner“}$ FI
--	---

In dem obigen Beispiel ist die Sequenz von Befehlen durch einen IF-Block eingeraht. Innerhalb des Blocks wird nur derjenige Befehl ausgeführt, dessen logische Eingangsbedingung (*Wächter*) C_i erfüllt ist. Im Beispiel sind nur Bedingungen angegeben, die nicht gleichzeitig gelten können. Was passiert, wenn mehrere Bedingungen erfüllt sind? In diesem Fall wird zufällig eines der gültigen Befehle ausgewählt und die Kontrolle geht zum Ende des Blocks bei FI. Bei dem ähnlichen, von Dijkstra zusätzlich vorgeschlagenen DO/OD-Konstrukt werden die Bedingungen aller aufgeführten Kommandos in einer Endlosschleife solange durchgegangen, bis mindestens eine davon erfüllt ist und ein Kommando ausgeführt wurde.

Diese von Hoare und Dijkstra erdachten Konstruktionen von bedingten Befehlen wurden teilweise in die parallele Programmiersprache OCCAM (Inmos 1988) übernommen, die für die kommunizierenden Prozessoren (*Transputer*) der Firma Inmos geschaffen wurde. Die kommunizierenden Leichtgewichtsprozesse können dabei nur aus wenigen Anweisungen einer Zeile bestehen. Die Kommunikation zwischen den Prozessen wird durch Konstrukte der Form

```
Kanal!data1  für die Prozedur  send(Kanal, data1)
und Kanal?data2  für die Prozedur  receive(Kanal, data2)
```

Beide Kommunikationspartner, Sender und Empfänger, müssen zum Nachrichtenaustausch über den Kanal aufeinander warten. Dies entspricht dem Rendez-vous-Konzept, einer Kommunikation ohne Pufferung. Nach der Synchronisation wird die Datenzuordnung $data2 := data1$ durchgeführt, wobei $data1$ und $data2$ vom gleichen Datentyp sein müssen, beispielsweise INTEGER oder REAL. *Kanal* ist ein Name, der innerhalb des Programms deklariert wurde und bekannt ist.

Beispielsweise lässt sich das *Erzeuger-Verbraucher*-Modell in OCCAM als ein Senden vom *Erzeuger* an einen Pufferprozess darstellen, von dem der *Verbraucher* wiederum empfängt. Dies ist in Abb. 2.38 gezeigt.

Der Puffer wird als Ringpuffer mit 10 Elementen gekapselt in einem Prozess mit dem Code

```
CHAN OF item producer, consumer :
INT in, out :
SEQ
  in := 0
  out := 0
  WHILE TRUE
    ALT
      IF (in < out + 10) AND (producer ? item (in REM 10))
        in := in + 1
      IF (out < in) AND (consumer ? more)
        consumer ! item (out REM 10)
        out := out + 1
```

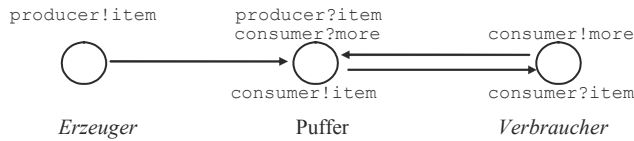


Abb. 2.38 Erzeuger-Verbraucher-Modell mit kommunizierenden Prozessen

Dabei ist der korrekte Index des Ringpuffers für die Ein- und Ausgabe jeweils mittels der Modulo-Operation REM (*remainder*) gegeben. Die Notation SEQ gibt eine sequentielle und ALT eine beliebige Reihenfolge an; nicht gezeigt wurde PAR, das eine parallele Ausführung aller Befehlszeilen spezifiziert.

Im Beispiel gibt es also zwei Kommandos: das Abspeichern im Ringpuffer beim Index *in*, und das Weitersenden eines *items* vom Index *out* zum Verbraucher.

Da nur eine Eingabe in der Bedingung abgefragt werden kann, ist ein zusätzliches Signal *more* des Verbrauchers nötig, um das nächste Datum aus dem Puffer abzurufen. Das Ganze wird ergänzt durch zusätzliche, hier nicht abgebildete Prozesse für den Erzeuger und den Verbraucher, die das Erzeugen bzw. Verbrauchen der *items* beinhalten.

Ein neueres Beispiel, das durch die kommunizierenden sequentiellen Prozesse CSP beeinflusst wurde, ist die Nebenläufigkeit in der **Programmiersprache GO**, die in Google 2009 von R. Griesemer, R. Pike, K. Thompson geschaffen wurde. Hier können Kanäle definiert werden, die als eigenständige Speicherbereiche automatisch durch Semaphore geschützt sind, und Threads, die mit einem Befehl `go` erzeugt werden. Ein Beispiel in Abb. 2.39 soll dies illustrieren.

Im Beispiel wird parallel zum Haupt-Thread *main* mittels `go` ein weiterer Thread erzeugt, der sequentiell eine Fibonacci-Folge errechnet und die Werte im *Integer*-Kanal *c* aus 10 Einträgen speichert. Nachdem der Kanal geschlossen wurde, ist er für den Leser freigegeben und kann sequenziell ausgelesen werden.

```
func fibonacci(n int, c chan int) {
    x, y := 0, 1           // Startwerte
    for i := 0; i < n; i++ { // Bis zur größten Zahl
        c <- x             // speichere im Kanal
        x, y = y, x+y      // Erzeuge eine Zahl
                            // aus 2 vorigen Zahlen.
    }                      // Sender schließt ab.
    close(c)
}

func main() {
    c := make(chan int, 10) // Erzeuge int-Kanal und
    go fibonacci(cap(c), c) // parallelen Thread.
    for i := range c {      // Wenn fertig,
        fmt.Println(i)      // drucke Kanal aus
    }
}
```

Abb. 2.39 Fibonacci-Folge mittels nebenläufiger Berechnung in GO

```

package main
import ( "fmt", "time" )

type Produkt struct {
id int          // einfache Beispiel-Datenstruktur
}

func main() {
/* erzeuge Puffer als Kanal vom Typ *Produkt (Kanal sendet
 * also Zeiger auf Produkte umher) mit Kapazität 10 */
buffer := make(chan *Produkt, 10)
/* go startet eine Goroutine */
go consume(buffer, 1)
go consume(buffer, 2)
go produce(buffer, 1)
go produce(buffer, 2)
/* Hauptthread noch ein wenig am Leben halten, damit nicht
 * alles sofort vorbei ist */
time.Sleep(60 * time.Second)
}

func consume(buffer chan *Produkt, nr int) {
for true { // ewige Schleife
// blockiere, bis Daten verfügbar
consumed := <-buffer
fmt.Println("consumer", nr, "consuming", consumed.id)
}
}

func produce(buffer chan *Produkt, nr int) {
for true { // ewige Schleife
fmt.Println("producer", nr, "producing")
p := new(Produkt)
p.id = nr
/* Produkt über den Kanal senden. Blockiert,
falls schon 10 Produkte im Kanal gepuffert sind,
bis jemand aus dem Kanal liest. */
buffer <- p
time.Sleep(1 * time.Second)
}
}

```

Abb. 2.40 Producer-Consumer-System in GO

Ein weiteres Beispiel in [Abb. 2.40](#) zeigt die Konstruktion eines *producer-consumer*-Systems.

Der reservierte Puffer umfasst 10 Einheiten und wird (verzögert) gefüllt. Man beachte, dass hier jeweils zwei Erzeuger und zwei Verbraucher aufgesetzt werden, die alle parallel agieren, ohne dass dafür explizit Semaphore aufgesetzt werden müssen.

Der obige Code hat beispielsweise folgende Ausgabe:

```

producer 1 producing
consumer 1 consuming 1
producer 2 producing
consumer 2 consuming 2

```

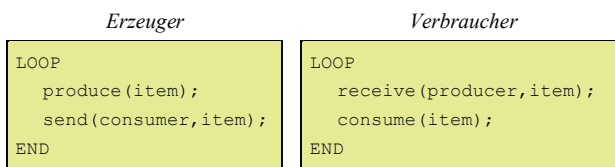
```

producer 2 producing
producer 1 producing
consumer 2 consuming 2
consumer 1 consuming 1
producer 1 producing
consumer 2 consuming 1
producer 2 producing
consumer 1 consuming 2
producer 2 producing
producer 1 producing
consumer 1 consuming 2
consumer 2 consuming 1
.....

```

2.4.5 Implizite und explizite Kommunikation

In unserem Beispiel des *Erzeuger-Verbraucher*-Modells aus [Abschn. 2.3.5](#) auf Seite 89 wurden Daten von einem Prozess über einen gemeinsamen Speicherbereich an einen anderen Prozess übergeben, der sie verarbeitet. Diese Informationsübergabe lässt sich als implizite Kommunikation auffassen, die man auch explizit formulieren kann: Anstatt `putInBuffer(item)` und `getFromBuffer(item)` verwendet man dann `send(consumer, item)` und `receive(producer, item)`



mit

- `send(consumer, item)` übergibt `item` an einen systeminternen Puffer, der mittels einer `P()`- und `V()`-Operation gefüllt wird. Ist der Puffer voll, so wird der Prozess verzögert, bis wieder Platz für das `item` vorhanden ist.
- `receive(producer, item)` liest ein `item`, geschützt durch `P()`- und `V()`-Operationen, aus dem Puffer. Ist kein `item` verfügbar, wird der Prozess so lange verzögert, bis eines eintrifft.

Diese Art des *Erzeuger-Verbraucher*-Modells hat den Vorteil, dass der Synchronisationsmechanismus des *Erzeuger-Verbraucher*-Modells mit dem Synchronisationsmechanismus des Botschaftenaustauschs implementiert werden kann und auch über Rechnergrenzen hinweg funktioniert.

Wir sparen uns also die Semaphoreoperationen in unserem Beispiel, indem wir entsprechende Funktionalitäten der Nachrichtenprozeduren `send(Msg)` und `receive(Msg)`

voraussetzen. Konkret lässt sich dies beispielsweise auf einem Monoprozessorsystem durch lokale Semaphoreoperationen implementieren. Der gemeinsame Pufferbereich ist in diesem Fall eine Mailbox, in die die Nachrichten eingehängt werden.

Allgemein gilt, dass fast alle Kommunikationsmechanismen, die zwischen Prozessen mit Hilfe gemeinsamer Speicherbereiche funktionieren (*shared memory*, s. [Abschn. 3.3.3](#)), besser durch expliziten Nachrichtenaustausch formuliert werden sollten. Der geringfügige Zusatzaufwand zahlt sich aus, wenn die darauf aufbauende Software nicht nur auf einem Monoprozessor, sondern auch in verteilten Systemen ablaufen soll. Hier kann man leicht von der Interprozesskommunikation zur Interprozessorkommunikation überwechseln, indem bei gleicher Schnittstelle eine andere, auf Netzkommunikation basierende Implementierung der `send()`- und `receive()`-Funktionen als Bibliothek verwendet wird.

2.4.6 Aufgaben zur Prozesskommunikation

Aufgabe 2.4-1 Prozesskommunikation

- a) Beschreiben Sie detailliert, was folgende Kommandozeile in UNIX bewirkt:

```
grep deb xyz | wc -l
```

- b) In UNIX ist die Kommunikation durch die Signale auf eine Eltern/Kind-Prozessgruppe beschränkt. Warum könnten die Implementatoren diese Einschränkung getroffen haben?
- c) Formulieren Sie den Übergang zwischen Prozesszuständen aus [Abschn. 2.1](#) mit Hilfe von Nachrichten und Mailboxen. Wer schickt an wen die Nachrichten?

Aufgabe 2.4-2 Pipes

Betrachten Sie ein Prozesssystem, das nur mit UNIX-pipes kommuniziert, deren Puffer aus einem gemeinsamen Speicherplatz alloziert werden. Jede *pipe* hat genau einen Sende- und Empfangsprozess.

- a) Unter welchen Umständen wird ein Prozess gezwungen, zu warten?
- b) Skizzieren Sie einen geeigneten Mechanismus, um für dieses System Verklemmungen festzustellen oder zu vermeiden.
- c) Gibt es eine Regel in diesem System, um den „Opferprozess“ auszuwählen, wenn eine Verklemmung existiert? Wenn ja, begründen Sie die Regel.

Literatur

Albers, S.: An experimental Study of new and known online packet buffering algorithms. *Algorithmica* 57:724–746 (2010)

- Albers, S.: Energy-efficient algorithms. *Commun. ACM* 53(5): 86–96 (2010)
- Brinch Hansen P.: Structured Multiprogramming. *Commun. of the ACM* 15(7), 574–578 (1972)
- Brinch Hansen, P.: *Operating System Principles*. Prentice Hall, Englewood Cliffs, NJ 1973
- Coffman E. G., Elphick M. J., Shoshani A.: System Deadlocks. *ACM Computing Surveys* 3, 67–78 (1971)
- Cooling J.: *Real-time Operating Systems*. Lindentree Associates, 2014
- Cristian F., Aghili H., Strong R., Dolev D.: Atomic Broadcast: From Simple Message Diffusion to Byzantine Agreement. *IEEE Proc. FTCS-15*, pp. 200–206 (1985)
- Dal Cin M., Brause R., Lutz J., Dilger E., Risse Th.: ATTEMPTO: An Experimental Fault-Tolerant Multiprocessor System. *Microprocessing and Microprogramming* 20, 301–308 (1987)
- Dijkstra E. W.: Cooperating Sequential Processes, Technical Report (1965). Reprinted in Genuys (ed.): *Programming Languages*. Academic Press, London 1985
- Dijkstra E. W.: Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, August 1975
- Furht B., Grostick D., Gluch D., Rabbat G., Parker J., McRoberts M.: *Real-Time UNIX System Design and Application Guide*. Kluwer Academic Publishers, Boston 1991
- Gonzalez, M.: Deterministic Processor Scheduling. *ACM Computing Surveys* 9(3), 173–204 (1977)
- Gottlieb A., Lubachevsky B., Rudolph, L.: Coordinating Large Numbers of Processors. *Proc. Int. Conf. on Parallel Processing*, 1981
- Gottlieb A., Grishman R., Kruskal C., McAuliffe K., Rudolph L., Snir M.: The NYU Ultracomputer-Designing an MIMD Shared Memory Parallel Computer. *IEEE Transactions on Computers* C-32(2), 175–189 (1983)
- Graham R. L.: Bounds on Multiprocessing Anomalies and Packing Algorithms. *Proc. AFIPS Spring Joint Conference 40*, AFIPS Press, Montvale, N.J., pp. 205–217 (1972)
- Habermann A. N.: Prevention of System Deadlocks. *Commun. of the ACM* 12(7), 373–377, 385 (1969)
- Hewlett-Packard: *How HP-UX Works*. Manual B2355-90029, Hewlett Packard Company, Corvallis, OR 1991
- Hoare C. A. R.: Towards a Theory of Parallel Programming. In Hoare, C. A. R., Perrot, R. H. (Eds): *Operating System Techniques*. Academic Press, London 1972
- Hoare C. A. R.: Monitors: An Operating System Structuring Concept. *Commun. of the ACM* 17(10), 549–557 (1974)
- Hoare C. A. R.: Communicating Sequential Processes. *Commun. of the ACM* 21(8), 666–677 (1978), auch unter <http://www.usingcsp.com/>
- Holt R. C.: Some Deadlock Properties of Computer Systems. *ACM Computing Surveys* 4, 179–196 (1972)
- IEEE: Threads extension for portable operating systems. P1003.4a, D6 draft, Technical Committee on Operating Systems of the IEEE Computer Society, Report-Nr. ISC/IEC JTC1/SC22/WG15N-P1003, New York, 1992
- INMOS Ltd.: *occam® 2 Reference Manual*. Prentice Hall, Englewood Cliffs, NJ 1988
- Keedy J. L.: On Structuring Operating Systems with Monitors. *ACM Operating Systems Rev.* 13(1), 5–9 (1979)
- Lampson B.W., Redell D. D.: Experience with Processes and Monitors in Mesa. *Commun. of the ACM* 23(2), 105–117 (1980)
- Laplanche Ph.: *Real-Time Systems Design and Analysis*. IEEE Press 1993
- Lister A.: The Problem of Nested Monitor Calls. *ACM Operating Systems Rev.* 11(3), 5–7 (1977)
- Liu C.L., Layland J.W.: Scheduling Algorithms for Multiprogramming in Hard Real-Time Environment. *Journal of the ACM* 20(1), 46–61 (1973)
- Liu J.W.S., Liu C.L.: Performance Analysis of Multiprocessor Systems Containing Functionally Dedicated Processors. *Acta Informatica* 10(1), 95–104 (1978)
- Love R.: *Linux Kernel Development*, Sams Publ. 2003, ISBN 0-672-32512-8

- Mattern F.: Verteilte Basisalgorithmen, Informatik-Fachberichte 226, Springer-Verlag Berlin, 1989
- Maurer C.: Grundzüge der Nichtsequentiellen Programmierung, Springer Verlag Berlin Heidelberg 1999
- Maurer W.: Linux Kernelarchitektur – Konzepte, Strukturen, Algorithmen von Kernel 2.6, Carl Hanser Verlag München, 2004, ISBN 3-446-22566-8
- Muntz R., Coffman, E.: Optimal Preemptive Scheduling on Two Processor Systems. IEEE Trans. on Computers C-18, 1014–1020 (1969)
- Northrup Ch.: Programming with Unix Threads. John Wiley & Sons, New York 1996
- Peterson G. L.: Myths about the Mutual Exclusion Problem. Information Processing Letters 12, 115–116 (1981)
- Robinson, J.T.: Some Analysis Techniques for Asynchronous Multiprocessor Algorithms. IEEE Trans. on Software Eng. SE-5(1), 24–30 (1979)
- Stankovic J., Ramamritham K.: The Spring Kernel: A New Paradigm for Real Time Systems. IEEE Software 8(3), 62–72 (1991)
- Stankovic J., Spuri M., Ramamritham K., Buttazzo G.: Deadline Scheduling for Real-time Systems, Kluwer Academic Publishers, 1998
- Teorey T. J., Pinkerton T. B.: A Comparative Analysis of Disk Scheduling Policies. Commun. of the ACM 15(3), 177–184 (1972)
- Wörn H., Brinkschulte U.: Echtzeitsysteme, Springer Verlag, Berlin 2005

Inhaltsverzeichnis

3.1 Direkte Speicherbelegung 122

3.1.1 Zuordnung durch feste Tabellen. 123

3.1.2 Zuordnung durch verzeigerte Listen 123

3.1.3 Belegungsstrategien 125

3.1.4 Aufgaben zur Speicherbelegung 128

3.2 Logische Adressierung und virtueller Speicher 129

3.2.1 Speicherprobleme und Lösungen. 129

3.2.2 Der virtuelle Speicher 130

3.3 Seitenverwaltung (*paging*) 132

3.3.1 Prinzip der Adresskonversion 132

3.3.2 Adresskonversionsverfahren 133

3.3.3 Gemeinsam genutzter Speicher (*shared memory*) 137

3.3.4 Virtueller Speicher in UNIX und Windows NT 138

3.3.5 Aufgaben zu virtuellem Speicher. 142

3.3.6 Seitenersetzungsstrategien 144

3.3.7 Modellierung und Analyse der Seitenersetzung 151

3.3.8 Beispiel UNIX: Seitenersetzungsstrategien 159

3.3.9 Beispiel Windows NT: Seitenersetzungsstrategien 161

3.3.10 Aufgaben zur Seitenverwaltung 162

3.4 Segmentierung 164

3.5 Cache 167

3.6 Speicherschutzmechanismen 170

3.6.1 Speicherschutz in UNIX 171

3.6.2 Speicherschutz in Windows NT 172

3.6.3 Sicherheitsstufen und *virtual mode* 173

Literatur 174

Eines der wichtigsten Betriebsmittel ist der Hauptspeicher. Die Verwaltung der Speicherressourcen ist deshalb auch ein kritisches und lohnendes Thema, das über die Leistungsfähigkeit eines Rechnersystems entscheidet. Dabei unterscheiden wir drei Bereiche, in denen unterschiedliche Strategien zur Speicherverwaltung angewendet werden:

- *Anwenderprogramme*

Die Hauptaufgabe besteht hierbei darin, interne Speicherbereiche des Prozesses (beispielsweise den Heap) mit dem Wissen um die speziellen Speicheranforderungen des Programms optimal zu verwalten. Dies erfolgt durch spezielle Programmteile, z. B. den Speichermanager, oder durch sog. *garbage collection*-Programme.

- *Hauptspeicher*

Hier besteht das Problem in der optimalen Aufteilung des knappen Hauptspeicherplatzes auf die einzelnen Prozesse. Die Aufgabe wird in der Regel durch spezielle Hardwareeinheiten unterstützt. Besonders in Multiprozessorsystemen ist dies wichtig, um Konflikte zu vermeiden, wenn mehrere Prozessoren auf dieselben Speicherbereiche zugreifen wollen. In diesem Fall helfen Techniken wie *Non-Uniform Memory Access* (NUMA) weiter, die zwischen einem Zugriff des Prozessors auf lokalen und auf globalen Speicher unterscheiden.

- *Massenspeicher*

Abgesehen von der Dateiverwaltung, die im nächsten [Kap. 4](#) besprochen wird, gibt es auch spezielle Dateien (z. B. bei Datenbanken), bei denen der Platz innerhalb der Datei optimal eingeteilt und verwaltet werden muss. Ein Beispiel dafür ist die Swap-Datei, auf die Prozesse verlagert werden, die keinen Platz mehr im Hauptspeicher haben.

Betrachten wir zunächst die direkten Techniken der Speicherbelegung, die meist in Anwenderprogrammen und kleinen, einfachen Betriebssystemen Verwendung finden.

3.1 Direkte Speicherbelegung

In den frühen Tagen der Computernutzung war es üblich, für jeden Job den gesamten Computer mit seinem Speicher zu reservieren. Der Betriebssystemkern bestand dabei oft aus einer Sammlung von Ein- und Ausgabeprozeduren, die als Bibliotheksprozeduren zusätzlich an den Job angehängt waren. Musste man einen Job zurückstellen, um erst einen anderen durchzuführen, so wurden alle Daten des dazugehörigen Prozesses auf den Massenspeicher verlagert und dafür die neuen Prozessdaten vom Massenspeicher in den Hauptspeicher befördert. Dieses Ein- und Auslagern (**swapping**) benötigt allerdings Zeit. Nehmen wir beispielsweise eine mittlere Zugriffszeit der Festplatte von 10 ms an und transferieren die Daten direkt (DMA-Transfer) mit einer Transferrate von 500 KB/s, so benötigen wir für ein 50 KB großes Programm 110 ms um es auf die Platte zu schreiben, und die gleiche Zeit, um ein ähnliches Programm dafür von der Platte zu holen, also zusammen 220 ms, die für die normale Abarbeitung verlorengehen. Das *swapping* behindert also das Umschalten zwischen verschiedenen Prozessen sehr stark.

Aus diesem Grund wurde es üblich, die Daten mehrerer Prozesse gleichzeitig im Speicher zu halten, sobald genügend Speicher vorhanden war. Allerdings brachte dies weitere Probleme mit sich: Wie kann man den Speicher so aufteilen, dass möglichst viele Prozesse Platz haben? Diese Frage stellt sich prinzipiell nicht nur für den Hauptspeicher, sondern gilt natürlich auch für den Auslagerungsplatz auf der Festplatte, den *swap space*.

3.1.1 Zuordnung durch feste Tabellen

Die einfachste Möglichkeit besteht darin, ein verkleinertes Abbild des Speichers zu erstellen: die **Speicherbelegungstabelle**. Jede Einheit einer solchen Tabelle (beispielsweise ein Bit) ist einer größeren Einheit (z. B. einem 32-Bit-Wort) zugeordnet. Ist das Wort belegt, so ist das Bit Eins, sonst Null.

In Abb. 3.1 ist eine solche Zuordnung gezeigt. Als Speichereinheit wurde ein größeres Stück (z. B. 4 KiB) gewählt. Die Anordnung der Datenblöcke A, B, C, beispielsweise Programme, wird vom Betriebssystem verwaltet.

Eine solche Belegungstabelle benötigt also bei 32 MiB Speicher selbst 1 MiB. Soll ein freier Platz belegt werden, so muss dazu die gesamte Tabelle auf eine passende Anzahl von aufeinanderfolgenden Nullen durchsucht werden.

3.1.2 Zuordnung durch verzeigerte Listen

Der belegte bzw. freie Platz ist meist wesentlich länger als die Beschreibung davon. Aus diesem Grund lohnt es sich, anstelle einer festen Tabelle eine Liste von Einträgen über die Speicherbelegung zu machen, die in der Reihenfolge der Speicheradressen über Zeiger miteinander verbunden sind. In Abb. 3.2 ist dies für das obige Beispiel aus Abb. 3.1 dargestellt.

Ein Eintrag der Liste besteht aus drei Teilen: der Startadresse des Speicherteils, der Länge und dem Index (Zeiger) des nächsten Eintrags.

Eine solche Belegungsliste lässt sich auch in zwei Listen zerteilen: eine für die belegten Bereiche, deren Einträge im Fall von Prozessen mit dem Prozesskontrollblock PCB verzeigert sind, und eine nur für die unbelegten Bereiche. Ordnen wir diese Liste noch zusätzlich nach

Abb. 3.1 Eine direkte Speicherbelegung und ihre Belegungstabelle

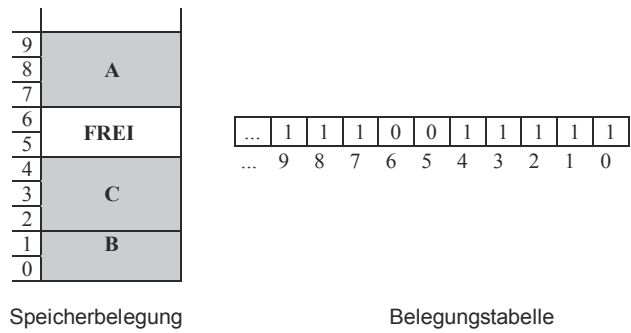
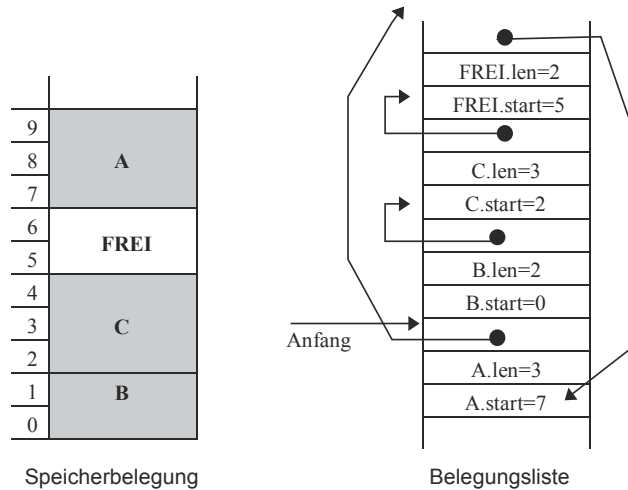


Abb. 3.2 Eine Speicherbelegung und die verzeigte Belegungsliste



der Größe der freien Bereiche, so muss normalerweise nie die ganze Liste durchsucht werden. Allerdings wird auch dadurch das Verschmelzen angrenzender, freier Bereiche erschwert. Eine doppelte Verzeigerung ermöglicht das Durchsuchen der Liste in beiden Richtungen.

Beispiel *Heap management*

Eine listenbasierte Speicherverwaltung ist beispielsweise für das Speichermanagement eines Heaps sinnvoll, der vom jeweiligen Benutzerprogramm verwaltet wird. In MODULA-2 bedeutet dies beispielsweise, eine alternative Implementierung der Prozeduren `ALLOCATE(.)` und `DEALLOCATE(.)` zu schreiben. Der freie Platz kann durch Einträge der Art

```

TYPE      EntryPtr  = POINTER TO EntryHead;
          EntryHead = RECORD
                                next:  EntryPtr;
                                size:  CARDINAL;
                                END;
VAR       rootPtr:   EntryPtr;
          nextStart: EntryPtr;
          FirstFree, NoOfEntryHeads: CARDINAL;
```

verwaltet werden, die als verkettete Liste miteinander verbunden sind; eingeleitet durch den Anker, einem Zeiger namens `rootPtr`. Jeder freie Platz wird am Anfang mit einem solchen Eintrag überschrieben. Ein freier Platz für die Liste kann also nicht kleiner als die Länge `TSIZE(EntryHead)` eines Eintrags sein. Wird ein freier Platz registriert, so wird

ein neuer Eintrag vom Typ `EntryHead` darin erstellt und in die *frei*-Liste eingehängt; ist der Platz zu klein, so wird er ignoriert.

Es ist einfacher und schneller, nur eine einzige Liste zu führen, die Liste der freien Plätze. Allerdings ist es sicherer, auch eine Liste der belegten Plätze zu führen, um bei der Platzrückgabe die Angaben über die Lage und die Länge des zurückgegebenen Platzes überprüfen zu können und so Fehlprogrammierungen des Benutzerprogramms aufzudecken. Besonders in der Debugging-Phase ist dies durchaus sinnvoll; eine solche Überprüfung kann dann in späteren Phasen bei der Laufzeitoptimierung abgeschaltet werden.

3.1.3 Belegungsstrategien

Unabhängig von dem Mechanismus der Speicherbelegungslisten gibt es verschiedene Strategien, um aus der Menge der unbelegten Speicherbereiche den geeignetsten auszusuchen. Ziel der Strategien ist es, die Anzahl der freien Bereiche möglichst klein zu halten und ihre Größe möglichst groß.

Die wichtigsten Strategien sind

- *FirstFit*
Das erste, ausreichend große Speicherstück, das frei ist, wird entsprechend belegt. Dies bringt meist ein Reststück mit sich, das unbelegt übrig bleibt.
- *NextFit*
Die *FirstFit*-Strategie führt dazu, dass in den ersten freien Bereichen nur Reststücke (*Verschnitt*) übrigbleiben, die immer wieder durchsucht werden. Um dies zu vermeiden, geht man wie *FirstFit* vor, setzt aber beim nächsten Mal die Suche an der Stelle fort, an der man aufgehört hat.
- *BestFit*
Die gesamte Liste bzw. Tabelle wird durchsucht, bis man ein geeignetes Stück findet, das gerade ausreicht, um den zu belegenden Bereich aufzunehmen.
- *WorstFit*
Sucht das größte vorhandene freie Speicherstück, so dass das Reststück möglichst groß bleibt.
- *QuickFit*
Für jede Sorte von Belegungen wird eine extra Liste unterhalten. Dies gestattet es, schneller passende Freistellen zu finden. Werden beispielsweise in einem Nachrichtensystem regelmäßig Nachrichten der Länge 1 KiB verschickt, so ist es sinnvoll, eine extra Liste für 1-KiB-Belegungen zu führen und alle Anfragen damit schnell und ohne Verschnitt zufriedenzustellen.
- *Buddy-Systeme*
Den Gedanken von *QuickFit* kann man nun dahin erweitern, dass für jede gängige Belegungsgröße, am besten Speicher in den Größen von Zweierpotenzen, eine eigene Liste vorsieht und nur Speicherstücke einer solchen festen Größe vergibt. Alle Anforderungen

müssen also auf die nächste Zweierpotenz aufgerundet werden. Ein Speicherplatz der Länge 280 Bytes = $256 + 16 + 8 = 2^8 + 2^4 + 2^3$ Bytes muss also auf $2^9 = 512$ Bytes aufgerundet werden.

Ist kein freies Speicherstück der Größe 2^k vorhanden, so muss ein freies Speicherstück der Größe 2^{k+1} in zwei Stücke von je 2^k Byte aufgeteilt werden. Beide Stücke, die **Partner (Buddy)**, sind genau gekennzeichnet: Ihre Anfangsadressen sind identisch bis auf das k -te Bit in ihrer Adresse, das invertiert ist. Beispiel: ... XYZ0000 ... und ... XYZ1000 ... sind die Anfangsadressen von Partnern.

Dies lässt sich ausnutzen, um sehr schnell (in einem Schritt!) prüfen zu können, ob ein freigewordenes Speicherstück einen freien Partner in der Belegungstabelle hat, mit dem es zu einem (doppelt so großen) Stück verschmolzen werden kann, ohne freie Reststücke zu hinterlassen.

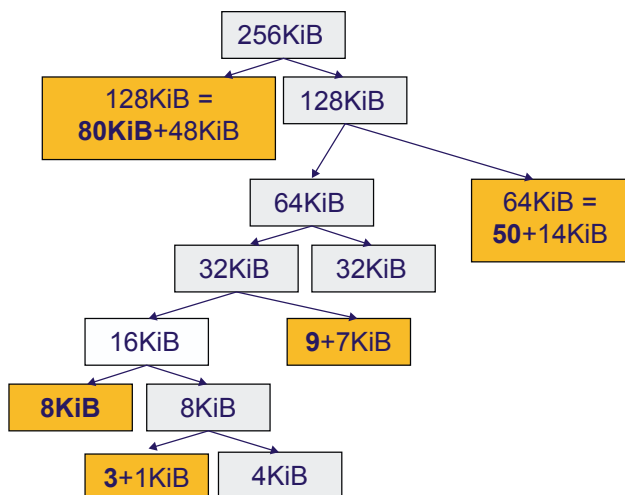
Beispiel Buddy-Speicherzuweisung

Angenommen, wir benötigen Speicherstücke der Größen 80 KiB, 8 KiB, 9 KiB, 3 KiB und 50 KiB in der genannten Reihenfolge von einem Speicherstück von 256 Kibibyte. Dann lässt sich die Zuordnung ganz gut mit Hilfe eines Binärbaumes visualisieren, bei dem jedes Speicherstück durch seine beiden *Buddy* notiert ist, siehe Abb. 3.3. Da es sich jedes Stück in zwei Unterstücke aufspalten lässt, bilden alle Speicherstücke zusammen einen Binärbaum.

Man sieht, dass jedes Speicherstück, das benutzt wird, meist noch ein unbenutztes Reststück, den Verschnitt, enthält.

Beide Vorgänge, sowohl das Suchen eines passenden freien Stücks (bzw. das dafür nötigen Auseinanderbrechen eines größeren) als auch das Zusammenfügen zu größeren Einheiten lässt sich rekursiv über mehrere Partnerebenen (mehrere Zweierpotenzen) durchführen, siehe Knolton (1965).

Abb. 3.3 Visualisierung einer Speicherzuteilung im Buddy-System



Bewertung der Strategien

Die vergleichende Simulation der verschiedenen Strategien führte zu einer differenzierten Bewertung. So stellte sich heraus, dass *FirstFit* von der Platzausnutzung etwas besser als *NextFit* und *WorstFit* ist und überraschenderweise auch besser als *BestFit*, das dazu neigt, nur sehr kleine, unbelegbare Reste übrig zu lassen. Eine Anordnung der Listenelemente nach Größe der Bereiche erniedrigt dabei die Laufzeit von *BestFit* und *WorstFit*.

Wissen wir mehr über die Verteilung der Speicheranforderungen nach Zeit oder Beleggröße, so können mit *QuickFit* und anderen, speziellen Algorithmen bessere Resultate erzielt werden.

Die Leistungsfähigkeit des *Buddy*-Systems lässt sich mit folgender Rechnung kurz abschätzen: Nehmen wir an, für den Hauptspeicher der Größe 2^n werden alle 2^n möglichen Stückgrößen S daraus mit gleicher Wahrscheinlichkeit verlangt (was sicher nicht vorkommt, aber mangels plausiblerer Annahmen vorausgesetzt sei). Dann ist die mittlere Speicheranforderung $\langle S(n) \rangle$ mit

$$\langle S(n) \rangle = \sum_{S=1}^{2^n} P(S)S = \frac{1}{2^n} \sum_{S=1}^{2^n} S = \frac{1}{2^n} \cdot \frac{2^n(2^n + 1)}{2} = 2^{n-1} + 1/2 \approx 2^{n-1},$$

$$\text{da } \sum_{i=1}^m i = \frac{m(m+1)}{2}$$

Für die tatsächliche Belegung muss auf Zweierpotenzen aufgerundet werden, d. h. bei der Anforderung eines Stückes der Größe $S_a = 2^{k-1} + \text{Rest}$ wird stattdessen ein Stück der Größe $S_b = 2^k$ reserviert. Der Rest besteht im Erwartungswert nach obiger Rechnung näherungsweise aus der Hälfte des 2^{k-1} großen Stückes. Das Verhältnis zwischen der mittleren Anforderung S_a und tatsächlicher Belegung S_b ist somit für die Speichergröße der Stufe k

$$\text{Effizienz} = \frac{S_a}{S_b} = \frac{2^{k-1} + 2^{k-2}}{2^k} = \frac{2 \cdot 2^{k-2} + 2^{k-2}}{4 \cdot 2^{k-2}} = \frac{3}{4} = 75\%$$

Das Ergebnis unserer Überschlagsrechnung: Ein Viertel des Platzes wird unabhängig von k auf jeder Stufe ungenutzt dazugeschlagen, so dass dies für das ganze *Buddy*-System gilt. Dadurch charakterisiert sich das Partnersystem als ein Belegungsverfahren, das zwar schnell, aber nicht sehr effizient ist. Die Ursache liegt in der groben Partitionierung durch Speicherplatzverdoppelung. Korrigiert man dies, indem man nicht nur gleich große, sondern auch halbe oder viertel Partner zulässt, so erhöht sich der Ausnutzungsgrad für den Speicher; die Verwaltung wird aber auch komplizierter, siehe Peterson und Norman (1977).

Fragmentierung und Verschnitt

Obwohl im allgemeinen die Belegungslisten eine gute Zuordnung der Belegungswünsche zu den freien Speicherstücken gestalten, neigen trotzdem die direkten

Speicherbelegungsverfahren dazu, den Speicher in viele kleine, unbelegbare Reststücke zu zersplittern (**Fragmentierung**). Dies geschieht unabhängig davon, ob es bei der programminternen Speicherbelegung (z. B. Heap) vom **internen Verschnitt** herrührt oder bei der Speicherzuweisung für das gesamte Programm im Hauptspeicher von den freien Stücken zwischen den Programmen, dem **externen Verschnitt**. Aus diesem Grund ist die Verschmelzung freier Bereiche eine der wichtigsten Funktionen der Speicherverwaltung.

Für das spezielle Problem der Speicherzuweisung bei der Einlagerung von Prozessen vom Massenspeicher gibt es mehrere, spezielle Strategien. Belegen wir den Speicher zuerst mit den größten Prozessen, um die kleineren Prozesse zum „Stopfen“ der Lücken zu verwenden, so resultiert dies in einer Schedulingstrategie, die gerade das Gegenteil von der *Shortest-Job-First*-Strategie bedeutet und damit besonders lange Laufzeiten garantiert.

Abweichend davon kann man ein faires Scheduling für jede Prozessgröße erreichen, indem man den Gesamtspeicher in eine feste Anzahl unterschiedlich großer Partitionen unterteilt. Jede Partitionsgröße erhält dann ihre eigene Warteschlange, so dass die Speicherzuordnungstabellen fest sind und eine Fragmentierung nicht auftritt. Eine solche Lösung war beispielsweise im IBM-Betriebssystem OS/MFT (*Multiprogramming with Fixed number of Tasks*) für das OS/360 (Großrechnersystem) vorhanden. Natürlich bringt ein solches starres, inflexibles System eine schlechte Betriebsmittelauslastung mit sich.

3.1.4 Aufgaben zur Speicherbelegung

Aufgabe 3.1-1 Belegungsstrategien

Gegeben sei ein Swapping-System, dessen Speicher aus folgenden Löchergrößen in ihrer Speicherreihenfolge besteht: 10 KiB, 4 KiB, 20 KiB, 18 KiB, 7 KiB, 9 KiB, 12 KiB und 15 KiB. Welches Loch wird bei der sukzessiven Speicherplatzanforderung von 12 KiB, 10 KiB, 9 KiB mit *First-Fit* ausgewählt? Wiederholen Sie die Anforderungen für *Best-Fit*, *Worst-Fit* und *Next-Fit* (Skizze).

Aufgabe 3.1-2 Speicherreservierung im Buddy-System

Ein Buddy-System verwalte einen Speicher der Größe 256 KiB. Zu Beginn ist lediglich ein Speichersegment von 50 KiB genutzt, was durch das Buddy-System mit einem Segment der Größe 64 KiB (und somit 14 KiB Verschnitt) bedient wurde:

0 KiB	128 KiB	256 KiB
50+14 KiB	64 KiB frei	128 KiB frei

Erfüllen Sie nun die folgenden Anfragen in der angegebenen Reihenfolge und zeichnen Sie den Speicher nach jedem Schritt neu:

17 KiB allozieren, 33 KiB allozieren, 50 KiB freigeben, 17 KiB freigeben, 30 KiB allozieren, 48 KiB allozieren, 60 KiB allozieren, 48 KiB freigeben, 33 KiB freigeben, 78 KiB allozieren, 50 KiB allozieren, 32 KiB allozieren.

Falls sich eine Anforderung nicht erfüllen lassen sollte, so machen Sie dies deutlich. In unserem Szenario wollen wir annehmen, dass der anfragende Prozess in diesem Fall eine Fehlermeldung erhalten würde.

Geben Sie den Gesamtverschnitt am Ende aller Anfragen an und zeichnen Sie den dazugehörigen Binärbaum.

3.2 Logische Adressierung und virtueller Speicher

Bedingt durch die schlechte Speicherausnutzung der Speicherverwaltung gab es verschiedene Bemühungen, neue Lösungen für diese Probleme zu finden.

3.2.1 Speicherprobleme und Lösungen

Eine Lösungsmöglichkeit für das Problem der Speicherfragmentierung besteht in der Kompaktifizierung des freien Speichers durch „Zusammenschieben“ der belegten Bereiche. Diese Lösung ist allerdings sehr aufwendig und verlangt für die praktische Anwendung eine spezielle Hardware.

Reloizierung von Programmcode

Dabei tritt außerdem ein Problem in den Vordergrund, das bisher noch nicht angesprochen wurde: das Problem der „absoluten Adressierung“. Beim Binden der übersetzten Programmteile weist der Binder allen Sprungbefehlen und Variablenreferenzen eine eindeutige, absolute Speicheradresse zu, wobei immer von einer Basisadresse, beispielsweise Null, im Programm hochgezählt wird. Möchte man nun ein Programm in einem anderen Speicherstück mit anderen Adressen benutzen, als für das es gebunden wurde, müssen die Adressreferenzen vorher geeignet bearbeitet (**reloziert**) werden. Dafür gibt es verschiedene Lösungen:

- Der erzeugte Programmcode darf nur relative Adressen (z. B. Adresse = absolute Adresse – Programmzähler PC) enthalten. Dies ist nicht immer möglich für alle Prozessortypen und alle Befehle und verlangt zusätzliche Adressarithmetik zur Laufzeit.
- Die Verschiebungsinformation wird zusätzlich beim Programm auf der Platte gespeichert. Beim Laden in den Hauptspeicher muss jede Adressreferenz einmal errechnet und dann an die entsprechende Stelle geschrieben werden. Dies erfordert bis zum doppelten Speicherplatz auf dem Massenspeicher und ist für *swapping* nicht geeignet. In einigen Kleinrechnern (z. B. ATARI ST) ist es implementiert.
- Die Verschiebungsinformation ist als Basisadresse in einem speziellen Hardwareregister der CPU vorhanden und wird bei jedem Zugriff genutzt.

Der erhöhte Aufwand, dieses Problem zu lösen, muss nun für die Lösung anderer Probleme weiter vergrößert werden.

Speicherzusatzbelegung

So ist zum Beispiel unklar, wie die Anforderungen eines Prozesses behandelt werden sollen, der bereits im Hauptspeicher ist und zusätzlichen Speicher benötigt. Lösungen dafür sind:

- Der anfordernde Prozess wird stillgelegt, seine neue Speichergröße wird notiert, und er wird ausgelagert. Beim nächsten Mal, wenn er eingelagert wird, berücksichtigt man bereits die neue Größe und schafft damit Raum für die zusätzlichen Anforderungen. Dies ist die Strategie der alten UNIX-Versionen.
- Die freien Bereiche (externer Verschnitt) zwischen den Prozessen im Hauptspeicher werden dem jeweiligen Prozess hinzugeschlagen. So wird der freie Platz nach „unten“ (zu kleineren Adressen) durch die Stackerweiterung und nach „oben“ durch die Heapvergrößerung besetzt.

Speicherschutz

Ein weiteres Problem ist die Tatsache, dass von dem Speicherplatz ein bestimmter Teil dauerhaft vom Betriebssystemkern, den Systempuffern usw. belegt ist. Dieser Teil darf weder freigegeben noch irrtümlich vom Benutzerprogramm verwendet werden. Dazu werden in vielen Betriebssystemen spezielle Speicheradressen (*fences, limits*) vorgesehen, die nicht über- bzw. unterschritten werden dürfen. Sorgt ein spezielles Hardwareregister im Prozessor dafür, dass dies bei der Adressierung auch beachtet wird, so müssen die Grenzen nicht als Konstanten fest in der Adressierlogik eingebaut werden.

Ein ähnliches Problem ist der Schutz des Speichers, den der Prozess eines Benutzers verwendet, vor den Programmen der anderen Benutzer. Nicht nur aus Datenschutzgründen, sondern auch aus praktischen Überlegungen heraus sollten fehlerhafte Programme anderer Nutzer die eigene Arbeit nicht stören dürfen. Der Zugriff eines Prozesses auf den Speicher eines anderen Prozesses muss also verhindert werden.

Zu den bisher genannten Problemen kommt nun noch die Erkenntnis, dass es wichtig ist, die Adressierung auf einem Rechner für den Programmierer so einfach wie möglich zu machen und damit eine bessere Portabilität und Wartungsfreundlichkeit für die Programme zu erreichen. Der gesamte rechnerspezifische Adressierungsaufwand soll versteckt werden zugunsten eines klaren, einfachen, abstrakten Speichermodells: des virtuellen Speichers.

3.2.2 Der virtuelle Speicher

Das Wunschbild des Programmierers besteht in einem Speicher, der mit der Adresse Null anfängt und sich kontinuierlich bis ins Unendliche erstreckt. Leider ist dem aber in der Realität nicht so: Meist ist in den unteren Speicherbereichen, wie erwähnt, das Interruptsystem und das Betriebssystem untergebracht; die Speicherbereiche sind nicht kontinuierlich, da andere Programme noch im Speicher vorhanden sind und dynamisch Platz anfordern und zurückgeben; und letztendlich spielt eine Rolle die Tatsache, dass Hauptspeicher (noch) teuer ist und deshalb meist nicht genug vorhanden ist, um alle Programme gleichzeitig zu versorgen.

Aus diesen Gründen hat sich das Konzept des **virtuellen Speichers** entwickelt, bei dem die Wunschvorstellung des Programmierers als Zielvorgabe dem System aus Prozessorhardware und Betriebssystem vorgegeben wird. In den meisten heutigen Betriebssystemen gibt es dazu zwei Dienstleistungen, die von der Hardware gemeinsam mit dem Betriebssystem für die Speicherverwaltung bereitgestellt werden müssen:

- Mehrere Fragmente (Speicherbereiche) müssen für das Programm so dargestellt werden, als ob sie von einem kontinuierlichen Bereich, anfangend bei null, stammen.
- Verlangt das Programm mehr Speicher als vorhanden, wird nicht das ganze Programm, sondern es werden nur inaktive Speicherbereiche auf einen Sekundärspeicher (Massenspeicher, z. B. Magnetplatte) ausgelagert (*geswapped*) und die frei werdenden Bereiche zur Verfügung gestellt.

In der folgenden **Abb. 3.4** ist links der gewünschte, virtuelle Speicherbereich gezeigt, der dem Programm von der Speicherverwaltung vorgespiegelt wird, und rechts der tatsächliche, **physikalische Speicher**.

Die Aufgabe, aus der logischen die physikalische Adresse zu generieren, muss sehr schnell gehen und wird deshalb im Programmablauf für jede Speicherreferenz von einer speziellen Hardwareeinheit, der **Memory-Management-Unit MMU**, durchgeführt. In vielen modernen Prozessoren, so z. B. im Motorola MC68040 und im Intel Pentium Prozessor, ist die MMU bereits auf dem Prozessorchip enthalten.

Welche Mechanismen gibt es nun für die gewünschte Abbildung von virtuellem auf tatsächlich vorhandenen Speicher?

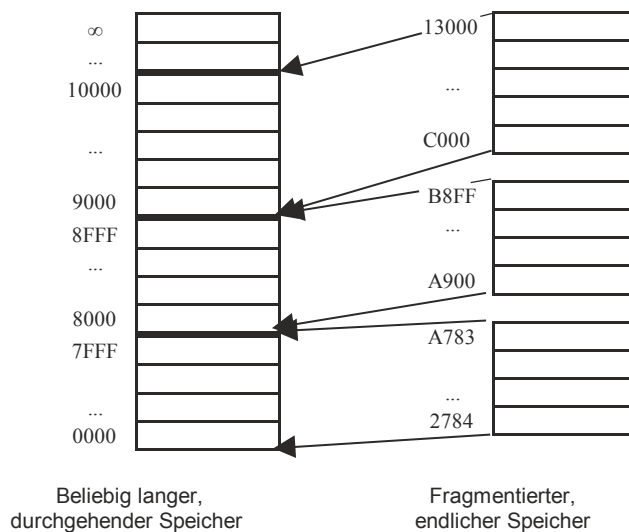


Abb. 3.4 Die Abbildung physikalischer Speicherbereiche auf virtuelle Bereiche

3.3 Seitenverwaltung (paging)

Einer der einfachsten Mechanismen der Implementierung eines virtuellen Adressraums besteht darin, den Speicher in gleich große Einheiten, sogenannte **Seiten (pages)**, einzuteilen. Übliche Seitengrößen sind z. B. 1 KiB, 4 KiB oder 8 KiB. Die Adresse und der Zustand jeder Seite werden in einer **Seitentabelle (page table)** geführt, die für jedes Programm im Hauptspeicher vorhanden ist.

3.3.1 Prinzip der Adresskonversion

Zur Umrechnung der im Programm verwendeten virtuellen Adresse auf die tatsächliche physikalische Adresse des Hauptspeichers wird die virtuelle Adresse zunächst in zwei Teile geteilt, s. Abb. 3.5. Der eine Teil mit den geringstwertigen Bits (*Least Significant Bits* LSB) wird meist *offset* genannt und stellt den relativen Abstand der Adresse zu einer Basisadresse dar. Den Wert dieser Basisadresse erhält man, indem der zweite Teil (links in der Abbildung) mit den höchstwertigen Bits (*High Significant Bits* HSB) als ein Index verwendet wird (in unserem Beispiel die Zahl 6), der einen Eintrag in der Seitentabelle bezeichnet. Dieser Eintrag enthält die gesuchte Basisadresse. Die vollständige physikalische Adresse erhält man durch Verkettung der Basisadresse mit dem *offset*. In Abb. 3.5 ist dieser zweistufige Übersetzungsprozess, bei dem für jeden Prozess andere Seitentabellen

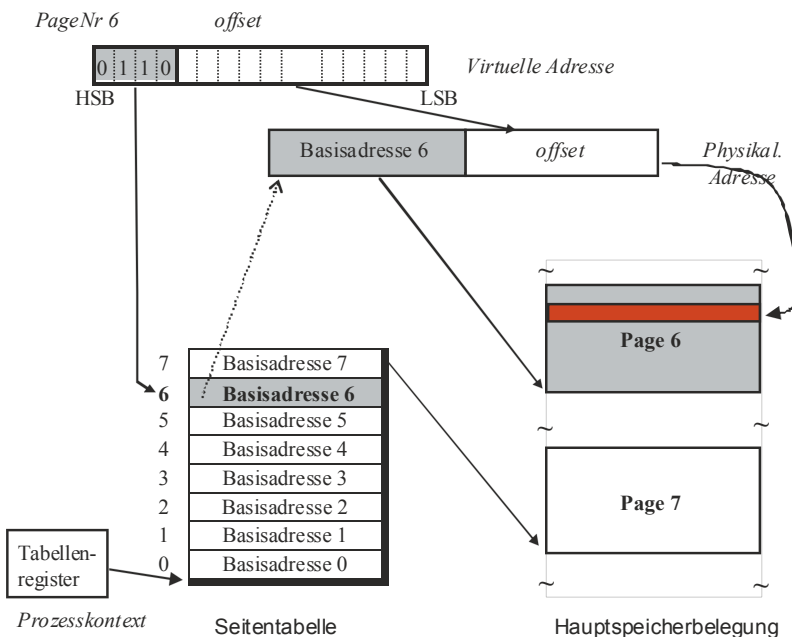


Abb. 3.5 Die Umsetzung einer virtuellen Adresse in eine physikalische Adresse

benutzt werden, visualisiert. Dabei ist die Grenze zwischen *PageNr* und *offset* innerhalb der Binärzahl der virtuellen Adresse abhängig von der jeweils verwendeten Hardware.

Man beachte, dass der tatsächlich vorhandene Hauptspeicher zwar ebenfalls in Seiten (Speicherpartitionen) eingeteilt sein kann, die effektive Lage und Größe der virtuellen Seite durch den Adressierungsmechanismus aber unabhängig davon ist. Die dadurch definierte, physikalische Speicherportion wird deshalb auch als **Seitenrahmen (page frame)** bezeichnet. Um die Übersetzung möglichst schnell durchzuführen, wird meist die Prozessspezifische Adresse der Seitentabelle in einem speziellen Register gehalten; die gesamte Adresszusammensetzung findet in der MMU statt.

Für die Auslagerung und Ergänzung der Seiten ist in hohem Maße das Betriebssystem zuständig. Zeigt das Statusbit einer Seite an, dass sie nicht im Speicher ist, so muss sie erst vom Massenspeicher übertragen werden. Dazu generiert die MMU ein Signal „**Seitenfehler**“ (**page fault**) in Form einer Programmunterbrechung, d. h. eines Interrupts. Dieser Interrupt wird vom Betriebssystem behandelt, das in der Interruptroutine eine wenig benutzte Seite auswählt, sie auf die Platte zurückschreibt, dafür die benötigte Seite einliest und die Tabelle entsprechend korrigiert. Anschließend wird nach dem Rücksprung vom Interrupt der Maschinenbefehl mit der vorher fehlgeschlagenen Adresskonversion wiederholt und dann das Programm weiter ausgeführt.

3.3.2 Adresskonversionsverfahren

Bisher betrachteten wir den Fall, dass eine beliebige Adresse des virtuellen Adressraums auf eine korrespondierende Adresse des physikalischen Raums direkt abgebildet wurde. Dies ist nicht immer möglich. Nehmen wir beispielsweise an, dass für jede Seite ein Eintrag in der Seitentabelle nötig ist, so ergeben sich bei einer 16-Bit-Wortbreite und einer Seitengröße von 12 Bit $\cong 4$ KiB genau $16 - 12 = 4$ Bit $\cong 16$ Einträge. Eine solche Tabelle ist leicht zu handhaben und zu speichern und war deshalb früher bei den PDP-11-Rechnern der Fa. Digital weit verbreitet. Schwieriger wird es nun bei 32-Bit-Wortbreiten, bei denen $20 \text{ Bit} \approx 10^6 = 1$ Million Einträge nötig sind. Bei der neuesten 64-Bit-Architektur mit $52 \text{ Bit} = 4 \cdot 10^{15}$ Seiteneinträgen stoßen wir allerdings an die Grenzen des Systems: Dies ist mehr, als Neuronen in einem Gehirn sind, und sogar mehr als die öffentliche Verschuldung in der BRD.

Das Problem rührt daher, dass für den gesamten virtuellen Adressraum eine Aussage über eventuell vorhandenen, korrespondierenden physikalischen Speicher gemacht werden muss, obwohl für den größten Teil der virtuellen Adressen überhaupt kein Speicher da ist. Dieses Dilemma lässt sich auf verschiedene Weise lösen.

- *Adressbegrenzung*

Die erste Idee, die man zur Lösung dieses Problems haben kann, besteht in der Begrenzung des virtuellen Adressraums auf eine maximal sinnvolle Größe. Begrenzen wir beispielsweise den Adressraum auf 30 Bit, was einem virtuellen Speicher von ca. 1 GB

pro Prozess entspricht, so benötigen wir bei 4 KiB = 12 Bit Seiteneinteilung eine Seitentabelle mit 18 Bit $\cong 256$ K Einträgen pro Prozess, was durchaus im Bereich des Möglichen ist.

Allerdings ist diese Maximalgröße unabhängig von der tatsächlichen Größe des Prozesses. Für die meisten (!) Prozesse ist sie viel zu groß; der Tabellenplatz wird unnütz verschenkt: Für einen 1 GB großen Prozess ist eine 1 MB große Seitentabelle vielleicht angemessen, nicht aber für einen kleinen Prozess von 50 KB!

Auch für den Compiler ist die Adressbeschränkung nicht sinnvoll: Für das Anwachsen des Stacks ist es vorteilhaft, ihn von sehr hohen Adressen nach unten wachsen zu lassen und damit von vornherein eine große Lücke zwischen Stack und der untersten möglichen Adresse vorzusehen.

- **Multi-level-Tabellen**

Aus diesen Gründen haben sich andere Ansätze durchgesetzt. Eine der wichtigsten Ideen versucht, die Information, *ob* ein Speicherbereich überhaupt genutzt wird, von der Information, *wie* bei einem genutzten und existierenden Bereich die Adresse transformiert wird, getrennt zu halten und damit Platz zu sparen. Dazu wird die Gesamtadresse in mehrere Teile unterteilt.

Beispiel Adressunterteilung

32-Bit-Wort, Seitengröße = 8 KiB $\cong 13$ Bit

<i>ungenutzt</i>										Tab 1	Tab 2	<i>offset</i>																0
										0	0	1	0	1	0	0	0	1	0	0	0	0	1	0	0	0	0	0

Aufteilung von 32 Bit in 13 bei diesem Beispiel ungenutzten Bits, einer 3-Bit-Tabelle1, einer 2-Bit-Tabelle2 und 13-Bit-*offset*. Die Adresse $41488_{10} = 121020_8$ teilt sich also auf in einen Index = 1 für Tabelle 1, Index = 1 für Tabelle 2 und einen *offset* = 528.

Jeder Adressteil erhält eigene Tabellen, wobei bei der Tabelle für die hohen Adressen (*page base table*) nur vermerkt ist, ob für diese virtuelle Adresse Speicher existiert und wenn ja, wo die dazugehörige Tabelle (*page table*) der Seitenbelegung steht. In [Abb. 3.6](#) ist dies für die zweigeteilte Adresse (zweistufige Tabelle) gezeigt, wobei die Suche durch die Tabellen mit dickeren Pfeilen angedeutet ist.

Jeder Teil der Adresse wirkt als Index in derjenigen Tabelle, die durch gestrichelte Pfeile angedeutet ist; der *offset* ist ein Index auf der tatsächlich existierenden Seite im Hauptspeicher. Da jede Adresse zu einer Seite gehören kann, die nicht im Hauptspeicher ist, kann auch die Adresse der ersten Tabelle zu einem *page fault* führen. In diesem Fall wird dann die Seite eingelagert, die die gewünschte Tabelle enthält und dann die Instruktion erneut wiederholt, bis die eigentliche virtuelle Seite ermittelt ist, sie sich im Speicher befindet, die physikalische Adresse ermittelt und die gewünschte Speicherzelle angesprochen wurde.

Unter Linux wird ein zweistufiges Tabellensystem ähnlich wie in [Abb. 3.6](#) verwirklicht. Im 32-Bit i386-Linux sind die beiden Tabellen durch je 10 Bit indiziert; der

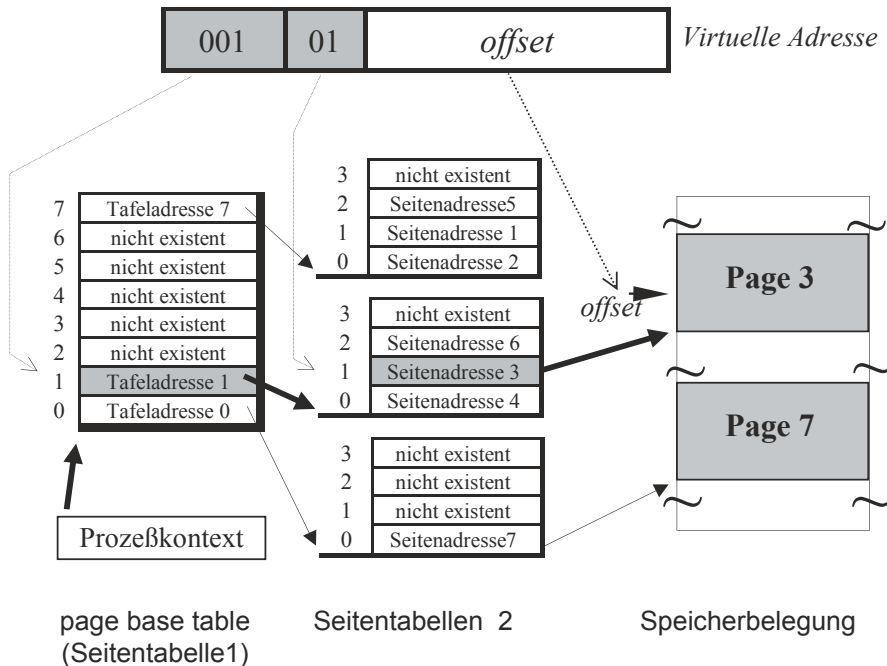


Abb. 3.6 Eine zweistufige Adresskonversion

12-Bit *offset* entspricht gerade einer 4KiB-Seitengröße. Die erste Tabelle der Größe 1KiB enthält 256 Adressen auf die 256 Tabellen á 4 KiB der zweiten Stufe, die zusammen 1 MB belegen und die Verweise auf die max. 1 Mill. Seiten des max. 4 GiB großen Speichers enthalten.

Analog existiert ein solcher Ansatz auch für dreistufige (SPARC Rechner von SUN) und vierstufige Seitentabellen (MC 68030 von Motorola und AMD Athlon 64), wobei das sequentielle Durchsuchen mehrerer Verweise, nur um eine Adresse herauszufinden, auch bei einer Hardwareunterstützung sehr lange dauert und deshalb die Programmausführung stark verzögert, beim AMD Athlon beispielsweise um ca. 80 %. Hier kommen spezielle Cache-Techniken zum Einsatz, die wir uns im Folgenden genauer ansehen.

Besitzen Betriebssysteme intern ein bestimmtes Zugriffsmodell, so stellt es eine besondere Aufgabe dar, die Software auf die zu verwendende Hardware abzubilden. Ein Beispiel dafür sind die vierstufigen Seitentabellen ab Linux 2.6, die auf die zweistufigen Seitentabellen der Intel IA32-Architektur („x86“) abgebildet werden müssen.

- **Invertierte Seitentabellen**

Die mehrstufigen Seitentabellen führten zu verschiedenen Versuchen, die lange Suche durch unbenutzte Tabellen abzukürzen. Ein Weg besteht darin, nur eine Tabelle für die relativ wenigen, tatsächlich existierenden Speicherseiten des physikalischen Speichers aufzustellen. Anstelle also als Schlüssel alle möglichen (sehr vielen) virtuellen Adressen auf die linke Seite zu schreiben und die tatsächliche, physikalische Adresse (falls vorhanden) auf die rechte Seite, vertauscht man die rechte mit der linken Seite und listet in

ProzeßId=1

virt.	reell
0	5
1	-
2	2
3	-
4	7
5	-

ProzeßId=2

virt.	reell
0	0
1	-
2	-
3	-
4	1
5	3

→

reell	virtuell	ProcId
0	0	2
1	4	2
2	2	1
3	5	2
4	-	-
5	0	1
6	-	-
7	4	1

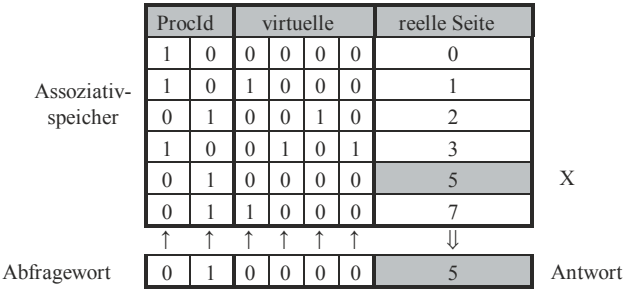
Abb. 3.7 Zusammenfassung von Seitentabellen zu einer inversen Seitentabelle

fortlaufender Reihenfolge links alle (wenigen) existierenden Seiten auf und rechts dazu die zugeordnete virtuelle Adresse der Seite (**inverse Seitentabelle**). Um mit einer solchen verkürzten, invertierten Tabelle eine Adressierung umzusetzen, muss man alle virtuellen Adressen rechts durchsuchen, bis man die passende, aktuelle Seite gefunden hat, und dann links die physikalische Seite dazu ablesen. In Abb. 3.7 ist eine solche Inversion für ein einfaches Beispiel von zwei Prozessen mit ihren Seitentabellen gezeigt. Beide Tabellen sind rechts zu einer einzigen zusammengefasst. Wie man sieht, reicht die virtuelle Adresse allein nicht aus, da der Adressraum bei allen Prozessen gleich ist, z. B. bei den virtuellen Seiten 0 und 4. Um die gleichen virtuellen Adressen trotzdem eindeutig den unterschiedlichen Adressen im Hauptspeicher (*page frames*) zuordnen zu können, muss zusätzlich ein Prozessindex, beispielsweise die Prozessnummer, hinzugenommen werden.

- **Assoziativer Tabellencache**
Sowohl bei den Multi-level-Tabellen als auch bei den invertierten Seitentabellen stellt sich das Problem, dass die Auswertung der Tabelleninformation zur Adressumsetzung viel Zeit kostet. Aus diesem Grund ist es üblich geworden, dass die letzten Zuordnungen von virtuellen zu reellen Seiten in einem kleinen, schnellen Speicher (Cache) gespeichert werden, wie z. B. beim MC 68030. Dieser Speicher wird zuerst durchsucht, bevor auf die regulären Tabellen zugegriffen wird.

Der Cache ist in der Regel speziell für einen inhaltsorientierten Zugriff (**Assoziativspeicher**) gebaut; nach dem Präsentieren der virtuellen Seitenadresse wird in einem Zeittakt ausgegeben, ob die Umsetzung dieser Seitenadresse gespeichert ist und wenn

Abb. 3.8 Funktion eines assoziativen Tabellencache



ja, wie die physikalische Adresse dazu lautet. In [Abb. 3.8](#) ist dies schematisch gezeigt. Die Bits der virtuellen Seitennummer sind in Spalten extra aufgeführt, die der realen Seiten nur pauschal als Dezimalzahl.

Um eine Abfrage in einem Zeitschritt zu erreichen, ist eine Zusatzelektronik an den Speicherzellen integriert, die für den ganzen Cache spaltenweise (bitweise) als Anforderung die Bits der virtuellen Adresse (hier: ProzessId = 1, virt. Seite = 0) jeweils mit den Werten des Eintrags vergleicht. Sind bei einem Eintrag alle Vergleiche mit den Anforderungsbits erfolgreich, so wird ein *enthalten*-Flag für diesen Eintrag gesetzt (X in obiger Abbildung) und der korrespondierende physikalische Adresswert (hier: reelle Seite = 5) ausgelesen. Eine solche zusätzliche Hardwareeinheit ergänzt hervorragend die tabellenorientierte Multi-Level Adresskonversion und beschleunigt den durchschnittlichen Zugriff auf den Speicher stark.

Ein solcher Assoziativspeicher wird auch als **translation lookaside buffer TLB** bezeichnet. Dabei trägt der assoziative Cachespeicher so viel zu der Adressübersetzung bei, dass man bei einem großen Cache auch auf die übrige Hardware zum Auslesen der Seitentabellen verzichten kann. Statt dessen führt man die Adressübersetzung bei den wenigen Ereignissen, bei denen die Übersetzung im Cache fehlt, wie beim MIPS R2000 RISC-Prozessor mit Software durch, ohne dadurch die Leistung des Prozessors stark zu beeinträchtigen.

3.3.3 Gemeinsam genutzter Speicher (*shared memory*)

Eine weitere wichtige Möglichkeit, die das virtuelle Speichermodell eröffnet, ist die dynamische, kontrollierte gemeinsame Nutzung von Speicherbereichen durch mehrere Prozesse. Dies ist besonders dann sinnvoll, wenn bei vielen Benutzerprozessen derselbe Code verwendet wird, beispielsweise wenn mehrere Benutzer denselben Texteditor benutzen, oder wenn mehrere Benutzerprogramme dieselben Bibliotheken (z. B. C-Bibliothek) verwenden. Aber auch gemeinsame, globale Datenbereiche sind sinnvoll, beispielsweise für die Interprozesskommunikation. Dazu können diese Speicherbereiche als Nachrichtepuffer (*Mailboxen*) verwendet werden.

Um bestimmte physikalische Speicherbereiche in die virtuellen Adressräume mehrerer Prozesse abbilden zu können, sind verschiedene Maßnahmen notwendig. Zum einen müssen Betriebssystemaufrufe geschaffen werden, um einen Speicherbereich eines Prozesses als globalen gemeinsamen Speicher (**shared memory**) zu deklarieren. Den Bezeichner dafür, der vom Betriebssystem zur Verfügung gestellt wird, dient dann allen anderen Prozessen als Referenz. Zusätzlich muss aber auch gewährleistet sein, dass diese Seiten nicht gelöscht werden, wenn einer der Prozesse, der diese als eigene Seiten in seinem Adressraum führt, beendet wird. Für die Synchronisation der Zugriffe auf den gemeinsamen Speicherbereich sollten auch Semaphoroperationen vom Betriebssystem bereitgestellt werden. In [Abb. 3.9](#) ist eine solche Situation für drei Prozesse gezeigt.

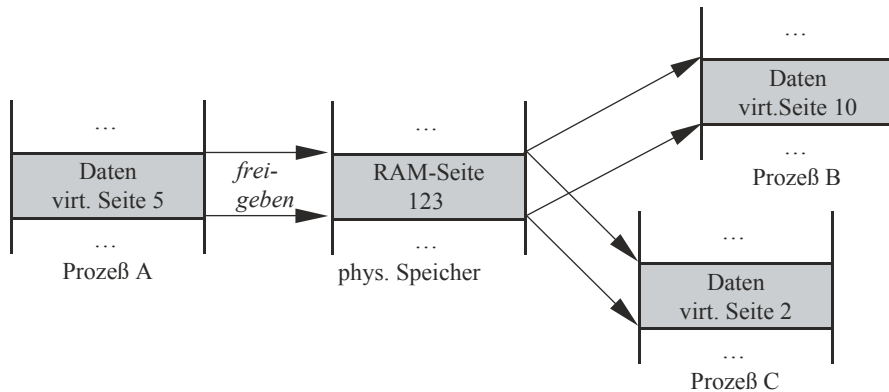


Abb. 3.9 Die Abbildung auf gemeinsamen Speicher

3.3.4 Virtueller Speicher in UNIX und Windows NT

Das Konzept des virtuellen Speichers ist in UNIX und Windows NT verschieden implementiert und ändert sich von Version zu Version. Im Allgemeinen sind die 32 Bit-Versionen durch die 4 GB-Adressierungsgrenze begrenzt, während die 64 Bit-Versionen nur durch den aktuellen Speicherausbaubzw. maximale interne Tabellenlängen bestimmt werden.

Virtueller Adressraum in UNIX

In Linux (i386) ist der virtuelle Adressraum 32 Bit, also 4 GB groß. Der gesamte virtuelle Adressraum von 4 GB ist in zwei einzelne Unterabschnitte aufgeteilt: den *kernel space* und den *user space*. Der *user* kann dabei nur auf die ersten 3 GB zugreifen; erst im *kernel mode* ist auch der Betriebssystemkern für die Systemaufrufe ansprechbar.

Im Benutzerprozess wächst der Stack von oberen Adressbereichen nach unten; der Heap von unteren Adressbereichen durch die Systemdienste `sbrk()` und `malloc()` nach oben. Dies ist in folgender [Abb. 3.10](#) gezeigt.

Im höchsten Adressbereich existiert ein Bereich, der für besonders schnellen Zugriff auf Ein- und Ausgabegeräte (*I/O map*) vorgesehen ist, deren Puffer oder Register unter einer Adressreferenz ausgelesen und beschrieben werden können (**memory mapped devices**). Natürlich kann dies der normale Benutzerprozess mit den normalen Zugriffsrechten nicht selbst machen, sondern wird unter der Kontrolle des Betriebssystems im *kernel mode* durchgeführt. Physikalisch ist der Kern im untersten Adressbereich ab der physikalischen Adresse null eingelagert, so dass für den Kern immer automatisch auch im realen Adressmodus ohne MMU das lineare Adressmodell gilt.

Virtueller Adressraum in Windows NT

Windows NT war bereits ab der ersten Version für einen 64 Bit virtuellen Adressraum vorgesehen, aber bis NT 5.0 nur in 32 Bit realisiert. Diese 4 GB sind in zwei Bereiche

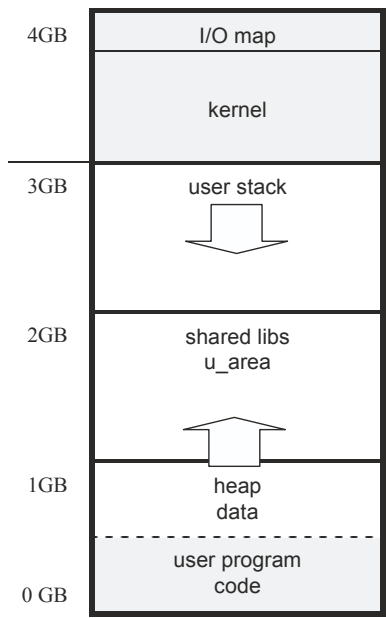


Abb. 3.10 Die Auslegung des virtuellen Adressraum bei Linux

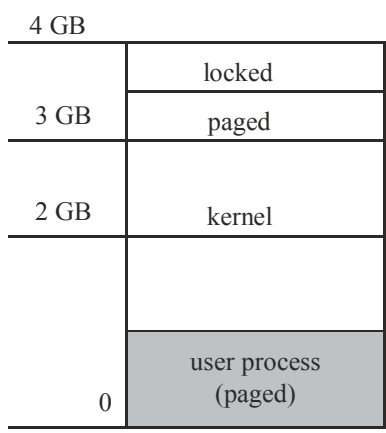


Abb. 3.11 Die Unterteilung des virtuellen Adressraums in Windows NT 32 Bit

unterteilt: einen 2 GB großen Adressraum für den Benutzerprozess und einen 2 GB Raum für alle restlichen Funktionen. Die Unterteilung ist in Abb. 3.11 gezeigt. Oberhalb von 2 GB sind der Systemkern und alle Systemtabellen lokalisiert, die sich durch besonders kurze Zugriffszeiten auszeichnen. Da alle Seiten des Systemkerns immer im Hauptspeicher stehen und für das Betriebssystem keine Schutzmechanismen beim Zugriff darauf nötig sind, wird der Zugriff auf die Kernadressen dadurch erreicht, dass im *kernel mode*

die obersten beiden Bits der Adresse unterdrückt werden und der Rest der virtuellen Adresse als physikalische Adresse interpretiert wird. Die Implementierung von Windows NT muss sicherstellen, dass dies Erfolg hat.

Im Gegensatz dazu liegt dem übrigen Speicher eine Seitenstruktur und -verwaltung zugrunde, die von dem *virtual memory*-Manager (VM) durchgeführt wird. Diese unterstützt Hardware-definierte Seitengrößen von 4 KiB–64 KiB; im Normalfall 4 KiB.

Bei 64 Bit-Versionen („X64“) fällt die 4 GB-Adressierungsobergrenze weg. Dafür machen sich interne Begrenzungen bemerkbar: So können unterschiedliche Versionen unterschiedliche Obergrenzen haben. In Windows 7 beispielsweise hat die X64-Home Basic-Version eine Obergrenze von 8 GiB, die Home-Premium-Version 16 GB und die Enterprise-Version 192 GB. In Windows 10 gilt für fast alle X64-Versionen eine Obergrenze von 2 TB, bis auf die Home-Version mit 128 GB.

Die Seitenverwaltung des VM sieht eine zweistufige Seitentabelle vor: Die erste Stufe enthält als Tabelle ein sog. „page directory“, das die Adressen der Tabellen der zweiten Stufe enthält. Erfolgt ein *page fault* beim Zugriff auf eine solche Adresse, so wird die Seite eingelagert, in der die fehlende Tabelle enthalten ist. In der zweiten Tabelle ist dann die physikalische Seitennummer (*page frame*) enthalten, verbunden mit weiteren Informationen wie Zugriffsrechte etc.

Für den gemeinsam genutzten Speicher (*shared memory*) gibt es eine Sonderregelung. Anstelle der physikalischen Seitennummern enthält die zweite Tabelle einen Index einer besonderen Tabelle: der *Prototype Page Table PPT*. In dieser Tabelle sind die Adressen aller Seiten enthalten, die zur gemeinsamen Nutzung deklariert werden. Die Einführung einer dritten Stufe im Adressierungsschema verlangsamt zwar den Zugriff beim ersten Mal (bei den weiteren Zugriffen steht die direkte Referenz im assoziativen Cache TLB), gestattet aber eine einfache Organisation der Seiten. Wird eine Seite nach dem Auslagern wieder vom Massenspeicher geholt und an einen anderen Platz im Hauptspeicher gelegt als vorher, so müsste das Betriebssystem alle Tabellen aller Prozesse durchsuchen und die physikalische Adresse, wenn gefunden, entsprechend abändern. Dies entfällt bei dem zentralen Management: Das Betriebssystem kann sich auf die PPT beschränken.

Speicherbereiche können von einem Prozess zur gemeinsamen Nutzung mit anderen Prozessen deklariert werden. Dazu wird ein sog. *section*-Objekt erzeugt, das als eine spezielle Datei betrachtet wird (s. nächster Abschnitt) und folgende Eigenschaften aufweist:

- Objektattribute: maximale Größe, Seitenschutz, *paging/mapped file*: JA/NEIN, gleicher Adressbeginn bei allen Prozessen: JA/NEIN
- Objektmethoden: Erzeugen, Öffnen, Ausweiten, Ausschnitt wählen, Status abfragen

Möchten die anderen Prozesse einen deklarierten Datenbereich auch nutzen, so müssen sie das mit einem Namen und Zugriffsrechten wie bei einer Datei versehene *section*-Objekt

öffnen und sich einen Ausschnitt daraus wählen. Dieser Ausschnitt erscheint dann in ihrem virtuellen Adressraum so, als ob er ein normaler Speicherabschnitt wäre.

Zur Verwaltung der physikalischen Seiten (*page frames*) gibt es in Windows NT eine besondere Datenstruktur im Kern: die *Page Frame Database* (PFD). Sie enthält für jede existierende Seite einen Eintrag, so dass also jeder Seitennummer in den Tabellen der zweiten Stufe ein Index in der PFD entspricht und umgekehrt. Im Unterschied zu den Statusbits einer virtuellen Seite stehen in der PFD die Statusinformationen der reellen Seite, die eine der folgenden Zustände haben kann:

<i>Valid</i>	wird benutzt; es existiert ein gültiger <i>page table</i> -Eintrag
<i>Zeroed</i>	ist frei und ist mit Nullen initialisiert
<i>Free</i>	frei, aber noch nicht initialisiert
<i>Standby</i>	im Zustand „transition“: Die Seite gehört offiziell nicht mehr zum Prozess, kann aber zurückgeholt werden
<i>Modified</i>	<i>Standby</i> + beschrieben
<i>Bad</i>	physikalisch fehlerhaft (<i>parity errors</i> etc.) und kann nicht benutzt werden.

Durch Zeiger in jedem Eintrag sind gleichartige Einträge zu einer Liste miteinander verbunden. Neben den normalen (*valid*) Seiten gibt es also fünf Listen. Die *Page Frame Database* wird von allen Prozessen benutzt und ist deshalb für den Multiprozessorbetrieb mit einem Semaphor (*spin lock*) gesichert.

Da allerdings nur ein Semaphor für die gesamte Datenstruktur benutzt wird, kann bei hoher *paging*-Frequenz die PFD zu einem Flaschenhals der Systemleistung werden. Eine parallele Version mit mehreren, jeweils *spin lock* gesicherten Abschnitten wäre zwar im Parallelbetrieb schneller, aber nicht im Monoprozessorbetrieb und wurde deshalb nicht implementiert.

Virtueller Adressraum in Großrechnern

In Großrechnern haben wir sehr große Hauptspeichergrößen von einigen tausend Gigabyte. Da auf diesen Großrechnern meist mehrere virtuelle Betriebssysteme laufen (s. [Kap. 1](#)), deren 32-Bit-Versionen auf max. 4 GB begrenzt sind, muss der Hauptspeicher in mehrer Untereinheiten zerlegt werden können. Dazu existiert meist eine hardware-unterstützte, logische Partitionierung. Jede logische Partition LPAR hat entsprechende Register und CPU-gestützte Adressumsetzung, um die von der Software angesprochenen Hardware-Adressen in die tatsächlichen Hardwareadressen der LPAR umzusetzen, so dass das virtuelle Betriebssystem und der Anwenderjob zusammen in einer eigenen Partition ablaufen. Die gesamte I/O-Verwaltung aller logischen Partitionen durch den Hypervisor ist davon getrennt und läuft in einer eigenen Partition (*hardware system area HSA*) ab.

Durch die Hardwareunterstützung ist der Leistungsverlust mit ca. 1,5 % nur gering.

3.3.5 Aufgaben zu virtuellem Speicher

Aufgabe 3.3-1 Virtuelle Adressierung

Wir haben einen Computer mit virtuellem Speicher.

Folgende Daten sind bekannt:

- 32 virtuelle Seiten
- 8 physische Seiten
- 11 Bit Seitenoffset

Beantworten Sie damit folgende Fragen:

- a) Wie viele Bits werden für die Adressierung von virtuellen Seiten benötigt?
- b) Wieviel physischen Speicher hat der Computer?
- c) Wie groß ist der virtuelle Adressraum des Computers?
- d) Wie viele Bits werden für die Adressierung der physischen Seiten benötigt?

Aufgabe 3.3-2 Adresstabellen

Eine Maschine hat virtuelle 128-Bit-Adressen und physikalische 32-Bit-Adressen. Die Seiten sind 8 KW groß.

- a) Wie viele Einträge werden für eine konventionelle bzw. für eine invertierte Seitentabelle benötigt?
- b) Wie viele Stufen werden für eine mehrstufige Seitentabelle benötigt, um unter 1 MW (wobei 1 W = 1 Eintrag) Seitentabellenlänge zu bleiben?

Aufgabe 3.3-3 Einstufige Adressumrechnung

- a) Berechnen Sie zu den gegebenen virtuellen Adressen 2204_H und $A226_H$ jeweils die physikalische Adresse. Der *offset* beträgt 13 Bit und die Seitennummer hat 3 Bit. Verwenden Sie folgende Basis-Seitentabelle:

Seite	Seitenrahmen
0	7
1	0
2	1
3	6
4	4
5	2
6	3
7	5

- b) Wie müsste eine Speicherverwaltung erweitert werden, damit die Seiten ausgelagert werden können?

Aufgabe 3.3-4 Zweistufige Adressumrechnung

Gegeben seien die folgenden 4 virtuellen 16-Bit Speicheradressen in Hexadezimaldarstellung:

- a. 75B4
- b. 8AC6
- c. 1E9C
- d. 5B3E

Die Adresse ist von links nach rechts kodiert. Das erste Bit wird nicht benutzt. Die nächsten 3 Bits kodieren den jeweiligen Index in der Basis-Seitentabelle, die weiteren 3 Bits den Index in der entsprechenden Tafel. Die übrigen 9 Bits bilden den *Offset*.

	Index 0			Index 1			Offset								
1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0
5	4	3	2	1	0										

Bestimmen Sie anhand der folgenden Abbildung die zu a)–d) gehörenden physikalischen 32 Bit-Adressen. Geben Sie diese in Hexadezimaldarstellung an.

Die physikalische Speicheradresse der Basis-Seitentabelle ist 9A38B75A.

7	NIL
6	39BE
5	B3C7
4	5B4C
3	NIL
2	1A75
1	87D3
0	A380

B47AB75A

7	B47AB75A
6	NIL
5	5A37B75A
4	A3EBB75A
3	B47AB75A
2	A3EBB75A
1	A3EBB75A
0	5A37B75A

9A38B75A

7	NIL
6	B125
5	4CC1
4	3333
3	B97C
2	NIL
1	5E4A
0	NIL

5A37B75A

7	NIL
6	D2A6
5	8AC6
4	2BE9
3	NIL
2	D17F
1	NIL
0	7A34

A3EBB75A

Aufgabe 3.3-5 Mehrstufige Seitentabelle

Eine Maschine habe einen virtuellen Adressraum. Die Speicherverwaltung benutzt eine zweistufige Seitentabelle mit einem assoziativen Cache (Translation Lookaside Buffer TLB) mit einer durchschnittlichen Trefferrate von 90 %. Beachten Sie, dass ein Zugriff zwei „Wege“ benutzen kann: TLB oder über die Seitentabelle.

- a) Wie groß sind die mittleren Zeitkosten für den Zugriff auf den Hauptspeicher HS, wenn die Zugriffszeit des Speichers 100 ns und die Zugriffszeit des TLB 10 ns beträgt? (Es wird angenommen, dass keine Seitenfehler auftreten!)
- b) In a) wurden Seitenfehler ausgeschlossen. Wir wollen nun den vereinfachten Fall untersuchen, in dem Seitenfehler nur beim Hauptspeicherzugriff auf das Datum auftreten sollen (vereinfachende Annahme: Seitentabellen sind im HS und werden nicht ausgelagert!). Die Häufigkeit von Seitenfehlern sei $1:10^5$. Ein Seitenfehler koste 100 ms. Wie ist dann die mittlere Zugriffszeit auf den Hauptspeicher *page fault*?

Hinweis:

$$\text{mittlere Zugriffszeit} = \text{Trefferzeit} + \text{Fehlzugriffsrate} \times \text{Fehlzugriffszeit}.$$

- c) Welche Probleme verursacht Mehrprogrammbetrieb für den TLB und für andere Caches? Denken Sie vor allem an die Identifikation der Blöcke.

3.3.6 Seitenersetzungsstrategien

Ergibt sich nach der Adressübersetzung, dass eine benötigte Seite fehlt (*page fault*), so muss sie vom Massenspeicher gelesen und in den physikalischen Hauptspeicher geladen werden. Dabei stellt sich aber die Frage: an welche Stelle? Welche der vorhandenen Seiten soll damit überschrieben werden?

Ersetzen wir eine beispielsweise häufig benutzte Seite, so müssen wir sie bald wieder nachladen, was die Programmlaufzeit erhöht. Es ist deshalb Aufgabe einer guten Strategie, diejenige Seite zu finden, die ersetzt werden kann, ohne solche Probleme zu erzeugen. Eine solche Strategie kann man auch als Schedulingstrategie für einen Seitentauschprozessor ansehen, der in der Warteschlange Anforderungen für die benötigten Seiten enthält. Je besser die Strategie, umso schneller die Abarbeitung der Seitenanforderungen.

Dabei kommt uns die Tatsache zur Hilfe, dass die meisten Referenzen in einem Programm nur lokal erfolgen, also keine Folge beliebiger Seiten vorhanden ist (**Lokalitätseigenschaft**). Diese Lokalität kommt zum einen dadurch zustande, dass die Programme meist sequentiell mit hintereinander abgelegten Befehlen abgearbeitet werden und Programmsprünge bei Schleifen lokal erfolgen, zum anderen sind auch die Einzeldaten einer Datenstruktur auch in einem Stück hintereinander abgelegt. Bei einer völlig zufälligen Folge der Befehle oder Daten hätten wir ein Problem: Wir könnten nichts über die Seite des nächsten Zugriffs vorhersagen; alle Strategien für eine Seitenersetzung wären vergebens. Stattdessen können wir nun versuchen, die Kurzzeitstatistik auszunutzen.

Die Folge der Nummern der benötigten, referenzierten Seiten, die **Referenzfolge (reference string)**, charakterisiert dabei den Weg des Kontroll- und Datenflusses während der Programmausführung. Wüssten wir diese Folge im Voraus, so könnten wir uns bei der Seitenersetzung darauf einstellen. Dies ist die Grundlage der optimalen Strategie zur Seitenersetzung.

- **Die optimale Strategie**

Belady (1966) konnte zeigen, dass genau dann die wenigsten Ersetzungen vorgenommen werden, wenn man folgende Strategie benutzt:

Wähle die Seite zum Ersetzen, die am spätesten in der Zukunft benutzt werden wird.

In [Abb. 3.12](#) ist dies an der Referenzfolge 1,2,3,1,4,1,3,2,3, ... für einen Hauptspeicher mit nur drei Seiten zu den verschiedenen Zeitpunkten $t = 1, 2, \dots$ gezeigt.

Abb. 3.12 Die optimale Seitenersetzung

Ziel	1	2	3	1	4	1	3	2	3
RAM1	1	1	1	1	1	1	1	②	2
RAM2	-	2	2	2	④	4	4	4	4
RAM3	-	-	3	3	3	3	3	3	3

t = 1 2 3 4 5 6 7 8 9

Die Tabelle ist dabei wie folgt zu lesen: Jede Spalte zeigt den Zustand des Speichers zu einem Zeitpunkt, wobei die Zeit von links nach rechts in der Tabelle zunimmt. Die neuen, ersetzenden Seiten sind in der Abbildung mit umkreisten Ziffern notiert. Im Beispiel wird bei $t = 5$ die Seite 2 überschrieben (ersetzt), da sie von den möglichen Seiten $\{1,2,3\}$ diejenige ist, die in der Folge 4,1,3,2, ... am spätesten benutzt wird.

Wir erkennen, dass bei dieser Strategie im Beispiel nur 2 Ersetzungen vorgenommen werden musste. Allerdings kann die optimale Strategie so nur bei deterministischen Programmen benutzt werden, für die alle Seitenanforderungen im voraus bekannt sind. Da es solche Programme praktisch nicht gibt, wird die Strategie nur als Referenz benutzt, um alle anderen Strategien in ihrer Leistungsfähigkeit zu vergleichen.

Verschiedene Zusatzinformationen führen uns zu verschiedenen Strategien (*page replacement-Algorithm*en) zur Seitenersetzung. Eine der einfachsten Strategien, die wir immer dann verwenden, wenn wir keine weiteren Informationen haben, ist die

• **FIFO (First-In-First-Out)– Strategie**

Ersetze die älteste Seite

Die neu eingelagerten Seiten lassen sich in der Reihenfolge ihres Einlesens jeweils ans Ende einer Liste stellen, ohne die Statusbits R und M zu berücksichtigen. Wählen wir als Kandidat zum Ersetzen jeweils die Seite am Anfang der Liste, so haben wir die „älteste“ Seite. Ihre Ersetzung, so vermuten wir, wird am wenigsten auffallen. In [Abb. 3.13](#) ist die einfache FIFO-Strategie an unserem vorigen Beispiel gezeigt.

Im Vergleich zur optimalen Strategie aus [Abb. 3.12](#) sind hier zwei zusätzliche Seitenersetzungen, also ein Maximum von vier Seitenersetzungen nötig nach dem initialen Füllen der RAM-Seiten.

Leider ist die Vermutung über den Wert älterer Seiten nicht ganz zutreffend, beispielsweise für die Seite, die das Hauptprogramm enthält. Deshalb ist es sinnvoll, statt dessen

Abb. 3.13 Die FIFO-Seitenersetzung

Folge	1	2	3	1	4	1	3	2	3
RAM1	1	1	1	1	④	4	4	4	③
RAM2	-	2	2	2	2	①	1	1	1
RAM3	-	-	3	3	3	3	3	②	2

t = 1 2 3 4 5 6 7 8 9

mehr oder weniger detaillierte Kurzzeitstatistiken über die benutzten Seiten aufzustellen und Strategien zur Seitenersetzung entwickelt, die auf der Auswertung der Statistik basieren und dies als Prognose für die Zukunft ansehen. Damit wird versucht, die optimale Strategie so gut wie möglich anzunähern.

Eine der einfachsten Auswertungen stützt sich auf die Statusbits R (= *referenced*, Seite benutzt, auch manchmal als A-Bit *accessed* bezeichnet) und M (= *modified*, Seite wurde verändert und muss zurückgeschrieben werden, auch manchmal als D-Bit *dirty* bezeichnet), die neben der Adressinformation im Eintrag der Seitentabelle existieren. Wird das R-Bit nach Ablauf einer Zeitspanne durch einen Zeitzähler (Überschreiten einer Zeitschranke) zurückgesetzt, so sagt $R = 1$ aus, dass die entsprechende Seite erst kürzlich benutzt wurde (die Zeitschranke ist noch nicht überschritten) und deshalb möglichst nicht ersetzt werden soll. Wird die Seite referiert, so muss mit dem Setzen von $R = 1$ auch der Zeitzähler initialisiert werden. Dabei ist für jede Seite ein Zähler nötig. Dies ist allerdings etwas teuer, so dass man als Annäherung auch einen Zeitzähler für alle R-Bits verwenden kann, der diese periodisch alle auf einmal zurücksetzt. Da für den Algorithmus nicht das absolute Zeitintervall, sondern nur die Benutzungsunterschiede der Seiten in einem Intervall nötig sind, kann die Rücksetzung der R-Bits auch bei einem besonderen Ereignis, beispielsweise einem Seitenfehler, erfolgen.

Die Verfügbarkeit von elementarer Statistikinformation führt uns nun zu einer Abwandlung des Clock-Algorithmus.

- **Second Chance– Strategie**

Ersetze die älteste, unbenutzte Seite

Wir betrachten noch zusätzlich zum FIFO-Eintrag den Zustand des R-Bits für die älteste Seite zu. Ist es 1, so wird die Seite noch benötigt. In Abwandlung der reinen FIFO-Strategie kann man deshalb diese Seite wieder ans Ende der Liste stellen, $R = 0$ setzen und damit die Seite so behandeln, als ob sie neu angekommen wäre: Sie erhält eine „zweite Chance“ (*second chance*-Algorithmus). War dagegen $R = 0$, so wird sie ersetzt; bei $M = 1$ wird sie vorher zurückgeschrieben.

- **Clock-Algorithmus**

Dieses Verfahren kann man vereinfacht implementieren, indem man die FIFO-Liste zu einem Ring schließt. Da nur eine bestimmte, maximale Seitenzahl im Hauptspeicher Platz hat, verändert sich die Ringgröße nicht. Nur die Markierung „älteste Seite“ verschiebt sich bei jeder Seiteneinlagerung und läuft im Ring der Tabelleneinträge um einen Eintrag weiter, wie der Zeiger einer Uhr (*clock*-Algorithmus). In [Abb. 3.14](#) ist dies visualisiert.

Hier ist gezeigt, wie der Zeiger auf die älteste Seite (gestrichelter Pfeil) nach der Ersetzung durch die neueste Seite um eine Stelle weiterrückt (ununterbrochener Pfeil) und nun die nächst-älteste Seite als „Älteste Seite“ ausweist. Wie beim *second chance*-Algorithmus wird geprüft, ob die älteste Seite ein R-Bit = 1 aufweist. Wenn ja, wird es zurückgesetzt und nicht ausgelagert.

- **LRU (Least Recently Used) -Strategie**

Genauer als eine FIFO-Liste und als die qualitative Existenz einer Seitenreferenz in einem Intervall ist es auch, quantitativ die Zeit im Intervall zu messen, die eine Seite

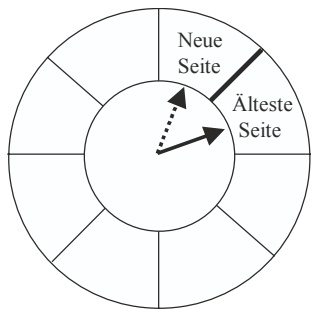


Abb. 3.14 Der Clock-Algorithmus

unbenutzt im Speicher zugebracht hat. Damit kann bei mehreren Seiten mit $R = 0$ die älteste ermittelt und ersetzt werden:

Ersetze die Seite, die die längste Zeit nicht benutzt wurde.

Dies klingt einfach, ist aber nicht einfach zu implementieren. Für die Referenzfolge ist die Forderung klar: Wähle die Seite, die in der Folge den größten Abstand zur aktuellen Seite hat. Wurde sie noch nicht benutzt, so ist der Abstand unendlich. Die Strategie nimmt an, dass aus einem großen Abstand in der Referenzfolge in die Vergangenheit auch ein großer Abstand in die Zukunft folgt, also implizit die optimale Strategie damit verwirklicht wird. In Abb. 3.15 ist die LRU-Strategie an unserem vorigen Beispiel gezeigt.

Die Seitenfolge 1,2,3 füllt zunächst nur den Speicher. Bei Seite 4 kommt die Vergangenheit ins Spiel: Obwohl Seite 2 neuer als Seite 1 ist, wird sie als die am wenigsten benutzte Seite ersetzt, im Unterschied zu FIFO. Mit dieser Strategie erreichen wir mit zwei Ersetzungen ein Ergebnis, das (in diesem Beispiel) so gut wie die Optimalstrategie ist.

Für die Implementierung dieser Strategie gibt es verschiedene Wege, die aber alle einigen Aufwand bereiten. So könnte eine Hardwarelösung aussehen, dass ein Zähler hochläuft, z. B. der normale Zeitzähler für Uhrzeit und Datum. Bei jedem Zeittick wird der Zeitstand automatisch in den Tabelleneintrag der gerade aktiven Seite übernommen. Alle inaktiven Seiten konservieren also einen alten Zählerstand. Wird nun die älteste Seite gesucht, so muss nur nach der kleinsten Zeitzahl in den Tabelleneinträgen gesucht werden.

Abb. 3.15 Die LRU-Seitenersetzung

Folge	1	2	3	1	4	1	3	2	3
RAM1	1	1	1	1	1	1	1	1	1
RAM2	-	2	2	2	④	4	4	②	2
RAM3	-	-	3	3	3	3	3	3	3

➔

$t =$ 1 2 3 4 5 6 7 8 9

Steht keine Hardware dafür zur Verfügung, so kann man das Altern der inaktiven Seiten auch approximativ durch ein Register pro Seite simulieren. Dazu wird in regelmäßigen Zeitabständen als oberstes Bit des Registers das R-Bit der jeweiligen Seite gesetzt und der ganze Registerinhalt um ein Bit nach rechts verschoben (*shift right*). In Abb. 3.16 ist dies für drei Seiten gezeigt. Das Register wirkt dabei wie ein gewichtetes Kurzzeitfenster für die Seitenaktivität; bei 8 Bits ist es 8 Zeittakte weit. Die Verschiebung nach rechts bewirkt ein *Altern* der Information. Dabei ist meist derjenige Registerwert am größten, der die häufigste Aktivität in letzter Zeit verzeichnet hat; der Wert der Aktivität sinkt mit wachsendem Zeitabstand, da eine *shift right* – Operation ein Teilen durch zwei der korrespondierenden Binärzahl im Register bedeutet. Allerdings ist diese Abbildung nicht immer plausibel: Hat Seite A den Registerwert 11111000 so ist sie sicher neuer als 00000111 der Seite B und damit vorzuziehen, aber 00100000 von Seite A deutet gegenüber 00011111 von B nicht auf eine heftige Benutzung hin, obwohl die Binärzahl größer ist.

Alternativ könnte man auch bei $R = 1$ eins auf den Wert des Registers addieren und bei $R = 0$ eins subtrahieren, aber dies dauert länger als eine einfache *shift*-Operation.

Die zu ersetzende Seite nach der LRU-Strategie weist also meistens die kleinste Zahl in ihrem Schieberegister auf.

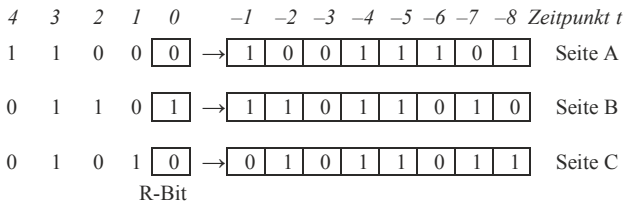
• **NRU (Not Recently Used)-Strategie**

Die FIFO-Strategie nutzt die Seitenstatistik nur wenig oder gar nicht aus. Dies kann man mit einfachen Mitteln verbessern. Mit Hilfe der zwei Bits (R und M) lassen sich dazu die Seiten in folgende vier Klassen einteilen:

- 0) $R = 0, M = 0$
- 1) $R = 0, M = 1$
- 2) $R = 1, M = 0$
- 3) $R = 1, M = 1$

Es ist klar, dass die Seiten der Klasse 0 ($R = 0, M = 0$) am wenigsten benutzt sind und deshalb zuerst ersetzt werden sollten, eher noch als die modifizierten, aber länger nicht referenzierten Seiten der Klasse 1 ($R=0, M=1$), die vielleicht nochmals benötigt werden. Wichtiger noch aber sind die tatsächlich referenzierten Seiten mit $R=1$ (Klasse 2), auf die mit $M = 1$ (Klasse 3) sogar geschrieben wurde.

Abb. 3.16 Realisierung von LRU mit Schieberegistern



Damit ist eine Gewichtung oder Priorität für die Seiten festgelegt worden:

Ersetze die Seiten der Klasse mit der kleinsten Nummer

Dies ist auch als **RNU (Recently Not Used)** bekannt. Da der Zustand der beiden Bits nur vier Möglichkeiten unterscheidet, muss bei einer größeren Zahl von Seiten bei gleicher Nummer durch Zufall entschieden werden. Diese Strategie kann deshalb nicht sehr differenziert wirken.

- **NFU (Not Frequently Used)** bzw. **LFU (Least Frequently Used)**

Bei der NRU-Strategie wurde die in einem gemeinsamen Intervall am wenigsten benutzte Seite ersetzt. Verfügt man über eine bessere Statistik, so kann man diejenige Seite ersetzen, die unter allen Seiten am geringsten benutzt wurde mit der Hoffnung, dass sie durch das Lokalitätsprinzip auch in Zukunft am wenigsten gebraucht werden wird:

Ersetze die Seiten mit der geringsten Benutzungsfrequenz

Dazu wird für jede Seite die Häufigkeit gemessen, mit der sie in der Zeitspanne seit der Existenz des Prozesses benutzt wird. Dazu wird für jede Seite ein Zähler geführt, der periodisch bei Benutzung ($R = 1$) erhöht und nie zurückgesetzt wird. Die Seite mit dem kleinsten Wert wird dann ersetzt. Für die Referenzfolge bedeutet dies, diejenige Seite zu wählen, die bisher am wenigsten referiert wurde. Bei gleich häufiger Nutzung wird die zu ersetzende Seite zufällig ausgewählt.

Problematisch an dieser Strategie ist, dass früher stark benutzte Seiten, die inzwischen obsolet geworden sind (etwa der Initialisierungscode), schwer aus dem Hauptspeicher verdrängt werden können, da ihr Zahlenwert zu hoch ist. Es ist deshalb sinnvoll, als Zeitspanne nicht die gesamte Existenzdauer zu wählen, sondern ein kürzeres Intervall. Dies lässt sich dadurch erreichen, dass man einen Alterungsmechanismus (periodisches Nullsetzen, Dekrementieren etc.) vorsieht, um die aktuelle Nutzung pro Zeiteinheit (Frequenz) zu bestimmen.

Die LFU-Strategie bestimmt die Nutzungsfrequenz einer Seite. Dies ist proportional zu der aktuellen Wahrscheinlichkeit, die Seite zu nutzen, und ermöglicht über das Lokalitätsprinzip eine Aussage über die nahe Zukunft. Sie kommt damit der optimalen Strategie sehr nahe. Wird allerdings ein zu großes Intervall gewählt, so werden beim Wechsel der Seitenstatistik (nichtstationäre Wahrscheinlichkeitsverteilung) die neu eingelagerten Seiten zuerst überschrieben; das System reagiert zu träge und damit nicht optimal.

- Die *working set* – Strategie

Die Menge $W(t, \Delta t)$ der Seiten, die zum Zeitpunkt t in einem Δt Zeitticks breiten Zeitfenster benutzt werde, ist ziemlich wichtig, enthält sie doch unter anderem die besonders häufig benutzten Seiten. Ohne sie kann der Prozess nicht effizient arbeiten; sie wird deshalb als **Arbeitsmenge** oder **working set** (Denning 1980) bezeichnet. Das

mittlere, erwartete *working set* $\langle W(t, \Delta t) \rangle_t$ charakterisiert einen gesamten Prozessverlauf. Man beachte, dass diese Definition von der ursprünglichen Definition von Denning abweicht, der damit nur die minimale Anzahl der Seiten bezeichnete, die überhaupt für die Ausführung eines Prozesses notwendig sind.

Beispiel Die Arbeitsmenge bei Denning

Haben wir in einem Programm die Maschinenbefehle

```
MOVE A, B
MOVE C, D
```

und die Speicherplätze der Variablen A, B, C, D befinden sich auf verschiedenen Seiten, so kann der Prozess nur arbeiten, wenn er neben der Codeseite auch die vier Seiten der Adressreferenzen zur Verfügung hat. Er benötigt also minimal 5 Seiten, so dass nach Denning sein *working set* = 5 Seiten beträgt, unabhängig wie oft weitere Seiten benutzt werden.

Nachdem wir ein Verständnis für den Begriff der Arbeitsmenge gewonnen haben können wir nun die entsprechende Strategie formulieren:

Ersetze eine der Seiten, die nicht zum augenblicklichen working set gehören.

Da in dem Fenster alle zuletzt benötigten Seiten enthalten sind, könnte man denken, dass die Strategie identisch zu der LRU-Strategie ist. Dies stimmt aber nicht: Bei LRU wird die älteste unbenutzte Seite überschrieben, hier aber eine Seite außerhalb des Fensters, die durchaus vorher heftig benutzt worden sein kann. Die Ersetzungsstrategie geht also ebenfalls wie bei LRU in der Referenzfolge zurück, betrachtet aber nur die Seiten außerhalb des Fensters. In Abb. 3.17 ist die Strategie an unserem Beispiel gezeigt. Sie unterscheidet sich in diesem Beispiel nicht von der LRU-Strategie, da nur eine Seite außerhalb des *working set* ist.

Da wir maximal drei Seiten im Speicher haben, muss hier die Fenstergröße $\Delta t = 3$ sein. Wiederholen sich Seitenanforderungen innerhalb des Fensters, so ist das *working set* zu diesem Zeitpunkt kleiner als das Fenster, etwa im Zeitpunkt $t = 6$, wo das *working set* nur aus den Seiten eins und vier besteht.

Die Implementierung dieser Strategie ist nicht einfach, da die Größe des *working set* unbekannt ist und ständig wechselt. Eine gute Näherung besteht darin, mit Hilfe einer Kurzzeitstatistik, beispielsweise der R- und M-Bits, das *working set* zu bestimmen.

Abb. 3.17 Die *working set* – Seitenersetzung bei $\Delta t = 3$

Ziel	1	2	3	1	4	1	3	2	3
RAM1	1	1	1	1	1	1	1	1	1
RAM2	-	2	2	2	④	4	4	②	2
RAM3	-	-	3	3	3	3	3	3	3

t =

123456789

>

Ein anderer Weg besteht darin, für das *working set* zunächst einen Wert anzunehmen. Über die gemessene Zahl F der Seitenersetzungen pro Zeiteinheit (Seitenauscharge) bzw. die Zeit zwischen zwei Ersetzungen kann man nun diesen Wert so justieren, dass die Ersetzungsrate (*page fault frequency*) um einen Richtwert pendelt. Die Ersetzungsstrategie wird so dynamisch von der Lastverteilung zwischen den Prozessen abhängig gemacht und deshalb im Unterschied zu den statischen Algorithmen mit festen Warteschlangenlängen bzw. Fenstergrößen arbeiten als „Dynamischer Seitenersetzungsalgorithmus“ bezeichnet.

Die Anwendung der verschiedenen Strategien ist sehr unterschiedlich. Die Auswahl der besten Strategie und das Design weiterer Techniken werden verständlicher, wenn wir etwas tiefer in die Mechanismen der Seitenersetzung hineinschauen.

3.3.7 Modellierung und Analyse der Seitenersetzung

Dazu versuchen wir in diesem Abschnitt, aus der Beobachtung heraus Modelle für die Strategien und das Verhalten der Algorithmen zu finden. Betrachten wir dazu zuerst die Frage, wie groß eigentlich eine Seite sein sollte.

Die optimale Seitenlänge

Sei k die Hauptspeichergröße und s die Größe einer Seite. Hierbei handelt es sich bei s nicht um die Hardwaregröße einer Seite, sondern die Größe, die das Betriebssystem einer Seite zuweist; z. B. indem es mehrere HW-Seiten für eine SW-Seite verwendet.

Nehmen wir an, dass

- pro Prozess jede einstufige Seitentabelle $\lceil k/s \rceil$ Einträge aufweist, wobei jeder Eintrag eine Speichereinheit (z. B. Wort) in Anspruch nimmt.
- die Gesamtlänge der Daten pro Prozess zufallsmäßig über alle Prozesse uniform verteilt ist, so dass bei den Prozessen bei der letzten Seite alle Werte aus dem Intervall $]0, s]$ als Reststück des Verschnitts auftreten können, egal, welche Gesamtlänge gefragt war. Der über alle Prozesse gemittelte Verschnitt pro Prozess ist damit $s/2$ Speichereinheiten, z. B. Worte pro Prozess.

Dann entsteht ein mittlerer Verlust von

$$V = (k/s + s/2) \equiv k f_v$$

Speichereinheiten pro Prozess mit einem Verlustfaktor f_v . Einerseits wird bei größeren Seiten die Seitentabelle kleiner, andererseits aber auch der Verschnitt größer. Umgekehrt wird bei kleineren Seiten der Verschnitt geringer, aber die Tabellengröße steigt dafür an. Zwischen beiden Extremen gibt es ein lokales Minimum. Das Minimum an Verlust erhalten wir, wenn die Ableitung bezüglich s null wird:

$$\frac{\partial V(s_{\text{opt}})}{\partial s} = 0 \Leftrightarrow \left(\frac{1}{2} - \frac{k}{s_{\text{opt}}^2} \right) = 0 \Leftrightarrow \frac{1}{2} = \frac{k}{s_{\text{opt}}^2} \Leftrightarrow s_{\text{opt}} = \sqrt{2k} \text{ und } f_v = 2/s_{\text{opt}}$$

Beispiel

Sei $k = 5000$ (KW), so ist $s_{\text{opt}} = 100$ (KW) mit dem Verlustfaktor $f_v = 2/100 = 2\%$ des Hauptspeichers.

Die Seitengröße, die tatsächlich in Betriebssystemen verwendet wird, richtet sich aber noch an weiteren Kriterien aus. Beispielsweise bedeuten große Seiten bei geringen zusätzlichen Speichieranforderungen (Erweiterung des Stacks oder Heaps) einen hohen, unnützen Verschnitt, der mit dem Modell einer pro Seite gleichverteilten Speichieranforderung nicht übereinstimmt. Aus diesem Grund sind die Seitengrößen meist kleiner.

Weitere Faktoren für die Seitengröße sind die Zeit, die benötigt wird, um eine Seite auf den Massenspeicher zu verlagern (große Seiten haben eine relativ kleinere Transferzeit, da die Suchzeit hinzukommt), sowie der Speicherverschnitt des Massenspeichers bei einer mittleren Dateigröße. Da die meisten Dateien ca. 1 KiB groß sind, liegt für eine effiziente Ausnutzung der Massenspeicher die tatsächlich verwendete Seitengröße in diesem Bereich. Würden wir nämlich bei einer mittleren Dateigröße von 1 KiB eine Seitengröße von 100 KiB festlegen, so sind im Durchschnitt bei jeder Datei 99 KiB = 99 % ungenutzt – eine sehr schlechte Betriebsmittelauslastung.

Um beiden Forderungen nach kleinen Speichereinheiten und großen Transfermengen Rechnung zu tragen, versuchen deshalb viele Betriebssysteme, bei kleiner Seitengröße (1 KiB) immer große Portionen (mehrere Seiten) bei der Ein- und Ausgabe zu berücksichtigen (*read ahead*).

Die optimale Seitenzahl

Eine weitere wichtige Sache, die alle Systemadministratoren interessiert, ist die Antwort auf die Frage, wie viele Prozesse im Hauptspeicher zugelassen werden sollten. Dies entspricht der Frage, wie viele Seiten pro Prozess im Hauptspeicher reserviert sein müssen.

Betrachten wir dazu ein Beispiel. Wir vergleichen den FIFO-Algorithmus für ein System mit 4 (linke Tabelle) und 5 (rechte Tabelle) RAM-Seiten.

Wir finden eine erstaunliche Tatsache: Bei 4 RAM-Seiten sind nur 7 Ersetzungen nötig, bei 5 Seiten dagegen sind es 8 Ersetzungen, obwohl mehr Hauptspeicher verfügbar ist! Diese Tatsache, die bei manchen Algorithmen auftreten kann, heißt nach ihrem Entdecker Belady *Beladys Anomalie*.

Betrachten wir im Gegensatz dazu den LRU-Algorithmus. Dazu führen wir zunächst eine andere Form der Notation ein, indem wir die RAM-Seiten und die auf den Massenspeicher ausgelagerten Seiten (DISK-Seiten) gemeinsam so anordnen, dass in der Tabelle immer die in der spezifischen Reihenfolge des Algorithmus ersten Seiten ganz oben stehen. Rückt eine Seite auf die erste Zeile der jeweiligen Speicherart, so werden alle anderen Seitennummern entsprechend in die unteren Zeilen verschoben. Beim FIFO-Algorithmus

steht also oben immer die zuletzt eingelagerte Seite, beim LRU-Algorithmus immer die zuletzt benutzte.

Jede Spalte ist horizontal in zwei Teile geteilt: Im oberen Teil stehen die Seitennummern, bei denen die Seiten im Hauptspeicher (RAM) sind, und im unteren Teil stehen die Nummern der Seiten, die sich ausgelagert auf dem Massenspeicher (DISK) befinden. Die Reihenfolge dehnt sich dabei über die Markierung vom RAM in den DISK-Speicherbereich aus. Dabei ist nicht die tatsächliche Lage notiert, sondern nur die Prioritätsreihenfolge der jeweiligen Strategie. Die angeforderte Seite ist identisch mit der ersten Zeile des RAM-Bereichs; die ausgelagerte (ersetzte) Seite ist ans Ende der Warteschlange in den Zeilen des DISK-Bereichs notiert. Die Reihenfolgen von RAM und DISK gehen ineinander über.

Die Anomalie aus Abb. 3.18 ist zur Verdeutlichung nochmals nun in dieser Notation in Abb. 3.19 gezeichnet.

Betrachten wir nun die LRU-Seitenersetzung. Hier bietet sich ein etwas anderes Bild, siehe Abb. 3.20.

Die zusätzliche Verfügung über eine RAM-Seite ändert (im Unterschied zur FIFO) nicht die Seiten, die im Hauptspeicher stehen. Dies ist durchaus logisch: Bei der LRU-Strategie stehen die am meisten benutzten Seiten immer ganz oben in der Prioritätsliste, unabhängig davon, ob sie ausgelagert oder im RAM sind und damit unabhängig von der Grenze zwischen RAM und DISK.

Die Prioritätsliste des LRU lässt sich so auch mit Hilfe eines Stack-Mechanismus implementieren: Hochprioritäre Seiten stehen ganz oben und verschieben alle anderen früher benutzte nach unten. Die Seite, die über die RAM-DISK-Grenze hinweg verschoben wird,

Folge	3	4	0	1	5	6	0	1	2	3	5	6		3	4	0	1	5	6	0	1	2	3	5	6	Folge
RAM1	0	④	4	4	4	⑥	6	6	6	6	6	6		0	0	0	0	⑤	5	5	5	5	③	3	3	RAM1
RAM2	1	1	①	0	0	0	0	0	②	2	2	2		1	1	1	1	⑥	6	6	6	6	⑤	5		RAM2
RAM3	2	2	3	①	1	1	1	1	1	③	3	3		2	2	2	2	2	①	0	0	0	0	⑥		RAM3
RAM4	3	3	2	2	⑤	5	5	5	5	5	5	5		3	3	3	3	3	3	①	1	1	1	1		RAM4
														-	4	4	4	4	4	4	4	②	2	2	2	RAM5

Abb. 3.18 Die FIFO-Seitenersetzung

Folge	3	4	0	1	5	6	0	1	2	3	5	6		Folge	3	4	0	1	5	6	0	1	2	3	5	6	
RAM	3	④	①	①	⑤	⑥	6	6	②	③	3	3		RAM	3	4	4	4	⑤	⑥	①	①	②	③	⑤	⑥	
RAM	2	3	4	0	1	5	5	5	6	2	2	2		RAM	2	3	3	3	4	5	6	0	1	2	3	5	
RAM	1	2	3	4	0	1	1	1	5	6	6	6		RAM	1	2	2	2	3	4	5	6	0	1	2	3	
RAM	0	1	2	3	4	0	0	0	1	5	5	5		RAM	0	1	1	1	2	3	4	5	6	0	1	2	
DISK			0	1	2	3	4	4	4	0	1	1	1	RAM	-	0	0	0	1	2	3	4	5	6	0	1	
DISK						2	3	3	3	4	0	0	0	DISK					0	1	2	3	4	5	6	0	
DISK							2	2	2	3	4	4	4	DISK						0	1	2	3	4	4	4	

a) 4 RAM-Seiten

b) 5 RAM-Seiten

Abb. 3.19 Die FIFO-Seitenersetzung in Reihenfolge-Notation

Folge	3	4	0	1	5	6	0	1	2	3	5	6		3	4	0	1	5	6	0	1	2	3	5	6	Folge
RAM	③	④	⑥	①	⑤	⑥	0	1	②	③	⑤	⑥		3	4	0	1	⑤	⑥	0	1	②	③	⑤	⑥	RAM
RAM	2	3	4	0	1	1	6	0	1	2	3	5		2	3	4	0	1	1	6	0	1	2	3	5	RAM
RAM	1	2	3	4	0	0	1	6	0	1	2	3		1	2	3	4	0	0	1	6	0	1	2	3	RAM
RAM	0	1	2	3	4	5	5	5	6	0	1	2		0	1	2	3	4	5	5	5	6	0	1	2	RAM
DISK		0	1	2	3	4	4	4	5	6	0	1			0	1	2	3	4	4	4	5	6	0	1	RAM
DISK					2	3	3	3	4	5	6	0						2	3	3	3	4	5	6	0	DISK
DISK					2	2	2	3	4	4	4	4						2	2	2	3	4	4	4	4	DISK

Abb. 3.20 Die LRU-Seitenersetzung

wird ausgelagert auf den Massenspeicher. Die Stack-Liste ist dabei gleich, egal wie viel RAM bereit steht, siehe Abb. 3.20. Derartige Algorithmen, die immer die gleiche Liste aufbauen, also die gleichen m Seiten im Speicher halten, unabhängig davon, ob sie mit m oder mit $m + 1$ RAM-Seiten die gleiche Referenzfolge abarbeiten, heißen deshalb *Stack-Algorithmen*. Es lässt sich zeigen, dass bei Ihnen Beladys Anomalie nicht auftreten kann.

Das bedarfsgetriebene Einlagern und Ersetzen von Seiten wird als **demand paging** bezeichnet. Im Gegensatz dazu kann man das (wahrscheinlich) nötige Einlagern der Seiten des *working sets* voraussagen und diese Seiten eines schlafenden Prozesses laden, bevor der Prozess wieder aktiviert wird (**prepaging**).

Thrashing und seine Ursachen

Wenn ein Rechner mit vielen Aufgaben beladen wird, können wir manchmal beobachten, dass er plötzlich sehr langsam wird; alle Aktivität scheint im Zeitlupentempo zu erfolgen, der Rechner „quält“ sich. Dabei ist allerdings die Plattenaktivität schlagartig stark angestiegen. Dieses Verhalten wird als **thrashing** bezeichnet. Warum ist das so? Und wie können wir es verhindern?

Dazu müssen wir wissen, dass *thrashing* zwei verschiedene Ursachen haben kann: Zum einen tritt es kurzzeitig auf, wenn ein schlechter Seitenersetzungsalgorithmus Seiten auf die Platte schiebt, weil zu wenig Hauptspeicherplatz da ist, und danach wenn mehr Platz da ist, meint, nun kann er die Seiten wieder hereinholen – und damit wieder den Speicher füllt. Hier „flattern“ dauernd dieselben Seiten hinaus und hinein. Eine solche Instabilität kann kurzzeitig auftreten, dauert aber nicht lange.

Zum anderen aber kann bei der Rechner durch zu viele Prozesse und ihre Speicheranforderungen überlastet sein – und dies geht von allein nicht weg. Betrachten wir deshalb diesen Fall etwas genauer. Sehen wir in einem Speicher von k Seiten einen Prozess vor, der $m < k$ Seiten benötigt, so wird der Prozess erst einmal durch Seitentauschen nicht verzögert. Seine Bearbeitungsdauer habe den Wert B_1 . Angenommen, wir starten nun weitere derartige Prozesse. Obwohl nach n Prozessen der gesamte Speicherbedarf mit $n \cdot m > k$ größer ist als das Speicherangebot, ist die gesamte Bearbeitungsdauer $B_G \approx n \cdot B_1$ nur linear erhöht. Der zusätzliche, notwendige Austausch der Seiten scheint keine Zeit zu kosten – warum?

Betrachten wir dazu die Phasen, in denen Seiten vorhanden sind (mittlere Seitenverweilzeit t_s , Zustand *running*), und Phasen, in denen auf Seiten gewartet werden muss (t_w , mittlere Wartezeit, Zustand *blocked*), jeweils zusammengefasst zu kompakten Zeiten,

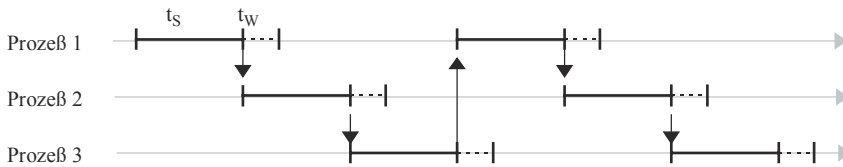


Abb. 3.21 Überlappung von Rechen- und Seitentauschphasen bei $t_s > t_w$

inklusive des Betriebssystemaufwands für Scheduling und Dispatching. In [Abb. 3.21](#) ist für drei Prozesse ein solcher Phasenablauf gezeichnet, wobei der Zustand *ready* ohne Linie gezeichnet ist. Wie man sieht, sind die Wartezeiten mit $t_s > t_w$ für den Seitenaustausch so kurz, dass immer ein rechenbereiter Prozess gefunden werden kann, der in der Wartephase der anderen abgearbeitet werden kann. Obwohl in jedem Prozess bei B_1/t_s Phasen eine Wartezeit t_w entsteht, wirkt sich diese nicht aus: Die Gesamtdauer ist durch die Summe der t_s bestimmt. Allerdings geht dies nicht für beliebig viele Prozesse gut.

Füllen wir den Rechner mit immer weiteren Prozessen, so beobachten wir einen erstaunlichen Effekt: Nach einer gewissen Zahl von Prozessen geht die Bearbeitungsdauer der einzelnen Prozesse plötzlich schlagartig hoch – die Prozesse „quälen“ sich durch die Bearbeitung. Durch die zusätzliche Belastung mit Prozessen nimmt einerseits die verfügbare Seitenzahl im Speicher pro Prozess ab (und damit die Länge der Phase t_s) und andererseits die Zahl der Seitenaustauschaktivitäten zu (und damit t_w). Da zwei Seitenaustauschprozesse nicht gleichzeitig ablaufen können, tritt bei $t_w > t_s$ an die Stelle des am wenigsten verfügbaren Betriebsmittels „Hauptprozessor“ (CPU) das Betriebsmittel „Seitenaustauschprozessor“ (PPU, meist mit DMA realisiert). In [Abb. 3.22](#) ist gezeigt, wie nun jede Austauschphase auf die andere warten muss; die Gesamtbearbeitungsdauer G wird überproportional größer als $n \cdot B_1$ und ist hauptsächlich durch die Summe der t_w bestimmt.

Eine genauere Analyse mit Hilfe einer Modellierung der Wahrscheinlichkeit des Seitenwechsels bei linear steigender Speicherauslastung ist im Anhang gegeben. Die einfache Modellierung zeigt, dass dabei die Bearbeitungszeit dramatisch nicht-linear ansteigen kann. Grundlage dafür ist die sog. **Lokalitätseigenschaft** von Programmen: Programmcode hat meist nur lokale Bezüge zu Daten und weist geringe Sprungweiten auf. Ist der Speicher aber zu gering, um den meisten lokalen Referenzen zu genügen, so steigt die Notwendigkeit stark an, neue Seiten einzulagern. Die Einlagerung einer ausreichenden Anzahl benachbarter Seiten ist also die Grundlage jeder Seitenwechselstrategie. Ohne die Lokalitätseigenschaft wären alle Seitenersetzungsstrategien wirkungslos und damit sinnlos.

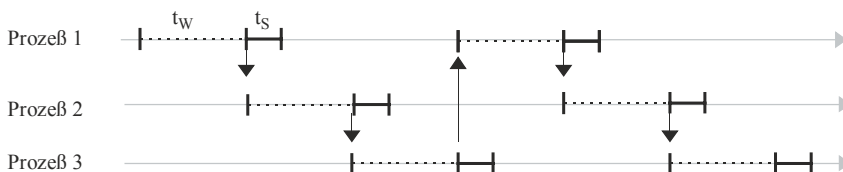


Abb. 3.22 Überlappung von Rechen- und Seitentauschphasen bei $t_s < t_w$

Schlussfolgerungen und Strategien

Aus den bisherigen Untersuchungen ergibt sich folgendes:

- Es ist wichtig, dass die Seiten der Arbeitsmenge aller Prozesse (*working set*) Platz im Speicher finden. Ist dies nicht möglich, so sollte die Zahl der Prozesse so lange verringert werden, bis dies der Fall ist.
- Allerdings reicht dies nicht aus. Entscheidend ist auch, das Speicherangebot auf das Verhältnis von Seitenverweilzeit zu Seitenwechseldauer abzustimmen. Ist dies nicht der Fall, so erhalten wir den *thrashing*-Effekt, selbst wenn wir genügend Speicher für das *working set* bereitstellen.

Wie können wir nun ein günstiges Systemverhalten zu erzielen? Dazu sind Hardware- und Softwaremaßnahmen nötig.

- *Hardwaremaßnahmen*

Auf dem Gebiet der Hardwaremaßnahmen lassen sich zum einen große Seitenverweilzeiten t_s erreichen, indem man die Seiten groß macht. Dies ist so lange sinnvoll, wie die Wartezeit für eine Seite überwiegend von einer initialen Zugriffsverzögerung (Festplatten: ca. 10 ms) und nicht von der Übermittlungszeit abhängt. Besser wären verzögerungslose Massenspeicher mit schnellem Zugriff, wie dies bei Festkörperspeichern angestrebt wird. Allerdings übernimmt dann in diesem Fall der Massenspeicher die Rolle des Hauptspeichers, so dass die gesamte Problematik sich nur verschiebt.

Zum anderen kann man t_T verkleinern, indem man mehrere Massenspeicher parallel für die Seitenauslagerung vorsieht (mehrere *swapping*-Platten).

- *Programmierungsmaßnahmen*

Kleine Seitenwechselwahrscheinlichkeit wird bei einer starken lokalen Ausrichtung des Programmcodes auf der Programmierenebene erreicht. Dies lässt sich beispielsweise durch Kopieren von Prozeduren der untersten Schicht einer Aufrufhierarchie derart durchführen, dass sie in der Nähe der aufrufenden Prozeduren stehen. Ein Beispiel dafür ist die Einrichtung von „Inline-Prozeduren“ anstelle eines Prozeduraufrufs. Dies ist auch bei großen Schleifen zu beachten, die besser in mehrere Schleifen kleinen Codeumfangs aufgeteilt werden sollten.

Auch der Algorithmus kann geeignet geändert werden, z. B. kann eine Matrizenmultiplikation so aufgeteilt werden, dass sie zeilenweise arbeitet, falls die Matrix zeilenweise abgespeichert ist. Bei spaltenweiser Abarbeitung würden sonst häufiger die Seiten gewechselt werden.

- *Working Set und Page Fault Frequency-Modell*

Auch das Betriebssystem kann einiges tun, um den *thrashing*-Effekt zu verhindern. Eine Strategie wurde schon angedeutet: Können wir für jeden Prozess mit Hilfe einer Kurzzeitstatistik, beispielsweise der R- und M-Bits, das *working set* bestimmen, so müssen wir dafür sorgen, dass nur so viele Prozesse zugelassen werden, wie Hauptspeicher für das *working set* vorhanden ist (**working-set-Modell**). Dies ist beispielsweise bei BS2000 (Siemens) und CP67 (IBM) der Fall.

Eine andere Strategie untersucht, ob die gemessene Zahl F der Seitenersetzungen pro Zeiteinheit (Seitentauschrate) bzw. die Zeit zwischen zwei Ersetzungen einen vorgegebenen Höchstwert F_0 überschreitet (**page-fault-frequency-Modell, PFF**) bzw. unterschreitet. Ist dies der Fall, so müssen Prozesse kurzzeitig stillgelegt werden. Ist dies nicht der Fall, so können mehr Prozesse aktiviert werden. Simulationen (Chu und Opderbek 1976) zeigen ein leicht besseres Verhalten gegenüber dem *working set*-Modell.

- *Das Nutzungsgradmodell*

Eine andere Idee besteht darin, die Systemleistung als solche direkt zu steuern. Dazu betrachten wir nochmals die Ausgangssituation. Je mehr Prozesse wir dazu nehmen, umso höher ist die Auslastung der CPU – so lange, bis die Wartezeit t_w größer als t_s wird. In diesem Fall staut sich alles bei der Seitenaustauscheinrichtung. Wir können dies als einen „Seitenaustauschprozessor PPU“ modellieren (Abb. 3.23) und die Systemleistung $L(n,t)$ als gewichtete Summe der beiden Auslastungen η_{CPU} und η_{PPU} schreiben:

$$L(n,t) = w_1 \eta_{\text{CPU}} + w_2 \eta_{\text{PPU}} \quad \text{Nutzungsgrad}$$

mit den Gewichtungen w_1 und w_2 .

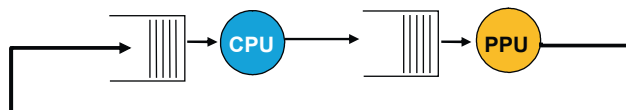
Simulationen (Badel et al. 1975) zeigen, dass ein solches Modell tatsächlich zunächst ein Ansteigen und dann rasches Abfallen des Nutzungsgrads bei Überschreiten einer kritischen Prozesszahl beinhaltet. Wie bei der adaptiven Ermittlung der Prozessparameter in Abschn. 2.2.2 kann auch hier $L(n,t)$ aus gemessenen Größen bestimmt werden. Sinkt L ab, so muss die Zahl der Prozesse vermindert, ansonsten erhöht werden. Auch eine verzögerte Reaktion (Hysterese) ist denkbar, um schnelle, kurzzeitige Änderungen zu vermeiden.

- *Globale vs. lokale Strategien*

Eine weitere Möglichkeit besteht darin, die Prozesse und ihren Platzbedarf nicht isoliert voneinander zu sehen, sondern den vorhandenen Platz dynamisch zwischen ihnen aufzuteilen. Dazu kann man beispielsweise nicht jedem Prozess den gleichen Speicherplatz, sondern den Platz proportional zu der Prozessgröße zuweisen. Auch eine initiale Mindestspeichergröße sollte eingehalten werden. Allerdings beachtet diese Strategie nicht die tatsächlich benötigte Zahl von Seiten, die (dynamisch wechselnde) Größe des *working set*.

Über die lokale Festlegung dieser Größe für jeden Prozess hinaus kann man versuchen, eine globale Strategie über alle Seiten aller Prozesse zu verfolgen, die i. allg. erfolgreicher ist. Dabei arbeiten die vorher eingeführten Algorithmen wie LRU und LFU auf der Gesamtmenge aller Seiten und legen damit die Arbeitsmenge der Prozesse dynamisch fest. Für den PFF-Algorithmus bedeutet dies, dass das Betriebssystem den

Abb. 3.23 Das Nutzungsgradmodell



Speicherplatz derart verteilen sollte, dass die Seitenersetzungen pro Zeiteinheit (*page-fault-frequency*) bei allen Prozessen gleich werden.

Der Nachteil der globalen Strategien liegt darin, dass ein Prozess Auswirkungen auf alle anderen haben kann. Existiert beispielsweise ein Prozess mit großen Platzansprüchen, so werden alle anderen gezwungen, öfter die Seiten zu wechseln und dabei zu warten. Bei einer lokalen Strategie dagegen muss nur der große Prozess warten; alle anderen können mit ihrem freien Platz weiterarbeiten. Dies ist zwar für die Gesamtheit aller Prozesse nicht optimal, es wird aber in Multi-user-Systemen von den einzelnen Benutzern als gerechter empfunden.

Die Strategie der „lazy evaluation“

Eine weitere Idee, um den Arbeitsaufwand für die Seiten zu reduzieren liegt in dem Prinzip der **lazy evaluation**. Hierbei werden alle Aktionen so lange wie möglich hinausgeschoben; es könnte ja sein, dass sie unterdessen überflüssig werden. Beispiele dafür sind

- *Copy On Write*

Wird die Kopie einer Seite benötigt, so wird die Originalseite nur mit einer Markierung (Statusbit **copy on write** = on) versehen. Erst wenn auf die Seite zum Schreiben zugegriffen wird, wird vorher die ursprünglich benötigte Kopie erstellt und dann die Seite modifiziert.

- *Page Out Pool*

Statt die Seiten, die auf den Massenspeicher verlagert werden, sofort hinauszuschreiben, kann man sie in einem Depot zwischenslagern (Zustand *standby*). Damit kann eine Seite, die nochmals benötigt wird, sofort wieder benutzt werden, ohne sie extra von der Platte holen zu müssen.

Ein solcher **page out pool** bildet ein Reservoir an Seiten, die unterschiedlich verwendet werden können. Gibt es sehr wenig freie Seiten, so kann man sie sofort für neue Seiten wiederverwenden; ist mehr Platz da, so lässt man sie etwas länger liegen und betrachtet sie als „potentiell aktive Reserve“.

Randprobleme

Neben dem Hauptproblem, wie der begrenzte Speicherplatz auf die konkurrierenden Prozesse aufgeteilt werden soll, gibt es auch andere Probleme, die eng mit dem Seitenmanagement zusammenhängen.

- *Instruktionsgrenzen*

Der Seitenfehler-Interrupt, der von der Adressierungshardware ausgelöst wird, wenn die Zieladresse nicht im Speicher ist, führt zum Abbruch des gerade ausgeführten Maschinenbefehls.

Ist durch die Seitenersetzungsmechanismen die gewünschte Seite im Hauptspeicher vorhanden, so muss dieselbe Instruktion erneut ausgeführt werden. Dazu muss allerdings das BS wissen, wo eine aus mehreren Bytes bestehende Instruktion angefangen

hat und nicht nur, wo sie eine ungültige Adresse referiert hatte. Aus diesem Grund sollte sie zum einen atomar sein, also keine Auswirkungen haben, wenn sie abgebrochen wird, und zum anderen ihre Anfangsadresse (PC) an einen Platz (in einem Register) abgespeichert sein (z. B. wie bei der PDP11/34).

Ist dies nicht der Fall und das BS muss sich erst mühsam aus den Daten eines Microcode-Stacks oder anderen Variablen die nötigen Daten erschließen, so wird eine Seitenersetzung unnötig verzögert.

- *I/O Pages und Shared Pages*

Ein weiteres Problem stellt die Behandlung spezieller Seiten dar. Wird für einen Prozess ein I/O-Datenaustausch angestoßen und ein anderer Prozess erhält den Prozessor, so kann es sein, dass die I/O-Seite des wartenden Prozesses im Rahmen einer globalen Speicherverteilung ausgelagert und die physikalische Seite neu vergeben wird. Erhält nach dem Seitentausch der I/O-Prozessor die Kontrolle, so wird die falsche Seite für I/O benutzt – fatal für beide Prozesse. Abhilfe schafft in diesem Fall eine Markierung von I/O-Seiten und der Ausschluss markierter Seiten vom Seitenaustausch.

Ein weiteres Problem sind Seiten, die von mehreren Prozessen benutzt werden (*shared pages, shared memory*), beispielsweise Codeseiten einer gemeinsamen Bibliothek (z. B. *C-library, Dynamic Link Library DLL*). Diese Seiten haben erhöhte Bedeutung und dürfen nicht mit einem Prozess ausgelagert oder bei Beendigung gestrichen werden, falls sie auch von einem anderen Prozess referiert werden. Auch dies muss bei den Seitenersetzungsalgorithmen berücksichtigt werden.

- *Paging Demon*

Die Arbeit der Seitenersetzungsalgorithmen kann effizienter werden, wenn man den Code als einen Prozess kapselt (*paging demon*) und ihn regelmäßig in Lastpausen ablaufen lässt. Dabei können nicht nur Anforderungen entsprochen werden, sondern auch vorausschauend Platz freigeräumt, wahrscheinlich benötigte Seiten eingelagert (*prepaging*), Statistikinformationen aktualisiert und regelmäßige Systemdienste ausgeführt werden, wie beispielsweise das Management des vorher erwähnten *page out pools*.

3.3.8 Beispiel UNIX: Seitenersetzungsstrategien

In Linux wird ein historisch gewachsener Algorithmus angewendet. Dazu existieren mehrere Prozesse: der *kswapd* wird für jeden Speichergerät aufgesetzt und prüft periodisch, ob noch genügend Seiten zur Verfügung stehen. Ist dies nicht mehr der Fall, so wird der *page frame reclaiming algorithm* PFRA (Gorman 2004) aufgerufen, um in den Seitenlisten geeignete Seiten zu finden, gekennzeichnet durch die *Read-* und *Modified-Bits*. Insgesamt unterscheidet er dabei vier Kategorien: a) löschbare, nicht referenzierte Seiten, b) synchronisierbare Seiten, etwa von Festplatten, c) auslagerbare Seiten (*swappable*), sowie d) nicht anforderbare Seiten (vom Kern benutzte Seiten und *locked pages*). Die globale Liste der zur Verfügung stehenden Seiten ist in LRU-Manier geordnet.

Der Algorithmus versucht maximal 32 Seiten auf einmal zu ersetzen und beginnt damit bei Kategorie a) zuerst. Ist dies nicht ausreichend, so werden die Seiten von Kategorie b) mit einem Clock-ähnlichen Algorithmus durchsucht; im ersten Durchgang wird das R-Bit zurückgesetzt und im zweiten Durchgang auf Änderung überprüft. Dies geschieht zuerst für die nicht beschriebenen, dann die allgemein referierten und zuletzt für die Einzelseiten der Prozesse. Ist auch dies nicht ausreichend, dann wird Kategorie c) durchsucht und bei mehr als 10 % beschriebener Seiten die auslagerbaren und synchronisierbaren Seiten von dem *pdflush*-Prozess in den Seitenpool (*paging area*) verschoben. Damit wird die Festplattenaktivität für die Seitenersetzung möglichst gering gehalten. Gesperrte Seiten bleiben unbeachtet. Die Belegung der Seitenrahmen selbst wird übrigens mit dem Buddy-Algorithmus durchgeführt.

Auch HP-UX Unix unterstützt die beiden Mechanismen des *swapping* und *paging*. Die direkte Speicherzuordnung des *swapping* wird immer dann benutzt, wenn schnelle Zugriffszeiten benötigt werden. So wird beispielsweise bei der Erzeugung von jedem Prozess automatisch eine Platzreservierung im *swap*-Bereich für die Auslagerung des Prozesses vorgenommen. Der *swap*-Bereich selbst kann verschieden organisiert sein: als eigenständige physikalische Einheit (*swap disk*), als logische Einheit (logisches Laufwerk), als Teil der Platte (*swap section*) oder als Teil eines Dateisystems (*swap directory*), wobei die Alternativen in der Reihenfolge ihrer Zugriffsauern aufgeführt wurden.

Für die Seitenersetzung gibt es in UNIX ein Zusammenspiel von zwei Mechanismen, die als zwei Hintergrundprozesse (**Dämonen**) ausgeführt sind.

Der Dämon zur Seitenauslagerung (*pageout demon*) setzt in regelmäßigen Zeitabständen das R-Bit (reference bit) jeder Seite zurück. Nach dem Zurücksetzen wartet er eine feste Zeitspanne Δt ab und lagert dann alle Seiten aus, deren R-Bit immer noch 0 ist.

Dieser Mechanismus tritt allerdings nur dann in Kraft, wenn weniger als 25 % des „freien“ Speicherplatzes (d. h. der Hauptspeichergröße, verringert um die Betriebssystemkerngröße, die Dämonen, die Gerätetreiber, die Benutzeroberfläche usw.) verfügbar ist. Da das Durchlaufen aller Seiten für einen Zeiger zu lange dauert, wurde in Berkeley UNIX für das Nachprüfen der R-Bits und das Auslagern ein zweiter Zeiger eingeführt, der dem ersten um Δt nachläuft. Dieser „zweihändige“ Uhrenalgorithmus wurde in das UNIX System V nicht übernommen. Statt dessen, um die Auslagerungsaktivität nicht schnellen, unnötigen Fluktuationen zu unterwerfen, werden die Seiten erst ausgelagert, wenn sie in n aufeinanderfolgenden Durchgängen unbenutzt bleiben.

Ist noch weniger Platz verfügbar, so tritt ab einem bestimmten Schwellwert *desfree* der *swapper demon* auf den Plan, deaktiviert so lange Prozesse und verlagert sie auf den Massenspeicher, bis das Minimum an freiem Speicher wieder erreicht ist. Ist es überschritten, so werden wieder Prozesse aktiviert. Im System V wurden dafür zwei Werte, *min* und *max*, eingeführt, um Fluktuationen an der Grenze zu verhindern. Betrachten wir beispielsweise einen großen Prozess, der ausgelagert wird, so erhöht sich der freie Speicher deutlich. Wird der Prozess wieder hereingeholt, so erniedrigt sich schlagartig der freie Speicher bis unter die Interventionsgrenze, und der Prozess wird wieder ausgelagert. Dieses fortdauernde Aus- und Einlagern („Seitenoszillation“) wird verhindert, wenn der

resultierende freie Speicher zwar größer als $\min$ ist (so dass kein weiterer Prozess ausgelagert wird), aber noch nicht größer als $\max$ (so dass auch noch kein Prozess eingelagert wird).

Hilft auch dies nicht, um den *thrashing*-Effekt zu verhindern, so muss vom Systemadministrator eingegriffen werden. Maßnahmen wie der Einsatz eines separaten *swap device* (Trennung von *swap system* und normalem Dateisystem), die Verteilung des *swapping* und *paging* auf mehrere Laufwerke oder die Installation von zusätzlichem Hauptspeicher kann die gewünschte Erleichterung bringen.

3.3.9 Beispiel Windows NT: Seitenersetzungsstrategien

In den vorhergehenden Abschnitten wurde ausgeführt, dass globale Strategien und Seitenersetzung, die auf der Häufigkeit der Benutzung der Seiten beruht, am günstigsten sind und die besten Ergebnisse bringen. Im Gegensatz dazu beruht die normale Seitenersetzungsstrategie in Windows NT auf lokaler FIFO-Seitenersetzungsstrategie. Die Gründe dafür sind einfach: Da globale Ersetzungsstrategien Auswirkungen von einem sich problematisch verhaltenden Prozess auf andere Prozesse haben, wählten die Implementatoren zum einen eine Strategie, die sich nur lokal auf den einzelnen Prozess beschränkt, um das subjektive Benutzerurteil über das Betriebssystem positiv zu stimmen. Zum anderen ist von den Seitenersetzungsmechanismen der einfachste und mit dem wenigsten Zeitaufwand durchführbare der FIFO-Algorithmus, so dass im Normalfall sehr wenig Zeit für Strategien verbraucht wird. Der Fehler, dabei auch häufig benutzte Seiten auszulagern, wird dadurch gemildert, dass die Seiten einen „zweite Chance“ bekommen: Sie werden zunächst nur in einen *page-out-pool* im Hauptspeicher überführt, aus dem sie leicht wieder zurückgeholt werden können. Mit dieser Maßnahme reduziert man auch den Platten I/O, da dann Cluster von modifizierten Seiten mit $M = 1$, die vor einer Benutzung auf Platte zurückgeschrieben werden müssen, in einem Arbeitsgang beschrieben werden können.

Ist allerdings zu wenig Speicher im System, so tritt ein zweiter Mechanismus in Kraft: das *automatic working set trimming*. Dazu werden systematisch alle Prozesse durchgegangen und überprüft, ob sie mehr Seiten zur Verfügung haben als eine feste, minimale Anzahl. Diese Minimalgröße kann innerhalb bestimmter Grenzen von autorisierten Benutzern (Systemadministrator) eingestellt werden. Die aktuelle Anzahl der benutzten Seiten eines Prozesses wird in Windows NT als „working set“ bezeichnet (im Gegensatz zu unserer vorigen Definition!). Verfügt der Prozess über mehr Seiten als für das *minimal working set* nötig ist, so werden Seiten daraus ausgelagert und das *working set* verkleinert.

Hat nun ein Prozess nur noch sein Minimum an Seiten, produziert er Seitenfehler und ist wieder zusätzlicher Hauptspeicherplatz vorhanden, so wird die *working set*-Größe wieder erhöht und dem Prozess damit mehr Platz zugestanden.

Bei der Ersetzung wird ein Verfahren namens „clustering“ angewendet: Zusätzlich zu der fehlenden Seite werden auch die anderen Seiten davor und danach eingelagert, um

die Wahrscheinlichkeit eines weiteren Seitenfehlers zu erniedrigen. Dies entspricht einer größeren effektiven Seitenlänge.

Eine weitere Maßnahme bildet die Anwendung des *copy on write*-Verfahrens, das besonders beim Erzeugen neuer Prozesse beim POSIX-Subsystem mit `fork()` wirksam ist. Da die Seiten des Elternprozesses nur mit dem *copy on write*-Bit markiert werden und der Kindsprozess meist danach gleich ein `exec()`-Systemaufruf durchführt und sich mit einem anderen Programmcode überlädt, werden die Seiten des Elternprozesses nicht unnötig kopiert und damit Zeit gespart.

Der gleiche Mechanismus wird auch bei den gemeinsam genutzten Bibliotheksdateien *Dynamic Link Libraries* DLL genutzt, die physisch nur einmal geladen werden. Ihre statischen Daten werden durch einen *copy on write*-Status gesichert. Greifen nun mehrere verschiedene Prozesse darauf zu und verändern die Daten, so werden nur die Daten davor auf private Datensegmente kopiert.

3.3.10 Aufgaben zur Seitenverwaltung

Aufgabe 3.3-6 Seitenersetzung

Was sind die grundlegenden Schritte einer Seitenersetzung?

Aufgabe 3.3-7 Referenzketten

Gegeben sei folgende Referenzkette: 0 1 2 3 3 2 3 1 5 2 1 3 2 5 6 7 6 5. Jeder Zugriff auf eine Seite erfolgt in einer Zeiteinheit.

- a) Wie viele Working-Set-Seitenfehler ergeben sich für eine Fenstergröße $h = 3$?
- b) Welches weitere strategische Problem ergibt sich, wenn innerhalb eines Working-Sets eine neue Seite eingelagert werden muss? Überlegen Sie dazu, wann es sinnvoll ist, eine Seite des Working-Sets zu setzen, und wann es sinnvoll ist, eine Seite hinzuzufügen und ggf. die Fenstergröße h dynamisch zu verändern, falls die maximale Working-Set-Größe noch nicht ausgeschöpft ist!

Aufgabe 3.3-8 Auslagerungsstrategien

Ein Rechner besitzt vier Seitenrahmen. Der Zeitpunkt des Ladens, des letzten Zugriffs und die R und M Bits für jede Seite sind unten angegeben (die Zeiten sind Uhrticks):

Seite	Geladen	Letzte Referenz	R	M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

- a) Welche Seite wird NRU ersetzen?
- b) Welche Seite wird FIFO ersetzen?
- c) Welche Seite wird von LRU ersetzt?
- d) Welche Seite wird die Second-Chance ersetzen?

Aufgabe 3.3-9 Working-Set

- a) Was versteht man unter dem *working set* eines Programms, wie kann es sich verändern?
- b) Welchen Effekt hat eine Erhöhung der Anzahl der Seitenrahmen im Arbeitsspeicher in Bezug auf die Anzahl der Seitenfehler? Welchen Effekt hat das auf die Seitentabellen?

Aufgabe 3.3-10 Seitenersetzung vs. swapping

Was sind die wichtigsten Eigenschaften von *page faults*, *working set* und *swapping*? Beschreiben Sie deren Vor- und Nachteile.

Aufgabe 3.3-11 Thrashing

- a) Was ist die Ursache von *thrashing*?
- b) Wie kann das BS dies entdecken, und was kann es dagegen tun?

Aufgabe 3.3-12 Thrashing selbst erleben

Erzeugen Sie nun selbst eine Thrashing-Situation. Besorgen Sie sich dafür eine möglichst große Bilddatei (am besten mehr als 1 GB) und öffnen sie diese mehrfach als Kopie, bis ihr Arbeitsspeicher voll ist.

- a) Beobachten und dokumentieren sie dabei jeweils, wie sich ihr PC im Laufe des Experiments verhält bezüglich der CPU – Auslastung, des freien und virtuellen Speichers und der I/O-Aktivität.
- b) Nachdem der Hauptspeicher voll ist, öffnen Sie weiterhin Bilder (ohne die alten zu schließen), bis Sie fast nicht mehr arbeiten können. Dokumentieren Sie auch hier die Leistungsdaten.
- c) Schließen Sie danach alle Bilder wieder und dokumentieren die Veränderungen (Prozesse, CPU-Auslastung etc. ...), die Sie erkennen, im Vergleich zu dem Stand, bevor sie mit Thrashing angefangen haben. Sollten sich Bilder nicht mehr öffnen lassen, nehmen sie ein anderes Anzeigeprogramm.
- d) Nennen Sie Lösungswege, wie man die Probleme des beobachteten Thrashing beheben kann.

3.4 Segmentierung

Unsere bisherige Sicht eines virtuellen Speichers modelliert den Speicher als homogenes, kontinuierliches Feld. Tatsächlich wird er aber in dieser Form von Programmierern nicht verwendet. Statt dessen gibt es sowohl benutzerdefinierte als auch immer wiederkehrende Datenabschnitte (**Segmente**) wie Stack und Heap, die sich dynamisch verändern und deshalb einen „Abstand“ im Adressraum zueinander benötigen. In [Abb. 3.24a](#) ist übersichtsweise die logische Struktur der Speicherauslegung eines Programms in UNIX gezeigt. In [Abb. 3.24b](#) ist die Speichereinteilung für das Beispiel eines Compilers gezeigt. Hier kommt das Problem hinzu, dass jedes Segment der Arbeitsdaten im Umfang (schraffierter Bereich in der Zeichnung) während der Programmausführung dynamisch wachsen und schrumpfen kann. Als Beispiel kann die Symboltabelle in den Quellcodebereich hineinwachsen: Es kommt zur Kollision der Adressräume.

Aus diesem Grund verfeinern wir unser einfaches Modell des virtuellen Speichers mit Hilfe der logischen Segmentierung zu einem **segmentierten** virtuellen Speicher. Zur Implementierung verwendet das Modell eine Speicherverwaltung, die an Stelle der Seitentabelle eine Tabelle der Segmentadressen (**Segmenttabelle**) benutzt. Die für ein Programm oder Modul jeweils gültige Segmenttabelle wird dann in entsprechenden Registern, den **Segmentregistern**, als Kopie gehalten, um die Umrechnung zwischen virtuellen und physikalischen Adressen zu beschleunigen. Bei der Umschaltung zwischen Programmen oder Moduln, die gleichzeitig im Hauptspeicher liegen, wird dann nur noch die Gruppe der Segmentregister neu geladen.

In der folgenden [Abb. 3.25](#) ist als einfaches Beispiel die Segmentadressierung beim INTEL 80286 gezeigt, bei dem zwei Codesegmente und zwei Datensegmente von Moduln existieren. Dazu kommen noch ein Stacksegment und zwei große, globale Datensegmente. Wendet man dies auf mehrere Programme an, so benötigt man dazu jeweils eine

Abb. 3.24 Die logische Unterteilung von Programmen

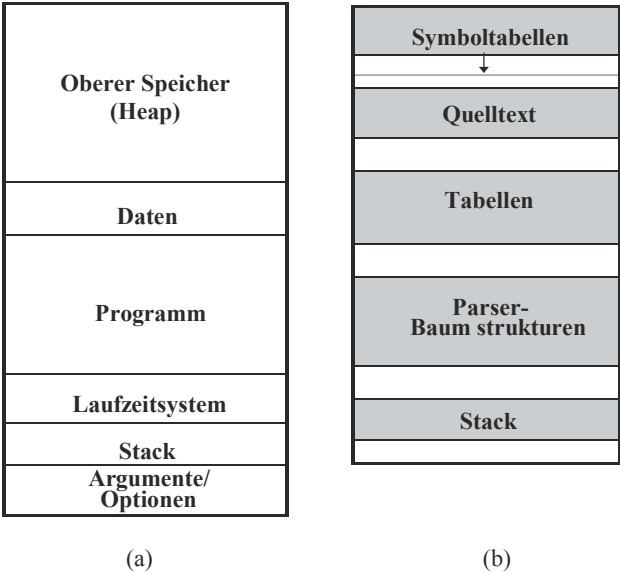
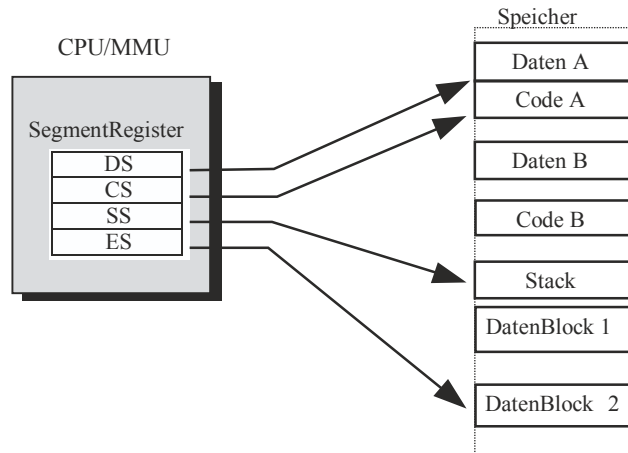


Abb. 3.25 Segment-Speicher-
verwaltung beim Intel 80286



Segmenttabelle und zusätzliche (Betriebs-)Systemstack- und globale Datenbereiche, z. B. zur Interprogrammkommunikation. Es ist günstig, die Zeiger zu den Segmenten in eigenen Registern zu halten. Beim 80286 sind dies je ein *CodeSegment*-Register CS, *DataSegment*-Register DS, *StackSegment*-Register SS und ein *ExtendedDataSegment*-Register ES.

Da die Segmente größere, unregelmäßige Speichereinheiten darstellen, kann der Speicher beim Ausladen eines größeren Segments und Einladen eines kleineren Segments von dem Massenspeicher nicht optimal gefüllt werden: Aus diesem Grund kann es beim häufigen *Swappen* zu einer Zerstückelung des Speichers kommen; die kleinen Reststücke können nicht mehr zugewiesen werden und sammeln sich verstreut an. Deshalb werden gern Paging und Segmentierung miteinander kombiniert, um die Vorteile beider Verfahren zu gewinnen: Jedes Segment wird in regelmäßige Seiten unterteilt.

Der gesamte Ansatz sei am Beispiel des Intel 80486 in [Abb. 3.26](#) übersichtsweise gezeigt. Die Datenstrukturen sind in zwei Teile gegliedert: in lokale Segmente, beschrieben durch die *Local Description Table LDT*, die für jeden Prozess privat existieren und die

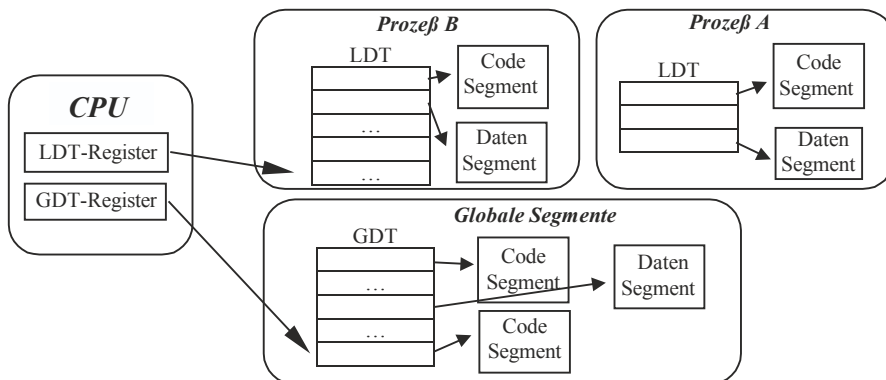


Abb. 3.26 Lokale und globale Seitentabellen beim INTEL 80486

Übersetzung der virtuellen Segmentadressen in physikalische Seitenrahmen durchführen, sowie in globale Segmente der *Global Description Table GDT*, die den Zugriff auf die globalen, für alle Prozesse gemeinsamen Daten ermöglichen. Dies ist vor allem der Betriebssystemcode, der vom Prozess im *kernel mode* abgearbeitet wird, sowie die gemeinsamen Speicherbereiche im System (*shared memory*).

In der Abbildung sind einige Segmente von einem aktiven Prozess B gezeigt; ein inaktiver Prozess A wartet im Speicher auf die Prozessorzuteilung. Die Segmente sind als Blöcke gezeichnet, wobei dies die Seitenbeschreibungstabellen mit den *page frames* zusammenfasst. Man sieht, dass bei der Prozessumschaltung der Übergang auf den virtuellen Adressraum von Prozess A sehr einfach ist: Man muss nur im LDT-Register den anderen Zeiger zu der LD-Tabelle von Prozess A laden – das ist alles.

Der Vorteil des segmentierten virtuellen Speichers liegt in einer effektiveren Adressierung und Verwaltung des Speichers. Die Segmentbasisadressen sind fest; existieren dafür Register in der CPU, so ist die erste Stufe der Adresskonversion schnell durchgeführt. Ist ein Segment homogen konstruiert (also ohne „Adresslöcher“), so ist es in dem verfeinerten Modell möglich, je nachdem ob das Speichersegment wächst oder abnimmt, in mehrstufigen Tabellen die dynamische Erzeugung und Vernichtung zusätzlicher Tabellen vorzusehen.

In diesem Fall ergänzen sich beide Modelle, der segmentierte Speicher und die Seiten des virtuellen Speichers, in geeigneter Weise: Das Modell der Segmente gibt eine inhaltliche Grobstrukturierung der Programmdaten vor, der virtuelle Speicher ermöglicht die fragmentfreie Implementierung der Segmente.

Der gemischte Ansatz von Segmenten und Seiten ist allerdings ziemlich aufwendig, da beide Verwaltungsinformationen (Segmenttabellen und Seitentabellen) geführt, erneuert und genutzt werden müssen. Aus diesem Grund nehmen bei heutigen Prozessoren das Adresswerk und die MMU einen wichtigen Platz neben der reinen Rechenleistung ein: Sie entscheiden mit über die Abarbeitungsgeschwindigkeit der Programme.

Aufgabe 3.4-1 Segmenttabellen

Es sei folgende Segment-Tabelle gegeben:

Segment	Segmentanfangsadresse	Segmentlänge
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96
5	900	30

Finden Sie heraus, ob es bei folgenden Adressen Speicherzugriffsfehler gibt oder nicht.

- a) 649
- b) 2310
- c) 195
- d) 1727

3.5 Cache

Die Vorteile einer schnellen CPU-Architektur kommen erst dann richtig zum Tragen, wenn der Hauptspeicher nicht nur groß, sondern auch schnell ist. Dies ist bei den heutigen dynamischen Speichern (*DRAM*) aber nicht der Fall. Um die preiswerten, aber relativ zum Prozessor langsamen Speicher trotzdem einsetzen zu können, benutzt man die Beobachtung, dass Programmcode meist nur lokale Bezüge zu Daten und geringe Sprungweiten aufweist. Diese **Lokalitätseigenschaft** der Programme ermöglicht es, in einem kleinen Programmabschnitt, der in einen schnellen Hilfsspeicher, den **Cache**, kopiert wird, große Abarbeitungsgeschwindigkeiten zu erzielen.

Die einfachste Möglichkeit, den Cache zu füllen, besteht darin, anstatt den Inhalt nur einer Adresse die Inhalte der darauf folgenden Adressen gleich mit in den Cache auslesen, bevor sie explizit verlangt werden. Der Datentransport vom langsamen DRAM (Dynamic Random Access Memory) wird hierdurch nur einmal ausgeführt. Der Datenblock (z. B. 4 Bytes) wird in einem schnellen Transport (*burst*) über den Bus in den Cache (*pipeline cache*) geschafft, wobei der Cache bei einem schnellen Bus und langsamen RAM auch beim RAM lokalisiert sein kann. Diese Art von Cache ermöglicht eine ca. 20 %ige Steigerung der Prozessorleistung. In Abb. 3.27 ist dies schematisch dargestellt.

Enthält der einfache Cache allerdings viele Daten, so kann es leicht sein, dass sie veralten und nur unbrauchbare, nichtaktuelle Daten darin stehen. Eine komplexere Variante des Caches enthält deshalb zusätzliche Mechanismen, um den Cache-Inhalt zu aktualisieren. Dazu werden alle Adressanfragen der CPU zuerst vom Cache bearbeitet, wobei für Instruktionen und Daten getrennte Cachespeicher (**instruction cache**, **data cache**) benutzt werden können (*Harvard-Architektur*). Ist der Inhalt dieser Adresse im Cache vorhanden (**Hit**), so wird schnell und direkt der Cache ausgelesen, nicht der Hauptspeicher. Ist die Adresse nicht vorhanden (**Miss**), so muss der Cache aus dem Hauptspeicher nachgeladen werden. In Abb. 3.28 ist das prinzipielle Schema dazu gezeigt.

Allerdings bereitet der Cache jedem Betriebssystementwickler ziemlich Kopfschmerzen. Nicht nur in Multiprozessorumgebungen, sondern auch in jedem Monoprozessorcomputer gibt es nämlich auch Einheiten, die parallel und unabhängig zur CPU arbeiten, beispielsweise die Ein- und Ausgabegeräte (Magnetplatte etc.), s. Abb. 3.29. Greifen diese direkt auf den Speicher zu, um Datenblöcke zu lesen oder zu schreiben (*direct memory*

Abb. 3.27 Benutzung eines pipeline-burst-Cache

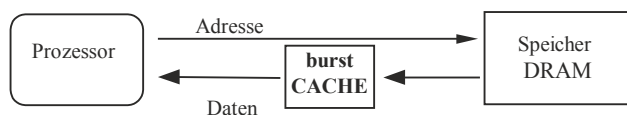


Abb. 3.28 Adress- und Datenanschluss des Cache

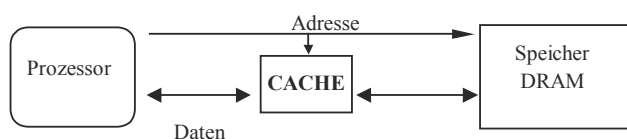
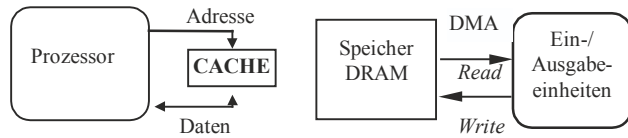


Abb. 3.29 Dateninkonsistenz bei unabhängigen Einheiten



access, DMA), so beeinflusst diese Unterbrechung die Abarbeitung des Programms normalerweise nicht weiter. Nicht aber in Cache-Systemen: Hier werden Cache und Hauptspeicher unabhängig voneinander aktualisiert.

Dies führt zu **Dateninkonsistenzen** zwischen Cache und Hauptspeicher, für deren Behebung es verschiedene Strategien gibt. Dazu unterscheiden wir den Fall, dass der zweite Prozessor (DMA-Transfer) den Speicher ausliest (*Read*) von dem, dass er hineinschreibt (*Write*). Im ersten Fall werden anstelle der aktuellen Daten des Caches die alten Daten aus dem Hauptspeicher vom zweiten Prozessor gelesen und z. B. ausgegeben.

Für dieses Problem existieren zwei Lösungen:

- *Durchschreiben (write through)*: Der Cache wird nur als Lesepuffer eingesetzt; geschrieben wird vom Prozessor über den Systembus direkt in den Hauptspeicher. So sind die Daten im Hauptspeicher immer aktuell. Dies ist allerdings langsam, es verzögert den Prozessor.
- *Rückschreiben (write back)*: Der Cache besorgt das Zurückschreiben selbst, ohne den Prozessor damit zu belästigen. Dies geht schneller für den Prozessor, verlangt aber einen zusätzlichen Aufwand beim Cache.

Beide Strategien verhindern aber auch die zweite Situation nicht, dass ein anderes Gerät durch DMA den Speicher verändert, ohne dass dies im Cache reflektiert wird. Um diese Art von Dateninkonsistenzen zu umgehen, muss der Cache mit einem Zusatzteil (*snooper*) versehen werden, der registriert, ob eine Adresse seines Bereiches auf dem Adressbus des Hauptspeichers zum Lesen oder Schreiben referiert wird. In diesem Fall gibt es eine dritte Lösung:

- *Aktualisieren (copy back)*. Wird der Hauptspeicher beschrieben, so wird für die entsprechende Speicherzelle im Cache vom *snooper* ein spezielles Bit gesetzt. Liest die CPU die Zelle, so veranlasst das gesetzte Bit vorher einen Cachezugriff auf den Hauptspeicher und stellt so die Konsistenz im Bedarfsfall her.

Für das Lesen durch die DMA-Einheit aus dem Hauptspeicher kann man zum einen die beiden obigen Strategien verwenden, die allerdings bei der ersten Lösung Prozessorzeit kosten oder bei der zweiten Lösung zu Dateninkonsistenzen führen, wenn der DMA-Transfer vor der Hauptspeicheraktualisierung stattfindet. Zur Prozessorentlastung ist deshalb ein erhöhter Aufwand sinnvoll, bei dem nicht nur das Beschreiben, sondern auch jedes Lesen des Hauptspeichers vom *snooper* überwacht wird. Wird eine Adresse

angesprochen, deren Datum sich im Cache befindet, so antwortet stattdessen der Cache und übermittelt den korrekten Wert an den DMA-Prozessor.

Diese Hardwarestrategien werden unterschiedlich in Systemen realisiert, beispielsweise durch *Cache Coherent Non-uniform Memory Access* (cc-NUMA). Das Betriebssystem muss sich darauf einstellen und alle Cache-Effekte, die nicht transparent für die Software sind (Dateninkonsistenzen), bei dem Aufsetzen der Aktionen berücksichtigen. Beispielsweise ist der Zustand des *shared memory* in einem Multiprozessorsystem ohne *write through*- oder *copy back*-Einrichtung nicht immer aktuell. Eine Kommunikation über *shared memory*, beispielsweise zwischen einem Debugger und dem überwachten Prozess, muss dies berücksichtigen und nach Aktionen alle Daten des Prozesses, in dem Programmierfehler gesucht werden, neu einlesen. Abhilfe schaffen hier Mechanismen, die in Hardware implementiert werden.

Beispiel Intel MESI

In symmetrischen Multiprozessorsystemen ist die für ungehinderten, gleichzeitigen Zugriff aller Prozessoren nötige Bustransferrate des zentralen Systembusses zu teuer und deshalb nicht vorhanden. Diese Begrenzung wird meist dadurch kompensiert, dass die Benutzerdaten dafür in einem prozessorspezifischen Cache gehalten und dort lokal bearbeitet werden. Die für globale Daten entstehenden Cacheinkonsistenzen werden beim MESI-Protokoll der Fa. Intel folgendermaßen gelöst: Jede Speichereinheit im Cache (*cache line*) hat ein Statuswort, das die vier Gültigkeitsstufen M (*modified*), E (*exclusive*), S (*shared*) und I (*invalid*) bezeichnet. Jeder Cache enthält einen *snooper*, der prüft, ob ein Datum, das im Cache ist, von einem anderen Prozessor in dem gemeinsamen Speicher geändert worden ist. Bei der Anforderung eines Datums wird deshalb zuerst geprüft, ob das Datum modifiziert wurde. Wenn ja, so muss es neu vom globalen Speicher eingelesen werden, sonst nicht. Wichtig dabei ist, dass vor allem die Software nichts davon mitbekommt, also die lokalen Caches sowohl für das Betriebssystem als auch für die Benutzerprogramme transparent funktionieren.

Es gibt noch eine weitere Zahl von Cachestrategien, auf die aber hier nicht weiter eingegangen werden soll.

Die Cacheproblematik ist nicht nur in Multiprozessorsystemen, sondern auch in vernetzten Systemen sehr aktuell. Hier gibt es viele Puffer, die als Cache wirken und deshalb leicht zu Dateninkonsistenzen führen können, wenn dies vom Nachrichtentransportsystem (Betriebssystem) nicht berücksichtigt wird.

Aufgabe 3.5-1 (Cache):

Der dem Hauptspeicher vorgeschaltete Cache hat eine Zugriffszeit von 50 ns. In dieser Zeit sei die Hit/Miss-Entscheidung enthalten. Der Prozessor soll auf den Cache ohne Wartezyklen und auf den Hauptspeicher mit drei Wartezyklen zugreifen können.

Die Trefferrate der Cachezugriffe betrage 80 %. Die Buszykluszeit des Prozessors ist 50 ns. Berechnen Sie

- a) die durchschnittliche Zugriffszeit auf ein Datum sowie
- b) die durchschnittliche Anzahl der benötigten Wartezyklen.

3.6 Speicherschutzmechanismen

Eine der wichtigsten Aufgaben, die eine Speicherverwaltung erfüllen muss, ist die Isolierung der Programme voneinander, um Fehler, unfaires Verhalten und bewusste Angriffe zwischen Benutzern bzw. ihren Prozessen zu verhindern.

Bei dem Modell eines virtuellen, segmentierten Speichers haben wir verschiedene sicherheitstechnische Mittel in der Hand. Die Vorteile einer virtuellen gegenüber einer physikalischen Speicheradressierung geben uns die Möglichkeit,

- eine vollständige Isolierung der Adressräume der Prozesse voneinander zu erreichen. Da jeder Prozess den gleichen Adressraum, aber unterschiedlichen physikalischen Speicher hat, „sehen“ die Prozesse nichts voneinander und können sich so auch nicht beeinflussen.
- einem jeden Speicherbereich bestimmte Zugriffsrechte (*lesbar, beschreibbar, ausführbar*) konsistent zuordnen und dies auch hardwaremäßig überwachen zu können. Wird beispielsweise vom Programm aus versucht, den Programmcode selbst zu überschreiben (bedingt durch Speicherfehler, Programmierfehler, Compilerfehler oder Viren), so führt dies zu einer Verletzung der Schutzrechte des Segments und damit automatisch zu einer Fehlerunterbrechung (*segmentation violation*), also einem speziellen Systemaufruf, der vom Betriebssystem weiter behandelt werden kann. Auch der Gebrauch von Zeigern ist so in seinen Grenzen überprüfbar; eine beabsichtigte und unbeabsichtigte Beeinflussung anderer Programme und des Betriebssystems wird unterbunden.
- die einfachen Zugriffsrechte (*read/write*) für verschiedene Benutzer verschieden zu definieren und so eine verfeinerte Zugangskontrolle auszuüben.

Die Sperrung von Seiten gegen unerlaubten Zugriff, etwa die Ausführung von Code auf einem Stack, kann man also durch Setzen von speziellen Bits, den NX (*no execute*) Bits, verhindern. Leider ist es bei dem heutzutage vorherrschenden linearen Programmiermodell, das Code und Daten vermischt, schwer möglich, einzelnen Seiten eine exklusive Verwendung zuzuordnen. So lässt sich eine missbräuchliche Befehlsausführung auf Daten nicht verhindern, wenn auf derselben Seite auch Code steht und dasselbe Segmentregister verwendet wird.

Ein weiteres Problem stellt die Hardwareunterstützung dar. Die weit verbreitete Intel 32 Bit x86-Architektur unterstützt solchen Zugriffsschutz durch Extrabits nur dann, wenn

auch eine PEA (*physical address extension*) existiert und aktiviert werden kann, also eine Erweiterung des 32 Bit-Adressraums um bis zu 10 weiteren Bits. Allerdings besitzen alle modernen 64Bit-Rechnerarchitekturen inzwischen die Möglichkeit, den Hardwarezugriff auf den Hauptspeicher mittels spezieller Zugriffsrechte (Bits) abzusichern.

Außerdem gibt es noch weitere Speicherschutzmechanismen, die hardwaremäßig abgesichert werden können:

- *Memory Curtaining*

Spezielle Speicherstellen, die hochsensitive Informationen enthalten wie Schlüssel von Verschlüsselungssystemen etc. müssen besonders geschützt werden. Dies kann man mit speziellen Hardwaremaßnahmen erreichen, etwa wie Intel mit der „*Trusted Execution Technology TXT*“. Hier wird neben einem sicheren Bootstrap durch den TPM-Chip (s. Kapitel „Sicherheit“) auch alle Hardwareressourcen eines Prozesses (z. B. Maus oder Keyboard) exklusiv während der Ausführung alleinig dem Prozess gewidmet.

- *Sealed Storage*

Persönliche Informationen können gestohlen werden. Eine Idee, dies zu verhindern, besteht darin, sie zu kodieren und nur dann eine Dekodierung zu ermöglichen, wenn der Hardwarekontext (CPU-Nummer, Hash-Wert der verwendeten Hardwareplattform aus Mainboard, Festplatten und Hauptspeicher) stimmt. Dies kann zum Schutz vertraulicher Informationen genutzt werden, aber auch, um Musikstücke beim *Digital Rights Management DRM* nur auf einem einzigen Rechner abspielbar zu machen.

3.6.1 Speicherschutz in UNIX

In UNIX gibt es über die üblichen, oben beschriebenen Speicherschutzmechanismen hinaus spezielle Sicherheitskonzepte. So werden unter Linux generell die NX-Bits unterstützt. Unter OpenBSD kommen noch zusätzliche Mechanismen zum Einsatz:

- „W^X“ (W XOR X): Jede Seite ist entweder schreib- oder ausführbar (via NX-Bit), aber nicht beides. Dies ist auch unter Linux mit Kernel-*patches* (PAX etc.) möglich.
- Wird bei der Speicheranforderung mittels des Systemaufrufs *malloc* mehr als eine Seite Speicher angefordert, so wird dahinter eine „*guard page*“ Sicherung gesetzt, so dass ein *segmentation fault* ausgelöst wird, wenn die Seite dahinter referiert wird.
- Der Compiler nutzt eine spezielle „*ProPolice*“-Taktik: lokale Variable etwa Puffer, werden auf dem Stack nach den lokalen Variablen mit Zeigern angeordnet, so dass ein Pufferüberlauf keine Sprungadressen überschreiben kann. Außerdem wird ganz am Schluss ein sog. „*Canary*“-Wert auf den Stack gelegt, der vor dem Rücksprung der Funktion auf Unversehrtheit getestet wird. Wenn er verändert ist, bedeutet dies, dass ein Overflow aufgetreten war. Das Programm wird in diesem Fall sicherheitshalber abgebrochen.

3.6.2 Speicherschutz in Windows NT

Die Speicherschutzmechanismen in Windows NT ergänzen die dem virtuellen Speicher inhärenten Eigenschaften wie die Isolierung der Prozesse voneinander durch die Trennung der Adressräume, die Einführung eines *user mode*, in dem nicht auf alle möglichen Bereiche im virtuellen Speicher zugegriffen werden darf, sowie die Spezifizierung der Zugriffsrechte (Read/Write/None) für *user mode* und *kernel mode*.

Zusätzlich gibt es eine Sicherheitsprüfung, wenn der Prozess einen Zugang zu privatem oder gemeinsam genutztem Speicher versucht. Die Schwierigkeit, derartige Mechanismen portabel zu implementieren, liegen in den enormen Unterschieden, die durch verschiedene Hardwareplattformen (wie beispielsweise der RISC-Prozessor MIPS R4000, der bis Windows NT Version 4 unterstützt wurde, im Unterschied zum Intel Pentium) bei den hardwareunterstützten Mechanismen existieren. Beispielsweise kann beim R4000 im *user mode* prinzipiell nur auf die unteren 2 GB zugegriffen werden, ansonsten wird eine Fehlerunterbrechung (*access violation*) wirksam. Ab Windows 8 ist eine Installation auf Hardware ohne NX-Bits nicht mehr möglich.

Auf den Hardwaremechanismen baut nun der VM Manager auf, der bei einem Seitenfehler eingeschaltet wird. Zusätzlich zu den einfachen Zugriffsrechten (Read Only, Read/Write) verwaltet er noch die Informationen (*execute only*, *guard page*, *no access* und *copy on write*), die mit der Prozedur `VirtualProtect()` gesetzt werden können. Diese werden nun näher erläutert.

- *Execute Only*

Das Verbot, Daten zu schreiben oder zu lesen ist besonders bei gemeinsam genutzten Programmcode (z. B. einem Editor) sinnvoll, um unerlaubte Kopien und Veränderungen des Originalprogramms zu verhindern. Da dies von der Hardware meist nicht unterstützt wird, ist es i.d.R. mit *ReadOnly* identisch.

- *Guard Page*

Wird eine Seite mit dieser Markierung versehen, so erzeugt der Benutzerprozess eine Fehlerunterbrechung (*guard page exception*), wenn er versucht, darauf zuzugreifen. Dies ermöglicht beispielsweise einem Subsystem oder einer Laufzeitumgebung, ein dynamisches Feld, hinter dem die Seite markiert wurde, automatisch zu erweitern, wenn auf Feldelemente zugegriffen wird, die noch nicht existieren. Nach der Fehlerunterbrechung kann der Prozess normal auf die Seite zugreifen.

Ein gutes Beispiel dafür ist der Mechanismus zur Regulierung der Stackgröße, die mit Hilfe einer *guard page exception* dynamisch bei Bedarf erhöht wird.

- *No Access*

Diese Markierung verhindert, dass auf nichtexistierende oder verbotene Seiten zugegriffen wird. Dieser Status wird meist verwendet, um bei der Fehlersuche (*debugging*) einen Hinweis zu erhalten und den Fehler einzukreisen.

- *Copy on Write*

Den Mechanismus des *copy on write*, der auf Seite 150 erklärt wurde, kann auch für den Schutz von Speicherabschnitten verwendet werden. Wird ein gemeinsamer

Speicherabschnitt von Prozessen nur als „privat“ deklariert, so wird zwar eine Zuordnung in den virtuellen Adressraum wie in [Abb. 3.9](#) vorgenommen, aber die virtuellen Seiten des Prozesses werden „*copy on write*“ markiert. Versucht nun ein Prozess, eine solche Seite zu beschreiben, so wird zuerst eine Kopie davon erstellt (mit *read/write*-Rechten, aber ohne *copy on write* auf der Seite) und dann die Operation durchgeführt. Alle weiteren Zugriffe erfolgen nun auf der privaten Kopie und nicht mehr auf dem Original – für diese eine Seite.

Eine weitere wichtige Speicherschutzeinrichtung ist die für die C_2 -Sicherheitsstufe geforderte Löschung des Seiteninhalts, bevor eine Seite einem Benutzerprozess zur Verfügung gestellt wird. Dies wird vom VM-Manager automatisch beim Übergang einer Seite vom Zustand *free* zum Zustand *zeroed* durchgeführt.

3.6.3 Sicherheitsstufen und *virtual mode*

Einen zusätzlichen Schutz bietet das Konzept, Sicherheitsstufen für die Benutzer einzuführen. Die in [Kap. 1](#) eingeführten Zustände *user mode* und *kernel mode* sind eine solche Einteilung. Sie besteht hier aus zwei groben Stufen, einem privilegierten Modus (*kernel mode*), in dem der Code alles darf, beispielsweise auf alle Speicheradressen zugreifen, und einem nicht-privilegierten Modus (*user mode*), in dem ein Prozess keinen Zugriff auf das Betriebssystem und alle anderen Prozesse hat. Diese Sicherheitsstufen lassen sich noch weiter differenzieren, wobei die Stufen nach dem Vertrauen angeordnet sind, das man in die jeweiligen Repräsentanten hat. Ihre Wirkung hängt sehr stark davon ab, ob sie auch von der Hardware unterstützt werden. Da man zu jeder Stufe nur über die vorhergehende kommt, kann man dies auch mit dem Zwiebelschalenmodell aus [Abschn. 1.2](#) als ein System von *Ringen* modellieren, um ihre Abgeschlossenheit zu zeigen.

Beispiel: Intel 80x86-Architektur

Die weitverbreitete Intel-80X86-Prozessorlinie besitzt in den moderneren Versionen mehrere Sicherheitsstufen. Wird der Prozessor durch ein elektrisches Signal zurückgesetzt, so befindet er sich im *real mode* und ist damit 8086 und 80186 kompatibel. Der wirksame Adressraum ist mit dem physikalischen Adressraum identisch; alle Prozesse können sich gegeneinander stören. Üblicherweise wird nach einem *reset* zuallererst das Betriebssystem eingelesen, das in diesem Modus seine Tabellen einrichtet. Dann aber setzt es das *virtual mode*-Bit (das nicht zurückgesetzt werden kann) und setzt so die MMU in Betrieb und damit den Zugriffsschutz der Ringe. Alle weiteren Speicherzugriffe benutzen virtuelle Adressen, die nun erst übersetzt werden müssen; alle nun aufgesetzten Prozesse sind in ihrem jeweiligen virtuellen Adressraum isoliert voneinander, laufen im *user mode* und müssen die kontrollierten Übergänge benutzen, um in den *kernel mode* von Ring 0 zu kommen.

Literatur

- Badel M., Gelenbe E., Leroudier I., Potier D.: Adaptive Optimization of a Time-Sharing System's Performance. Proc. IEEE 63, 958–965 (1975)
- Belady L. A.: A Study of Replacement Algorithms for a Virtual Storage Computer. IBM Systems Journal 5, 79–101 (1966)
- Chu W. W., Opderbek H.: Analysis of the PFF Replacement Algorithm via a Semi-Markov Model. Commun. of the ACM 19, 298–304 (1976)
- Denning P. J.: Working Sets Past and Present. IEEE Trans. on Software Eng. SE-6, 64–84 (1980)
- Gorman M.: Understanding the Linux Virtual Memory Manager. Prentice Hall 2004, <https://www.kernel.org/doc/gorman/pdf>, [24.4.2016]
- Knolton K. C.: A Fast Storage Allocator. Commun. of the ACM 8, 623–625 (1965)
- Peterson J.L., Norman, T.A.: Buddy Systems. Commun. of the ACM 20, 421–431 (1977)

Inhaltsverzeichnis

4.1 Dateisysteme 176

4.2 Dateinamen 178

 4.2.1 Dateitypen und Namensbildung..... 178

 4.2.2 Pfadnamen 182

 4.2.3 Beispiel UNIX: Der Namensraum..... 183

 4.2.4 Beispiel Windows NT: Der Namensraum 185

 4.2.5 Aufgaben 187

4.3 Dateiattribute und Sicherheitsmechanismen 188

 4.3.1 Beispiel UNIX: Zugriffsrechte..... 188

 4.3.2 Beispiel Windows NT: Zugriffsrechte 190

 4.3.3 Aufgaben 191

4.4 Dateifunktionen 192

 4.4.1 Standardfunktionen 192

 4.4.2 Beispiel UNIX: Dateizugriffsfunktionen..... 193

 4.4.3 Beispiel Windows NT: Dateizugriffsfunktionen 194

 4.4.4 Strukturierte Zugriffsfunktionen 195

 4.4.5 Gemeinsame Nutzung von Bibliotheksdateien 202

 4.4.6 Speicherabbildung von Dateien (*memory mapped files*) 204

 4.4.7 Besondere Dateien (*special files*)..... 206

 4.4.8 Aufgaben zu Dateikonzepten..... 207

4.5 Implementierung der Dateiorganisation..... 209

 4.5.1 Kontinuierliche Speicherzuweisung 209

 4.5.2 Listenartige Speicherzuweisung 210

 4.5.3 Zentrale indexbezogene Speicherzuweisung..... 210

 4.5.4 Verteilte indexbezogene Speicherzuweisung..... 212

 4.5.5 Beispiel UNIX: Implementierung des Dateisystems..... 213

 4.5.6 Beispiel Windows NT: Implementierung des Dateisystems 215

 4.5.7 Aufgaben zu Dateimplementierungen 219

Literatur..... 219

Bei einer Unterbrechung oder nach dem Ablauf eines Prozesses stellt sich die Frage: Wie speichere ich meine Daten so ab, dass ich später wieder damit weiterarbeiten kann? Diese als „**Persistenz** der Daten“ bezeichnete Eigenschaft erreicht man meist dadurch, dass man die Daten vor dem Beenden des Programms auf einen Massenspeicher schreibt. Allerdings haben wir das Problem, dass bei großen Massenspeichern die Liste der Dateien sehr lang werden kann. Um die Zugriffszeiten nicht ebenfalls zu lang werden zu lassen, ist es deshalb günstig, eine Organisation für die Dateien einzuführen.

4.1 Dateisysteme

Dazu schreiben wir uns zunächst alle Dateien in eine Liste und vermerken dabei, was wir sonst noch über die Datei wissen, z. B. das Datum der Erzeugung, die Länge, Position auf dem Massenspeicher, Zugriffsrechte usw.

Eine solche Tabelle ist nichts anderes als eine relationale Datenbank. Geben wir zu jedem Datensatz (*Attribut*) auch Operationen (*Prozeduren, Methoden*) an, mit denen er bearbeitet werden kann, so erhalten wir eine objektorientierte Datenbank. Haben wir außerdem noch Bild- und Tonobjekte, die das Programm erzeugt oder verwendet, so benötigen wir eine Multimedia-Datenbank, in der zusätzlich noch die Information zur Synchronisierung beider Medien enthalten ist. Allgemein wurden für kleine Objekte die *Persistent Storage Manager PSM* und für große Datenbanken die *Object-Oriented Database Management Systems OODBMS* entwickelt.

Wir können alle Mechanismen, die zur effizienten Organisation von Datenbanken erfunden wurden, auch auf das Dateiverzeichnis von Massenspeichern anwenden, beispielsweise die Implementierung der Tabelle in Datenstrukturen (z. B. Bäumen), mit denen man sehr schnell alle Dateien mit einem bestimmten Merkmal (Erzeugungsdatum, Autor, ...) heraussuchen kann.

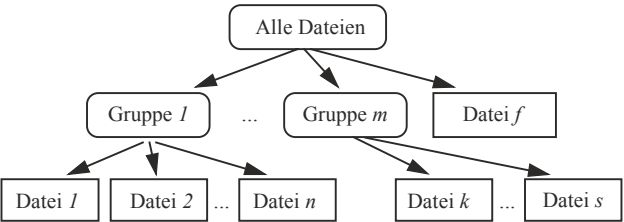
Allerdings müssen wir dabei auch die Probleme berücksichtigen, die Datenbanken haben. Ein wesentliches Problem ist die Datenkonsistenz: Erzeugen, löschen oder ändern wir die Daten einer Datei auf dem Massenspeicher, so muss dies auch im Datenverzeichnis vermerkt werden. Wird diese Operation abgebrochen, beispielsweise weil der schreibende Prozess abgebrochen wird (z. B. bei Betriebsspannungsausfall, *power failure*), so sind die Daten im Verzeichnis nicht mehr konsistent. Nachfolgende Operationen können Dateien überschreiben oder freien Platz ungenutzt lassen. Einen wichtigen Mechanismus, um solche Inkonsistenzen auszuschließen, haben wir in [Abschn. 2.3.2](#) kennengelernt: das Zusammenfassen mehrerer Operationen zu einer *atomaren Aktion*. Ausfalltolerante Dateisysteme müssen also alle Operationen auf dem Dateiverzeichnis als atomare Aktionen implementieren.

Die Bezeichnung der Dateien mit einem Namen anstatt einer Nummer ist eine wichtige Hilfe für den menschlichen Benutzer. In der Dateitabelle in [Abb. 4.1](#) können nun zwei Dateien mit gleichem Namen vorkommen. Da sie zwei verschiedene Nummernschlüssel haben, sind sie trotzdem für die Dateiverwaltung klar unterschieden, nicht aber für den

Abb. 4.1 Dateiorganisation als Tabelle

Nummer	Name	Position	Länge	Datum	...
0	Datei1.dat	264	1024	11.12.87	
1	MyProgram	234504	550624	23.4.96	
2	Datei2.dat	530	2048	25.1.97	
...	...	...	...	...	...

Abb. 4.2 Hierarchische Dateiorganisation



menschlichen Benutzer, der seine Operationen (Löschen, Beschreiben usw.) nur mit dem Namen referenziert. Wie kann man nun eindeutige Namen bilden, um dieses Problem zu lösen?

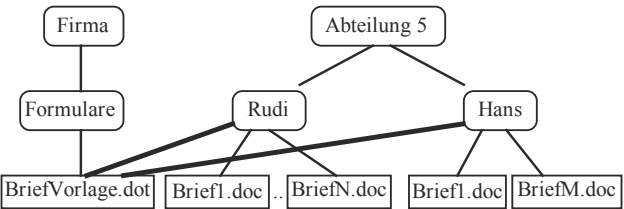
Eine Möglichkeit besteht darin, die Dateien hierarchisch in Untergruppen zu gliedern und daraus eindeutige Namen abzuleiten. In [Abb. 4.2](#) ist eine solche Organisation gezeigt.

Die Gruppen können ebenfalls wieder zu einer Großgruppe zusammengestellt werden, so dass schließlich eine Baumstruktur resultiert. Diese Art der Dateiorganisation ist sehr verbreitet. Die Gruppenrepräsentanten werden als **Dateiordner** oder als Kataloge/ Verzeichnisse (**directory**) bezeichnet. Dabei können auch Einzeldateien und Ordner gemischt auf einer Stufe der Hierarchie vorhanden sein, wie dies oben mit „Datei f“, angedeutet ist.

Eine alternative Dateiorganisation wäre beispielsweise die Einrichtung einer Datenbank, in der alle Dateien mit ihren Eigenschaften und Inhaltsangaben verzeichnet sind. Die Übersicht über alle Dateien ist dann in einem *data directory* vorhanden. In den meisten Betriebssystemen werden aber nur einfache, hierarchische Dateisysteme benutzt.

Eine weitere Möglichkeit in hierarchischen Systemen ist die zusätzliche Vernetzung. Betrachten wir dazu das Beispiel in [Abb. 4.3](#). Hier sind zwei Verzeichnisse gezeigt: das von Rudi und das von Hans, die beide Briefe abgespeichert haben.

Abb. 4.3 Querverbindungen in Dateisystemen



Da die Briefvorlage, die sie dafür benutzen, immer aktuell bleiben soll, haben sie eine Querverbindung zu der verbindlichen Firmenbriefvorlage gezogen, so dass sie alle dieselbe zentral verwaltete Vorlage mit demselben Briefkopf und den aktuellen Angaben benutzen. Die ursprüngliche Baumstruktur der Verzeichnishierarchie wird so zu einem azyklischen Graphen.

4.2 Dateinamen

Aus der Notwendigkeit heraus, die Daten der Prozesse nicht nur dauerhaft auf Massenspeichern zu sichern, sondern auch den Zugriff darauf durch andere Prozesse zu ermöglichen, wurde jeder Datei ein eindeutiger Schlüssel zugewiesen. Da die Organisation der Dateien meist von Menschen durchgeführt wird, ebenso wie die Programmierung der Prozesse, die auf die Dateien zugreifen, hat sich anstelle einer Zahl (wie z. B. die Massenspeicherposition) als Schlüssel eine symbolische Namensgebung durchgesetzt, wie „Datei1“ oder „Brief an Alex“. Dies hat den Vorteil, dass die interne Verwaltung der Datei (Position, Länge usw.) nach außen verborgen bleibt; die Datei kann aktualisiert und an einer anderen Stelle auf dem Massenspeicher abgelegt werden, ohne dass sich ihr Name (und damit die Referenz in allen darauf bezogenen Programmen!) ändert.

4.2.1 Dateitypen und Namensbildung

Historischerweise bestehen Namen aus zwei Teilen, die durch einen Punkt getrennt sind: dem eigentlichen Namen und einem Anhang (**Extension**), der einen Hinweis auf den Verwendungszweck der Datei geben soll. Beispielsweise soll der Name „Brief.txt“ andeuten, dass die Datei „Brief“ von der Art „txt“ ist, eine einfache Textdatei aus ASCII-Buchstaben. Es gibt verschiedene Namenskonventionen für die Anhänge wie beispielsweise

.odt,.doc,.docx	Textdokument in dem speziellen Format des Texteditors
.c,.cc,.java,.py	Quellcodes in verschiedenen Programmiersprachen
.pdf	<i>Portable Document File</i> : Dateien nur zum Ausdrucken
.zip,.gz,.7z	Komprimierte Dateien
.tar, rar	ein gesamtes Dateisystem, in einer Datei abgespeichert
.html	Textdatei für das <i>world wide web</i> -Hypertextsystem
.jpg,.gif,.png	Bilddateien in verschiedenen Formaten
.tif,.bmp,.raw	
.dat	Daten; Format hängt vom Erzeugerprogramm ab

Wie man bemerkt, besteht die Extension meist nur aus wenigen, manchmal etwas kryptischen Buchstaben. Dies ist darin begründet, dass zum einen manche Dateisysteme nur wenige Buchstaben für die Extension (MS-DOS: 3!) erlauben, und zum anderen in der Bequemlichkeit der Benutzer, nicht zu viel einzutippen.

Es gibt noch sehr viele weitere Endungen, die von dem jeweiligen Anwendersystem erzeugt werden und zur Gruppierung der Dateien dienen. Da der gesamte Dateiname beliebig ist und vom Benutzer geändert werden kann, sind einige Programmhersteller dazu übergegangen, die Dateien „ihrer“ Werkzeuge noch intern zu kennzeichnen. Dazu wird am Anfang der Datei eine oder mehrere, typische Zahlen (*magic number*) geschrieben, die eine genaue Kennzeichnung des Dateiinhalts erlauben.

Beispiel UNIX Dateinamen

Dateinamen in UNIX können bis zu 255 Buchstaben haben. In der speziellen System-V-Version sind es weniger: 14 für den Hauptnamen und 10 für die Extension.

Ausführbare Dateien, Bibliotheken, übersetzte Objekte und Speicherabzüge (*core dump*) in UNIX haben meist eine feste, genormte Struktur, die *Executable and Linkable Format* (ELF)-Struktur. Sie ist unabhängig von der Wortbreite (32 oder 64 Bit) und der CPU-Architektur. Der Dateianfang ist (in Pseudocode-Notation)

```

TYPE File Header =      RECORD
    ei_magic :           4BYTES      /*Magische Zahl 0x 7F 45 4c 46 */
    ei_class :           BYTE        /* 1 = 32 Bit, 2 = 64Bit */
    ei_data :           BYTE        /* 1 = little endian, 2 = big endian */
    ei_version :        BYTE        /* 1 = original ELF */
    ei_osabi :           BYTE        /* OSABI */
    ei_abiversion:       BYTE        /* OSABI-Version */
    ei_pad:             7BYTES      /* nicht benutzt */
    ei_abiversion:       2BYTES      /* relocatable /executable /shared /core */
    ei_machine:         2BYTES      /* Maschinencode-Architektur */
    ...
END

```

Als allererstes steht eine „magische Zahl“ am Dateikopf; sie enthält nach der Hexzahl 7F die Buchstaben ELF. Nach einigen allgemeinen Angaben über Wortlänge und Bytefolgen (*little/big endian*: s. [Abschn. 6.2](#)) folgt dann die Angabe, welches Betriebssystem verwendet werden kann, also welches ABI (*application binary interface*) verwendet wurde. Dabei bedeuten

<code>ei_osabi = 0x00</code>	System V	<code>ei_machine = 0x00</code>	No spec.
<code>0x01</code>	HP-UX	<code>instruct.set</code>	
<code>0x02</code>	NetBSD	<code>0x02</code>	SPARC
<code>0x03</code>	Linux	<code>0x03</code>	x86
<code>0x06</code>	Solaris	<code>0x08</code>	MIPS
<code>0x07</code>	AIX	<code>0x14</code>	PowerPC
<code>0x08</code>	IRIX	<code>0x28</code>	ARM
<code>0x09</code>	FreeBSD	<code>0x2A</code>	SuperH
<code>0x0C</code>	OpenBSD	<code>0x32</code>	IA-64
<code>0x0D</code>	OpenVMS	<code>0x3E</code>	x86-64
<code>0x0E</code>	NSK op.system	<code>0xB7</code>	AArch64
<code>0x0F</code>	AROS		
<code>0x10</code>	Fenix OS		
<code>0x11</code>	CloudABI		

Eine spezielle Kennung am Anfang der Datei ist nur eine mögliche Implementierung, um verschiedene Dateiarten voneinander zu unterscheiden. Diese Art der Unterscheidung ist allerdings sehr speziell. Man könnte auch direkt einen Dateityp zusätzlich zum Dateinamen angeben und damit – unabhängig von der Extension – implizit sinnvolle Operationen für die Datei angeben. Diese Art der Typisierung ist allerdings nicht unproblematisch.

Beispiel Dateityp durch Namensgebung

In UNIX und Windows NT wird der Typ einer Datei mit durch den Namen angegeben. Beispielsweise bedeutet eine Datei namens *Skript.doc.zip*, dass dies eine Datei „Skript“ ist, deren editierbare Version im PDF-Format mit einem Programm ins Format *zip* komprimiert worden ist. Um diese Datei zu lesen, muss man also zuerst ein *unzip*-Programm aufrufen (meist kann das auch das Komprimierungsprogramm) und dann die entstandene Datei einem passenden Editor übergeben. Diese Anweisungsfolge ist Konvention – sie ist weder in der Datei noch in dem Dateityp *komprimiert* enthalten. Wäre für die Datei ein Typ im Verzeichnis angegeben, so könnte man nur die erste Operation automatisch durchführen, aber nicht die zweite. Eine Typisierung müsste also in der Datei vermerkt oder es müssten geschachtelte Typen möglich sein.

Ein Problem bei typisierten Dateien ist die Vielfalt der möglichen Typen. Es muss auch möglich sein, neue Typen zu deklarieren und die entsprechenden sinnvollen Aktionen dafür anzugeben. Dazu existiert eine Konvention für die inhaltliche Charakterisierung von Dateien, die übers Internet verschickt werden können: der Medientyp MIME (*Multipurpose Internet Mail Extensions*). Er gibt an, welchen Inhalt eine Datei hat, etwa *text/html* für Webseiten oder *audio/mpeg* für eine MP3-Audiodatei.

Beispiel Email Dateitypen und Programmaktionen

Jedes Email-Programm, das zum Ausführen ein Doppelklick auf einen Dateianhang ermöglicht, hat vorher das jeweilige Anwendungsprogramm registriert, das für eine Datei dieses MIME-Typs ausgeführt werden muss. Der MIME-Typ wird dabei aus der Datei erschlossen, etwa aus der Dateiextension, oder per Hand vom Benutzer in eine Tabelle eingetragen.

Dies kann zu Problemen führen, etwa wenn eine Textdatei in einem MAC OS-System nur intern als Text gekennzeichnet ist und keine Extension besitzt. Wird sie an ein MS Windows-System geschickt, so ist unklar, was damit zu tun ist. Da dort der Inhalt aus der Extension gefolgert wird, die aber in dem Beispiel fehlt, führt ein Doppelklick zu nichts und der Benutzer kann sie so nicht bearbeiten. Erst eine manuelle Umbenennung mit einer. *doc*-Extension ermöglicht die gewohnte Bearbeitung.

Beispiel UNIX Dateitypen und Programmaktionen

Im UNIX gibt es verschiedene grafische Benutzeroberflächen, in der ein Dateimanager enthalten ist. Damit ist es möglich, für speziellen Dateien für jede Dateart (Extension) verschiedene Aktionen (Programme) anzugeben. Beim direkten Doppelklick mit der linken Maustaste auf den Dateinamen wird die als erste angegebene Applikation gestartet. Im Debian-System ist dies beispielsweise in der Datei */usr/share/applications/mimeinfo.cache* enthalten.

Die Liste aller Extensionen und der dazu gehörenden Aktionen bzw. Programme wird dazu beim Starten des Fenstermanagers für eine Benutzersitzung eingelesen.

Beispiel Windows NT Dateitypen und Programmaktionen

Unter Windows NT gibt es einen sogenannten *Registration Editor*, der eine eigene Datenbank aller Anwendungen, Treiber, Systemkonfigurationen usw. verwaltet. Alle Programme im System müssen dort angemeldet werden, ebenfalls alle Extensionen. Ein Doppelklick auf den Dateinamen im Dateimanager-Programm (*Explorer*) ruft das assoziierte Programm auf, mit dem Dateinamen als Argument.

Man beachte, dass in allen Beispielen die Typinformation der Datei nicht bei der Dateispezifikation vom Betriebssystem verwaltet wird, sondern in extra Dateien steht und mit extra Programmen aktuell gehalten und verwendet wird. Dies ist nicht unproblematisch. Die beste Art der Dateiverwaltung ist deshalb die Organisation aller Daten als allgemeine Datenbank, in der zusätzliche Eigenschaften von Dateien wie Typ durch zusätzliche Merkmale (Spalten in [Abb. 4.1](#)) verwirklicht werden. Damit wird der Dateityp mit dem Dateinamen inhaltlich verbunden, so dass das Betriebssystem beim Zugriff von Programmen auf die Datei vorher prüfen kann, ob das Dateiformat dem einlesenden Programm bekannt sein kann (Konsistenzcheck) und fehlerhafte oder unzulässige Zugriffe verhindert. Dies ist ähnlich wie die Überprüfung von Datentypen von Übergabeparametern einer Prozedur oder Methode durch den Compiler oder Interpreter.